Анатомия Tor

Протокол анонимности побайтно

Alessandro Greco

Первое издание — версия 1.0.0

Анатомия Tor — Протокол анонимности побайтно
Первое издание, версия 1.0.0. Перевод на русский язык.

Copyright © 2026 Alessandro Greco.

Это свободное произведение: текст, иллюстрации, исходные файлы LaTeX и код лабораторных работ можно распространять и изменять на условиях Стандартной общественной лицензии GNU (GNU General Public License), опубликованной Фондом свободного программного обеспечения, версии 3 либо (по вашему выбору) любой более поздней версии (GPL-3.0-or-later). Произведение распространяется в надежде, что оно будет полезным, но *без каких-либо гарантий*, в том числе подразумеваемых гарантий товарной пригодности или пригодности для определённой цели; подробности приведены в лицензии: `https://www.gnu.org/licenses/gpl-3.0.html`.

Исходная форма, пригодная для изменения (исходные файлы LaTeX, иллюстрации TikZ, код и сеть лабораторных работ), опубликована по адресу `https://github.com/aleff-github/onion-routing-anatomy`.

Книга распространяется бесплатно; права на её использование и распространение определяются указанной выше лицензией.

Этот перевод полностью создан с помощью инструментов искусственного интеллекта на основе оригинального итальянского издания, которое остаётся эталонным текстом произведения.

«Tor» и логотип Tor являются товарными знаками The Tor Project, Inc. Эта книга — независимое произведение; она не одобрена и не спонсируется Tor Project.

Упомянутые названия продуктов и сервисов принадлежат их владельцам и используются только в описательных целях.

Оглавление

Предисловие **xi**

I История и основы **1**

1 Анонимность, наблюдение и метаданные **3**
1.1 Содержание и метаданные . . . 3
1.1.1 Что раскрывают метаданные . . . 4
1.2 Анализ трафика . . . 4
1.3 Точная терминология . . . 5
1.4 Кому нужна анонимность? . . . 5
1.4.1 Анонимность любит компанию . . . 6

2 Истоки: mix-сети, ремейлеры, Crowds и Freedom **9**
2.1 Mix Чаума (1981) . . . 9
2.2 Высокая и низкая задержка . . . 10
2.3 Ремейлеры . . . 10
2.4 К низкой задержке . . . 11

3 От луковой маршрутизации NRL к Tor Project **15**
3.1 В Naval Research Laboratory . . . 15
3.2 Три поколения . . . 15
3.3 Сеть Tor . . . 17
3.4 Организация и сообщество . . . 17

4 Модель угроз Tor **19**
4.1 Цели . . . 19
4.2 Что не является целью . . . 20
4.3 Противник . . . 20
4.3.1 Подтверждение и анализ . . . 21
4.4 От чего Tor защищает, а от чего нет . . . 21

II Низкоуровневый протокол **23**

5 Криптография, необходимая для Tor **25**
5.1 Пять задач . . . 25
5.2 Шифры . . . 26
5.2.1 AES в режиме счётчика . . . 26
5.2.2 TLS . . . 26
5.3 Хеш-функции . . . 27

5.4 MAC и выработка ключей . . . 27
5.4.1 HMAC . . . 27
5.4.2 Функции выработки ключей . . . 28
5.5 Обмен ключами: X25519 . . . 28
5.6 Подписи: Ed25519 и RSA . . . 28
5.7 Современная криптография . . . 28

6 Архитектура сети 31
6.1 Роли . . . 31
6.2 Уровни протокола . . . 32
6.3 Веб-страница от начала до конца . . . 33
6.4 Реализации и версии . . . 34
6.4.1 Две реализации . . . 34
6.4.2 Субпротоколы . . . 34

7 Каналы и протокол соединения 37
7.1 Что такое канал? . . . 37
7.2 Идентичность реле . . . 38
7.3 Процедура установления канала в общих чертах . . . 39
7.4 Уровень TLS . . . 41
7.5 CERTS: подтверждение идентичности . . . 41
7.5.1 Ячейка . . . 41
7.5.2 Сертификат Tor Ed25519 . . . 42
7.5.3 Проверка . . . 43
7.5.4 Устаревшая цепочка RSA . . . 43
7.5.5 Что реле действительно отправляет . . . 43
7.6 AUTH_CHALLENGE и AUTHENTICATE . . . 44
7.6.1 Кто аутентифицируется? . . . 44
7.6.2 Вызов . . . 44
7.6.3 Ответ . . . 45
7.7 NETINFO: время и адреса . . . 45
7.8 После установления канала . . . 46
7.9 Лабораторная работа: открытие проверенного канала . . . 47

8 Ячейки 51
8.1 Зачем нужны ячейки? . . . 51
8.2 От потока TLS к ячейкам . . . 52
8.3 Форматы ячеек . . . 52
8.3.1 Константы . . . 52
8.3.2 Ячейки фиксированной длины . . . 53
8.3.3 Ячейки переменной длины . . . 53
8.3.4 Как различать два формата . . . 53
8.3.5 Особый случай VERSIONS . . . 54
8.4 CircID . . . 54
8.4.1 Кто выбирает CircID? . . . 55
8.5 Команды ячеек . . . 55
8.5.1 DESTROY . . . 56
8.5.2 RELAY_EARLY . . . 57
8.6 Внутри RELAY-ячеек . . . 57
8.6.1 Recognized и Digest: двухступенчатый фильтр . . . 58
8.6.2 Заполнение сообщений RELAY . . . 58

8.6.3 Команды RELAY . . . 59
8.7 К новому формату: предложение 340 и CGO . . . 59
8.8 Анатомия сеанса . . . 60
8.9 Лабораторная работа: кодировщик ячеек . . . 62

9 Построение цепи: TAP, CREATE_FAST, ntor и ntor-v3 67
9.1 По одному переходу . . . 67
9.2 Ячейки и сообщения . . . 68
9.2.1 CREATE2 и CREATED2 . . . 68
9.2.2 EXTEND2 и EXTENDED2 . . . 69
9.3 Что должно обеспечивать согласование ключей . . . 69
9.4 TAP: первый протокол согласования ключей . . . 70
9.5 CREATE_FAST: когда открытый ключ не нужен . . . 70
9.6 ntor . . . 71
9.6.1 Компоненты . . . 71
9.6.2 Протокол . . . 71
9.7 ntor-v3 . . . 72
9.7.1 Что меняется . . . 72
9.7.2 Протокол . . . 72
9.7.3 Расширения . . . 72
9.7.4 Какой протокол согласования ключей использует tor сегодня? . . . 73
9.8 От согласованных секретов к ключам цепи . . . 73
9.9 Лабораторная работа: построение реальных цепей . . . 74

10 «Луковица»: шифрование ячеек RELAY и CGO 79
10.1 Традиционная схема . . . 79
10.1.1 Прямое направление: от клиента внутрь цепи . . . 80
10.1.2 Обратное направление: от цепи к клиенту . . . 80
10.1.3 Цепь как «труба с промежуточными выходами (leaky pipe)» . . . 80
10.2 Распознавание ячейки . . . 80
10.3 Слабости tor1 . . . 81
10.3.1 Пластичность . . . 81
10.3.2 Атака маркирования (tagging) . . . 81
10.3.3 Другие ограничения . . . 82
10.4 Counter Galois Onion . . . 82
10.4.1 Свойства . . . 82
10.4.2 Как это работает на высоком уровне . . . 82
10.4.3 CGO на практике . . . 83
10.5 Лабораторная работа: трёхпереходная цепь . . . 84

11 Потоки, SENDME и управление перегрузкой 89
11.1 Потоки . . . 89
11.1.1 Открытие потока: BEGIN и CONNECTED . . . 89
11.1.2 Передача данных: DATA . . . 91
11.1.3 Закрытие потока: END . . . 91
11.1.4 DNS на выходном реле . . . 91
11.1.5 Потоки каталогов: BEGIN_DIR . . . 91
11.2 Исходное управление потоком . . . 91
11.2.1 Проблема . . . 91
11.2.2 Окна и SENDME . . . 92
11.2.3 Аутентифицированные SENDME . . . 92

11.3 Управление перегрузкой . . . 92
11.3.1 Сигнал: время туда и обратно (RTT) . . . 93
11.3.2 Алгоритмы . . . 93
11.3.3 Параметры: почему 31? . . . 93
11.3.4 Больше никаких окон потока: XON и XOFF . . . 93
11.4 Лабораторная работа: загрузка файла через Tor . . . 94

12 Система каталогов 97
12.1 Проблема . . . 97
12.2 Участники . . . 98
12.3 Документы . . . 98
12.4 Голосование . . . 100
12.4.1 Расписание . . . 100
12.4.2 Объединение голосов . . . 100
12.4.3 Флаги . . . 101
12.4.4 Пропускная способность . . . 101
12.5 Анатомия консенсуса . . . 101
12.6 Проверка консенсуса . . . 103
12.7 Как клиент остаётся актуальным . . . 103
12.8 Лабораторная работа: загрузка и проверка консенсуса . . . 104

13 Выбор маршрута, входные узлы и vanguards 107
13.1 Порядок и ограничения . . . 107
13.2 Взвешивание по пропускной способности . . . 108
13.2.1 Зачем нужны веса . . . 108
13.2.2 Позиции цепи неравноценны . . . 108
13.2.3 Веса реальной сети . . . 109
13.3 Входные узлы (guards) . . . 109
13.3.1 Какую проблему они решают . . . 109
13.3.2 Три, один, два . . . 110
13.3.3 Алгоритм . . . 110
13.4 Vanguards . . . 111
13.5 Лабораторная работа: исследование публичной сети . . . 112

14 Onion-сервисы v3 115
14.1 Что предоставляет onion-сервис . . . 115
14.2 Адрес . . . 116
14.3 Ослеплённые ключи . . . 117
14.3.1 Проблема . . . 117
14.3.2 Решение . . . 117
14.3.3 Креденциалы и ключи, хранимые вне сети . . . 117
14.4 Определение местоположения дескриптора . . . 118
14.4.1 Временные периоды . . . 118
14.4.2 Кольцо HSDir . . . 118
14.4.3 Только анонимные запросы . . . 119
14.5 Дескриптор . . . 119
14.6 Введение и встреча (rendezvous) . . . 120
14.7 Ограниченное обнаружение (restricted discovery) . . . 122
14.8 Защитные механизмы . . . 122
14.9 Лабораторная работа: поиск и чтение дескриптора . . . 123

15 Защита от DoS и управление ресурсами 127
15.1 Почему DoS — проблема анонимности . . . 127
15.2 Физические ресурсы . . . 128
15.2.1 Память . . . 128
15.2.2 Процессор . . . 128
15.2.3 Пропускная способность . . . 128
15.3 Ограничения по IP-адресам . . . 128
15.3.1 Код и консенсус . . . 129
15.4 Onion-сервисы под атакой . . . 129
15.4.1 Ограничения в пунктах введения . . . 130
15.4.2 Доказательство выполнения работы . . . 130
15.4.3 Линейная шкала усилия . . . 131
15.4.4 Управление усилием . . . 131
15.5 Лабораторная работа: вызов защитных механизмов . . . 132

16 Цензура, мосты и подключаемые транспорты 135
16.1 Как блокируют Tor . . . 135
16.2 Мосты . . . 136
16.2.1 Проблема распространения . . . 136
16.2.2 Lox . . . 137
16.3 Подключаемые транспорты . . . 137
16.3.1 Архитектура . . . 137
16.3.2 Протокол между tor и PT . . . 137
16.3.3 Lyrebird . . . 138
16.4 Стратегии . . . 138
16.4.1 obfs4: не быть похожим ни на что распознаваемое . . . 138
16.4.2 meek: за крупным сервисом . . . 139
16.4.3 Snowflake: множество эфемерных прокси . . . 139
16.4.4 WebTunnel: выглядеть как веб-сайт . . . 139
16.4.5 Conjure: скрытие в адресном пространстве . . . 139
16.5 Лабораторная работа: obfs4-мост глазами цензора . . . 140

III Tor на практике 143

17 Установка и torrc 145
17.1 Какой вариант Tor устанавливать? . . . 145
17.2 Файл torrc . . . 146
17.2.1 Проверка перед использованием . . . 146
17.3 Конфигурация клиента . . . 146
17.3.1 Изоляция потоков . . . 147
17.3.2 Выбор реле: соблюдайте осторожность . . . 148
17.3.3 Другие порты . . . 148
17.4 Порт управления . . . 148
17.5 Журналы . . . 148

18 Порт управления и Stem 151
18.1 Протокол управления . . . 151
18.1.1 Цепи с точки зрения tor . . . 152
18.1.2 Основные команды . . . 153
18.2 Stem . . . 153

18.3 Лабораторная работа: изоляция потоков в реальном времени 154

19 Приватная сеть Tor: Chutney и Shadow 157
19.1 Почему нужна приватная сеть? 157
19.2 Эмуляция или симуляция? 158
19.3 Лабораторная среда книги 159
19.3.1 Требования 159
19.3.2 Сборка образа и запуск сети 159
19.4 Топология сети 160
19.5 Тестовая сеть — не реальная сеть 161
19.6 Внутри каталога узла 162
19.7 Использование сети 163
19.7.1 Доступ через приватную сеть 163
19.7.2 Доступ к onion-сервису 163
19.7.3 Наблюдение за цепями с помощью Stem 163
19.7.4 Работа внутри контейнера 164
19.8 Первый контакт с реле 164
19.9 Shadow: сеть внутри симулятора 165

20 Наблюдение за трафиком: Wireshark и журналы 169
20.1 Что может видеть наблюдатель 169
20.2 Инструменты 170
20.2.1 Расшифрование собственного TLS 170
20.3 Лабораторная работа: ячейки внутри TLS 171
20.4 Журналы tor 172

21 Публикация onion-сервиса 175
21.1 Первый onion-сервис 175
21.1.1 Как не раскрыть сервер 176
21.2 Onion-Location 177
21.3 Ограниченное обнаружение 177
21.4 Другие требования 179
21.4.1 Адреса с узнаваемым префиксом 179
21.4.2 Одношаговые сервисы 179
21.4.3 Onionbalance 179

22 Эксплуатация реле или моста 181
22.1 Какой тип узла выбрать? 181
22.2 Базовая конфигурация 182
22.2.1 Жизненный цикл нового реле 182
22.2.2 Ключи 183
22.3 Политика выхода 183
22.4 Лабораторная работа: проверка политики выхода 184
22.5 Семейства реле 185
22.6 Запуск моста 186

23 Tor Browser, Tails и Whonix 189
23.1 Почему одного прокси недостаточно 189
23.2 Tor Browser 190
23.2.1 Одна цепь на сайт 190
23.2.2 Защита от цифровых отпечатков 191

23.2.3 Уровни безопасности . . . 191
23.3 За пределами браузера . . . 192
23.3.1 Tails . . . 192
23.3.2 Whonix . . . 193

24 Измерение сети: Tor Metrics и OONI 195
24.1 Источники данных . . . 195
24.2 Подсчёт анонимных пользователей . . . 196
24.2.1 Выявление цензуры . . . 197
24.3 Производительность . . . 197
24.4 Лабораторная работа: чтение реального консенсуса . . . 197
24.5 OONI . . . 198

25 Правовые и этические аспекты в Италии и ЕС 201
25.1 Инструмент и способы его использования . . . 201
25.2 Операторы ретрансляторов . . . 202
25.2.1 Простая передача (mere conduit) . . . 202
25.2.2 Гражданская ответственность и уголовные расследования . . . 203
25.2.3 Хранение данных . . . 203
25.2.4 Журналы — это персональные данные . . . 203
25.3 Исследователи . . . 204

IV Атаки и защита 207

26 Корреляция трафика, распознавание сайтов и атаки Sybil 209
26.1 Позиция атакующего . . . 209
26.2 Сквозная корреляция . . . 210
26.3 Лабораторная работа: корреляция собственного трафика . . . 211
26.4 Распознавание сайтов по трафику . . . 212
26.5 Sybil: размножение идентичностей ретрансляторов . . . 213
26.6 Обнаружение сторожевого узла . . . 213

27 Реальные атаки на сеть 215
27.1 2014: сигнализация через типы ячеек . . . 215
27.2 Лабораторная работа: лимит RELAY_EARLY . . . 216
27.3 2020–2021: KAX17 и здоровье сети . . . 217
27.4 2022–2023: отказ в обслуживании . . . 218
27.5 Выводы . . . 218

28 Ошибки операционной безопасности и деанонимизация: уроки 221
28.1 Четыре уровня . . . 221
28.2 Ошибки приложений . . . 222
28.3 Лабораторная работа: tor обнаруживает признак DNS leak . . . 222
28.4 Ошибки содержимого . . . 223
28.5 Поведенческие ошибки: слишком малое множество анонимности . . . 224
28.6 Контрольный список . . . 224

V Исследования и будущее **227**

29 Академические исследования Tor **229**
29.1 Где публикуются исследования Tor . . . 229
29.2 Измерение анонимности . . . 230
29.3 Четыре метода . . . 230
29.4 Лабораторная работа: время до первого компрометационного события . . . 231
29.5 Безопасное проведение исследований . . . 233

30 Arti: Tor на Rust **235**
30.1 Зачем переписывать tor? . . . 235
30.2 Архитектура . . . 236
30.2.1 Сравнение: запись ячейки . . . 237
30.3 Поддерживаемые функции . . . 237
30.4 Лабораторная работа: Arti в приватной сети . . . 238

31 Открытые проблемы и текущие проекты **243**
31.1 Как меняется протокол . . . 243
31.2 Квантовые компьютеры . . . 244
31.3 Более крупные сообщения — и более мелкие . . . 245
31.4 Производительность: несколько путей . . . 245
31.5 Масштабируемость: Walking Onions . . . 246
31.6 Другие открытые направления . . . 246
31.7 Как внести вклад . . . 246

A Глоссарий терминов Tor **249**

B Дополнительная криптография **253**
B.1 Одна кривая, две формы . . . 253
B.1.1 Фиксация битов ключей X25519 . . . 253
B.1.2 Точка на бесконечности . . . 254
B.2 Из формы Монтгомери в форму Эдвардса: перекрёстный сертификат ntor . . . 254
B.3 Изменяемость шифртекста потокового шифра . . . 255

C Руководство по спецификациям torspec **257**
C.1 Где их найти . . . 257
C.2 Соглашения . . . 257
C.3 Карта документов . . . 258
C.4 Как читать раздел спецификации . . . 258
C.5 Arti и спецификации . . . 259

D Решения упражнений **261**

Литература **285**

Предметный указатель **289**

Предисловие

Tor ежедневно используют миллионы людей, однако о нём почти всегда говорят через призму его реальных или предполагаемых применений: обход цензуры, защита источников, «тёмная сеть». Эта книга подходит к теме иначе: она объясняет, как работает Tor. Мы начинаем с байтов, которыми обмениваются два реле, а затем, глава за главой, переходим к архитектурным решениям, атакам, реальным случаям и вопросам, которые по-прежнему остаются открытыми. И читателю не предлагается принимать изложенное на веру: каждый механизм, описанный в технической части, создаётся и проверяется в лаборатории, в частной сети Tor, которую можно запустить на обычном компьютере.

Происхождение, метод и приглашение к проверке

В основу этой книги легли учебные заметки и документы, которые я составлял на протяжении многих лет, начиная с бакалаврской выпускной работы. Чтобы превратить эти материалы в связное техническое изложение, я руководил использованием инструментов искусственного интеллекта. Они внесли существенный вклад в разработку объяснений, аргументацию и написание текста, а также в создание иллюстраций, упражнений, кода и лабораторных работ. ИИ не просто редактировал мой текст и не заменяет независимую экспертную проверку.

Несмотря на технические проверки и мою собственную редактуру, в книге могут оставаться ошибки, упрощения и устаревшие сведения. Прошу читателей сообщать о них и предлагать исправления через репозиторий проекта. Я решил распространять книгу бесплатно через официальные каналы проекта, а её исходные материалы — под лицензией GNU GPL-3.0-or-later, следуя принципу «diffusione dei saperi senza fondare poteri»: распространять знания, не создавая центров власти. Условия повторного использования, в том числе коммерческого, определяет лицензия.

Для кого эта книга

Книга предназначена для студентов, изучающих информатику и информационную безопасность, специалистов, которые сталкиваются с Tor в работе, людей, уже управляющих реле или планирующих это делать, а также исследователей. От читателя требуется базовое понимание сетей (TCP/IP и общая идея TLS), знакомство с командной строкой и, для лабораторных работ, Python. Необходимые криптографические основы вводятся в гл. 5, а дополнительные сведения приведены в прил. B.

Как читать эту книгу

Книга состоит из пяти частей. Их можно читать последовательно или выбрать один из следующих маршрутов:

Исторический и концептуальный маршрут Часть I (главы 1–4) и часть V (главы 29–31): что означает анонимность, откуда появился Tor, от каких противников он защищает и в каком направлении развивается.

Протокольный маршрут Часть II (главы 5–16), которую лучше читать по порядку вместе с лабораторными работами: это техническое ядро книги.

Практический маршрут Часть III (главы 17–25): настройка tor, наблюдение за его трафиком, публикация onion-сервиса, эксплуатация реле, использование Tor Browser, измерение сети и понимание правовых аспектов. Глава 19 является обязательной основой для всех лабораторных работ.

Маршрут по безопасности Часть IV (главы 26–28), перед которой следует прочитать гл. 4: атаки рассматриваются через исследования, реальные случаи и эксплуатационные ошибки.

Условные обозначения

Каждая глава начинается с учебных целей и строки *Последняя проверка*, где указаны дата, версии программного обеспечения и спецификаций, по которым проверялось содержание. Tor меняется, поэтому читателю важно понимать, какую версию Tor описывает текст. Во всей книге *Tor* обозначает сеть и проект, *tor* — программу на C, а *Arti* — реализацию на Rust. Ссылки на спецификации записываются в форме [document § section]; предложения об изменении протокола идентифицируются по номеру. Принятые в русском издании технические термины вместе с их английскими эквивалентами собраны в прил. A.

Врезки отделяют основной текст от отступлений: *подробных разборов*, *исторических справок* и *предупреждений*. Сложность упражнений обозначается звёздами: одна — на понимание, две — на применение, три — лабораторная задача или небольшой исследовательский проект; решения находятся в прил. D. Лабораторные работы пронумерованы от 1 до 20. В части II они постепенно создают небольшой клиент Tor на Python (`torlab`); код и лабораторная сеть сопровождают книгу, а автоматизированные проверки можно повторно запускать после обновлений. Все эти ресурсы — пакет `torlab`, двадцать лабораторных работ, тестовая сеть Docker и инструменты проверки — доступны в репозитории книги: `https://github.com/aleff-github/onion-routing-anatomy`.

Этическая оговорка

Все эксперименты в этой книге выполняются в частной лабораторной сети Tor либо используют действующую сеть только как обычный клиент, без наблюдения за чужим трафиком и без его изменения. Именно такой принцип рекомендует исследователям Tor Research Safety Board; он также имеет правовое значение (гл. 25). Главы об атаках написаны с аналитической и защитной точки зрения: они объясняют, что происходило и как от этого защищаться, а не как атаковать конкретного человека.

Благодарности

Эта книга опирается на работу Tor Project и его сообщества. Открытые спецификации, исходный код `tor` и Arti, предложения об изменении протокола и документация лежат в основе всех последующих технических утверждений. Благодаря их публичной доступности утверждения можно сверять с первоисточниками, а не принимать на веру. Благодарности заслуживают и разработчики свободных инструментов, с помощью которых создавалась

книга и тестировались её лабораторные работы: LaTeX, Python, Docker, Chutney, Wireshark и многие другие проекты, упомянутые в тексте.

Часть I

История и основы

Глава 1

Анонимность, наблюдение и метаданные

Последняя проверка: 23 сентября 2026 г. — данные Tor Metrics по 20 сентября 2026 г.

Цели главы

К концу этой главы вы сможете:

- различать конфиденциальность, анонимность, псевдонимность, несвязываемость и ненаблюдаемость;
- объяснять, почему метаданные коммуникации способны раскрывать не меньше, а иногда и больше, чем её содержание;
- описывать анализ трафика и объяснять, почему одного шифрования недостаточно для защиты от него;
- определять, кто использует Tor и почему разнообразие пользователей само по себе является формой защиты.

Эта книга посвящена системе, предназначенной для защиты особого вида информации: не того, *что* мы говорим в сети, а того, *с кем* мы общаемся. Прежде чем разбирать устройство системы, необходимо понять решаемую ею задачу и то, почему она отличается от более привычной задачи, которую решает шифрование.

1.1. Содержание и метаданные

Когда два компьютера обмениваются данными через Интернет, каждый пакет несёт два вида информации. **Содержание** — это то, чем обмениваются приложения: текст сообщения или содержимое веб-страницы. **Метаданные** описывают *сам факт коммуникации*: кто отправляет данные, кому, когда, как часто и в каком объёме. Часть метаданных необходима для работы сети, поскольку маршрутизаторы должны знать, куда доставлять пакеты. IP-адрес назначения и IP-адрес источника, на который следует отправлять ответы, передаются открытым текстом в каждом пакете.

Шифрование, ставшее повсеместным благодаря HTTPS, защищает содержание, но оставляет метаданные видимыми. Любой, кто наблюдает сеть, — от интернет-провайдера пользователя до оператора крупной точки обмена трафиком — всё ещё может видеть, что пользователь подключается к определённому сайту, когда устанавливается соединение, сколько оно длится и какой объём трафика создаёт.

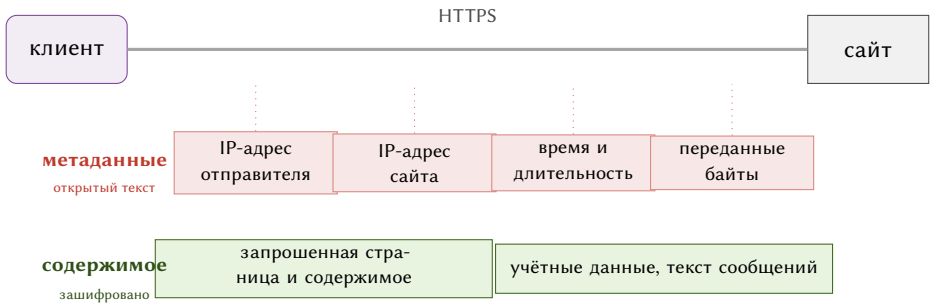


Рис. 1.1: Что может видеть наблюдатель при HTTPS-соединении. Содержание зашифровано, но метаданные остаются видимыми; именно эту информацию и пытается защищать Tor.

1.1.1. Что раскрывают метаданные

Метаданные легко принять за безобидную информацию: «Они знают только, с кем я разговариваю, но не знают, что я говорю». Это впечатление обманчиво. Достаточно нескольких примеров:

- человек в два часа ночи посещает сайт центра помощи жертвам домашнего насилия, а затем сайт адвоката по бракоразводным делам;
- за несколько дней до публикации расследования журналист обменивается сообщениями с сотрудником министерства, которого касается расследование;
- сотрудник регулярно посещает сайт профсоюза или сайт ассоциации пациентов, посвящённой определённому заболеванию.

Ни в одном из этих случаев не требуется читать содержание, чтобы узнать очень многое. У метаданных есть ещё одно преимущество для тех, кто их собирает: они структурированы, их легко хранить и анализировать в больших масштабах, а также сопоставлять с другими данными. Генерал Майкл Хейден, бывший директор как NSA, так и CIA, выразил эту мысль в ставшей известной реплике во время дискуссии в Университете Джонса Хопкинса 1 апреля 2014 года: «Мы убиваем людей на основании метаданных» (перевод), после чего сразу уточнил: «но это не то, что мы делаем с этими метаданными» [Jus14]. Уточнение не меняет сути: те, кто собирает метаданные, считают их достаточными для решений такой степени серьёзности.

1.2. Анализ трафика

Анализ трафика изучает метаданные коммуникаций, чтобы делать выводы, не читая само содержание. Он так же стар, как телекоммуникации. Даже во время войн XX века наблюдение за тем, как часто передаёт вражеская радиостанция, с кем она связывается и в каком темпе работает, позволяло выявлять структуру армии, не расшифровывая сообщения.

В Интернете анализ трафика имеет две основные формы:

Анализ адресов Наблюдатель на линии связи видит IP-адреса источника и назначения. Это простейшая форма и именно с ней Tor борется наиболее непосредственно: ни одна отдельная точка сети не видит обе конечные стороны соединения.

Анализ объёма и времени Даже не имея полной адресной информации, наблюдатель может сравнивать потоки по объёму и временным характеристикам. Если пользователь загружает 3 MB в 10:15:02, а сервер в другой части сети в 10:15:02 отправляет 3 MB, весьма

вероятно, что наблюдения относятся к одной и той же коммуникации. Такая **корреляция трафика** является главным ограничением Tor, и мы будем неоднократно к ней возвращаться (гл. 4 и 26).

1.3. Точная терминология

Слово «анонимность» используется в нескольких значениях, и смешение этих значений неоднократно приводило к ошибкам в проектировании подобных систем. Научная литература, в частности терминология, предложенная Пфитцманном и Хансеном [PH10], различает несколько понятий:

Конфиденциальность Неавторизованная сторона не может прочитать содержание. Именно это обеспечивает шифрование.

Анонимность Субъект не может быть идентифицирован внутри группы субъектов, называемой **множеством анонимности**. Анонимность никогда не бывает абсолютной: она существует «среди» других людей. Чем больше и разнообразнее множество, тем выше защита.

Псевдонимность Человек действует под идентификатором, отличным от собственного имени, причём этот идентификатор может быть постоянным. Учётная запись с вымышленным именем — это псевдоним, а не анонимность: все её действия можно связать между собой.

Несвязываемость Наблюдатель не может определить, связаны ли два интересующих объекта — например, два сообщения, пользователь и веб-сайт или два посещения. Именно это свойство Tor пытается обеспечить между пользователями и их пунктами назначения.

Ненаблюдаемость Наблюдатель не может даже определить, *происходит ли* коммуникация. Это значительно более сильное свойство, которого Tor не обеспечивает: наблюдатель, следящий за пользователем, может увидеть использование Tor, если только не применяется подключаемый транспорт (гл. 16).

Историческая справка: Отделение идентификации от маршрутизации

Трое исследователей Naval Research Laboratory, придумавших onion routing в 1995 году, — Дэвид Голдшлаг, Майкл Рид и Пол Сайверсон — предпочитали не говорить об анонимности. Их цель, как они писали в статье 1996 года, к которой Сайверсон позднее вернулся в воспоминаниях об этом периоде, состояла в том, чтобы «отделить идентификацию от маршрутизации» [Syv11]. Пользователю может понадобиться подтвердить свою личность системе, с которой он взаимодействует, например рабочему компьютеру, не раскрывая каждой промежуточной точке сети, что кто-то из его местоположения подключается именно к этому компьютеру.

1.4. Кому нужна анонимность?

По оценкам Tor Metrics, в августе и сентябре 2026 года к сети Tor ежедневно подключались от 2,8 до 3,5 млн пользователей без использования мостов, в среднем около 3,2 млн. Для Италии оценка находилась в диапазоне от 44 000 до 83 000 пользователей в день, в среднем около 60 000. Эти оценки характеризуются значительной неопределённостью, поскольку сеть оценивает число пользователей, а не подсчитывает их напрямую (гл. 24). Кто эти люди? Они принадлежат к очень разным группам:

- обычные пользователи, которые не хотят, чтобы рекламные компании, интернет-провайдеры или операторы используемых ими сетей отслеживали их;
- журналисты и их источники, активисты и правозащитники, особенно в странах с авторитарными режимами;
- люди, живущие в условиях цензуры и использующие Tor для доступа к заблокированным сайтам (гл. 16);
- жертвы насилия или преследования, которым необходимо скрывать своё местоположение;
- компании, не желающие раскрывать конкурентам направления своих исследований;
- сотрудники правоохранительных органов и военнослужащие, которым требуется посещать сайты или собирать информацию, не раскрывая свою личность: onion routing возник в лаборатории ВМС США именно для этой цели.

Небольшая доля пользователей применяет Tor для незаконной деятельности, как это происходит с любым средством связи. Публичная дискуссия обычно сосредоточена именно на этой группе. В части IV и гл. 25 мы рассмотрим эту тему на основе фактических данных и конкретных случаев.

1.4.1. Анонимность любит компанию

Разнообразие пользователей важно и по более глубокой причине. Система анонимности, которой пользуется только одна группа — например, военные, — не защищала бы никого в этой группе: наблюдатель, увидев подключение к сети, сразу понял бы, что перед ним военный, а любой наблюдатель исходящего трафика знал бы, что тот пришёл от военных. Сайверсон приводит именно этот аргумент применительно к сети, которую ВМС США могли бы построить исключительно для себя: всё, что выходило бы из неё, распознавалось бы как трафик ВМС [Syv11]. Анонимность каждого человека зависит от количества других пользователей и от их разнообразия. Эту идею кратко выражает фраза «анонимность любит компанию» (в оригинале *Anonymity Loves Company*), которая также стала названием статьи Динглдайна и Мэтьюсона [DM06]. Именно поэтому Tor задуман как бесплатная и открытая система, доступная каждому: чем больше людей ею пользуются, тем лучше она защищает всех.

Итоги

- Шифрование защищает содержание; метаданные — кто, с кем, когда и в каком объёме общается — остаются видимыми и могут раскрывать очень многое.
- Анализ трафика извлекает сведения из адресов, объёмов и временных характеристик коммуникации.
- Анонимность означает невозможность идентификации внутри множества; её необходимо отличать от псевдонимности, несвязываемости и ненаблюдаемости.
- Tor обслуживает миллионы очень разных пользователей, и это разнообразие является частью его защиты.

Упражнения

Упражнение 1.1 ⋆ Перечислите метаданные, которые интернет-провайдер может наблюдать, когда его клиент читает статью на новостном сайте по HTTPS. Какие данные остаются видимыми, если клиент использует VPN? А если Tor?

Упражнение 1.2 ⋆ Отнесите каждый из следующих примеров к анонимности, псевдонимности или ни к одной из этих категорий: аккаунт в социальной сети под вымышленным именем; неподписанное сообщение на публичной доске объявлений; письмо, подписанное инициалами; пожертвование наличными, опущенное в ящик для сбора средств.

Упражнение 1.3 ⋆⋆ За один час сайт посещают 50 разных пользователей Tor. Один из них публикует комментарий с орфографической ошибкой, типичной для носителей итальянского языка, причём комментарий появляется в 3:00 по итальянскому времени. Как это изменяет множество анонимности пользователя? Что этот пример показывает о роли поведения самого пользователя?

Упражнение 1.4 ⋆⋆ Сравните коммерческий VPN и Tor с точки зрения того, кто способен связать пользователя с его пунктом назначения. Где в каждом случае сосредоточено доверие?

Дополнительная литература

- A. Pfitzmann, M. Hansen, *A terminology for talking about privacy by data minimization* [PH10]: базовая терминология.
- P. Syverson, *A Peel of Onion* [Syv11]: история onion routing, рассказанная одним из его создателей.
- Tor Metrics, `https://metrics.torproject.org/`: оценки числа пользователей по странам.

Глава 2

Истоки: mix-сети, ремейлеры, Crowds и Freedom

Последняя проверка: 23 сентября 2026 г. — исторические источники, цитируемые в этой главе

Цели главы

К концу этой главы вы сможете:

- объяснять идею *mix*, предложенную Дэвидом Чаумом, и её значение для систем анонимности;
- различать высоколатентные системы для электронной почты и низколатентные системы для веб-сёрфинга;
- проследить развитие ремейлеров от anon.penet.fi до Mixminion;
- определить место Crowds, PipeNet и Freedom в истории, приведшей к Tor.

Tor возник не на пустом месте. На сегодняшний день это очередной этап более чем сорокалетней истории исследований и реально развёрнутых систем. Некоторые из них добились успеха, многие оказались неудачными, но каждая чему-то нас научила. В этой главе прослеживаются основные этапы развития вплоть до появления onion routing.

2.1. Mix Чаума (1981)

В 1981 году Дэвид Чаум опубликовал пятистраничную статью, положившую начало целому направлению исследований: *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81]. Задача касалась электронной почты: как отправить сообщение так, чтобы наблюдатель не мог связать отправителя с получателем, даже если вся сеть находится под наблюдением?

Решением стал **mix**: сервер, который принимает сообщения, зашифрованные его открытым ключом, расшифровывает их, собирает в группу (*batch*) и пересылает вместе в порядке, отличном от порядка поступления. Связать входящее сообщение с исходящим мешают три приёма:

- исходящее сообщение выглядит совершенно иначе, чем соответствующее входящее, поскольку mix снял один слой шифрования;
- все сообщения имеют одинаковую длину;
- сообщения перемешиваются и выпускаются вместе, поэтому даже время передачи не связывает их друг с другом.

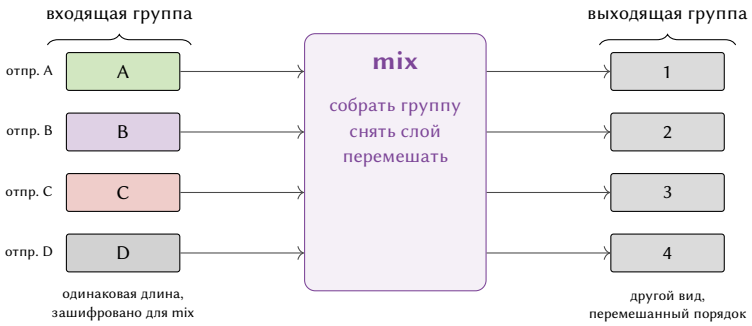


Рис. 2.1: Mix Чаума. Наблюдатель, видящий одновременно вход и выход, не может связать сообщения: все они имеют одинаковую длину, их внешний вид меняется после снятия mix-сервером слоя шифрования, а выходят они одновременно и в другом порядке.

Вместо доверия одному mix отправитель может пропустить сообщение через **цепочку** mix-серверов, послойно зашифровав его ключом каждого из них. Каждый mix снимает свой слой и узнаёт только следующий узел. Если хотя бы один mix в цепочке честен, связь между отправителем и получателем остаётся скрытой. Именно здесь находится зародыш идеи onion routing.

В той же статье Чаум предложил *анонимные обратные адреса*, позволяющие получателям отвечать, не зная отправителя, а также цифровые псевдонимы. В 1988 году он добавил второй механизм — *DC-nets* [Cha88], обеспечивающий безусловную анонимность ценой очень высоких затрат и оказавший преимущественно теоретическое влияние.

2.2. Высокая и низкая задержка

Mix Чаума обеспечивает сильную защиту, но работает **медленно**: сообщение должно ждать на каждом mix, пока сформируется пакет, и доставка может занять минуты или часы. Для электронной почты это приемлемо, но не для веб-сёрфинга, где страница должна загружаться за секунды.

Отсюда возникает фундаментальное разделение:

Высоколатентные системы (mix-сети, ремейлеры) задерживают и перемешивают сообщения и способны противостоять наблюдателю, видящему всю сеть, но подходят лишь для коммуникаций, допускающих задержки.

Низколатентные системы (onion routing, Tor) пересылают данные сразу после поступления, позволяя просматривать веб-страницы, вести чаты и поддерживать интерактивные соединения; однако наблюдатель, видящий оба конца, может сопоставлять временные характеристики и объём трафика.

Tor сознательно выбирает второй путь: система, которой почти никто не пользуется из-за чрезмерной медлительности, никого не защищает (разд. 1.4).

2.3. Ремейлеры

В 1990-х идеи Чаума воплотились в реальных сервисах, называемых **ремейлерами**: серверах, которые получают электронное письмо и пересылают его после удаления сведений об отправителе.

anon.penet.fi (1993–1996). Самый известный ремейлер работал в Финляндии под управлением Йохана «Julf» Хельсингиуса с 1993 по 1996 год. Это был не mix: сервис присваивал каждому пользователю *псевдоним* и хранил таблицу соответствий между псевдонимами и реальными адресами, чтобы доставлять ответы. Со временем система обрабатывала тысячи сообщений в день. Но именно эта таблица в итоге привела к закрытию сервиса. В 1995 и 1996 годах Церковь саентологии при содействии финской полиции получила личности некоторых пользователей, распространявших документы церкви. В 1996 году Хельсингиус закрыл сервис, сославшись на правовую неопределённость вокруг Интернета в Финляндии. Урок, который впоследствии прямо учитывали разработчики onion routing [Syv11], заключается в том, что система, сохраняющая соответствие между пользователями и их личностями, создаёт единую точку отказа: оператора можно обязать раскрыть эти данные.

Cypherpunk, Mixmaster, Mixminion. Тем временем сообщество *cypherpunk* разрабатывало ремейлеры без таблиц соответствий: отправители послойно шифровали сообщения для цепочки серверов. Последующие поколения — сначала Mixmaster, затем Mixminion [DDM03] — ввели сообщения фиксированной длины, настоящее пакетирование и защиту от повторной передачи сообщений. Роджер Динглдайн и Ник Мэтьюсон были среди авторов Mixminion, выпущенного в 2003 году; примерно в то же время они проектировали Tor.

2.4. К низкой задержке

PipeNet (1995). В феврале 1995 года Вэй Дай предложил в рассылке cypherpunk систему PipeNet — низколатентную сеть, в которой все участники непрерывно передавали бы трафик с постоянной скоростью, не оставляя признаков для корреляции. Такая система сопротивлялась бы временному анализу, но неприемлемой ценой: если хотя бы один участник прекращал передачу, останавливаться должна была вся сеть [Syv11]. PipeNet так и не был реализован, но остаётся ориентиром того, что потребовалось бы для противодействия корреляции трафика.

Onion routing (1996). В том же году в Naval Research Laboratory формировалась концепция onion routing. Эту историю мы разберём в гл. 3.

Crowds (1998). Майкл Рейтер и Авиэль Рубин предложили Crowds [RR98]. Пользователи образуют «толпу», а каждый веб-запрос передаётся случайно выбранному участнику. С некоторой вероятностью тот отправляет запрос на веб-сайт, иначе передаёт следующему участнику. Веб-сайт видит запрос, пришедший от произвольного члена группы, а ни один участник не может определить, был ли человек, передавший ему запрос, его первоначальным источником. Crowds не использует послойное шифрование и защищает только от веб-сайта и других участников, но не от наблюдателя сети. Тем не менее система ввела строгий способ выражать анонимность через вероятность, а также анализ атак сговаривающихся участников, предвосхитивший исследования guard-реле.

Freedom (1999–2001). Крупнейшей сетью анонимности до Tor была Freedom, которой управляла канадская компания Zero-Knowledge Systems. Её анонсировали в конце 1998 года, а работала она с конца 1999 до конца 2001 года [Syv11]. Это был коммерческий сервис: пользователи платили подписку, а серверы обслуживали компания и её партнёры. Сервис закрылся, потому что доходов от подписок оказалось недостаточно для его поддержки. Его история показывает два урока, которым Tor на практике последовал, как мы увидим далее: сеть анонимности устойчивее, когда ею управляет множество независимых сторон, а не одна компания, и модель финансирования важна не меньше криптографии (разд. 3.4).

Подробнее: Много «луковиц»

В тот же период предлагалось множество других систем onion routing, включая полностью децентрализованные Tarzan и MorphMix, а также Cebolla, у которой Tor заимствовал идею строить цепь по одному переходу за раз, выполняя обмен Диффи–Хеллмана для каждого перехода [Syv11]. Параллельно с Tor появились и другие проекты, пошедшие иными путями: I2P, ориентированный прежде всего на внутренние сервисы и действующий до сих пор, и AN.ON, созданный в Дрезденском университете вместе с клиентом JAP (и предлагавшийся также как коммерческий сервис под названием JonDonym), который использует фиксированные цепочки mix-серверов под управлением известных организаций. Каждый проект по-своему балансирует между децентрализацией, доверием и производительностью.

Итоги

- Mix Чаума (1981) скрывает связь между входящими и исходящими сообщениями, изменяя их внешний вид, выравнивая длину и перемешивая порядок. Цепочка mix-серверов с послойным шифрованием предвосхищает «луковицу».
- Высоколатентные системы противостоят сильным наблюдателям, но медленны; низколатентные пригодны для интерактивной работы, однако уязвимы к временной корреляции.
- anon.penet.fi продемонстрировал опасность хранения идентичностей пользователей; Mixmaster и Mixminion развили подход mix-сетей для электронной почты.
- PipeNet, Crowds и Freedom исследовали низкую задержку с разными компромиссами. Freedom показал важность сети, управляемой множеством сторон, и устойчивой модели финансирования.

Упражнения

Упражнение 2.1 ⋆ Mix собирает пакеты по 100 сообщений. Наблюдатель видит, как все 100 сообщений входят и как все 100 выходят. Если наблюдателю известен отправитель одного входящего сообщения, какова вероятность правильно угадать соответствующее исходящее сообщение при случайном выборе? Что изменится, если сам наблюдатель отправил 99 из этих 100 сообщений?

Упражнение 2.2 ⋆⋆ Набросайте архитектуру ремейлера, позволяющего отвечать пользователям без хранения таблицы соответствий между псевдонимом и адресом. Какие идеи Чаума для этого понадобятся?

Упражнение 2.3 ⋆⋆ В Crowds при вероятности пересылки $p_f = 3/4$ через сколько участников в среднем проходит запрос, прежде чем попасть на веб-сайт? Почему участник, получивший запрос, не может быть уверен, что передавший его участник и был первоначальным источником?

Дополнительная литература

- D. Chaum, *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81].
- M. Reiter, A. Rubin, *Crowds: Anonymity for Web Transactions* [RR98].

- G. Danezis, R. Dingledine, N. Mathewson, *Mixminion: Design of a Type III Anonymous Remailer Protocol* [DDM03].
- *Anonymity Bibliography* проекта Free Haven, `https://www.freehaven.net/anonbib/`: справочная коллекция научных публикаций по этой теме.

Глава 3

От луковой маршрутизации NRL к Tor Project

Последняя проверка: 23 сентября 2026 г. — Syverson, A Peel of Onion *(2011); страница истории Tor Project*

Цели главы

К концу этой главы вы сможете:

- проследить происхождение onion routing в Naval Research Laboratory и описать три поколения этой технологии;
- объяснить, чем Tor отличается от предыдущих поколений;
- перечислить основные этапы истории Tor Project с 2002 года до наших дней;
- объяснить происхождение названия «Tor».

3.1. В Naval Research Laboratory

В 1995 году трое исследователей Naval Research Laboratory (NRL) ВМС США — Дэвид Голдшлаг, Майкл Рид и Пол Сайверсон — задались вопросом, можно ли устанавливать интернет-соединения, не раскрывая, кто с кем общается [Thea]. Их целью была не анонимность сама по себе, а, как мы уже видели, стремление «отделить идентификацию от маршрутизации» (разд. 1.3). Первый вариант архитектуры они представили на *Information Hiding Workshop* в мае 1996 года [GRS96]. До конца того же года появился второй вариант, в котором пользовательское приложение, строящее цепи, было отделено от узлов, передающих трафик [Syv11]. Уже в 1996 году лаборатория получила разрешение опубликовать исходный код, ещё до широкого распространения самой идеи *программного обеспечения с открытым исходным кодом*. Разработчики понимали: никто не станет доверять сети, код которой нельзя изучить.

Название происходит от структуры данных: сообщение обёрнуто слоями шифрования, каждый из которых снимается очередным узлом, словно слои луковицы.

3.2. Три поколения

Сайверсон различает три поколения разработок NRL [Syv11].

Первое поколение. Использовались цепи фиксированной длины из пяти узлов, предназначенные, в частности, для соединения защищённых сетей двух организаций. При трёх переходах враждебный промежуточный узел сразу видел бы обе взаимодействующие сети. При четырёх атакующий, скомпрометировав узел рядом с одной конечной точкой, знал бы,

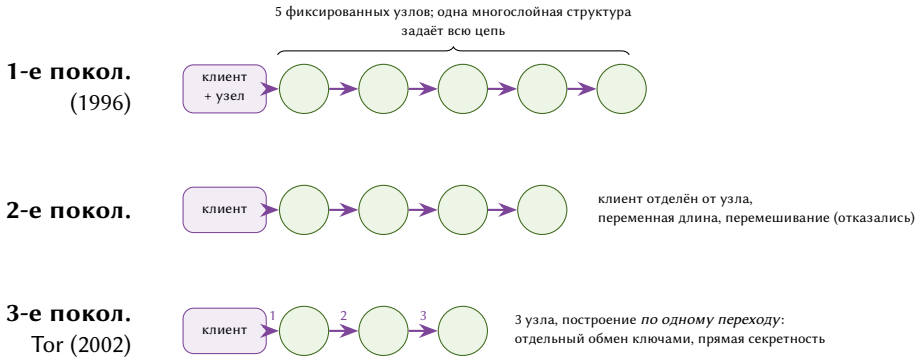


Рис. 3.1: Три поколения onion routing. Ключевое изменение в Tor состояло не в числе переходов, а в способе построения цепи: вместо подготовки клиентом одной полной «луковицы» цепь расширяется по одному переходу, с отдельным обменом ключами для каждого узла.

какой ещё узел нужно атаковать, чтобы восстановить всю цепь. При пяти такой короткий путь был невозможен без предварительной компрометации ещё одного узла. Клиент был встроен в сам узел. Сеть была статической: двадцать или сто узлов под управлением постоянных организаций, конфигурации и ключи распространялись вручную. Каждому приложению требовался собственный прокси — для веб-сёрфинга, удалённого входа или электронной почты, — поскольку SOCKS тогда ещё не был широко распространён. Цепь строилась как единая «луковица», зашифрованная открытыми ключами всех узлов.

Второе поколение. Клиент отделили от узла, длина цепи стала переменной, были введены механизмы перемешивания и заполнения между цепями, от которых впоследствии отказались. Распространение информации о сети было предусмотрено, но полностью так и не реализовано.

Третье поколение: Tor. Самым важным изменением стало построение цепи **по одному переходу**, с обменом Диффи–Хеллмана с каждым узлом (разд. 9.1) — идеей, заимствованной у проекта Cebolla. Это обеспечивает прямую секретность и позволяет проверять каждый переход по мере расширения цепи. Tor также добавил серверы каталогов, поддержку SOCKS, скрытые сервисы с точками рандеву и, прежде всего, архитектуру, рассчитанную на развёртывание и эксплуатацию добровольцами по всему миру.

Историческая справка: Название

Заголовок статьи 2004 года *Tor: The Second-Generation Onion Router* породил два заблуждения, которые позднее признал сам Сайверсон [Syv11]. Tor относится к *третьему*, а не ко второму поколению, и буква «r» в Tor не означает *router*. Когда Роджер Динглдайн начал работать над проектом NRL, существовало множество систем onion routing. На вопрос «какая именно?» он отвечал «*the onion routing*», имея в виду исходную систему. Рэйчел Гринстадт заметила, что первые буквы образуют удачный акроним. Динглдайн также обратил внимание на рекурсивное прочтение *Tor's onion routing* и решил, что название следует писать «Tor», а не «TOR»: это обычное слово, которое по-немецки также означает *ворота*, например городские. Во всей остальной книге мы пишем *Tor*, когда речь идёт о сети и проекте, и *tor* со строчной буквы, когда речь идёт о программе.

3.3. Сеть Tor

Роджер Динглдайн присоединился к проекту NRL в начале 2000-х годов, а вскоре к нему присоединился Ник Мэтьюсон. Сеть Tor начала работу в **октябре 2002 года**, причём исходный код с самого начала распространялся под лицензией свободного программного обеспечения. К концу 2003 года в сети было около дюжины узлов, управляемых добровольцами, почти все в США и один в Германии [Thea]. В августе 2004 года Динглдайн, Мэтьюсон и Сайверсон представили проект на USENIX Security Symposium [DMS04].

Основные последующие этапы были следующими [Thea]:

Год	Событие
2002	Сеть Tor начинает работу; исходный код свободно доступен
2004	Electronic Frontier Foundation начинает финансировать работу Динглдайна и Мэтьюсона
2006	Tor Project, Inc. учреждается как некоммерческая организация США (501(c)(3))
2007	Начинается разработка устойчивых к цензуре мостов (гл. 16)
2008	Начинается разработка Tor Browser (гл. 23)
2010–2011	Tor становится важным инструментом во время восстаний «арабской весны»
2013	Публикации Эдварда Сноудена о массовой слежке, в которых Tor играет центральную роль
2017	Onion-сервисы v3 (гл. 14)
2021	Onion-сервисы v2 отключены
2022–2023	Управление перегрузкой (разд. 11.3) и proof of work для onion-сервисов (гл. 15)
2024	WebTunnel; перенос распределения мостов на rdsys
2025–2026	CGO в tor (разд. 10.4); Arti как клиент и для onion-сервисов (гл. 30)

3.4. Организация и сообщество

Tor Project, Inc. разрабатывает программное обеспечение, поддерживает спецификации, совместно с независимыми операторами обслуживает доверенные серверы каталогов (directory authorities), измеряет сеть и поддерживает сообщество. Но организация не владеет сетью: тысячи добровольцев — частные лица, университеты и ассоциации — эксплуатируют реле в десятках стран. Это архитектурное решение учитывает опыт Freedom (разд. 2.4): ни одна организация не должна иметь возможность единолично контролировать сеть или быть вынужденной скомпрометировать её пользователей.

Финансирование организации неоднократно становилось предметом публичных споров. На протяжении многих лет средства поступали от государственных ведомств, особенно в США, частных фондов и отдельных жертвователей. Организация ежегодно публикует финансовую отчётность. Вопрос «Можно ли доверять программному обеспечению, которое частично финансировалось государством?» прежде всего технический, а не политический: код и спецификации публичны и проверяемы, архитектурные решения открыто обсуждаются в предложениях Tor (proposal) (гл. 31), а безопасность системы не зависит от доверия к Tor Project. Вместо этого она опирается на распределение сети между множеством независимых операторов (гл. 4).

Подробнее: Публикации 2013 года

Среди документов, опубликованных Сноуденом в 2013 году, были внутренние презентации NSA о Tor. В них признавалась сложность систематической деанонимизации пользователей Tor и описывались целевые атаки на отдельных пользователей, эксплуатировавшие уязвимости браузера, а не самой сети. Это косвенно подтверждает принцип, к которому мы будем неоднократно возвращаться: самые опасные слабости нередко находятся не в самом протоколе, а в окружающих его системах (гл. 28).

Итоги

- Onion routing появился в 1995 году в Naval Research Laboratory благодаря Голдшлагу, Риду и Сайверсону с целью отделить идентификацию от маршрутизации.
- Первое поколение использовало фиксированные цепи из пяти узлов и статическую сеть. Tor, третье поколение, строит цепи по одному переходу, обеспечивает прямую секретность и использует каталоги и точки рандеву.
- Название «Tor» происходит от *the onion routing*; заглавной является только первая буква.
- Сеть работает с 2002 года; Tor Project, Inc. существует с 2006 года; реле эксплуатируются независимыми добровольцами.

Упражнения

Упражнение 3.1 ⋆ Почему первое поколение использовало цепи из пяти узлов? Восстановите приведённые в этой главе аргументы для трёх и четырёх переходов и объясните, почему Tor использует только три.

Упражнение 3.2 ⋆⋆ Сравните построение цепи как единой «луковицы» (первое поколение) с телескопическим построением Tor с точки зрения прямой секретности и возможности немедленно определить не отвечающий узел.

Упражнение 3.3 ⋆⋆ Напишите краткий аргумент объёмом около одной страницы в ответ на вопрос «Можно ли доверять Tor с учётом того, кто его финансировал?». Разделите то, что можно проверить технически, и то, что действительно требует доверия.

Дополнительная литература

- P. Syverson, *A Peel of Onion* [Syv11].
- D. Goldschlag, M. Reed, P. Syverson, *Hiding Routing Information* [GRS96].
- Страница истории Tor Project [Thea].

Глава 4

Модель угроз Tor

Последняя проверка: 23 сентября 2026 г. — Dingledine, Mathewson, Syverson (2004), раздел 3

Цели главы

К концу этой главы вы сможете:

- объяснять, что такое модель угроз и зачем она нужна каждой системе безопасности;
- перечислять цели и «нецели», заявленные разработчиками Tor;
- описывать противника, от которого Tor должен защищать, и противника, которому он не способен противостоять;
- отличать то, что защищает Tor, от того, что остаётся ответственностью пользователей и приложений.

Ни одна система безопасности не защищает от всего. **Модель угроз** явно описывает, кого или что должна защищать система, каким противникам и угрозам она должна противостоять и какие предположения при этом делает. Именно по этому стандарту следует оценивать систему: успешная атака, укладывающаяся в модель, указывает на дефект, тогда как атака, прямо исключённая из модели, — нет. Однако и последняя остаётся ограничением, которое пользователь обязан понимать. В этой главе рассматривается модель угроз Tor в формулировке разработчиков 2004 года [DMS04] и её последующая эволюция.

4.1. Цели

Главная цель — не позволить противнику **связать** две стороны одной коммуникации либо несколько коммуникаций одного пользователя. Архитектуру направляли четыре критерия, сформулированные в статье 2004 года:

Развёртываемость Система должна работать в реальном мире: не обходиться добровольцам слишком дорого, не подвергать операторов чрезмерной юридической ответственности, не требовать изменений операционной системы и не вынуждать веб-сайты что-либо устанавливать.

Удобство использования Сложная система привлекает меньше пользователей, а меньше пользователей означает меньшую анонимность. Как объясняют авторы, удобство — не просто комфорт, а «требование безопасности».

Гибкость Протокол должен быть достаточно хорошо специфицирован, чтобы служить основой для будущих исследований.

Простота Следует использовать только хорошо изученные и проверенные методы. Каждая дополнительная функция усложняет анализ безопасности.

4.2. Что не является целью

Не менее важно понимать, чего Tor прямо *не* пытается делать:

Это не одноранговая сеть (peer-to-peer) Tor не пытается работать с тысячами краткоживущих узлов, многие из которых могут быть враждебными. Он использует набор реле, перечисленных и проверенных доверенными серверами каталогов (directory authorities) (гл. 12).

Он не противостоит атакам на обоих концах соединения (end-to-end) Tor не заявляет, что способен победить временную корреляцию или атаки пересечения, если противник наблюдает обе конечные точки.

Он не нормализует протоколы Tor переносит данные приложений такими, каковы они есть. Если приложение раскрывает личность пользователя, Tor не может этому помешать. Изначально для этого требовался фильтрующий прокси; сегодня эту роль выполняет Tor Browser (гл. 23).

Он не является стеганографическим Tor не скрывает сам факт подключения пользователя к сети. Разработанные позднее подключаемые транспорты смягчают это ограничение (гл. 16).

4.3. Противник

Во многих теоретических исследованиях систем анонимности предполагается **глобальный пассивный противник**: тот, кто способен наблюдать весь интернет-трафик. Ни одна практическая низколатентная система не защищает от такого противника, и Tor прямо это признаёт. Вместо этого Tor предполагает противника лишь с частичной видимостью, но в некоторых отношениях с более широкими активными возможностями [DMS04]. Он может:

- наблюдать *часть* сетевого трафика;
- генерировать, изменять, удалять или задерживать трафик;
- эксплуатировать собственные реле;
- компрометировать некоторые реле, управляемые другими.

С этими возможностями противник может пытаться проводить множество атак: коррелировать входящий и исходящий трафик, наблюдая края сети; компрометировать реле или ключи; повторно передавать трафик; выводить честные реле из строя, чтобы направлять пользователей к вредоносным (гл. 15); маркировать трафик так, чтобы распознавать его в другом месте (разд. 10.3); через систему каталогов показывать разным пользователям несовместимые представления сети (гл. 12); либо сделать сеть настолько ненадёжной, чтобы пользователи отказались от неё в пользу менее защищённых систем.

4.3.1. Подтверждение и анализ

Разработчики Tor различают два типа атак на основе трафика. При атаке **подтверждения** противник уже подозревает наличие связи («Алиса общается с Бобом») и наблюдает обе конечные точки, чтобы проверить гипотезу. Tor, как и любая низколатентная система, не способен полностью защищать от таких атак. При атаке **анализа** противник изучает трафик, чтобы выяснить, *где* наблюдать, то есть какие точки сети следует атаковать. В первую очередь Tor рассчитан на сопротивление именно этому классу. Весь процесс выбора маршрута, включая guard-реле и весовые коэффициенты пропускной способности (гл. 13), призван снизить вероятность того, что противник окажется одновременно на обоих концах цепи.

Внимание: Главный принцип

Tor защищает от того, кто наблюдает *часть* сети, но не от того, кому удаётся наблюдать *оба конца* коммуникации. Противник, контролирующий или наблюдающий одновременно соединение пользователя с guard-реле и соединение exit-реле с веб-сайтом, может коррелировать временные характеристики и объём трафика. Для организации, имеющей доступ к сетям целой страны, и пользователь, и веб-сайт могут находиться внутри этой страны. Это практический, а не только теоретический сценарий.

4.4. От чего Tor защищает, а от чего нет

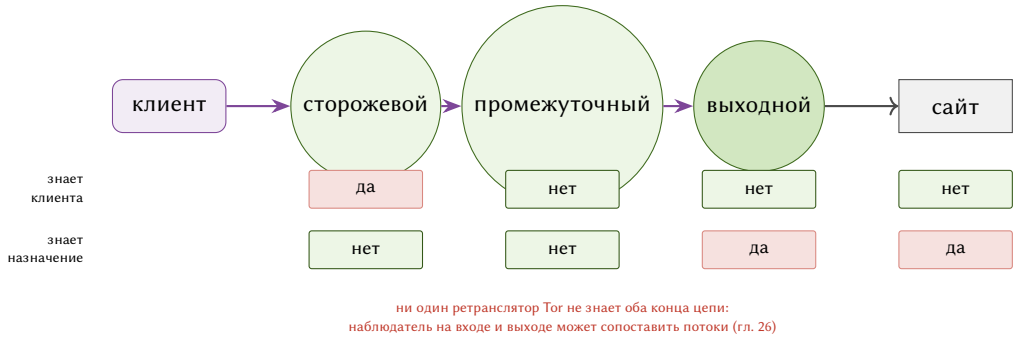


Рис. 4.1: Что знает каждая точка маршрута. Защита Tor основана на разделении: guard-реле знает клиента, но не пункт назначения; exit-реле знает пункт назначения, но не клиента.

Tor защищает от	Tor не защищает от
веб-сайта, пытающегося узнать IP-адрес посетителя	информации, которую сам пользователь раскрывает, например входя в аккаунт или сообщая личные сведения
наблюдателя в локальной сети или интернет-провайдера, видящего только соединение с Tor	наблюдателя, следящего за обоими концами коммуникации
отдельного реле, знающего не более одной конечной точки	противника, контролирующего одновременно guard- и exit-реле цепи
exit-реле, пытающегося определить отправителя запроса	exit-реле, читающего или изменяющего незашифрованный трафик без HTTPS
цензуры по IP-адресам при использовании мостов	уязвимого, скомпрометированного или неправильно настроенного приложения либо браузера

Со временем модель расширилась, чтобы учесть явления, которые статья 2004 года не могла предвидеть: цензуру в масштабах целых стран (гл. 16), защиту от *фингерпринтинга* браузера (гл. 23), защиту от DoS (гл. 15) и будущую угрозу квантовых компьютеров (гл. 31). Один принцип остаётся неизменным: понимание модели угроз — первый шаг к безопасному использованию Tor.

Итоги

- Модель угроз определяет противников и атаки, от которых система предназначена защищать.
- Tor стремится не позволить связать пользователя с пунктом назначения, отдавая приоритет развёртываемости, удобству, гибкости и простоте.
- Tor не является peer-to-peer системой, не побеждает end-to-end атаки, не нормализует прикладные протоколы и без подключаемых транспортов не скрывает факт использования Tor.
- Противник может наблюдать часть сети, эксплуатировать и компрометировать реле и предпринимать активные действия. Tor рассчитан прежде всего на сопротивление атакам анализа и слабее защищает от атак подтверждения.

Упражнения

Упражнение 4.1 ⋆ Для каждого сценария укажите, защищает ли Tor пользователя, и объясните почему: (a) оператор Wi-Fi в кафе хочет узнать, какие сайты посещает клиент; (b) веб-сайт хочет узнать IP-адреса посетителей; (c) пользователь через Tor входит в личный аккаунт социальной сети; (d) ведомство, контролирующее сети страны, подозревает, что пользователь общается с газетой в той же стране.

Упражнение 4.2 ⋆⋆ Приведите пример, показывающий, почему удобство использования в системе анонимности является требованием безопасности, а не просто вопросом комфорта.

Упражнение 4.3 ⋆⋆ Составьте модель угроз для журналиста, который использует Tor Browser для общения с источником в другой стране. Определите возможных противников, то, что они способны наблюдать, и аспекты, остающиеся за пределами защиты Tor.

Дополнительная литература

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], разделы 3 и 7.
- R. Dingledine, N. Mathewson, *Anonymity Loves Company: Usability and the Network Effect* [DM06].

Часть II

Низкоуровневый протокол

Глава 5

Криптография, необходимая для Tor

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; коммит torspec `8e1f245`

Цели главы

К концу этой главы вы сможете:

- определять криптографические примитивы, используемые Tor, и назначение каждого из них;
- различать шифрование, контроль целостности, выработку ключей, обмен ключами и цифровые подписи;
- понимать, почему Tor использует несколько семейств хеш-функций (SHA-1, SHA-2, SHA-3, Blake2b) и чем они различаются;
- уверенно читать формулы в последующих главах.

Эта глава — набор инструментов, а не курс криптографии. Здесь с минимально необходимой теорией вводятся средства, которые встретятся нам в части II, и каждый раз указывается, где именно они используются. В приложении B приведены дополнительные сведения для читателей, желающих глубже изучить тему. Все примеры можно повторить в Python с помощью стандартного модуля `hashlib` и библиотеки `cryptography` — тех же инструментов, которые используются в лабораторных работах.

5.1. Пять задач

В протоколе вроде Tor криптография решает пять разных задач:

Конфиденциальность Никто, кроме предполагаемых получателей, не должен иметь возможности читать данные: *шифры*.

Целостность Никто не должен иметь возможности незаметно изменить данные: *хеш-функции* и *коды аутентификации сообщений* (MAC).

Согласование ключей Две стороны, изначально не имеющие общего секрета, должны установить его, обмениваясь сообщениями по открытому каналу: *обмен ключами* Диффи–Хеллмана.

Аутентификация Одна сторона должна доказать владение определённой идентичностью: *цифровые подписи*.

Выработка Из общего секрета необходимо получить несколько независимых ключей: *функции выработки ключей* (KDF).

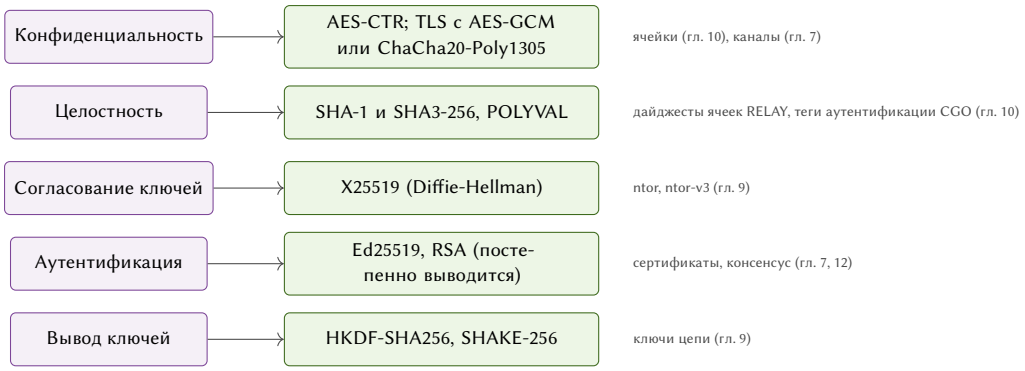


Рис. 5.1: Пять криптографических задач в Tor, примитивы, которые их решают, и главы, где мы увидим их применение.

Здравое правило проектирования, которому Tor со временем следует всё строже, — никогда не использовать один и тот же ключ для разных задач или контекстов.

5.2. Шифры

5.2.1. AES в режиме счётчика

Почти всё в Tor шифруется с помощью **AES**, стандартного блочного шифра с блоками по 16 байт и ключами длиной 128 или 256 бит. Сам по себе блочный шифр обрабатывает по одному блоку. Для шифрования потоков произвольной длины Tor использует **режим счётчика** (CTR): AES шифрует значение счётчика, увеличивающегося на единицу для каждого блока, а полученный *поток ключа* объединяется с данными операцией XOR:

$$C = P \oplus \mathrm{Trunc}_{|P|}\big(\mathrm{AES}_K(IV) \,\|\, \mathrm{AES}_K(IV+1) \,\|\, \cdots\big)$$

Расшифрование выполняется той же операцией. Два свойства режима счётчика необходимо помнить всегда:

- одна и та же пара ключ–счётчик **никогда не должна использоваться повторно**: два сообщения, зашифрованные одним потоком ключа, раскрывают XOR своих открытых текстов;
- режим счётчика **пластичен**: изменение бита шифротекста изменяет соответствующий бит открытого текста без знания ключа.

AES-128-CTR шифрует RELAY-ячейки в традиционной схеме (разд. 10.1); его пластичность делает возможными атаки маркирования (tagging). AES-256-CTR шифрует сообщения ntor-v3 (разд. 9.7) и дескрипторы onion-сервисов (разд. 14.5). AES-128 является базовым компонентом CGO (разд. 10.4).

5.2.2. TLS

Канал между двумя узлами защищён TLS (гл. 7), который независимо согласует аутентифицированные шифры, например AES-GCM или ChaCha20-Poly1305. Однако Tor не полагается на TLS для подтверждения идентичности реле: реле доказывают её с помощью собственных сертификатов Tor.

5.3. Хеш-функции

Криптографическая хеш-функция преобразует вход произвольной длины в отпечаток фиксированной длины. Она проектируется так, чтобы вычислительно невозможно было найти два входа с одинаковым отпечатком (*устойчивость к коллизиям*) либо вход, соответствующий заданному отпечатку. Tor использует четыре семейства, отражающие историю его развития:

Функция	Выход	Где используется
SHA-1	20 байт	RSA-отпечатки реле, накопительный дайджест RELAY-ячеек (tor1), KDF-TOR; считается слабой относительно коллизий, но сохраняется там, где это свойство не требуется
SHA-256	32 байта	ntor и его KDF, сертификаты Ed25519 и TLS, подписи консенсуса, отпечатки микродескрипторов
SHA3-256, SHAKE-256	32 байта / переменная длина	ntor-v3, onion-сервисы v3 (адреса, ослеплённые ключи, кольцо HSDir, дескрипторы), дайджесты цепей рандеву
Blake2b	переменная длина	доказательство выполнения работы (proof of work) для onion-сервисов (гл. 15)

SHA-3 — не улучшенная версия SHA-2, а совершенно иная конструкция (*губка (sponge)*). SHAKE-256 относится к тому же семейству и является функцией с расширяемым выходом: она выдаёт столько байт, сколько требуется, поэтому удобна для выработки ключей. В новых протоколах Tor каждое применение SHA-3 предваряется меткой домена, например `"ntor3-curve25519-sha3_256-1:kdf_phase1"`. Это не позволяет хешу, вычисленному для одной цели, совпасть с хешем, вычисленным для другой.

Подробнее: Накопительные хеши

Многие библиотеки позволяют приложению добавлять данные в хеш постепенно и в любой момент получать дайджест, не начиная вычисление заново. Tor использует это свойство для *накопительного дайджеста* RELAY-ячеек (разд. 10.2), который обобщает все ячейки, переданные с данным переходом. В Python объект `hashlib.sha1()` и его методы `update`, `digest` и `copy` делают именно это.

5.4. MAC и выработка ключей

5.4.1. HMAC

MAC — это отпечаток, который также зависит от ключа: вычислить или проверить его может только тот, кто знает ключ. **HMAC** строит MAC на основе хеш-функции. Tor использует HMAC-SHA256 в протоколе согласования ключей ntor (разд. 9.6) и в его функции выработки ключей. В протоколах на основе SHA-3 MAC можно получить проще: хешируя ключ, которому предшествует его длина, а затем сообщение. Длина однозначно указывает, где заканчивается ключ и начинается сообщение [rend-spec § Cryptographic building blocks]. В ntor-v3 длина также ставится перед меткой домена (разд. 9.7).

Практическая деталь: MAC всегда следует сравнивать за *постоянное время* (в Python — с помощью `hmac.compare_digest`); иначе время сравнения способно раскрыть, сколько байт совпало.

5.4.2. Функции выработки ключей

Из общего секрета требуется получить несколько ключей: один для шифрования в прямом направлении, один для обратного и два для контроля целостности (разд. 9.8). KDF расширяет секрет в поток необходимой длины, из которого извлекаются отдельные ключи. Tor использует три варианта:

KDF-TOR $\mathrm{SHA1}(K_0\|0) \,\|\, \mathrm{SHA1}(K_0\|1) \,\|\, \ldots$, историческая конструкция, до сих пор используемая **`CREATE_FAST`**;

HKDF-SHA256 стандартная конструкция RFC 5869, используемая ntor;

SHAKE-256 используется напрямую как KDF в ntor-v3 и onion-сервисах.

5.5. Обмен ключами: X25519

Обмен ключами Диффи–Хеллмана позволяет двум сторонам установить общий секрет, передавая друг другу только открытые значения. В Tor используется эллиптическая версия **X25519** (обмен ключами на *Curve25519*). У каждой стороны есть 32-байтовый закрытый ключ x и открытый ключ $X = x \cdot G$, где G — фиксированная точка кривой, а произведение обозначает умножение точки на целое число. Имея пары x, X и y, Y, обе стороны вычисляют одно и то же значение

$$x \cdot Y = y \cdot X = xy \cdot G,$$

которое наблюдатель, знающий только X и Y, получить не может. Ключи имеют длину 32 байта, а обмен выполняется очень быстро. Эти два преимущества привели Tor к отказу от 1024-битного модульного Диффи–Хеллмана, применявшегося в протоколе TAP (разд. 9.4).

Ключ, используемый только для одного обмена и затем удаляемый, называется **эфемерным**. Именно сочетание эфемерных и долговременных ключей позволяет протоколам согласования ключей Tor обеспечивать одновременно аутентификацию и *прямую секретность* (разд. 9.3).

5.6. Подписи: Ed25519 и RSA

Цифровая подпись создаётся закрытым ключом и проверяется соответствующим открытым ключом. Tor в основном использует **Ed25519** — схему подписи на кривой, эквивалентной Curve25519, с 32-байтовыми открытыми ключами и 64-байтовыми подписями: для идентичностей реле, сертификатов Tor (разд. 7.5) и дескрипторов onion-сервисов. У Ed25519 есть свойство, которое Tor использует необычным способом: умножение открытого ключа A на коэффициент h даёт другой допустимый открытый ключ $A' = h \cdot A$, соответствующий закрытый ключ которого может вычислить тот, кто знает исходный закрытый ключ. На этом основаны ослеплённые ключи onion-сервисов (разд. 14.3).

RSA с 1024-битными ключами — наследие ранних лет Tor: это устаревшие RSA-идентичности реле и ключи, которыми доверенные серверы каталогов (directory authorities) подписывают консенсус. Ключи серверов каталогов длиннее, и для двух типов действуют разные требования: спецификация требует не менее **3072** бит для ключа идентичности (`KP_auth_id_rsa`) и не менее 2048 для ключа подписи (`KP_auth_sign_rsa`) [dir-spec § Directory authority operation and formats]. Tor постепенно выводит RSA из употребления.

5.7. Современная криптография

В последних разработках появляются ещё две специализированные конструкции:

- **POLYVAL**, универсальная хеш-функция, также используемая в AES-GCM-SIV, вместе с AES образует один из строительных блоков CGO — новой схемы шифрования RELAY-ячеек (разд. 10.4);
- **Elligator2** кодирует точку Curve25519 так, чтобы она была неотличима от случайной строки; obfs4 использует его для сокрытия открытых ключей обмена (разд. 16.4).

Постквантовая криптография, которой придётся заменить X25519, когда квантовые компьютеры станут практической угрозой, является активным направлением исследований Tor и рассматривается в гл. 31.

Итоги

- AES в режиме счётчика шифрует данные; один поток ключа нельзя использовать повторно, а сам режим является пластичным.
- SHA-1, SHA-256, SHA3-256/SHAKE-256 и Blake2b сосуществуют в Tor: выбор зависит от времени создания протокола и назначения хеша.
- HMAC обеспечивает аутентификацию сообщений, а KDF (KDF-TOR, HKDF, SHAKE) служат для выработки ключей; MAC необходимо сравнивать за постоянное время.
- X25519 используется для обмена ключами во всех современных протоколах согласования ключей; Ed25519 подписывает идентичности, сертификаты и дескрипторы; RSA постепенно выводится из употребления.

Упражнения

Упражнение 5.1 ⋆ При AES-CTR противник знает открытый текст P_1 одного сообщения и перехватывает его шифротекст C_1, а также шифротекст C_2 другого сообщения, ошибочно зашифрованного тем же потоком ключа. Как противник может восстановить P_2?

Упражнение 5.2 ⋆ В Python вычислите `hashlib.shake_256(b"Tor").hexdigest(16)` и `hexdigest(32)`. Как связаны два результата? Что это означает для использования SHAKE в качестве KDF?

Упражнение 5.3 ⋆⋆ С помощью библиотеки `cryptography` сгенерируйте две пары ключей X25519 и убедитесь, что они дают одинаковые общие секреты. Что произойдёт, если одна сторона использует 32 нулевых байта как открытый ключ другой стороны? Свяжите результат с проверкой ntor на «точку на бесконечности».

Упражнение 5.4 ⋆⋆ Почему протоколы Tor на основе SHA-3 ставят перед каждым входом метку домена? Опишите возможную атаку, если два разных применения одной функции не были бы разделены.

Дополнительная литература

- J.-P. Aumasson, *Serious Cryptography* (No Starch Press): современное практическое введение во все примитивы этой главы.
- D. J. Bernstein et al., исходные статьи о Curve25519 и Ed25519, а также RFC 7748 (X25519) и 8032 (Ed25519).

- Раздел «Preliminaries» спецификации Tor [Thee], содержащий соглашения о шифрах и хеш-функциях.

Глава 6

Архитектура сети

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; публичный консенсус от 23 сентября 2026 г., 15:00 UTC

Цели главы

К концу этой главы вы сможете:

- перечислять роли узлов сети Tor и их обязанности;
- описывать уровни протокола от TLS-канала до потока приложения;
- на высоком уровне прослеживать всё, что происходит при открытии веб-страницы через Tor;
- понимать, как разные реализации и версии сосуществуют в одной сети.

Прежде чем переходить к деталям следующих глав, полезно взглянуть на сеть сверху. Эта глава — карта: позже мы вернёмся к каждому элементу и разберём его подробнее, а ссылки укажут соответствующие разделы.

6.1. Роли

Сеть Tor состоит из программ, выполняющих разные роли, зачастую несколько одновременно. Рис. 6.1 сводит их вместе.

Клиент Пользовательская программа, например входящая в Tor Browser. Она выбирает маршруты, строит цепи и предоставляет приложениям SOCKS-порт. Она не принимает входящие соединения из сети и не аутентифицирует себя (гл. 7).

Реле Узел, пересылающий чужой трафик. В зависимости от флагов, назначенных доверенными серверами каталогов, и своей конфигурации он может выполнять роль:

- **guard-реле**: первого перехода в цепях клиента, выбранного один раз и используемого длительное время (разд. 13.3);
- **промежуточного реле**: промежуточного перехода, не знающего ни источник, ни пункт назначения;
- **exit-реле**: последнего перехода, который открывает интернет-соединения согласно своей политике выхода (exit policy) (гл. 11);
- **HSDir**: хранилища дескрипторов onion-сервисов (разд. 14.4);
- **кэша каталогов**: узла, повторно распространяющего консенсус и дескрипторы (гл. 12).

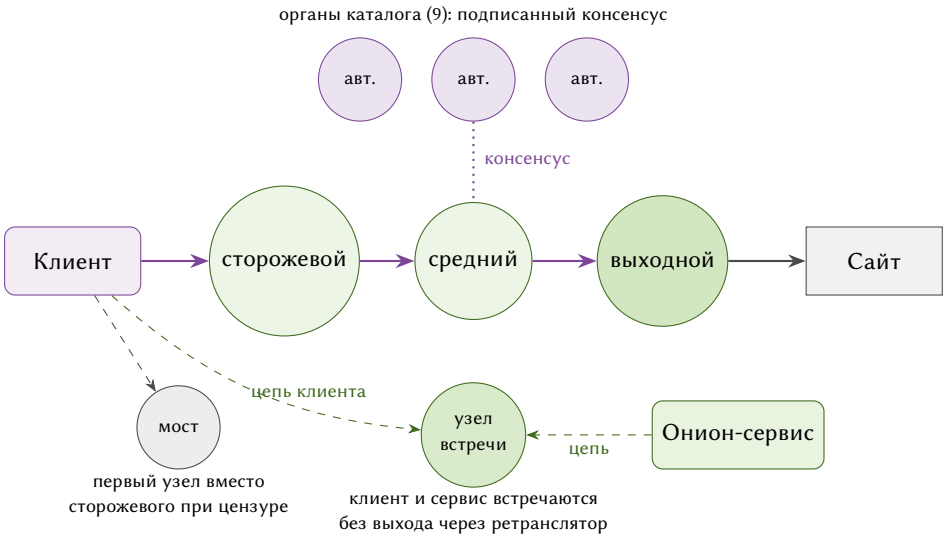


Рис. 6.1: Роли в сети Tor. Клиент строит цепь через guard-реле, промежуточное реле и exit-реле. Доверенные серверы каталогов (directory authorities) публикуют консенсус со списком реле. Для пользователей, сталкивающихся с цензурой, мосты заменяют guard-реле. Доступ к onion-сервисам выполняется через точку рандеву без использования exit-реле.

Доверенный сервер каталогов Один из девяти доверенных серверов, которые каждый час голосуют о состоянии сети и подписывают консенсус; некоторые также измеряют пропускную способность реле.

Мост Непубличное реле, используемое первым переходом теми, кто не может обращаться к публичным реле, часто вместе с подключаемым транспортом (гл. 16).

Onion-сервис Сервис, доступный только через Tor без раскрытия своего местоположения (гл. 14).

В публичном консенсусе от 23 сентября 2026 года насчитывается 9358 реле: 5306 с флагом `Guard` и 3177 с флагом `Exit`; 1615 имеют оба флага. На guard-реле приходится 70 % общей пропускной способности, на exit-реле — 33 % (гл. 13).

6.2. Уровни протокола

Протокол Tor можно представить как стек уровней, каждый из которых строится поверх нижележащего (рис. 6.2).

Канал TLS-соединение между двумя соседними узлами, в рамках которого проверяется идентичность реле (гл. 7).

Ячейки Единицы передачи длиной 514 байт, переносимые каналом (гл. 8).

Цепь Многошаговый маршрут, строящийся по одному переходу с помощью протоколов согласования ключей, вырабатывающих разные ключи для каждого перехода (гл. 9).

Послойное шифрование Сообщения RELAY, обёрнутые одним слоем шифрования для каждого перехода (гл. 10).

Поток TCP-соединения приложений, открываемые exit-реле и управляемые окнами и механизмом управления перегрузкой (гл. 11).



Рис. 6.2: Уровни протокола Tor и главы, в которых они описаны. Каждый уровень использует нижележащий: потоки передаются как сообщения RELAY, а сообщения RELAY помещаются в ячейки с послойным шифрованием, а ячейки идут внутри TLS-каналов.

Рядом с этим стеком существуют система каталогов, предоставляющая клиентам список реле, и протокол onion-сервисов, использующий цепи для встречи в точке рандеву.

6.3. Веб-страница от начала до конца

Проследим на высоком уровне, что происходит, когда пользователь впервые запускает Tor Browser и открывает страницу.

1. **Запуск.** tor связывается с доверенным сервером каталогов или одним из примерно двухсот *резервных серверов каталогов* (fallback directories), перечисленных в исходном коде, и загружает консенсус, подписи которого проверяет. Затем загружаются микродескрипторы реле (гл. 12).

2. **Выбор guard-реле.** tor случайным образом выбирает небольшую выборку guard-реле с учётом пропускной способности и использует два из них (разд. 13.3).

3. **Канал.** tor открывает TLS-соединение с guard-реле и проверяет его идентичность с помощью ячеек **`CERTS`** (гл. 7).

4. **Цепь.** tor выбирает exit-реле, подходящее для пункта назначения, и промежуточное реле, после чего строит трёхпереходную цепь с помощью **`CREATE2`** и `EXTEND2` (гл. 9 и 13). На практике некоторые цепи строятся заранее, чтобы быть готовыми к моменту запроса.

5. **Поток.** Через SOCKS-порт браузер запрашивает соединение с `www.example.com:443`. tor отправляет exit-реле сообщение `BEGIN`; exit-реле разрешает имя и открывает TCP-соединение (гл. 11).

6. **Данные.** Браузер общается с веб-сайтом по HTTPS через поток. Exit-реле видит имя сайта и TLS-трафик, но не знает, кто его запросил. Guard-реле видит, кто сделал запрос, но не знает, что именно было запрошено. Промежуточное реле не знает ни того, ни другого.

Весь процесс запуска обычно занимает лишь несколько секунд. Многие механизмы последующих глав, включая заполнение, управление перегрузкой и vanguards, незаметно работают вокруг этой последовательности.

6.4. Реализации и версии

6.4.1. Две реализации

Историческая реализация — **tor** на C, которая выполняет все роли: клиента, реле, доверенного сервера каталогов и onion-сервиса. Наряду с ней Tor Project разрабатывает **Arti**, переписанную на Rust реализацию (гл. 30), которая уже используется как клиент и для onion-сервисов. В публичном консенсусе от 23 сентября 2026 года Arti не используется ни одним реле: 6027 реле работают на tor 0.4.9.12, 2520 — на 0.4.9.11, а большинство остальных — на более старых выпусках серии 0.4.9. Сорок семь реле используют разрабатываемую версию 0.5.0.0-alpha-dev.

6.4.2. Субпротоколы

Как сеть из тысяч узлов, обновляющихся в разное время, вводит новые возможности? Tor не опирается для этого на номера версий программного обеспечения. Вместо этого используются **субпротоколы**: каждая возможность относится к субпротоколу с собственным номером версии, а каждое реле объявляет поддерживаемые версии в строке `pr` консенсуса [tor-spec § Subprotocol-based versioning]. Например, реле в публичной сети объявляет:

```
pr Conflux=1 Cons=1-2 Desc=1-4 DirCache=2 FlowCtrl=1-2 HSDir=2 HSIntro=4-5
   HSRend=1-2 Link=3-5 LinkAuth=3 Microdesc=1-3 Padding=2 Relay=2-6
```

Клиент, желающий использовать ntor-v3 с реле, проверяет, объявляет ли оно `Relay=4`; CGO требует `Relay=6`, а управление перегрузкой — `FlowCtrl=2`. Эти числа обозначают независимые *возможности*, а не накопительные версии: формально `Relay=6` не подразумевает `Relay=4` (Предложение 346, поэтому там предпочитается термин «возможности субпротокола»). На практике tor объявляет их непрерывным диапазоном, поскольку реализует функции по порядку. В консенсусе также перечислены *обязательные* субпротоколы: версия tor, не поддерживающая их, понимает, что устарела, и отказывается работать, вместо того чтобы подвергать пользователей риску.

Итоги

- Сеть состоит из клиентов, реле (guard, middle, exit, HSDir и кэшей каталогов), доверенных серверов каталогов, мостов и onion-сервисов.
- Протокол образует стек: TLS-каналы, ячейки, цепи, послойное шифрование и потоки.
- Guard-реле знает пользователя, но не пункт назначения; exit-реле знает пункт назначения, но не пользователя.
- tor (C) и Arti (Rust) сосуществуют. Возможности согласуются как субпротоколы, объявляемые в строке `pr` консенсуса.

Упражнения

Упражнение 6.1 ⋆ Для каждой роли — guard, middle, exit, HSDir и directory authority — укажите, что она знает о пользователе, посещающем веб-сайт через Tor.

Упражнение 6.2 ⋆ Какие функции, рассматриваемые в главах 9–11, поддерживает реле из строки `pr`, приведённой в этой главе? Можно ли использовать его как exit-реле с CGO?

Упражнение 6.3 ⋆⋆ Через Tor загружается веб-страница размером 100 KB. Оцените по уровням, сколько байт проходит по линии между клиентом и guard-реле, учитывая ячейки, заголовки сообщений RELAY и SENDME.

Дополнительная литература

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]: исходная архитектура, значительная часть которой применяется до сих пор.
- Tor Metrics, `https://metrics.torproject.org/`: актуальные сведения о реле, пользователях и версиях.

Глава 7

Каналы и протокол соединения

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; Arti 2.6.0; протокол соединения 4–5

Цели главы

К концу этой главы вы сможете:

- объяснять, что такое канал и почему Tor строит его поверх TLS;
- описывать ключи, образующие идентичность реле, и то, как они связываются между собой;
- прослеживать процедуру установления канала ячейка за ячейкой: VERSIONS, CERTS, AUTH_CHALLENGE, AUTHENTICATE и NETINFO;
- читать и проверять сертификат Tor Ed25519;
- понимать, почему клиент не аутентифицируется и что это означает для анонимности;
- открывать проверенный канал к реле лабораторной сети (лабораторная работа 1).

Цепь, центральная конструкция Tor, представляет собой последовательность переходов. Но каждый переход опирается на более простой элемент: прямое зашифрованное соединение между двумя соседними узлами. Спецификация называет это соединение **каналом**. В этой главе мы увидим, как устанавливается канал и, прежде всего, как каждая сторона проверяет идентичность того узла, с которым намеревалась соединиться. Формат ячеек, передаваемых по каналу, рассматривается в гл. 8, к которой мы будем часто обращаться.

7.1. Что такое канал?

Канал — это прямое зашифрованное соединение между двумя реле либо между клиентом и реле [tor-spec § Channels]. Технически это TLS-сессия поверх TCP, переносящая ячейки. Её характеризуют три свойства.

Асимметричные роли Открыть канал может кто угодно (*инициатор* (initiator)), но принять его может только реле (*ответчик* (responder)). Клиент не принимает соединения: у него нет порта, прослушивающего подключения из сети Tor.

Проверенная идентичность Ответчик всегда криптографически доказывает владение одной или несколькими идентичностями реле. Инициатор обычно уже знает, какую идентичность должен увидеть: он получает её из консенсуса, из списка доверенных серверов каталогов, встроенного в исходный код, либо, если канал открывается при

расширении цепи, из самого сообщения `EXTEND2`. Если предъявленная идентичность не совпадает с ожидаемой, канал должен быть закрыт.

Совместное использование Один канал переносит множество цепей, различаемых по CircID (разд. 8.4). Узлы предпочитают повторно использовать уже существующий канал, а не открывать второй к тому же реле. Канал без цепей закрывается после случайно выбранного периода бездействия, диапазон которого зависит от назначения канала (`channelpadding_get_channel_idle_timeout` в `src/core/or/channelpadding.c`): 3–4,5 минуты для клиентских каналов и 45–75 минут для каналов между двумя реле, где центральное значение задаётся параметром консенсуса `nf_conntimeout_relays` (один час). Пока канал используется, `KeepalivePeriod` вместо этого приводит к отправке ячейки заполнения (PADDING) каждые пять минут, чтобы межсетевые экраны не закрывали соединение.

Подробнее: Канал или соединение?

В старой документации и многих научных работах каналы называются *соединениями* (*OR connections*). Спецификация отказалась от этого термина, потому что его можно спутать с TCP-соединениями, которые exit-реле открывают для исходящих потоков. Оба термина сохранились в исходном коде tor: `or_connection_t` обозначает TLS-соединение, а `channel_t` — абстракцию, которая его использует.

7.2. Идентичность реле

Прежде чем разбирать процедуру установления канала, нужно понять, *что именно* должно быть доказано. Реле идентифицируется долговременными ключами. Спецификация использует систематические имена: префикс `KP_` обозначает открытый ключ, а `KS_` — соответствующий закрытый [tor-spec § Relay keys and identities]. Для канала важны следующие ключи:

`KP_relayid_ed` **Идентичность Ed25519**, или «главный ключ идентичности (master identity key)». Она никогда не истекает и служит одной цели: подписывать ключ подписи. Её можно хранить офлайн.

`KP_relayid_rsa` 1024-битная **устаревшая идентичность RSA**. На ней по-прежнему основаны 40-значные шестнадцатеричные отпечатки в Tor Metrics, но безопасность протокола больше от неё не зависит.

`KP_relaysign_ed` Среднесрочный **ключ подписи**, сертифицированный идентичностью. Именно этот ключ реле использует в обычной ежедневной работе онлайн.

`KP_link_ed` Краткосрочный ключ, используемый только тогда, когда реле инициирует канал и должно аутентифицироваться (разд. 7.6).

На рис. 7.1 показано, как сертификаты связывают эти ключи. Идея представляет собой *цепочку доверия*, начинающуюся с идентичности и доходящую до TLS-сертификата конкретного соединения.

В лаборатории эти ключи находятся в файлах каталога `keys/` каждого реле: `ed25519_master_id_public_key` и соответствующий секретный ключ идентичности, `ed25519_signing_cert` и `ed25519_signing_secret_key` для ключа подписи, а также `secret_id_key` для RSA-идентичности. Файл `fingerprint-ed25519` в каталоге узла содержит его Ed25519-идентичность в base64.

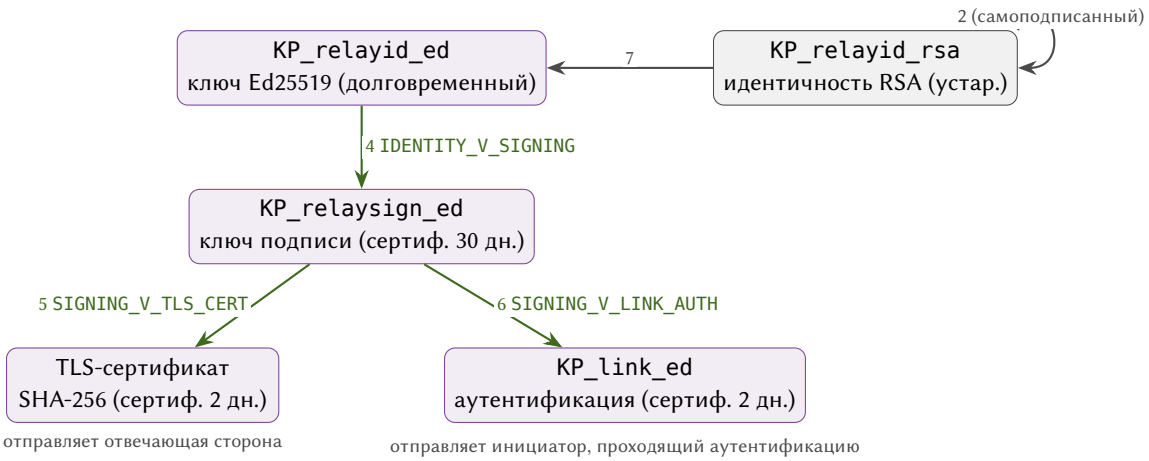


Рис. 7.1: Цепочка сертификатов, связывающая Ed25519-идентичность реле с TLS-сертификатом соединения. Каждая стрелка обозначает сертификат: ключ у её начала подписывает ключ или отпечаток у конца. Число указывает CertType в ячейке CERTS. Справа показана устаревшая цепочка RSA, связывающая старую RSA-идентичность с Ed25519-идентичностью.

Историческая справка: От RSA к Ed25519

До 2015 года единственной идентичностью реле был его 1024-битный RSA-ключ — размер, который сегодня считается недостаточным. Предложение 220 ввела Ed25519-идентичности и формат сертификатов, описываемый в этой главе; поддержка появилась в tor 0.2.7.2-alpha. RSA-ключ сохранили ради совместимости и связали с Ed25519-ключом перекрёстным сертификатом (cross-certificate).

7.3. Процедура установления канала в общих чертах

Рис. 7.2 кратко показывает установление канала [tor-spec § Negotiating and initializing channels].

1. Инициатор открывает TLS-соединение.

2. Обе стороны отправляют ячейку **`VERSIONS`** и согласуют версию протокола соединения (разд. 8.3).

3. Ответчик отправляет **`CERTS`** (сертификаты, подтверждающие идентичность), **`AUTH_CHALLENGE`** (вызов, который инициатор подпишет, если захочет аутентифицироваться) и **`NETINFO`** (время и адреса).

4. Инициатор проверяет сертификаты. Если он является публичным реле, то отвечает **`CERTS`** и **`AUTHENTICATE`**; в любом случае завершает установление канала ячейкой **`NETINFO`**.

`NETINFO` отмечает конец процедуры установления канала: после её отправки сторона больше не отправляет ячейки установления канала. Так ответчик понимает, решил ли инициатор не аутентифицироваться. Между ними может появляться любое число ячеек **`VPADDING`**, но никакие другие типы ячеек недопустимы.

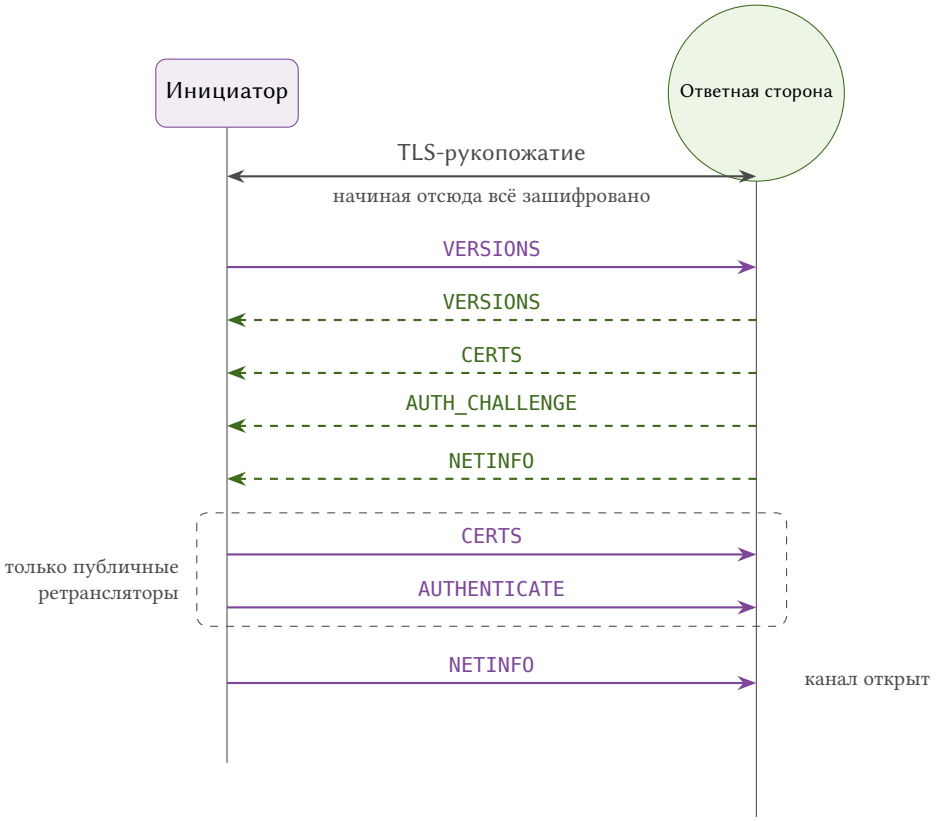


Рис. 7.2: Процедура установления канала. После установления TLS всё передаётся в зашифрованном виде. Ответчик всегда отправляет CERTS, AUTH_CHALLENGE и NETINFO именно в таком порядке. Пунктирный блок применяется только тогда, когда инициатор — публичное реле, которому необходимо подтвердить собственную идентичность; клиент или мост отправляет только NETINFO.

7.4. Уровень TLS

Tor использует TLS необычным образом. У веб-сайта сертификат сервера подписан центром сертификации и содержит доменное имя; браузер проверяет и подпись, и имя. Реле, напротив, предъявляет **самоподписанный** сертификат без значимого имени, а клиент не проверяет его по обычным правилам TLS. В коде лабораторной работы 1 это выражается двумя строками, которые почти в любом другом контексте были бы серьёзной ошибкой безопасности:

```
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
```

Здесь это не ошибка, потому что аутентификация выполняется *после*, внутри канала, с помощью ячейки **CERTS**. TLS обеспечивает шифрование и целостность канала и доказывает владение ключом TLS-сертификата; всё остальное устанавливают ячейки.

Спецификация добавляет несколько правил [tor-spec § The TLS handshake]:

- реализациям следует поддерживать TLS 1.3, а реле — также TLS 1.2, с современными наборами шифров (AES-GCM, ChaCha20-Poly1305 и обменом ключами X25519 или P-256);
- *возобновление TLS-сеанса (session resumption)* не следует разрешать: это ослабляет прямую секретность и уже делало возможными атаки вроде *Triple Handshake* (2013);
- сжатие TLS использовать не следует.

В лабораторной сети клиент Python с OpenSSL 3.5 согласует с tor TLS 1.3 и набор `TLS_AES_256_GCM_SHA384`.

Историческая справка: Зачем нужна такая необычная процедура установления канала?

Самые ранние версии протокола соединения помещали сертификаты Tor непосредственно в TLS-рукопожатие (версия 1), затем скрывали их с помощью повторного согласования TLS (версия 2), сигнализируя выбранный вариант через список предлагаемых наборов шифров. Причиной была цензура: до появления подключаемых транспортов (гл. 16) Tor пытался выглядеть как обычное HTTPS-соединение между браузером и веб-сервером. В TLS 1.2 сертификаты передавались открыто и легко распознавались. В версии 3 сертификаты переместили внутрь зашифрованного канала, в ячейки **CERTS**. Сегодня сопротивление цензуре возложено на подключаемые транспорты, а старые варианты устарели; спецификация отмечает, что по состоянию на март 2025 года tor всё ещё принимал некоторые из них, но они были запланированы к удалению.

7.5. CERTS: подтверждение идентичности

7.5.1. Ячейка

CERTS — ячейка переменной длины (команда 129), тело которой содержит список сертификатов:

Поле	Байты	Содержимое
N	1	число сертификатов
повторяется N раз:		
CertType	1	тип сертификата
CertLen	2	длина сертификата
Certificate	CertLen	сертификат

Ячейка **CERTS** не может содержать два сертификата одного типа. Определённые типы перечислены в табл. 7.1 [cert-spec § Certificate types (CERT_TYPE field)].

Тип	Имя	Формат	Что сертифицирует
1	`TLS_LINK_X509`	X.509	TLS-ключ, подписанный RSA-идентичностью (устарел)
2	`RSA_ID_X509`	X.509	самоподписанная RSA-идентичность
3	`LINK_AUTH_X509`	X.509	RSA-ключ аутентификации (устарел)
4	`IDENTITY_V_SIGNING`	Ed	ключ подписи, подписанный Ed25519-идентичностью
5	`SIGNING_V_TLS_CERT`	Ed	отпечаток TLS-сертификата, подписанный ключом подписи
6	`SIGNING_V_LINK_AUTH`	Ed	ключ `KP_link_ed`, подписанный ключом подписи
7	`RSA_ID_V_IDENTITY`	RSA→Ed	Ed25519-идентичность, подписанная RSA-идентичностью
8–12	onion-сервисы (8, 9, 11), перекрёстный сертификат ключа ntor (10), семейства (12)		

Таблица 7.1: Типы сертификатов. «Ed» обозначает собственный формат сертификата Tor Ed25519.

7.5.2. Сертификат Tor Ed25519

Tor не использует X.509 для сертификатов Ed25519. Вместо этого применяется собственный, гораздо более простой формат [cert-spec § Tor Ed25519 Certificates], показанный на рис. 7.3.

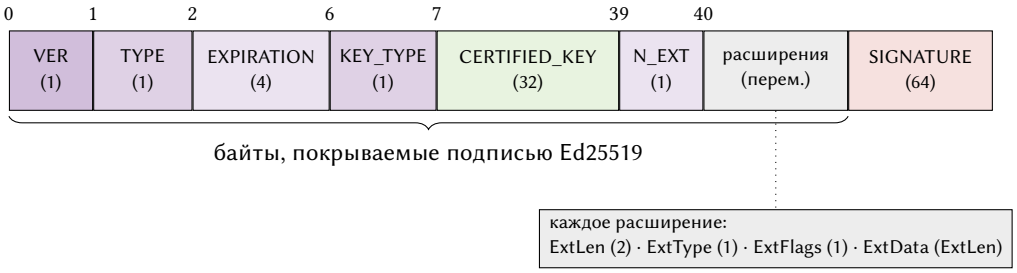


Рис. 7.3: Сертификат Tor Ed25519. Подпись охватывает каждое поле, расположенное перед ней. Срок действия выражается в *часах* с эпохи Unix: беззнаковое четырёхбайтовое поле не переполнится сотни тысяч лет.

VERSION имеет значение 1.

CERT_TYPE имеет то же значение, что CertType в табл. 7.1.

EXPIRATION_DATE задаёт срок действия в часах с 1 января 1970 года.

CERT_KEY_TYPE указывает содержимое следующего поля: 1 означает ключ Ed25519, а 3 — SHA-256-отпечаток X.509-сертификата. Версии tor до 0.4.5.1-alpha включительно всегда записывали 1, и спецификация требует, чтобы реализации это допускали.

CERTIFIED_KEY содержит 32 сертифицируемых байта.

Extensions Каждое расширение содержит длину, тип, флаги и данные. Сегодня определено только расширение типа 4, *signed-with-ed25519-key*, содержащее ключ, подписавший сертификат. Флаг 1 (`AFFECTS_VALIDATION`) определяет, как обрабатывать неизвестное расширение: если он установлен, сертификат необходимо отклонить; иначе расширение игнорируется.

SIGNATURE — подпись Ed25519 всех предыдущих полей.

7.5.3. Проверка

Ответчик должен отправить ровно один сертификат типа 4 и один типа 5. Инициатор проверяет их следующим образом [tor-spec § Authenticating the responder from its CERTS]:

1. Сертификат типа 4 содержит в расширении 4 ключ Ed25519: заявленную идентичность `KP_relayid_ed`. Сертификат должен быть подписан именно этим ключом, то есть быть *самоподписанным*; сертифицированный ключ — `KP_relaysign_ed`.

2. Сертификат типа 5 должен быть подписан `KP_relaysign_ed`, а его сертифицированным ключом должен быть SHA-256-отпечаток TLS-сертификата, полученного во время TLS-рукопожатия.

3. Ни один сертификат не должен быть просрочен.

4. Наконец, полученная идентичность сравнивается с ожидаемой.

Почему эта цепочка что-то доказывает? Шаг 2 связывает ключ подписи с TLS-сертификатом *этого* соединения. TLS-рукопожатие уже доказало, что ответчик владеет закрытым ключом, соответствующим этому сертификату. Шаг 1 связывает ключ подписи с идентичностью. Самозванец мог бы скопировать ячейки **`CERTS`** настоящего реле, поскольку они не содержат секретов, но не смог бы ими воспользоваться: во время TLS-рукопожатия ему пришлось бы предъявить TLS-сертификат настоящего реле и, следовательно, владеть его закрытым ключом.

Внимание: Проверяйте ожидаемую идентичность

Полностью корректная цепочка доказывает лишь то, что ответчик владеет *какой-то* идентичностью. Если инициатор пропустит шаг 4, любой обладатель любой идентичности сможет выдать себя за требуемое реле. В лабораторной работе 1 параметр `expected_identity` существует именно поэтому, а один из тестов проверяет, что реле с другой идентичностью отклоняется.

7.5.4. Устаревшая цепочка RSA

Инициатор при желании может также проверить RSA-идентичность. Сертификат типа 2 содержит 1024-битный RSA-ключ (самоподписанный, экспонента 65537), а сертификат типа 7 в формате «перекрёстного сертификата (cross-certificate)» связывает этот ключ с уже проверенной Ed25519-идентичностью. RSA-подпись перекрёстного сертификата вычисляется над SHA-256-отпечатком строки `"Tor TLS RSA/Ed25519 cross-certificate"`, за которой следуют поля сертификата. Префикс не позволяет повторно использовать ту же подпись в другом контексте.

7.5.5. Что реле действительно отправляет

В лабораторной сети реле `test003r` отправляет ячейку **`CERTS`** с пятью сертификатами:

```
type 1  584 bytes
type 2  442 bytes
type 4  140 bytes  expires 2026-10-24 16:00  key_type=1  ext=[4]
type 5  104 bytes  expires 2026-09-29 11:00  key_type=3  ext=[]
type 7  165 bytes
```

Помимо Ed25519-цепочки (типы 4 и 5) и RSA-цепочки (типы 2 и 7), реле всё ещё отправляет старый сертификат типа 1 для старых клиентов. Он не просто имеет ту же длину, что TLS-сертификат: совпадает и его отпечаток. Сравнение байтов показывает, что это *точно тот*

же сертификат. Его длину не стоит запоминать, и полезно понимать почему. Общее имя (common name) сертификата — фиктивный домен, который tor строит как `www.`, затем от 4 до 25 случайных символов и `.com` (`crypto_random_hostname` в `src/lib/tls/tortls_openssl.c`). Только это даёт разброс примерно в двадцать байт между реле. Длина меняется и *со временем* у одного реле, потому что tor обновляет TLS-контекст, а вместе с ним сертификат, каждые два часа (`MAX_SSL_KEY_LIFETIME_INTERNAL` в `src/core/or/or.h`). Повторное измерение через сорок минут дало тело **`CERTS`** длиной 1456 байт вместо 1451. Если ваши измерения отличаются, это ожидаемо: совпадать должна *структура*, а сумма длин сертификатов плюс один байт счётчика и три байта заголовка на каждый сертификат должна равняться длине тела ячейки. Две даты истечения показывают разные сроки жизни: ключ подписи сертифицируется на 30 дней, а привязка к TLS-сертификату — лишь на 2 дня. Это значения tor по умолчанию (параметры `SigningKeyLifetime` и `TestingLinkCertLifetime`, причём второй настраивается только в тестовой сети), поэтому они относятся и к публичной сети. Ключ идентичности остаётся постоянным, тогда как представляющие его онлайн-ключи часто обновляются, ограничивая ущерб от компрометации реле.

7.6. AUTH_CHALLENGE и AUTHENTICATE

7.6.1. Кто аутентифицируется?

Аутентификация инициатора необязательна, а правило, определяющее, кто должен её выполнять, относится к важнейшим архитектурным решениям протокола [tor-spec § Channels]:

- **Публичное реле**, открывающее канал к другому реле, *должно, как правило*, аутентифицироваться: тогда ответчик знает, что разговаривает с сетевым реле, а не клиентом.
- **Клиент** или **мост** *не должен, как правило*, аутентифицироваться.

Причина второго правила — анонимность. У клиента нет идентичности, и её создание было бы вредно: guard-реле могло бы узнавать того же клиента при повторном подключении с другого адреса. Мост, в свою очередь, не должен раскрывать своим пользователям, что является сетевым реле. Компромисс заключается в том, что реле может сделать вывод, что оно является первым переходом цепи, если канал не аутентифицирован (гл. 8, разд. 8.6). Это одна из причин, по которым реле отказываются выступать exit-реле для одношаговых цепей.

7.6.2. Вызов

Ответчик в любом случае отправляет ячейку **`AUTH_CHALLENGE`**. Она содержит 32 случайных байта и список допустимых методов аутентификации:

Поле	Байты	Содержимое
Challenge	32	случайное значение, отдельное для каждого канала
N_Methods	2	число методов
Methods	2× N_Methods	список методов

Определены методы 1 (`RSA-SHA256-TLSSecret`, устарел), 2 (никогда не был реализован) и 3 (`Ed25519-SHA256-RFC5705`). Ячейка лабораторного реле имеет тело длиной 36 байт: $32+2+2$, что соответствует одному методу, номер 3.

7.6.3. Ответ

Реле, желающее аутентифицироваться, отвечает собственной ячейкой **CERTS** (используя типы 4 и 6 вместо 4 и 5, поскольку инициатор не обязательно предъявлял TLS-сертификат), после чего отправляет **AUTHENTICATE**. Для метода 3 тело **AUTHENTICATE** представляет собой 352-байтовый документ, подписанный ключом KP_link_ed:

Поле	Байты	Содержимое
TYPE	8	строка AUTH0003
CID, SID	32 + 32	SHA-256-отпечатки RSA-идентичностей инициатора и ответчика
CID_ED, SID_ED	32 + 32	Ed25519-идентичности инициатора и ответчика
SLOG	32	SHA-256 всех байтов, *полученных* по каналу до AUTH_CHALLENGE включительно
CLOG	32	SHA-256 всех байтов, *отправленных* по каналу до AUTHENTICATE
SCERT	32	SHA-256 TLS-сертификата ответчика
TLSSECRETS	32	значение, выведенное из секретов TLS-сессии (*keying material exporter*)
RAND	24	случайные байты
SIG	64	подпись Ed25519 всех предыдущих полей

Каждое поле связывает подпись с определённым аспектом *этого* соединения: участвующими идентичностями (CID, SID и т. д.), предыдущим диалогом (SLOG и CLOG, включающими случайный вызов), TLS-сертификатом (SCERT) и самой TLS-сессией (TLSSECRETS). Такую подпись нельзя повторно использовать ни в другом соединении, ни посреднику, пересылающему сообщения между двумя разными TLS-сессиями (атака *man-in-the-middle*), поскольку TLSSECRETS и SCERT на двух сторонах будут различаться.

Подробнее: Ошибка, сохранённая в спецификации

TLSSECRETS вычисляется через TLS *exporter* (RFC 5705 и, для TLS 1.3, RFC 8446, § 7.5) с меткой `"EXPORTER FOR TOR TLS CLIENT BINDING AUTH0003"` и контекстным значением. Предполагалось использовать в качестве контекста Ed25519-идентичность инициатора, однако реализация по ошибке использовала CID, то есть отпечаток RSA-идентичности. Поскольку это не ослабляет безопасность, спецификация решила документировать фактическое поведение, а не менять его. Это хороший пример того, как для протокола с тысячами уже развёрнутых реализаций поведение реального кода может стать стандартом.

7.7. NETINFO: время и адреса

Последняя ячейка процедуры установления канала, отправляемая обеими сторонами, — **NETINFO**. Это ячейка фиксированной длины (команда 8) со следующим телом:

Поле	Байты	Содержимое
TIME	4	секунды с эпохи Unix, big-endian
OTHERADDR	2 + ALEN	адрес другой стороны глазами отправителя
NMYADDR	1	число собственных адресов отправителя
MYADDR	NMYADDR × (2 + ALEN)	собственные адреса отправителя

Каждый адрес кодируется типом (один байт: `0x04` для IPv4, `0x06` для IPv6), длиной (4 или 16) и значением. Эти поля решают две задачи:

- **Время** позволяет обнаруживать неверные часы: tor пишет предупреждение, если время, полученное от реле, отличается от собственного более чем на час (константа `NETINFO_NOTICE_SKEW` в `src/core/or/channeltls.c`).
- **Адреса** позволяют реле узнать публичный адрес, под которым оно видно, а инициатору — проверить, достиг ли он реле по одному из его *канонических* адресов, то есть адресов, опубликованных в дескрипторе.

Внимание: Клиент не раскрывает свои часы

Реле отправляют собственное время и адреса. Клиент, напротив, должен отправлять `00 00 00 00` как время и не должен отправлять ни одного собственного адреса. Причина — *идентификация по отпечатку*: часы компьютера почти всегда имеют небольшой характерный сдвиг. Если клиент сообщал бы время каждому guard-реле, этот сдвиг мог бы помочь узнавать клиента с течением времени. Это иллюстрирует общее правило, к которому мы ещё не раз вернёмся: клиент Tor не должен раскрывать ничего, что не является строго необходимым.

Беззнаковое четырёхбайтовое поле TIME переполнится в 2106 году; Предложение 338 рассматривает его расширение до восьми байт. Получатель ячейки **`NETINFO`** не должен слепо доверять значениям: другая сторона может солгать о времени и адресах.

Вот ячейка **`NETINFO`**, полученная от реле `test003r` и декодированная лабораторной работой 1:

```
Netinfo(timestamp=1790511791, peer_address='127.0.0.1',
        my_addresses=['127.0.0.1'])
```

Время соответствует 27 сентября 2026 года, 12:23:11 UTC. Оба адреса являются адресами обратной петли (loopback), потому что вся сеть работает на одном компьютере. Наш ответ, построенный `client_netinfo_cell`, имеет тело `00000000 04 04 7f000001 00`: нулевое время, адрес реле и ни одного собственного адреса.

7.8. После установления канала

Теперь канал *открыт* и может переносить цепи. Простой эксперимент обнаруживает интересную деталь. Если после открытия канала слушать его, не создавая цепей, реле не отправляет *ничего*: в лаборатории за пятнадцать секунд не приходит ни одной ячейки.

Можно было бы ожидать ячейки **`PADDING`**: начиная с версии 5 протокола соединения клиенты и guard-реле обмениваются ячейками заполнения на бездействующих каналах через случайные интервалы от 1,5 до 9,5 секунды, чтобы сетевой наблюдатель, например маршрутизатор, записывающий статистику NetFlow, не мог отличить периоды активности от тишины [padding-spec § Connection-level padding]. Исходный код tor объясняет, почему здесь этого не происходит. Функция, решающая, добавлять ли заполнение (`channelpadding_decide_to_pad_channel` в `src/core/or/channelpadding.c`), активирует его только на каналах, *уже использованных* для полных цепей или пользовательского трафика. Со стороны реле канал считается использованным так только тогда, когда через него проходит многошаговая цепь: первая ячейка **`RELAY_EARLY`**, расширяющая цепь ко второму переходу, либо **`RELAY`** в цепи, продолжающейся за пределы реле (`src/core/or/relay.c`). Новый канал остаётся безмолвным, как и канал, несущий только одношаговые цепи, что мы увидим в гл. 9. Заполнение появится в гл. 10, когда наш клиент расширит цепь.

Этот небольшой эпизод даёт методологический урок: *отсутствие наблюдения не является доказательством*. Прежде чем заключать, что заполнение не работает, нужно проверить, выполнены ли условия, при которых он вообще должен появляться.

7.9. Лабораторная работа: открытие проверенного канала

Лабораторная работа 1: Открыть канал к реле и проверить его идентичность

Цель. Написать `labs/ru/код/torlab/channel.py`, который открывает канал как клиент Tor: TLS, **`VERSIONS`**, проверка **`CERTS`**, чтение **`AUTH_CHALLENGE`** и **`NETINFO`** и отправка собственной **`NETINFO`**.
Предварительные условия. Лабораторная работа 2 (`cells.py`, гл. 8): обработка ячеек лежит в основе канала, поэтому логично сначала написать её, хотя объясняется она в следующей главе. Здесь нисходящее объяснение (сначала канал, затем ячейки) и восходящая реализация (сначала ячейки, затем канал) идут в противоположных направлениях, и это нормально. Также необходима работающая лабораторная сеть (гл. 19) и библиотека `cryptography` для Ed25519.
Оценочное время. 2–3 часа.

Шаг 1: декодировать сертификат Ed25519. Следуя рис. 7.3, читаем заголовок фиксированной длины, затем расширения и сохраняем байты, охваченные подписью:

```
def decode_ed_certificate(data):
    if len(data) < 40 + 64 or data[0] != 1:
        raise ChannelError("некорректный сертификат")
    _, kind, expiration, key_type = struct.unpack(">BBIB", data[:7])
    key = data[7:39]
    extension_count, pos, extensions = data[39], 40, {}
    for _ in range(extension_count):
        length, extension_type, flag = struct.unpack(">HBB", data[pos:pos+4])
        body = data[pos+4:pos+4+length]
        if extension_type != 4 and flag & 1:     # AFFECTS_VALIDATION
            raise ChannelError("неизвестное расширение влияет на проверку")
        extensions[extension_type] = (flag, body)
        pos += 4 + length
    return EdCertificate(kind, expiration, key_type, key,
                         extensions, data[:pos], data[pos:])
```

Шаг 2: проверить цепочку. Эта функция применяет правила разд. 7.5.3 и возвращает доказанную идентичность:

```
def verify_responder_identity(certificates, cert_tls_der):
    id_v_sign = decode_ed_certificate(certificates[4])
    sign_v_tls = decode_ed_certificate(certificates[5])
    identity = id_v_sign.signing_key               # extension 4
    id_v_sign.verify_signature(identity)             # self-signed
    sign_v_tls.verify_signature(id_v_sign.certified_key)
    if sign_v_tls.certified_key != sha256(cert_tls_der).digest():
        raise ChannelError("TLS-сертификат не совпадает")
    for c in (id_v_sign, sign_v_tls):
        if c.expired():
            raise ChannelError("сертификат просрочен")
    return identity
```

TLS-сертификат в DER-формате получаем из сокета вызовом `sock.getpeercert(binary_form=True)`. Python возвращает его даже несмотря на то, что в данном случае мы его ещё не проверяли.

Шаг 3: установление канала. Функция `open_channel` отправляет **`VERSIONS`**, предлагая версии 4 и 5 (как Arti, мы пропускаем 3). Затем она читает ячейки до появления **`NETINFO`**, обрабатывая каждый тип: игнорирует **`VPADDING`** и все **`VERSIONS`** после первой, а любую другую неожиданную ячейку отклоняет. Наконец, проверяет сертификаты, сравнивает идентичность с ожидаемой и отправляет собственную **`NETINFO`**. Полный код находится в репозитории; его центральный цикл:

```
while netinfo is None:
    cell, buffer = _read(sock, buffer, version)
    if cell.command == CellCommand.VERSIONS:
        if version:
            continue
        version = negotiate_version(SUPPORTED_VERSIONS, cell.body)
    elif cell.command == CellCommand.CERTS:
        certificates = decode_certs(cell.body)
    elif cell.command == CellCommand.AUTH_CHALLENGE:
        ...                                      # допустимые методы
    elif cell.command == CellCommand.NETINFO:
        netinfo = decode_netinfo(cell.body)
    elif cell.command != CellCommand.VPADDING:
        raise ChannelError("неожиданная ячейка во время handshake")
```

Шаг 4: тесты без сети. В `labs/ru/код/torlab/test/test_channel.py` генерируются два ключа Ed25519, строится корректная цепочка сертификатов и проверяется, что код её принимает. Затем цепочка нарушается всеми способами, описанными в спецификации: изменённая подпись, несертифицированный ключ подписи, другой TLS-сертификат, истёкший сертификат, повторяющийся тип; для каждого дефекта проверяется обнаружение. Тест только корректного случая ничего не доказывает: проверяющая функция, принимающая всё подряд, тоже его пройдёт.

Шаг 5: запуск в лабораторной сети. При работающей сети:

```
from torlab.channel import open_channel, identity_from_file

expected = identity_from_file("labs/ru/лаборатория/net/nodes/003r/fingerprint-ed25519"
    )
c = open_channel("127.0.0.1", 5103, expected_identity=expected)
print(c.version, c.ed_identity.hex(), c.netinfo, c.auth_methods)
print(c.registry)
```

Сокращённый вывод:

```
5 785d3199f21d20ad...  Netinfo(time=1790511793, ...)  [3]
['TLS TLSv1.3 TLS_AES_256_GCM_SHA384', '<- VERSIONS (6 байт)',
 '<- CERTS (1451 байт)', '<- AUTH_CHALLENGE (36 байт)',
 '<- NETINFO (509 байт)', '-> NETINFO: канал открыт']
```

Если вместо этого подключиться к порту 5104 (реле `test004r`), ожидая идентичность другого реле, получается корректная ошибка: `реле имеет идентичность, отличную от ожидаемой`.

Итоги

- Канал — это TLS-сессия между двумя соседними узлами. Каналы принимают только реле, а один канал переносит множество цепей.

- Идентичность реле — его Ed25519-ключ `KP_relayid_ed`, сертифицирующий ключ подписи `KP_relaysign_ed`; RSA-идентичность сохраняется ради совместимости.
- TLS-сертификат реле самоподписан. Аутентификация происходит внутри канала через ячейку **`CERTS`**, связывающую идентичность, ключ подписи и отпечаток TLS-сертификата.
- Инициатор обязан проверить подписи, сроки действия, привязку к TLS и **ожидаемую идентичность**.
- Реле, инициирующие канал, аутентифицируются с помощью **`AUTHENTICATE`**, подписи, привязанной к TLS-сессии и диалогу. Клиенты и мосты не аутентифицируются, чтобы не создавать узнаваемых идентичностей.
- **`NETINFO`** завершает процедуру установления канала. Клиент отправляет нулевую временную метку и ни одного собственного адреса.
- Заполнение на уровне канала требует, чтобы канал уже переносил *полную* цепь или пользовательский трафик. Новый канал или канал только с одношаговыми цепями остаётся безмолвным. После прохождения этого порога заполнение запускается также до первого прикладного трафика (`nf_pad_before_usage`, по умолчанию 1).

Упражнения

Упражнение 7.1 ⋆ Почему клиенту Tor не нужен прослушивающий порт, доступный из сети? В каких случаях реле инициирует каналы?

Упражнение 7.2 ⋆ Сертификат Ed25519 имеет EXPIRATION_DATE, равную `0x000772F7`. Когда истекает его срок? Почему спецификация использует часы, а не секунды?

Упражнение 7.3 ⋆ Тело ячейки **`AUTH_CHALLENGE`** имеет длину 38 байт. Сколько методов она предлагает? Корректна ли 36-байтовая ячейка с N_Methods, равным 2?

Упражнение 7.4 ⋆⋆ Противник записывает настоящую ячейку **`CERTS`** реле и без изменений воспроизводит её на собственном сервере, предъявляя собственный TLS-сертификат. Какой шаг проверки завершится ошибкой? Что изменится, если противник также предъявит TLS-сертификат настоящего реле?

Упражнение 7.5 ⋆⋆ Запишите в шестнадцатеричном виде тело ячейки **`NETINFO`**, отправленной реле с адресом `203.0.113.5` в 12:00:00 UTC 1 января 2026 года клиенту, который подключился с адреса `198.51.100.7`.

Упражнение 7.6 ⋆⋆ Предположим, клиенты Tor аутентифицируются перед guard-реле ключом, который генерируется один раз и хранится на диске. Опишите два конкретных способа, которыми это повредило бы анонимности пользователя.

Упражнение 7.7 ⋆⋆ Расширьте `labs/ru/код/torlab/channel.py`, добавив необязательную проверку RSA-идентичности (типы 2 и 7) по разд. 7.5. Сравните SHA-1-отпечаток полученного RSA-ключа с файлом `fingerprint` реле. *Подсказка:* отпечаток — SHA-1 DER-кодировки открытого ключа.

Упражнение 7.8 ⋆⋆⋆ При работающей лабораторной сети откройте канал с помощью `labs/ru/код/torlab/channel.py` и держите его открытым, пока клиент `test008c` загружает страницу через сеть. Получите ли вы ячейки **`PADDING`** по *своему* каналу? Сформулируйте гипотезу, проверьте её и объясните результат с учётом `channelpadding_decide_to_pad_channel`.

Упражнение 7.9 ⋆⋆⋆ Объясните, почему поле TLSSECRETS в **`AUTHENTICATE`** не позволяет посреднику ретранслировать аутентификацию реле A реле B через другую TLS-сессию, даже если посредник точно пересылает каждую ячейку. Что произошло бы без TLSSECRETS?

Дополнительная литература

- *Tor Protocol Specification*: главы «Channels», «Negotiating and initializing channels» и «Relay keys», а также *Tor certificate formats* (cert-spec) [Thee].
- Предложение 220, *Migrate server identity keys to Ed25519*: причины перехода от RSA.
- *Tor channel заполнение* в спецификации заполнение: модель противника на основе NetFlow и выбор интервалов.
- RFC 5705 и RFC 8446 (§ 7.5): функции TLS *keying material exporter*.

Глава 8

Ячейки

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; Arti 2.6.0; протокол соединения 3–5

Цели главы

К концу этой главы вы сможете:

- описывать побайтно форматы ячеек фиксированной и переменной длины;
- объяснять, почему Tor использует единицы фиксированного размера, от чего защищает это решение и от чего не защищает;
- интерпретировать CircID и правила их выбора на каждом канале;
- распознавать все команды ячеек и основные команды RELAY;
- читать заголовок сообщения RELAY и объяснять назначение `Recognized` и `Digest`;
- писать на Python кодировщик/декодировщик ячеек (лабораторная работа 2).

В предыдущей главе мы увидели, как два узла Tor открывают *канал*: TLS-соединение, по которому они согласуют версию протокола и аутентифицируют друг друга. Теперь рассмотрим, *что передаётся* внутри этого канала. Ответ прост: **ячейки**. Всё, что переносит Tor, от построения цепи до загруженных веб-страниц, разбивается на ячейки. Понимание ячеек даёт нам алфавит, на котором записан весь протокол.

8.1. Зачем нужны ячейки?

Низколатентная система анонимности не может скрыть сам *факт* общения двух узлов, но может пытаться скрыть *что* они передают и *сколько* промежуточные узлы способны об этом узнать. Если бы каждый узел пересылал данные ровно в том виде, в каком они поступили, пакетами разных размеров, наблюдатель мог бы проследить сообщение длиной 1337 байт через сеть, просто находя сообщение той же длины на каждой линии.

Поэтому исходная архитектура onion routing, которой следует Tor [DMS04], использует **единицу передачи фиксированного размера**: ячейку. Когда все ячейки имеют одинаковый размер:

- длина отдельного сообщения не раскрывает ни его содержимое, ни принадлежность к конкретному потоку;
- реле могут просто и эффективно управлять очередями и буферами;
- минимальный заголовок позволяет реле переслать ячейку, прочитав всего несколько байтов.

Внимание: Что ячейки не скрывают

Ячейки стандартизируют *размер* отдельных единиц, но не *количество* единиц и не *время* их передачи. Противник, наблюдающий оба конца цепи, всё ещё может коррелировать объёмы и временные характеристики трафика. Это фундаментальное ограничение Tor, которое мы рассмотрим в гл. 26. Частичные меры защиты — заполнение (padding) на уровне канала и цепи — обсуждаются в гл. 7 и 26 и части V.

Историческая справка: 512 байт

В статье 2004 года, представившей Tor, описаны ячейки по 512 байт: 2-байтовый идентификатор цепи, 1-байтовая команда и 509 байт данных [DMS04]. Тело длиной 509 байт сохранилось до наших дней. Однако идентификатор цепи в версии 4 протокола соединения вырос до 4 байт, увеличив общий размер ячейки до 514 байт.

8.2. От потока TLS к ячейкам

После завершения TLS-рукопожатия стороны обмениваются ячейками *последовательно*, одна за другой, как непрерывным потоком байтов [tor-spec § Cells (messages on channels)]. Границы TLS-записей и ячеек независимы: одна ячейка может быть разделена между несколькими записями, а одна TLS-запись может содержать несколько ячеек или их фрагментов. Тем не менее спецификация требует: «разбиение на TLS-записи не должно раскрывать сведений о типе или содержимом ячеек» (перевод).

Для разработчика это имеет практическое следствие: получатель не может ожидать, что сокет будет выдавать «по одной ячейке». Нужно накапливать байты в буфере и извлекать ячейку только после получения её целиком, оставляя лишние байты в буфере. Именно так будет работать декодировщик лабораторной работы 2.

8.3. Форматы ячеек

Существует два формата: **фиксированной длины** и **переменной длины**. Оба начинаются одинаковыми полями `CircID` и `Command`.

8.3.1. Константы

Спецификация определяет следующие длины [tor-spec § Message lengths], где v — версия протокола соединения, согласованная на канале:

Имя	Байты	Значение
`CELL_BODY_LEN`	509	тело ячейки фиксированной длины
`CIRCID_LEN`(v), $v < 4$	2	длина CircID
`CIRCID_LEN`(v), $v \geq 4$	4	
`CELL_LEN`(v), $v < 4$	512	размер ячейки фиксированной длины
`CELL_LEN`(v), $v \geq 4$	514	

Для любого v: `CELL_LEN`$(v) = 1 +$ `CIRCID_LEN`$(v) +$ `CELL_BODY_LEN`.

Подробнее: Старое имя

В старом коде и документации `CELL_BODY_LEN` называется `PAYLOAD_LEN`. Оба имени обозначают одно значение: 509.

8.3.2. Ячейки фиксированной длины

На рис. 8.1 показана ячейка фиксированной длины с современным протоколом соединения ($v \geq 4$).

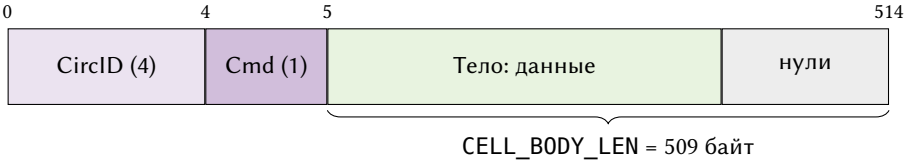


Рис. 8.1: Ячейка фиксированной длины (протокол соединения ≥ 4): 4-байтовый CircID, 1-байтовая команда и 509-байтовое тело, всего 514 байт. Если полезное содержимое короче 509 байт, остаток тела заполняется нулями. Числа обозначают смещения в байтах от начала ячейки.

Если передаваемая информация занимает меньше 509 байт, отправитель **обязан** заполнить оставшиеся байты тела нулями, а получатель **обязан** их игнорировать [tor-spec § Padding fixed-length cell bodies].

8.3.3. Ячейки переменной длины

Некоторым ячейкам, особенно используемым при установлении канала, требуется больше 509 байт, например для сертификата, или значительно меньше. Формат переменной длины решает эту задачу (рис. 8.2), добавляя 2-байтовое целое поле `Length` в порядке *big-endian* и не используя заполнение.

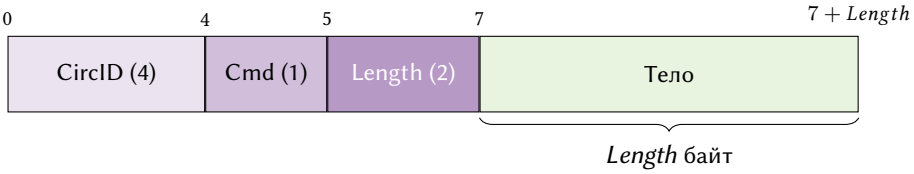


Рис. 8.2: Ячейка переменной длины (протокол соединения ≥ 4): после CircID и команды 2-байтовое big-endian поле Length задаёт число последующих байтов тела. Поэтому тело может содержать от 0 до 65 535 байт.

8.3.4. Как различать два формата

Получателю нужно знать, сколько байтов читать, *до* того как он получит всю ячейку. Правило полностью определяется командой [tor-spec § Cells (messages on channels)]:

- команда 7 (**`VERSIONS`**) означает ячейку переменной длины;
- любая другая команда меньше 128 означает фиксированную длину;
- любая команда не меньше 128 означает переменную длину.

Иначе говоря, старший бит байта команды служит индикатором формата, за единственным историческим исключением **VERSIONS**.

Историческая справка: Исключение VERSIONS

В версии 1 протокола соединения все ячейки имели фиксированную длину. Версия 2 протокола соединения ввела согласование версий, и **VERSIONS**, получившая значение 7 после уже существовавших команд 0–6, стала единственной ячейкой переменной длины. Версия 3 протокола соединения зарезервировала диапазон ≥ 128 для команд переменной длины. Версии 1 и 2 сегодня устарели, и поддерживать их в реализациях не следует.

8.3.5. Особый случай VERSIONS

Возникает циклическая зависимость: длина CircID определяется согласованной версией, но сама версия согласуется через ячейку **VERSIONS**. Решение — **всегда** отправлять **VERSIONS** так, будто $v = 0$, с 2-байтовым CircID. Её тело — последовательность 2-байтовых big-endian целых, по одному на каждую поддерживаемую версию. Клиент, поддерживающий версии 3, 4 и 5, отправляет:

Байты	Поле	Значение
`00 00`	CircID	0, 2-байтовый CircID
`07`	Command	**VERSIONS**
`00 06`	Length	тело 6 байт
`00 03 00 04 00 05`	Body	версии 3, 4, 5

Обе стороны выбирают **наибольшую версию, присутствующую в обеих** ячейках **VERSIONS**. Если общей версии нет или тело содержит нечётное число байтов, соединение должно быть закрыто [tor-spec § Negotiating versions with VERSIONS cells].

Версия	Описание
1	(устарела) первоначальное установление канала на основе сертификатов
2	(устарела) установление канала с повторным согласованием TLS; вводит ячейки переменной длины
3	(устарела) современное «внутрипротокольное» установление канала (гл. 7)
4	4-байтовый CircID
5	поддержка заполнения на уровне канала и его согласования

В современных реализациях tor предлагает версии 3, 4 и 5 (массив `or_protocol_versions` в `src/core/or/connection_or.c`), а Arti — только 4 и 5. Поэтому два актуальных экземпляра всегда согласуют версию 5.

8.4. CircID

Поле `CircID` идентифицирует цепь, к которой относится ячейка. Его значение равно 0 для ячеек, не принадлежащих ни одной цепи, например **VERSIONS**, **NETINFO** и **PADDING**.

Ключевой момент: **CircID локальны для канала**. Трёхпереходная цепь пересекает три канала (клиент–guard, guard–middle и middle–exit) и имеет *разный* CircID на каждом. Каждое реле поддерживает таблицу пересылки, сопоставляющую входящую пару (канал, CircID) с исходящей парой (канал, CircID), как показано на рис. 8.3.

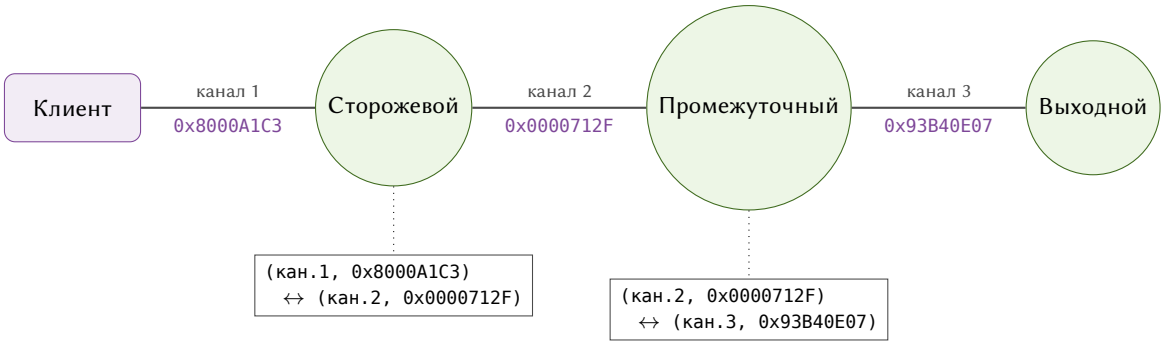


Рис. 8.3: Одна и та же цепь имеет различный CircID на каждом канале. Guard- и middle-реле переводят идентификатор по таблице пересылки. Поэтому наблюдатель на одной линии не может обнаружить ту же цепь на другой, просто сравнив CircID. Значения иллюстративны, но соблюдают правило старшего бита: на канале 2 бит равен нулю, то есть соединение открыл middle-реле, а guard-реле повторно его использует.

Это решение имеет два преимущества: узлам не нужно координировать выбор идентификаторов, а CircID не становится глобальной меткой, по которой можно связать разные сегменты одной цепи.

8.4.1. Кто выбирает CircID?

CircID выбирает узел, отправляющий ячейку создания цепи (**`CREATE2`**, гл. 9). Поскольку цепи может создавать любая сторона канала, нужно исключить одновременный выбор одинакового значения. Правило зависит от версии [tor-spec § Choosing circuit IDs in create cells]:

- **протокол соединения** ≥ 4: узел, который *открыл* соединение, использует CircID со старшим битом (MSB), равным 1; другая сторона — CircID с MSB, равным 0;
- **протокол соединения** ≤ 3: две половины пространства назначались сравнением открытых ключей идентичности двух реле; реле с меньшим ключом использовало MSB 0.

Значение 0 зарезервировано; никакое другое значение не зарезервировано. Существующие реализации выбирают CircID **случайно** из доступных значений, и спецификация рекомендует новым реализациям поступать так же, чтобы не становиться различимыми. Tor прекращает попытки создать новую цепь на канале после 64 последовательных неудач найти свободное значение.

Подробнее: Зачем 4 байта?

При 2 байтах и доступной только половине пространства канал может переносить чуть больше 32 000 цепей на сторону, а случайный выбор приводит к коллизиям задолго до исчерпания пространства. Предложение 214 (2012) формулирует проблему прямо: «у реле заканчиваются идентификаторы цепей» (перевод). Предложение ввело версию 4 протокола соединения с 4-байтовыми CircID, реализованную в tor 0.2.4.11-alpha.

8.5. Команды ячеек

Байт `Command` определяет интерпретацию тела. Следующие таблицы перечисляют все определённые значения [tor-spec § Interpreting the fields: Command]. В столбце C «да» означает,

что команде требуется ненулевой CircID, «нет» — нулевой. Столбец P показывает первую версию протокола соединения, поддерживающую команду; пустое значение означает как минимум версию 3.

Значение	Команда	C	P	Назначение
0	**PADDING**	нет		заполнение на уровне канала
1	**CREATE**	да		создать цепь (TAP, устарело)
2	**CREATED**	да		ответ на CREATE (устарело)
3	**RELAY**	да		end-to-end данные
4	**DESTROY**	да		уничтожить цепь
5	**CREATE_FAST**	да		создать цепь без криптографии с открытым ключом
6	**CREATED_FAST**	да		ответ на CREATE_FAST
8	**NETINFO**	нет		время и адреса
9	**RELAY_EARLY**	да		как RELAY, но в ограниченном количестве
10	**CREATE2**	да		создать цепь (расширяемый handshake)
11	**CREATED2**	да		ответ на CREATE2
12	**PADDING_NEGOTIATE**	да	5	согласование заполнения

Таблица 8.1: Команды ячеек фиксированной длины.

Значение	Команда	C	Назначение
7	**VERSIONS**	нет	согласование протокола соединения
128	**VPADDING**	нет	заполнение переменной длины
129	**CERTS**	нет	сертификаты
130	**AUTH_CHALLENGE**	нет	вызов для аутентификации
131	**AUTHENTICATE**	нет	аутентификация инициатора
132	**AUTHORIZE**	нет	зарезервировано

Таблица 8.2: Команды ячеек переменной длины.

Неизвестные команды. Другие значения не разрешены. Получатель неизвестной команды *может* молча отбросить ячейку и *может* закрыть канал. Такая строгость связана с тем, что каждая новая команда вводится с новой версией протокола соединения: корректная реализация никогда не отправит команду, поддержку которой другая сторона не объявляла. До появления версионирования протокола соединения неизвестные команды, напротив, игнорировались в предположении, что они пришли от более новой версии Tor.

Команды можно сгруппировать по назначению:

Управление каналом **VERSIONS**, **CERTS**, **AUTH_CHALLENGE**, **AUTHENTICATE**, **NETINFO**: установление канала из гл. 7.

Заполнение **PADDING**, **VPADDING**, **PADDING_NEGOTIATE**: фиктивный трафик, затрудняющий анализ трафика на линии.

Жизненный цикл цепи **CREATE2**/**CREATED2** и их исторические варианты, **DESTROY**: создание и уничтожение цепей (гл. 9).

Транспорт **RELAY** и **RELAY_EARLY**: всё, что передаётся *внутри* цепи.

8.5.1. DESTROY

Тело ячейки **DESTROY** содержит всего один байт, задающий *причину* закрытия цепи [torspec § Tearing down circuits]. Получив **DESTROY**, узел освобождает ресурсы цепи и, если сам не

является конечной точкой, передаёт ячейку **DESTROY** следующему узлу.

0	NONE	7	OR_IDENTITY
1	PROTOCOL	8	CHANNEL_CLOSED
2	INTERNAL	9	FINISHED
3	REQUESTED	10	TIMEOUT
4	HIBERNATING	11	DESTROYED
5	RESOURCELIMIT	12	NOSUCHSERVICE
6	CONNECTFAILED		

Внимание: Код причины как побочный канал

Спецификация рекомендует **всегда** отправлять причину NONE и не передавать далее полученный код причины. Если реле пересылало бы исходную причину в обоих направлениях, два сговорившихся реле одной цепи могли бы использовать её как сигнал, чтобы определить, что находятся в одной цепи: маленький пример того, как даже один байт способен стать *побочным каналом*.

8.5.2. RELAY_EARLY

RELAY_EARLY имеет тот же формат, что **RELAY**, но служит для **ограничения длины цепи** [tor-spec § Handling RELAY_EARLY cells]. Правила:

- клиент должен отправлять сообщения расширения цепи (EXTEND2) только внутри **RELAY_EARLY**; начиная с версии 0.2.3.11-alpha реле отклоняют расширения, полученные иначе;
- реле, получившее более 8 прямых **RELAY_EARLY** в одной цепи, должно, как правило, закрыть её; при получении хотя бы одной *обратной*, направленной к клиенту, ячейки оно обязано немедленно закрыть цепь;
- чтобы не раскрывать длину цепи, клиенты также должны, как правило, отправлять примерно первые 8 ячеек, не предназначенных первому переходу, как **RELAY_EARLY**.

Без этого ограничения противник мог бы строить чрезвычайно длинные цепи, многократно проходящие через одни и те же реле и тем самым усиливающие его собственный трафик. Это атака перегрузки длинными маршрутами, описанная Эвансом, Динглдайном и Гротхоффом [EDG09].

Историческая справка: Неожиданный сигнал

Правило, запрещающее обратные **RELAY_EARLY**, появилось после инцидента 2014 года: некоторые реле отправляли такие ячейки клиентам как скрытый канал для маркировки цепей пользователей onion-сервисов. Подробно этот случай разбирается в гл. 27.

8.6. Внутри RELAY-ячеек

Ячейки **RELAY** и **RELAY_EARLY** несут **сообщения RELAY**: команды, которыми обмениваются клиент и узлы цепи, от расширения цепи до данных TCP-потока. Их 509-байтовые тела зашифрованы слоями, по одному на каждый переход. Это и есть «луковица» из гл. 10. С точки зрения канала тело RELAY-ячейки поэтому выглядит полностью заполненным: нулевого padding не видно.

После снятия всех слоёв шифрования открытое тело следует традиционному формату на рис. 8.4 [tor-spec § Relay cells].

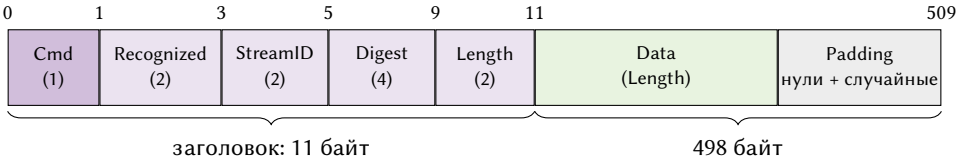


Рис. 8.4: *Расшифрованное* тело традиционной RELAY-ячейки: заголовок 11 байт, до 498 байт данных и, при наличии места, заполнение до 509 байт. Спецификация рекомендует до четырёх нулевых байтов вначале, затем случайные байты; если осталось менее четырёх байтов, все они нулевые, а при 498 байтах данных заполнения нет.

Команда RELAY (1 байт) Тип сообщения (табл. 8.3).

Recognized (2 байта) Ноль в открытом сообщении. Это поле позволяет реле быстро определить, адресована ли ячейка ему: если после снятия собственного слоя поле ненулевое, ячейка наверняка всё ещё зашифрована и должна быть переслана.

StreamID (2 байта) Поток, к которому относится сообщение, выбранный клиентом; ноль для сообщений о цепи в целом.

Digest (4 байта) Первые 4 байта *накопительного* дайджеста всех сообщений, которыми обменивались с этим переходом (SHA-1 для обычных цепей, SHA3-256 для цепей onion-сервисов). Он подтверждает, что ячейка предназначена этому реле и не была изменена.

Length (2 байта) Число значимых байтов в поле Data.

Data Содержимое сообщения.

Заполнение Остаток, доводящий тело до 509 байт.

11-байтовый заголовок оставляет максимум $509 - 11 = 498$ байт данных на ячейку.

8.6.1. Recognized и Digest: двухступенчатый фильтр

Зачем нужны два поля? Вычислять дайджест для каждой транзитной ячейки было бы дорого. Recognized — дешёвый фильтр: после расшифрования ячейки, которая всё ещё зашифрована, его 2 байта псевдослучайны и равны нулю лишь с вероятностью 2^{-16}. Только при нулевом Recognized реле вычисляет дайджест. Если дайджест совпадает, ячейка «распознана» и обрабатывается; иначе пересылается дальше. Вычисление и его слабости рассматриваются в гл. 10.

8.6.2. Заполнение сообщений RELAY

Заполнение сообщений RELAY отличается от заполнения нулями ячеек фиксированной длины. Спецификация рекомендует 4 нулевых байта, а затем **случайные байты** [tor-spec § Relay cells]. Случайные байты делают содержимое ячейки непредсказуемым. Это необходимо для аутентифицированных сообщений SENDME из Предложения 289, которые доказывают фактическое получение данных, включая дайджест полученных ячеек (гл. 11). Получатель не должен зависеть от содержимого заполнения.

8.6.3. Команды RELAY

Табл. 8.3 перечисляет определённые команды RELAY. Столбец Dir. задаёт направление: F (*forward*, только от клиента внутрь цепи), B (*backward*, только к клиенту) или F/B (оба направления). C означает *управляющее* сообщение, для которого StreamID должен быть нулевым.

Значение	Команда	Dir.	Назначение
Основной протокол			
1	BEGIN	F	открыть поток
2	DATA	F/B	данные потока
3	END	F/B	закрыть поток
4	CONNECTED	B	поток успешно открыт
5	SENDME	F/B, C?	подтверждение получения (управление потоком)
6–7	EXTEND/EXTENDED	F/B, C	расширение с TAP (устарело)
8–9	TRUNCATE/TRUNCATED	F/B, C	усечение цепи (не используется)
10	DROP	F/B, C	заполнение между концами цепи
11–12	RESOLVE/RESOLVED	F/B	удалённое разрешение DNS
13	BEGIN_DIR	F	поток к кэшу каталогов
14–15	EXTEND2/EXTENDED2	F/B, C	расширение цепи
16–18	*зарезервировано*		UDP поверх Tor (Предложение 339)
Conflux			
19–22	CONFLUX_…	F/B, C	LINK, LINKED, LINKED_ACK, SWITCH: несколько цепей (Предложение 329)
Onion-сервисы			
32	ESTABLISH_INTRO	F, C	создать точку введения
33	ESTABLISH_RENDEZVOUS	F, C	создать точку встречи
34–35	INTRODUCE1/INTRODUCE2	F/B, C	запрос введения
36–37	RENDEZVOUS1/RENDEZVOUS2	F/B, C	соединить стороны в rendezvous
38–40	…_ESTABLISHED, INTRODUCE_ACK	B, C	подтверждения
Заполнение цепи и управление потоком			
41–42	PADDING_NEGOTIATE/…D	F/B, C	заполнение цепи
43–44	XOFF/XON	F/B	управление потоком (Предложение 324)

Таблица 8.3: Команды RELAY. SENDME является управляющим сообщением только тогда, когда относится к цепи, а не к отдельному потоку (гл. 11).

Если ячейка распознана, но её relay command неизвестна, её необходимо отбросить; тем не менее её содержимое учитывается в дайджесте и окнах управления потоком.

8.7. К новому формату: предложение 340 и CGO

Традиционный формат имеет два известных ограничения. Первое связано с эффективностью: большое сообщение всегда занимает как минимум целую ячейку, тогда как несколько маленьких сообщений нельзя упаковать в одну. Второе связано с безопасностью: 4-байтовый дайджест и неаутентифицированное шифрование AES-CTR оставляют возможность атак маркирования (*tagging*), которые мы рассмотрим в гл. 10.

Предложение 340 (*Packed and fragmented relay messages*) разделяет понятия *ячейки* и *сообщения*: одно сообщение может фрагментироваться по нескольким ячейкам, а несколько сообщений — упаковываться в одну. Предложение 359 (*Counter Galois Onion, Updated*) вводит новую схему шифрования RELAY, CGO, и использует формат ячейки предложения 340. Расшифрованная CGO-ячейка имеет структуру:

Поле	Байты
тег аутентификации	16
тело	$509 - 16 = 493$
внутри тела: команда	1
длина	2
StreamID (только для некоторых команд)	2
данные	*длина*
заполнение (4 нуля, затем случайные байты)	остаток

StreamID присутствует только для команд, которым он требуется (BEGIN, DATA, END, CONNECTED, RESOLVE, RESOLVED, BEGIN_DIR, XON, XOFF). Recognized и Digest исчезают, их заменяет тег аутентификации. Клиент использует CGO с реле только тогда, когда его поддерживают обе стороны (субпротокол `Relay=6`) и согласуют его при создании цепи.

Подробнее: Сколько данных помещается в CGO-ячейку?

Из формата следует, что для сообщения DATA доступно $493 - 1 - 2 - 2 = 488$ байт данных, на 10 меньше традиционного формата: это цена 16-байтового тега аутентификации.

CGO уже не просто предложение: tor поддерживает его начиная с серии 0.4.9 (введён в 0.4.9.3-alpha, сентябрь 2025 года). В Arti поддержка всегда включена; параметр сборки `counter-galois-onion` сохраняется лишь для совместимости. Репозиторий спецификаций по-прежнему помечает оба предложения как «Open»: статус предложения отражает процесс стандартизации и не всегда совпадает с состоянием кода.

8.8. Анатомия сеанса

Теперь можно собрать части вместе и ячейка за ячейкой проследить, что клиент отправляет guard-реле, чтобы построить трёхпереходную цепь и загрузить веб-страницу. Последовательность показана на рис. 8.5; в следующих главах каждый шаг разбирается подробно.

1. **Установление канала**: **`VERSIONS`**, затем **`CERTS`**, **`AUTH_CHALLENGE`** и **`NETINFO`** от guard-реле; клиент отвечает **`NETINFO`**. CircID всегда равен 0.

2. **Первый переход**: **`CREATE2`** с новым CircID (MSB 1, поскольку соединение открыл клиент); guard-реле отвечает **`CREATED2`**.

3. **Второй и третий переходы**: две ячейки **`RELAY_EARLY`**, содержащие `EXTEND2`; guard-реле, а затем middle-реле преобразуют их в ячейки **`CREATE2`**, отправляемые следующему узлу. Ответы возвращаются как **`RELAY`** с `EXTENDED2`.

4. **Поток**: ячейка с `BEGIN` для exit-реле, которое отвечает `CONNECTED`; затем `DATA` в обоих направлениях вперемешку с `SENDME`.

5. **Закрытие**: `END` для потока, а в конце жизни цепи — **`DESTROY`**.

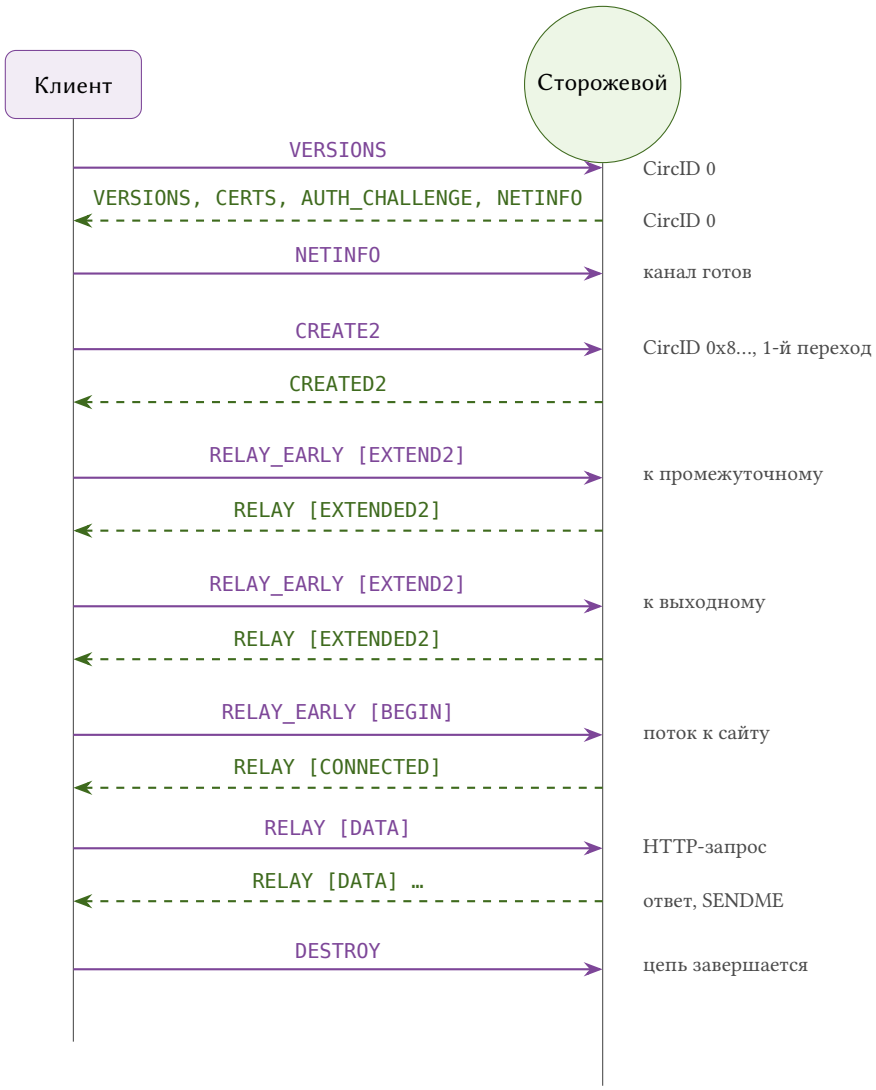


Рис. 8.5: Ячейки, которыми обмениваются клиент и guard-реле по TLS-каналу при создании трёхпереходной цепи и открытии потока. Guard-реле видит только ячейки собственного канала: оно не может читать содержимое RELAY-ячеек, предназначенных middle- и exit-реле, поскольку они зашифрованы ключами, которыми guard-реле не владеет.

8.9. Лабораторная работа: кодировщик ячеек

Лабораторная работа 2: Кодирование и декодирование ячеек

Цель. Написать модуль `labs/ru/код/torlab/cells.py`, который будет использоваться во всех последующих лабораторных работах.
Предварительные условия. Python 3.10 или новее, `pytest`. Сеть не нужна: лабораторная работа оперирует вручную построенными байтами.
Оценочное время. 1–2 часа.

Шаг 1: константы и команды. Превратим таблицы спецификации в перечисления Python, чтобы каждое числовое значение задавалось только в одном месте:

```
CELL_BODY_LEN = 509
RELAY_HEADER_LEN = 11
RELAY_DATA_MAX = CELL_BODY_LEN - RELAY_HEADER_LEN   # 498

class CellCommand(IntEnum):
    PADDING = 0
    CREATE = 1
    # ... все значения из таблиц 8.1 и 8.2
    AUTHORIZE = 132

def circid_len(version):
    return 2 if version < 4 else 4

def is_variable_length(command):
    return command == CellCommand.VERSIONS or command >= 128
```

Шаг 2: кодирование. Функция получает CircID, команду, тело и версию link protocol и выдаёт байты для отправки. Обратите внимание на особую обработку **`VERSIONS`** и нулевой padding:

```
def encode(cell, version):
    if cell.command == CellCommand.VERSIONS:
        version = 0                  # CircID всегда 2 байта
    n = circid_len(version)
    header = cell.circ_id.to_bytes(n, "big") + bytes([cell.command])
    if is_variable_length(cell.command):
        return (header + struct.pack(">H", len(cell.body))
                + cell.body)
    return header + cell.body.ljust(CELL_BODY_LEN, b"\x00")
```

Шаг 3: декодирование потока. Поскольку байты приходят из TLS-сокета фрагментами, декодировщик возвращает `None`, пока полной ячейки нет. Иначе он возвращает ячейку и неиспользованные байты:

```
def decode(buffer, version):
    n = circid_len(version)
    if len(buffer) < n + 1:
        return None
    circ_id = int.from_bytes(buffer[:n], "big")
    command = buffer[n]
    if is_variable_length(command):
```

```
        if len(buffer) < n + 3:
            return None
        (length,) = struct.unpack(">H", buffer[n+1:n+3])
        start, end = n + 3, n + 3 + length
    else:
        start, end = n + 1, n + 1 + CELL_BODY_LEN
    if len(buffer) < end:
        return None
    return Cell(circ_id, command, buffer[start:end]), buffer[end:]
```

Шаг 4: VERSIONS и согласование версии.

```
def negotiate_version(ours, received_body):
    if len(received_body) % 2:
        raise CellError("VERSIONS содержит нечётное число байтов")
    theirs = {v for (v,) in struct.iter_unpack(">H", received_body)}
    common = set(ours) & theirs
    if not common:
        raise CellError("нет общей версии link protocol")
    return max(common)
```

Диагностические строки в русской лабораторной среде локализованы, поскольку предназначены читателю; числовые значения протокола и API функции остаются неизменными.

Шаг 5: сообщение RELAY. Наконец, строим 11-байтовый заголовок. Строка формата `">BHH4sH"` для `struct` точно соответствует полям на рис. 8.4:

```
def encode_relay(msg, random_bytes=os.urandom):
    header = struct.pack(">BHH4sH", msg.command, msg.recognized,
                                msg.stream_id, msg.digest, len(msg.data))
    space = CELL_BODY_LEN - RELAY_HEADER_LEN - len(msg.data)
    padding = b"\x00" * min(4, space) + random_bytes(max(0, space - 4))
    return header + msg.data + padding
```

Параметр `random_bytes` позволяет тестам заменять случайные байты предсказуемыми значениями.

Проверка. Репозиторий содержит тесты в `labs/ru/код/torlab/test/test_cells.py`. Запустите их из каталога `labs/ru/код`:

```
$ python3 -m pytest -q torlab/test/test_cells.py
........                                                 [100%]
8 passed
```

Без имени файла `pytest` запускает тесты всех лабораторных работ репозитория. Например, один тест проверяет побайтное совпадение ячейки **`VERSIONS`** для версий 3, 4 и 5 с примером из разд. 8.3: `0000 07 0006 0003 0004 0005`.

Расширения. В лабораторной работе 1 (гл. 7) этот модуль используется для общения с настоящим реле частной сети. В лабораторной работе 4 мы добавим вычисление digest и послойное шифрование.

Итоги

- Ячейки — единицы передачи Tor по каналу. Начиная с версии 4 протокола соединения ячейка фиксированной длины содержит 514 байт (CircID 4 + команда 1 + тело 509); старые версии использовали 512 байт.
- Ячейки переменной длины добавляют 2-байтовое поле Length. **VERSIONS** (7) и все команды ≥ 128 имеют переменную длину.
- **VERSIONS** всегда использует 2-байтовый CircID; обе стороны выбирают наибольшую общую версию.
- CircID локален для каждого канала. Сторона, открывшая соединение, выбирает случайные значения с MSB 1.
- **DESTROY** должна, как правило, всегда нести причину NONE, чтобы не создавать побочные каналы; **RELAY_EARLY** ограничивает длину цепи.
- Расшифрованное тело RELAY-ячейки имеет 11-байтовый заголовок (command, Recognized, StreamID, Digest, Length) и максимум 498 байт данных. Padding состоит из 4 нулевых, затем случайных байтов.
- Предложения 340 и 359 (CGO) вводят новый формат с 16-байтовым тегом аутентификации и необязательным StreamID.

Упражнения

Упражнение 8.1 ⋆ Сколько байт занимает ячейка **CREATE2** на канале, согласовавшем link protocol 3? А версию 5? Сколько байт занимает **VERSIONS** с телом `00 04 00 05`?

Упражнение 8.2 ⋆ Для каждого значения команды 0, 7, 9, 12, 129 и 200 укажите, имеет ячейка фиксированную или переменную длину. Какое из этих значений недопустимо?

Упражнение 8.3 ⋆ Клиент открывает канал к guard-реле с link protocol 5. Какой из CircID он может использовать для новой цепи: `0x00000000`, `0x12345678` или `0x92345678`? Почему?

Упражнение 8.4 ⋆⋆ Вручную декодируйте следующую последовательность байтов, полученную на канале с link protocol 4. Для каждой ячейки укажите CircID, команду и длину тела:
`00 00 00 00 81 00 03 aa bb cc 80 00 00 07 04 00 …`
(Вторая ячейка продолжается оставшейся частью тела, которая не показана.)

Упражнение 8.5 ⋆⋆ Запишите в шестнадцатеричном виде первые 11 байт открытого тела relay message DATA в потоке 5, содержащего 300 байт данных, до вычисления digest. Сколько байт padding следует после данных и сколько из них гарантированно равны нулю?

Упражнение 8.6 ⋆⋆ Промежуточное реле обрабатывает 1 000 000 ячеек, которые ему не адресованы. Для скольких из них в среднем потребуется вычислять digest? Почему одного поля Recognized недостаточно, чтобы определить, предназначена ли ячейка этому реле?

Упражнение 8.7 ⋆⋆ Опишите конкретный сценарий, в котором два сговорившихся реле могли бы использовать код причины в **DESTROY**, чтобы узнать, что находятся в одной цепи. Почему использование причины NONE предотвращает этот сценарий?

Упражнение 8.8 ⋆⋆ Клиент загружает через цепь файл размером 1 MiB (1 048 576 байт). Сколько ячеек DATA требуется в традиционном формате? Сколько с CGO? В каждом случае

какой процент полезных данных приходится на накладные расходы заголовков ячеек (link protocol 4, без учёта TLS)?

Упражнение 8.9 ⋆⋆⋆ Расширьте лабораторную работу 2 *property-based* тестом, например с библиотекой Hypothesis, который генерирует случайные ячейки фиксированной и переменной длины, кодирует и объединяет их, а затем делит результат на фрагменты случайной длины. Проверьте, что декодирование возвращает в точности исходные ячейки.

Упражнение 8.10 ⋆⋆⋆ Найдите в исходном коде Arti функции кодирования и декодирования ячеек канала (crate `tor-cell`). Сравните обработку «неполной ячейки» с функцией `decode` лабораторной работы 2.

Дополнительная литература

- *Tor Protocol Specification*, разделы «Cells», «Relay cells» и «Negotiating channels» [Thee]: первичный источник для этой главы.
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], раздел 4: исходный формат ячеек и его мотивация.
- N. Evans, R. Dingledine, C. Grothoff, *A Practical Congestion Attack on Tor Using Long Paths* [EDG09]: атака, мотивировавшая **`RELAY_EARLY`**.
- Предложения 340 и 359 в torspec: будущий формат ячеек.

Глава 9

Построение цепи: TAP, CREATE_FAST, ntor и ntor-v3

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; предложения 216, 332, 350

Цели главы

К концу этой главы вы сможете:

- описывать, как клиент строит цепь по одному переходу с CREATE2 и EXTEND2;
- объяснять, что должен гарантировать протокол согласования ключей цепи: аутентификацию реле, секретность и прямую секретность;
- пошагово разбирать согласование ключей CREATE_FAST, ntor и ntor-v3 и понимать, почему TAP был выведен из эксплуатации;
- получать четыре ключа одного перехода (Df, Db, Kf, Kb) из согласованного ключевого материала;
- проверять реализацию по официальным тестовым векторам;
- строить реальные цепи к реле лабораторной сети (лабораторная работа 3).

Имея открытый канал (гл. 7) и алфавит ячеек (гл. 8), мы располагаем всем необходимым для построения центральной структуры Tor: **цепи**. В этой главе мы увидим, как клиент устанавливает с каждым реле маршрута общий секрет, неизвестный никому другому, и как превращает этот секрет в ключи, шифрующие трафик. Использование этих ключей для послойного шифрования рассматривается в гл. 10.

9.1. По одному переходу

Клиент строит цепь *постепенно*, или «телескопически» [DMS04]:

1. Выбирается маршрут, например входной узел R_1 (guard), промежуточный R_2 (middle) и выходной R_3 (exit) (гл. 13).

2. По каналу к R_1 клиент отправляет ячейку **`CREATE2`**, содержащую первую половину криптографического протокола согласования ключей. R_1 отвечает **`CREATED2`**, после чего клиент и R_1 имеют общий ключевой материал.

3. Клиент отправляет сообщение `EXTEND2`, зашифрованное для R_1, с просьбой расширить цепь до R_2. В сообщении находится первая половина нового протокола согласования ключей, предназначенного уже R_2.

4. R_1 открывает или повторно использует канал к R_2 и отправляет ему ячейку **CREATE2** с этими данными согласования ключей. Ответ **CREATED2** возвращается к R_1, который пересылает его клиенту в сообщении EXTENDED2.

5. Клиент завершает согласование ключей с R_2 и повторяет процедуру для R_3, на этот раз через R_1 и R_2.

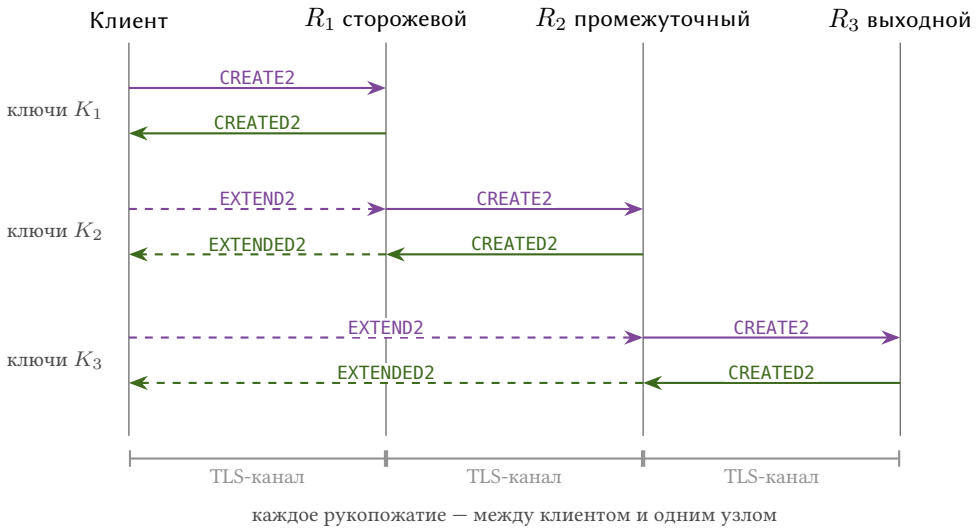


Рис. 9.1: Телескопическое построение трёхпереходной цепи. Каждый протокол согласования ключей выполняется только между клиентом и одним реле; промежуточные реле лишь переносят сообщения. Пунктирными стрелками показаны сообщения, идущие внутри цепи и зашифрованные ключами уже установленных переходов.

У такого устройства есть фундаментальное следствие. Согласование ключей с R_3 проходит через R_1 и R_2, но его сообщения зашифрованы для открытого ключа R_3: ни R_1, ни R_2 не могут получить его общий секрет. Поскольку каждый переход использует отдельный секрет, ни одно реле не может прочитать то, чем клиент обменивается с другими. Входной узел R_1 знает клиента, но не знает, куда направляется трафик; выходной R_3 знает пункт назначения, но не знает отправителя.

Историческая справка: Onion routing первого поколения

В исходной системе Naval Research Laboratory клиент строил одну «луковицу»: сообщение, послойно зашифрованное открытыми ключами каждого реле. Каждое реле снимало слой, чтобы узнать следующий переход и собственный сеансовый ключ (гл. 3). Реле, скомпрометированное в будущем, могло бы расшифровать ранее записанные «луковицы». Телескопическая цепь Tor с эфемерным обменом Диффи–Хеллмана для каждого перехода ввела *прямую секретность*: после удаления сеансовых ключей даже владение долговременными ключами не позволяет расшифровать прошлый трафик [DMS04].

9.2. Ячейки и сообщения

9.2.1. CREATE2 и CREATED2

Формат **CREATE2** намеренно универсален [tor-spec § Create and Created cells]: тип протокола согласования ключей (HTYPE, 2 байта), длина (HLEN, 2 байта) и данные (HDATA). **CREATED2**

содержит только HLEN и HDATA, поскольку тип уже известен. Всё помещается в ячейку фиксированной длины, поэтому данные согласования ключей в **CREATE2** не могут превышать $509 - 4 = 505$ байт.

HTYPE	Протокол	Статус
0x0000	TAP	устарел; удалён в tor 0.4.9.1-alpha
0x0001	—	зарезервировано
0x0002	ntor	используется
0x0003	ntor-v3	используется; предпочтителен

Если согласование ключей прошло успешно, реле отвечает **CREATED2**; при ошибке — **DESTROY**. CircID новой ячейки следует правилам разд. 8.4: начиная с версии 4 протокола соединения сторона, открывшая канал, использует значения со старшим битом 1.

9.2.2. EXTEND2 и EXTENDED2

Сообщение RELAY EXTEND2 (команда RELAY 14) помимо полей **CREATE2** содержит список *спецификаторов соединения (link specifiers)*, сообщающих реле, как достичь следующего перехода и как его идентифицировать [tor-spec § Extend and Extended messages]:

Тип	Содержимое	Длина
00	IPv4-адрес и ORPort	6
01	IPv6-адрес и ORPort	18
02	устаревшая RSA-идентичность (SHA-1, 20 байт)	20
03	идентичность Ed25519	32

Реле, расширяющее цепь, должно проверить *каждую* указанную идентичность и не должно расширять цепь, если достигнутое реле не докажет владение ими во время установления канала из гл. 7. Так клиент уверен, что сообщение согласования ключей дошло именно до выбранного реле, хотя напрямую с R_2 не общается. Реле также отклоняет запрос на расширение цепи к самому себе или к реле, от которого запрос поступил.

Две детали показывают, насколько внимательно протокол относится к *неразличимости*. Во-первых, спецификация рекомендует всегда отправлять спецификаторы в одном порядке (00, 02, 03, 01), чтобы порядок не выдавал реализацию. Во-вторых, EXTEND2 всегда передаётся внутри **RELAY_EARLY** (разд. 8.5); ограниченное число таких ячеек не позволяет строить произвольно длинные цепи.

EXTENDED2 (команда RELAY 15) имеет в точности то же тело, что **CREATED2**.

9.3. Что должно обеспечивать согласование ключей

Перед рассмотрением разных протоколов согласования ключей определим ожидаемые свойства. По завершении клиент и реле должны иметь общий секрет со следующими свойствами:

Аутентификация реле Клиент должен знать, что делит секрет именно с *конкретным* реле и никем больше. Реле, напротив, не должно узнавать ничего о клиенте: согласование ключей аутентифицировано *только в одном направлении*; иначе каждое реле узнавало бы идентичность клиента.

Секретность Сетевой наблюдатель или предыдущее реле цепи, пересылающее сообщения, не должны уметь вывести секрет.

Прямая секретность Даже тот, кто в будущем получит долговременные ключи реле, не должен иметь возможности восстановить прошлые сеансовые секреты.

Эффективность Нагруженное реле выполняет тысячи согласований ключей в секунду, поэтому вычислительная стоимость имеет значение.

Каждый рассматриваемый протокол согласования ключей по-своему балансирует эти требования.

9.4. TAP: первый протокол согласования ключей

Исходный протокол согласования ключей Tor TAP (*Tor Authentication Protocol*) сочетал обычный обмен ключами Диффи–Хеллмана в 1024-битной модульной группе с шифрованием открытым 1024-битным RSA-ключом. Клиент шифровал свою половину Диффи–Хеллмана RSA «onion»-ключом реле; расшифровать её и ответить могло только выбранное реле. Затем ключи вырабатывались функцией KDF-TOR на основе SHA-1 (разд. 9.8).

TAP удовлетворял требованиям, но с постоянно уменьшающимся запасом безопасности: 1024 бит недостаточно и для RSA, и для модульного Диффи–Хеллмана, гибридная схема шифрования считалась хрупкой, а согласования ключей были вычислительно дорогими. С 2013 года TAP дополнил ntor, а начиная с tor 0.4.9.1-alpha реле полностью перестали его поддерживать (Предложение 350). Один след остаётся: ради совместимости реле всё ещё публикуют старый RSA «onion»-ключ в дескрипторах. В лабораторной сети он виден в микродескрипторах как `onion-key`, рядом с реально используемым `ntor-onion-key`.

9.5. CREATE_FAST: когда открытый ключ не нужен

Есть один случай, когда согласование ключей с открытым ключом не нужно: *первый* переход. Клиент только что открыл TLS-канал к этому реле и проверил его идентичность, поэтому TLS уже даёт зашифрованный аутентифицированный канал. **`CREATE_FAST`** использует это обстоятельство [tor-spec § CREATE_FAST/CREATED_FAST cells]:

1. Клиент отправляет 20 случайных байт X.
2. Реле отвечает 20 случайными байтами Y и 20 байтами KH.
3. Обе стороны получают ключевой материал функцией KDF-TOR из $K_0 = X \| Y$. Первые 20 байт должны совпасть с KH, доказывая, что реле получило X.

Безопасность **`CREATE_FAST`** целиком зависит от TLS-канала: тот, кто способен прочитать TLS-трафик, увидит и X, и Y. Поэтому его следует использовать только для одношаговых цепей. Типичный случай — запускающийся клиент, которому нужно загрузить каталожную информацию с реле, onion key которого он ещё не знает. Исходный код tor говорит об этом явно: `should_use_create_fast_for_circuit` (в `src/core/or/circuitbuild.c`) выбирает **`CREATE_FAST`** только при отсутствии пригодного onion key первого перехода.

Внимание: CREATE_FAST раскрывает первый переход

Реле, получившее **`CREATE_FAST`**, достоверно знает, что является первым переходом цепи и, следовательно, что сторона на другом конце канала — клиент. Спецификация предлагает реле использовать именно этот признак, чтобы отклонять одношаговые цепи, пытающиеся выйти в Интернет.

9.6. ntor

Протокол ntor, предложенный Голдбергом, Стебилой и Устаоглу [GSU13] и введённый в Tor с Предложением 216 (tor 0.2.4.8-alpha, 2013), заменяет RSA и модульный Диффи–Хеллман на **Curve25519**, быстрый обмен ключами на эллиптической кривой с ключами всего по 32 байта (гл. 5).

9.6.1. Компоненты

Реле имеет ключ onion среднего срока действия на Curve25519: секретный b и открытый $B = bG$, опубликованный в дескрипторе как `ntor-onion-key`. Клиент знает B и идентичность реле (здесь 20-байтовый RSA-отпечаток `ID`). Через $\mathrm{EXP}(P, k)$ обозначим обмен Диффи–Хеллмана Curve25519 между точкой P и скаляром k, а через $H(x, t)$ — HMAC-SHA256 сообщения x с ключом t. Ключи t — постоянные строки, полученные из идентификатора протокола `"ntor-curve25519-sha256-1"`.

9.6.2. Протокол

1. Клиент генерирует эфемерную пару (x, X) и отправляет в **`CREATE2`**: `ID` (20 байт), B (32 байта, определяющие использованный ключ onion) и X (32 байта). Всего 84 байта.

2. Реле проверяет `ID` и B, генерирует эфемерную пару (y, Y) и вычисляет

$$\mathit{secret_input} = \mathrm{EXP}(X, y) \,\|\, \mathrm{EXP}(X, b) \,\|\, \texttt{ID} \,\|\, B \,\|\, X \,\|\, Y \,\|\, \texttt{PROTOID}.$$

   Из этого выводится $\mathit{KEY_SEED} = H(\mathit{secret_input}, t_{key})$ и подтверждающее значение $AUTH$. Ответ содержит Y (32 байта) и $AUTH$ (32 байта).

3. Клиент вычисляет тот же *secret_input* со своими секретами: $\mathrm{EXP}(Y, x) \,\|\, \mathrm{EXP}(B, x) \,\|\, \ldots$ — и проверяет совпадение $AUTH$.

В основе согласования ключей лежат два обмена Диффи–Хеллмана:

- $\mathrm{EXP}(X, y) = \mathrm{EXP}(Y, x)$ сочетает два **эфемерных** ключа и обеспечивает прямую секретность, поскольку x и y удаляются сразу после согласования ключей.

- $\mathrm{EXP}(X, b) = \mathrm{EXP}(B, x)$ включает ключ onion реле и обеспечивает **аутентификацию**: только сторона, знающая b, может вычислить значение и создать правильный $AUTH$.

Ни одного из этих обменов недостаточно по отдельности. Без первого тот, кто похитит b в будущем, сможет восстановить всё; без второго посредник сможет выдать себя за реле. Клиент при этом не использует долговременный ключ: реле знает лишь, что делит секрет с «тем, кто отправил X».

Подробнее: Проверка точки на бесконечности

Спецификация требует от обеих сторон проверять, что ни один обмен Диффи–Хеллмана не даёт «точку на бесконечности», представленную здесь полностью нулевым результатом. Противник мог бы отправить точку малого порядка как Y, заставив $\mathrm{EXP}(Y, x)$ принять предсказуемое значение. Для Curve25519 этой простой проверки достаточно вместо полной проверки принадлежности Y правильной группе. В лабораторной работе 3 это три строки функции `x25519_dh`.

9.7. ntor-v3

У ntor есть ограничение: кроме ключей он ничего не переносит. Со временем появилась необходимость *согласовывать* параметры при установлении перехода, например управление перегрузкой (гл. 11) или CGO (гл. 10). ntor-v3 (Предложение 332) расширяет ntor **зашифрованными сообщениями** в обоих направлениях и одновременно меняет ряд криптографических решений.

9.7.1. Что меняется

- Идентичность реле теперь — его **Ed25519**-идентичность (32 байта), а не RSA-отпечаток.
- Хеш-функции относятся к семейству SHA-3: SHA3-256 для хеширования и MAC, SHAKE-256 для выработки ключей переменной длины. Перед каждым входом добавляется доменная метка с кодированной длиной (функция ENCAP), поэтому значение для одной цели нельзя принять за значение для другой.
- Строка проверки VER, известная обеим сторонам, участвует в каждом вычислении: согласование ключей из одного контекста нельзя повторно использовать в другом. Для обычных цепей tor применяет `"circuit extend"` (константа `NTOR3_CIRC_VERIFICATION` в `src/core/crypto/onion_crypto.c`).
- Клиент может передать сообщение CM, а реле — сообщение SM; оба зашифрованы.

9.7.2. Протокол

1. Клиент генерирует (x, X) и сразу вычисляет $B_x = \mathrm{EXP}(B, x)$. Из B_x и публичных параметров он получает два ключа «первой фазы»: один для шифрования CM посредством AES-256-CTR, второй для его аутентификации MAC. Отправляются ID (32), B (32), X (32), зашифрованный CM и MAC (32).
2. Реле вычисляет $\mathrm{EXP}(X, b) = B_x$, получает те же ключи первой фазы, проверяет MAC и расшифровывает CM. Затем генерирует (y, Y) и объединяет оба обмена, $\mathrm{EXP}(X, y)$ и $\mathrm{EXP}(X, b)$, в *key_seed*. С помощью SHAKE-256 выводятся ключ для шифрования ответа SM и поток ключей цепи. Реле отвечает Y (32), $AUTH$ (32) и зашифрованным SM.
3. Клиент проверяет $AUTH$, расшифровывает SM и получает тот же поток ключей.

Внимание: Сообщение клиента не имеет прямой секретности

CM шифруется ключами, зависящими только от B_x, который выводится с использованием ключа onion реле. Тот, кто запишет обмен сообщениями и позже получит b, сможет расшифровать CM. Спецификация говорит об этом прямо: прямой секретностью обладают только данные реле. Поэтому CM должен содержать только параметры согласования, но никогда не пользовательские данные.

9.7.3. Расширения

Сообщения CM и SM используют общий формат [tor-spec § Sending extensions in the circuit extension handshake]: один байт задаёт число расширений, затем для каждого идут тип (1 байт), длина (1 байт) и данные. Для создания цепи определены:

Тип	Направление	Значение
1	клиент → реле	`CC_FIELD_REQUEST`: клиент запрашивает управление перегрузкой из Предложения 324; пустое тело
2	реле → клиент	`CC_FIELD_RESPONSE`: реле принимает; 1 байт, `sendme_inc`
3	клиент → реле	запрос субпротокола, например `[02 06]` для CGO (`Relay=6`)

9.7.4. Какой протокол согласования ключей использует tor сегодня?

В tor 0.4.9.13 тип согласования выбирается отдельно для каждого перехода (`circuit_pick_create_handshake`). ntor-v3 используется, когда реле поддерживает управление перегрузкой и эта возможность включена, как обычно происходит в современной сети, либо когда требуется согласовать CGO. Иначе tor возвращается к ntor. На практике, как отмечает спецификация, современные клиенты почти всегда используют ntor-v3, а ntor сохраняется как вариант совместимости.

9.8. От согласованных секретов к ключам цепи

Каждое согласование ключей создаёт поток псевдослучайных байтов, из которого извлекаются ключи традиционной схемы шифрования ячеек RELAY, называемой «tor1» [tor-spec § Setting circuit keys]:

Ключ	Байты	Назначение	Направление
Df	20	начальное значение накопительного дайджеста (SHA-1)	клиент → реле
Db	20	начальное значение накопительного дайджеста (SHA-1)	реле → клиент
Kf	16	ключ AES-128-CTR	клиент → реле
Kb	16	ключ AES-128-CTR	реле → клиент

Всего 72 байта (константа `CPATH_KEY_MATERIAL_LEN` в коде tor). Способ формирования потока зависит от типа согласования ключей:

KDF-TOR (CREATE_FAST и ранее TAP) $K = \mathrm{SHA1}(K_0 \| \texttt{00}) \,\|\, \mathrm{SHA1}(K_0 \| \texttt{01}) \,\|\, \ldots$. Первые 20 байт образуют KH, затем идут Df, Db, Kf и Kb. Однобайтовый счётчик ограничивает выход 5120 байтами, что значительно больше необходимого.

KDF-RFC5869 (ntor) использует HKDF с SHA-256 и `KEY_SEED` как ключом; Df, Db, Kf и Kb занимают первые 72 байта.

SHAKE-256 (ntor-v3) использует первые 32 байта для шифрования SM; далее идут Df, Db, Kf и Kb.

Разные ключи в двух направлениях не избыточны. Если бы клиент и реле применяли один AES-CTR ключ в обе стороны, зашифрованные потоки повторно использовали бы одну и ту же гамму шифрования, и XOR двух ячеек раскрыл бы XOR двух открытых текстов.

CGO меняет схему ключей; это рассматривается в гл. 10.

9.9. Лабораторная работа: построение реальных цепей

Лабораторная работа 3: Согласование ключей цепи

Цель. Написать `labs/ru/код/torlab/ntor.py` с CREATE_FAST, ntor и ntor-v3; проверить по официальным тестовым векторам; строить одношаговые цепи к реле лабораторной сети.
Предварительные условия. Лабораторные работы 1 и 2; работающая лабораторная сеть; библиотека `cryptography` (X25519, AES) и стандартный модуль `hashlib` (SHA-3, SHAKE).
Оценочное время. 3–4 часа.

Шаг 1: ntor-v3 в нескольких строках. Реализация почти буквально следует спецификации. Сначала вспомогательные функции:

```
NTOR3_PROTOID = b"ntor3-curve25519-sha3_256-1"
def _t(name):  return NTOR3_PROTOID + b":" + name.encode()
def _encap(s): return struct.pack(">Q", len(s)) + s
def _h3(s, t): return hashlib.sha3_256(_encap(t) + s).digest()
def _mac3(k, msg, t):
    return hashlib.sha3_256(_encap(t) + _encap(k) + msg).digest()
def _kdf3(s, t, n): return hashlib.shake_256(_encap(t) + s).digest(n)
```

Затем клиентская часть протокола согласования ключей:

```
def ntor3_client_start(ID, B, message, ver=b"circuit extend", x=None):
    sk, X = x25519_generate(x)
    Bx = x25519_dh(sk, B)
    phase1 = _kdf3(Bx + ID + X + B + NTOR3_PROTOID + _encap(ver),
                   _t("kdf_phase1"), 64)
    enc_k1, mac_k1 = phase1[:32], phase1[32:]
    encrypted = aes_ctr(enc_k1, message)
    mac = _mac3(mac_k1, ID + B + X + encrypted, _t("msg_mac"))
    state = Ntor3State(sk, X, B, ID, Bx, mac, ver)
    return state, ID + B + X + encrypted + mac

def ntor3_client_finish(st, response, length=72):
    Y, auth, msg = response[:32], response[32:64], response[64:]
    secret_input = (x25519_dh(st.x, Y) + st.Bx + st.ID + st.B + st.X + Y
                    + NTOR3_PROTOID + _encap(st.ver))
    key_seed = _h3(secret_input, _t("key_seed"))
    verify = _h3(secret_input, _t("verify"))
    auth_input = (verify + st.ID + st.B + Y + st.X + st.mac
                  + _encap(msg) + NTOR3_PROTOID + b"Server")
    if not hmac.compare_digest(_h3(auth_input, _t("auth_final")), auth):
        raise HandshakeError("неверный ntor-v3 AUTH")
    stream = _kdf3(key_seed, _t("kdf_final"), 32 + length)
    return aes_ctr(stream[:32], msg), stream[32:]
```

Сравнение *AUTH* выполняется `hmac.compare_digest`, который занимает одинаковое время независимо от позиции первого несовпавшего байта. Обычное сравнение могло бы по времени раскрывать число совпавших начальных байтов.

Шаг 2: тестовые векторы. Криптографический протокол согласования ключей нельзя проверять лишь тем, что он «работает»: две реализации с одной и той же ошибкой прекрасно взаимодействовали бы между собой. Поэтому используются **тестовые векторы** спецификации: фиксированные входы, включая якобы «случайные» ключи, и ожидаемые побайтовые

выходы. Предложение 332 содержит полный вектор ntor-v3, а Предложение 216 — несколько векторов HKDF ntor. Файл `labs/ru/код/torlab/test/test_ntor.py` воспроизводит их; например:

```
def test_ntor3_vectors_proposal_332():
    st, hdata = ntor3_client_start(h(V["ID"]), h(V["B"]), h(V["CM"]),
                                   ver=h(V["VER"]), x=h(V["x"]))
    assert hdata.hex() == V["client"]
    sm, stream = ntor3_client_finish(st, h(V["server"]), length=256)
    assert sm == h(V["SM"])
    assert stream[:32].hex() == V["keystream_start"]
```

Другие тесты проверяют, что изменённый *AUTH* или другая строка `VER` приводят к ошибке согласования ключей.

Шаг 3: ключи реле. Для согласования ключей нужны Ed25519-идентичность реле и `ntor-onion-key`. Настоящий клиент получает их из консенсуса и микродескрипторов (гл. 12); мы берём их из тех же источников, используя файл `cached-microdescs.new` лабораторного клиента, где для каждого реле есть запись:

```
onion-key
-----BEGIN RSA PUBLIC KEY-----
MIGJAoGBAJet1Oxp+cvg/vgryEmJuVv2s++WxQt5Id1BQ9UNwCOVFcYvEN9KVklM
...
-----END RSA PUBLIC KEY-----
ntor-onion-key gsi2OisnQE96UkjOVvBKtWDLfwRdYG9d4DScg2wz4Qs
id ed25519 Kz1dj/u6/g7WvhHrt9Ds0jVZjKC+KHo26hWViXAJ1Pg
```

Обратите внимание на старый TAP-ключ, всё ещё публикуемый, но уже не используемый, и две base64-строки, которые нам действительно нужны.

Шаг 4: против настоящего реле. Открываем канал к `test003r` с помощью лабораторной работы 1 и отправляем **`CREATE2`** для каждого типа согласования ключей:

```
c = open_channel("127.0.0.1", 5103, expected_identity=ed)
st, hd = ntor3_client_start(ed, B, encode_extensions([]))
c.send(Cell(0x80000001, CellCommand.CREATE2,
              create2_body(HTYPE_NTOR_V3, hd)))
r = c.receive()
sm, stream = ntor3_client_finish(st, hdata_created2(r.body))
keys = CircuitKeys.from_stream(stream)
```

Повторив опыт с другими вариантами, получаем:

```
<- CircID=0x80000001 CREATED2 тело=509 байт
   ntor-v3 ok, сообщение реле: []
<- CircID=0x80000002 CREATED2 тело=509 байт
   ntor-v3 с CC_FIELD_REQUEST ok, ответ: [(2, b'\x1f')]
<- CircID=0x80000003 CREATED2 тело=509 байт
   ntor ok
<- CircID=0x80000004 CREATED_FAST тело=509 байт
   CREATE_FAST ok
<- CircID=0x80000005 DESTROY тело=509 байт     (изменён MAC)
   причина: 1
```

Все неизменённые протоколы согласования ключей завершаются успешно. Во втором случае реле принимает управление перегрузкой и отвечает расширением 2 с `sendme_inc`, равным

31 (гл. 11). В последнем случае мы намеренно меняем один бит MAC: реле отклоняет согласование ключей ячейкой **DESTROY** с причиной 1, PROTOCOL. Здесь реальную причину можно сообщать, потому что ошибку породило это реле; оно не пересылает чужую **DESTROY** (разд. 8.5).

Неожиданное наблюдение. В гл. 7 мы увидели, что новый канал не получает **PADDING**. Теперь по этому каналу открыты четыре цепи, но за двадцать секунд ожидания по-прежнему не приходит ни одной ячейки. Причина находится в коде tor (`src/core/or/relay.c`): реле считает канал «используемым», а значит пригодным для заполнения (padding), только если по нему идёт цепь, продолжающаяся за пределы этого реле. Наши одношаговые цепи не учитываются. В гл. 10, после расширения цепи появится заполнение.

Итоги

- Цепь строится по одному переходу: **CREATE2**/**CREATED2** с первым реле, затем EXTEND2/EXTENDED2 через цепь для последующих. Каждое согласование ключей связывает клиента только с одним реле.
- EXTEND2 содержит спецификаторы соединения (адрес, RSA- и Ed25519-идентичности); расширяющее реле обязано проверять эти идентичности.
- Протокол согласования ключей цепи аутентифицирует реле, но не клиента, и должен обеспечивать секретность и прямую секретность.
- TAP (1024-битные RSA и DH) удалён в tor 0.4.9.1-alpha. **CREATE_FAST** зависит от TLS и должен использоваться только для одношаговых цепей.
- ntor сочетает два обмена Curve25519: эфемерный ключ с эфемерным для прямой секретности и эфемерный ключ с ключом onion для аутентификации.
- ntor-v3 использует Ed25519-идентичность и семейство SHA-3, добавляя зашифрованные сообщения для согласования параметров вроде управления перегрузкой и CGO; сообщение клиента не имеет прямой секретности.
- Поток ключей даёт Df и Db (по 20 байт) и Kf и Kb (по 16 байт): ключ и дайджест для каждого направления.

Упражнения

Упражнение 9.1 ⋆ Вычислите длину HDATA в **CREATE2** для ntor и для ntor-v3 с трёхбайтовым сообщением. Каков максимальный размер сообщения CM ntor-v3 внутри **CREATE2**?

Упражнение 9.2 ⋆ Почему **CREATE_FAST** нельзя использовать для второго перехода цепи? Что могло бы сделать первое реле?

Упражнение 9.3 ⋆ Почему клиент и реле используют разные AES-ключи в двух направлениях (Kf и Kb)? Опишите, что произошло бы при использовании одного ключа.

Упражнение 9.4 ⋆⋆ Запишите в шестнадцатеричном виде заголовок тела сообщения EXTEND2 — число спецификаторов и спецификатор IPv4 — для расширения цепи до лабораторного реле `test004r`. Сколько байт занимает полное сообщение с ntor-v3 и однобайтовым CM?

Упражнение 9.5 ⋆⋆ Предположим, в ntor клиент использует только $\mathrm{EXP}(Y, x)$ и опускает $\mathrm{EXP}(B, x)$. Опишите конкретную атаку промежуточного реле, пересылающего согласование

ключей следующему переходу. Теперь предположим, что клиент вместо этого опускает $\mathrm{EXP}(Y, x)$: какое свойство теряется?

Упражнение 9.6 ⋆⋆ Каково назначение строки `VER` в ntor-v3? Представьте, что другой протокол, не Tor, использует то же согласование ключей и те же ключи: что могло бы произойти без `VER`?

Упражнение 9.7 ⋆⋆ Повторите шаг 4 лабораторной работы с запросом субпротокола `[02 06]` (CGO) вместе с `CC_FIELD_REQUEST`. Принимает ли его реле? С помощью спецификации Предложение 359 объясните, почему CGO также требует управления перегрузкой.

Упражнение 9.8 ⋆⋆⋆ Репозиторий tor содержит эталонную реализацию ntor на Python (`src/test/ntor_ref.py`). Используйте её для создания полных тестовых векторов протокола ntor для клиента и реле и добавьте их в `test_ntor.py`.

Упражнение 9.9 ⋆⋆⋆ Реализуйте *сторону реле* ntor-v3 (функции `server_part1` и `server_part2`) и проверьте её по векторам Предложения 332. Затем выполните полный обмен сообщениями между своим клиентом и реле в памяти.

Дополнительная литература

- *Tor Protocol Specification*, главы «Creating and extending circuits» и «Setting circuit keys» [Thee].
- I. Goldberg, D. Stebila, B. Ustaoglu, *Anonymity and one-way authentication in key exchange protocols* [GSU13]: статья, определяющая ntor и доказывающая его безопасность.
- Предложения 216 (ntor), 332 (ntor-v3) и 350 (удаление TAP).

Глава 10

«Луковица»: шифрование ячеек RELAY и CGO

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; предложение 359

Цели главы

К концу этой главы вы сможете:

- описывать, как клиент оборачивает сообщение одним слоем шифрования для каждого перехода и как реле снимают эти слои;
- объяснять роль Recognized и накопительного дайджеста и почему нужны оба механизма;
- определять слабости традиционной схемы: пластичность и атаки *маркирования (tagging)*;
- описывать базовый принцип Counter Galois Onion (CGO), схемы, которая её заменяет;
- с помощью собственного кода расширять реальную цепь до трёх переходов (лабораторная работа 4).

В гл. 9 клиент согласовал с каждым реле цепи четыре ключа: Kf и Kb для шифрования, Df и Db для проверки целостности. Теперь задействуем их. Именно этот механизм дал onion routing его название: сообщение оборачивается в столько слоёв, сколько реле входит в маршрут, а каждое реле снимает один слой, словно очищая луковицу.

10.1. Традиционная схема

Схема, используемая с первых версий Tor и называемая в новом коде *tor1*, сочетает для каждого перехода и каждого направления два средства:

- потоковый шифр — **AES-128 в режиме счётчика** (CTR), с ключом Kf или Kb;
- **накопительный дайджест SHA-1**, инициализированный Df или Db.

Оба механизма сохраняют *состояние* в течение всей жизни цепи. Счётчик AES-CTR не начинается с нуля для каждой ячейки: первая использует первые 509 байт гаммы шифрования, вторая — следующие 509 и так далее. Аналогично дайджест вычисляется не для одной ячейки, а для *всех* ячеек, которыми до этого обменивались с данным переходом. Поэтому клиент и реле обязаны обрабатывать ячейки строго в одном порядке: потеря или дублирование ячейки рассинхронизирует оба состояния. TLS, гарантирующий порядок и целостность на канале, делает это возможным.

10.1.1. Прямое направление: от клиента внутрь цепи

Чтобы отправить сообщение переходу d, клиент [tor-spec § Routing relay cells]:

1. готовит открытое тело сообщения RELAY (разд. 8.6) с Recognized и Digest, равными нулю;
2. обновляет дайджест прямого направления перехода d этим телом и записывает первые четыре байта результата в поле Digest;
3. шифрует тело ключом Kf перехода d, затем Kf перехода $d-1$ и так далее до перехода 1: внешний слой принадлежит первому реле.

Каждое реле, получив ячейку, расшифровывает её своим Kf, снимая один слой, и проверяет, предназначена ли ячейка ему. Если нет, оно пересылает ячейку дальше, где следующий переход снимает ещё один слой. Слои показаны на рис. 10.1.

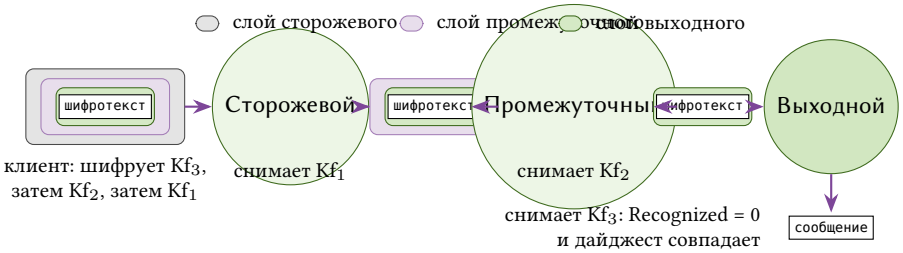


Рис. 10.1: Шифрование RELAY по традиционной схеме tor1: клиент применяет Kf_3, Kf_2, Kf_1, а каждый ретранслятор снимает свой слой. Выходной узел распознаёт сообщение при Recognized = 0 и правильном накопленном дайджесте; данные приложения могут оставаться зашифрованными HTTPS. Вложенные рамки условно изображают шифротекст.

10.1.2. Обратное направление: от цепи к клиенту

В обратном направлении роли меняются. Реле, порождающее сообщение, снабжает его дайджестом обратного направления и шифрует ключом Kb. Каждое следующее реле на пути к клиенту *добавляет* ещё один слой с собственным Kb. Клиент, знающий все ключи, снимает слои по одному, начиная с первого перехода, и после каждого слоя проверяет, не этим ли переходом была создана ячейка.

10.1.3. Цепь как «труба с промежуточными выходами (leaky pipe)»

Клиент может отправить сообщение *любому* переходу цепи, а не только последнему, просто остановив шифрование на нужном переходе. Именно так сообщение `EXTEND2` для третьего перехода доставляется второму реле. Спецификация называет это *leaky pipe topology*: цепь подобна трубе, из которой сообщения могут выйти до конца. Исходная статья Tor [DMS04] предлагала использовать это свойство и как защиту: отправка некоторых потоков через промежуточное реле затруднила бы наблюдателю на выходе определение соответствующей цепи. На практике tor выводит потоки только через последний переход.

10.2. Распознавание ячейки

Как реле после снятия своего слоя узнаёт, адресована ли ячейка ему? Нельзя просто «проверить, имеет ли содержимое смысл»: для программы и всё ещё зашифрованная, и

открытая ячейка являются последовательностями байтов. Традиционная схема выполняет две проверки подряд [tor-spec § Relay cells]:

1. **Recognized**: эти два байта должны быть нулевыми. Если ячейка всё ещё зашифрована, байты псевдослучайны и равны нулю лишь один раз на $65\,536$. Проверка почти бесплатна и сразу отбрасывает подавляющее большинство транзитных ячеек.

2. **Digest**: если Recognized равен нулю, реле вычисляет накопительный дайджест *так, как если бы* ячейка принадлежала ему, при этом поле Digest обнуляется, и сравнивает первые четыре байта с содержимым поля. Если они совпадают, ячейка «распознана»: реле обновляет состояние дайджеста и обрабатывает ячейку.

Деталь, которую легко упустить: если Recognized равен нулю, но дайджест не совпадает, это ложное срабатывание, и состояние дайджеста **не должно** обновляться. В лабораторной работе 4 мы вычисляем на копии состояния (`self.digest.copy()`) и сохраняем её только при успехе.

Дайджест охватывает всё тело, включая байты заполнения, которые поэтому являются случайными (разд. 8.6). Для цепей onion-сервисов, где клиент и сервис делят ключи «виртуального перехода» (гл. 14), вместо SHA-1 используется SHA3-256.

Что происходит, если ячейку никто не распознал? Если последнее реле получает прямую ячейку и не распознаёт её, произошла ошибка: оно уничтожает цепь посредством **`DESTROY`**. Если ни один переход не распознаёт входящую к клиенту ячейку, клиент делает то же самое.

10.3. Слабости tor1

10.3.1. Пластичность

У режима счётчика есть неудобное свойство: шифротекст равен $C = P \oplus S$, где S — гамма шифрования. Изменение одного бита C меняет тот же бит P, не требуя знания ключа. Если противник знает или может угадать часть открытого текста, он способен заменить её выбранным текстом: достаточно XOR-нуть шифротекст с разностью двух текстов. В лабораторной работе 4 есть тест, который делает именно это, меняя «заплати 10 евро» на «заплати 99 евро».

В tor1 целостность проверяется только *в пункте назначения*: целевое реле проверяет дайджест, а промежуточные реле этого не делают. Изменённая ячейка поэтому может беспрепятственно пройти до назначения, где дайджест перестанет совпадать.

10.3.2. Атака маркирования (tagging)

Вместе эти свойства допускают специфическую атаку *маркирования (tagging)*. Представим противника, контролирующего входное и выходное реле цепи:

1. Входное реле меняет несколько битов транзитной ячейки по узнаваемому шаблону — «метке», или *tag*.

2. Ячейка проходит через промежуточное реле, которое не может обнаружить изменение.

3. До проверки дайджеста выходное реле проверяет наличие метки. Если она есть, выходное реле узнаёт, что цепь проходит через сотрудничающий входной узел, и может связать пункт назначения трафика с адресом клиента, известным этому входному узлу.

Если выходное реле не участвует в сговоре, дайджест не совпадёт, цепь будет уничтожена, а клиент построит новую. Противник может повторить попытку, и каждая неудача что-то сообщает. Метку можно накладывать и в обратном направлении, от выходного реле к клиенту.

Подробнее: Насколько важно маркирование?

Противник, контролирующий одновременно входное и выходное реле, уже может коррелировать временные характеристики и объёмы трафика без метки (гл. 26). Поэтому разработчики Tor долго считали маркирование реальной, но менее приоритетной проблемой [DMS04]. Однако маркирование надёжнее статистической корреляции и работает даже при очень малом объёме трафика. Предложения по его устранению появлялись более десяти лет: предложения 202, 261 (на основе шифра AEZ), 295 и 308, а затем 359, приведшее к CGO.

10.3.3. Другие ограничения

Дайджест имеет длину лишь четыре байта, а лежащий в его основе SHA-1 больше не считается безопасным для новых применений. Кроме того, схема не обеспечивает прямую секретность *внутри* цепи: тот, кто получит ключи перехода, пока цепь активна, сможет расшифровать и все ранее записанные ячейки, поскольку гамма шифрования полностью определяется начальным ключом.

10.4. Counter Galois Onion

CGO (*Counter Galois Onion*) — схема шифрования ячеек RELAY, разработанная Дегабриэле, Меллони, Мюнхом и Стамом и принятая в Предложении 359. Здесь даётся функциональный обзор; полная криптографическая конструкция описана в предложении и работах авторов.

10.4.1. Свойства

Согласно предложению, CGO даёт три улучшения:

- схема **непластична**: любое изменение ячейки делает эту и все последующие ячейки цепи нерасшифровываемыми. Метку нельзя ни прочитать, ни удалить: она способна только разрушить цепь;
- обеспечивается **прямая секретность на уровне ячеек**: после обработки ячейки у стороны больше нет материала, необходимого для её повторного расшифрования;
- аутентификация имеет стойкость 128 бит вместо значительно более короткой проверки традиционной схемы.

Поскольку CGO больше не использует SHA-1, в целом она также должна быть быстрее.

10.4.2. Как это работает на высоком уровне

Базовые компоненты — AES-128 и POLYVAL, универсальная хеш-функция из того же семейства, которое используется режимом AES-GCM-SIV. На их основе предложение строит *широкоблочный настраиваемый шифр (wide-block tweakable cipher)*: все 509 байт ячейки обрабатываются как один блок, поэтому изменение хотя бы одного бита в любом месте непредсказуемо меняет весь результат. Расшифрованная ячейка состоит из *тега* (16 байт) и тела (493 байта), как мы видели в разд. 8.7.

Для каждого перехода и каждого направления клиент и реле разделяют:

- 64-байтовый ключ K и 16-байтовое значение N, полученные при согласовании ключей цепи;

- *тег* T' последней обработанной ячейки, изначально нулевой.

Три идеи делают схему устойчивой к маркированию:

Распознавание по тегу После снятия реле своего слоя ячейка принадлежит ему, если полученный тег совпадает с N. Это 16-байтовая проверка вместо двух байт Recognized и четырёх байт дайджеста.

Связывание Тег предыдущей ячейки входит в обработку следующей как параметр шифра. Изменённая ячейка создаёт неправильный тег, который, в свою очередь, делает невозможным расшифрование всех последующих ячеек: цепь перестаёт работать.

Обновление ключей Каждый раз, когда переход распознаёт или создаёт ячейку, он заменяет K и N новыми значениями, выведенными из предыдущих. Старые значения стираются, обеспечивая прямую секретность на уровне ячеек.

Байт команды ячейки (**RELAY** или **RELAY_EARLY**) включён в вычисления как *связанные данные (associated data)*: реле не может превратить один тип в другой, не сделав ячейку нерасшифровываемой.

Внимание: Направления нельзя менять местами

Конструкция, лежащая в основе CGO, непластична в одном направлении и пластична в другом. Предложение тщательно выбирает, какое направление использовать клиенту и какое реле, и прямо предупреждает разработчиков не менять их местами. Это показывает, как решение, кажущееся деталью реализации, способно подорвать безопасность всей криптографической схемы.

10.4.3. CGO на практике

- **Ключи**: из потока согласования ключей каждое направление получает 64-байтовый K и 16-байтовый N вместо Df, Db, Kf и Kb.

- **SENDME**: аутентифицированное управление потоком из Предложения 289 использовало накопительный дайджест как доказательство получения данных. В CGO такого дайджеста нет, поэтому SENDME вместо него несёт тег последней подтверждённой ячейки (гл. 11).

- **Согласование**: клиент использует CGO с реле только если реле объявляет субпротокол `Relay=6`, а клиент запрашивает его при согласовании ключей ntor-v3 (разд. 9.7). В гл. 9 мы видели, что реле отклоняет запрос, если он не сопровождается запросом управления перегрузкой.

- **Развёртывание**: tor поддерживает CGO начиная с серии 0.4.9, а в Arti он всегда включён. Одна цепь может содержать переходы с CGO и переходы, всё ещё использующие tor1: выбор делается независимо для каждого перехода.

10.5. Лабораторная работа: трёхпереходная цепь

Лабораторная работа 4: Послойное шифрование и расширение цепи

Цель. Написать `labs/ru/код/torlab/encryption.py`, реализующий клиентскую сторону tor1, и использовать его для расширения реальной цепи до выходного реле.
Предварительные условия. Лабораторные работы 1–3; работающая лабораторная сеть.
Оценочное время. 3 часа.

Шаг 1: одно направление одного перехода. В центре лабораторной работы находится небольшой класс, объединяющий AES-CTR и дайджест SHA-1 одного направления:

```
class Direction:
    @classmethod
    def new(cls, key, digest_seed):
        d = hashlib.sha1(); d.update(digest_seed)
        return cls(_aes_ctr(key), d)

    def apply(self, data):             # шифрование и расшифрование: CTR выполняет одну операцию
        return self.cipher.update(data)

    def sign(self, body):                 # записать поле Digest
        self.digest.update(_without_digest(body))
        return body[:5] + self.digest.digest()[:4] + body[9:]

    def recognizes(self, body):
        if body[1:3] != b"\x00\x00":
            return False
        trial = self.digest.copy()
        trial.update(_without_digest(body))
        if trial.digest()[:4] != body[5:9]:
            return False                # ложное срабатывание: состояние не меняется
        self.digest = trial
        return True
```

Объект шифра из `cryptography` сохраняет счётчик между вызовами: именно такое поведение здесь требуется.

Шаг 2: клиент. Имея по одному объекту `Direction` на направление и переход, шифрование и расшифрование превращаются в циклы:

```
def encrypt(self, msg, destination):
    body = self.hops[destination].forward.sign(encode_relay(msg))
    for i in range(destination, -1, -1):
        body = self.hops[i].forward.apply(body)
    return body

def decrypt(self, body):
    for i, hop in enumerate(self.hops):
        body = hop.backward.apply(body)
        if hop.backward.recognizes(body):
            return i, decode_relay(body)
    raise CellError("ячейка не распознана")
```

Шаг 3: тесты в памяти и их ограничения. Файл `labs/ru/код/torlab/test/test_encryption.py` строит имитированную цепь из трёх реле. Он проверяет, что каждый переход распознаёт только свои ячейки, промежуточные реле видят лишь псевдослучайные байты, а изменение обнаруживается в пункте назначения; также демонстрируется пластичность AES-CTR. Но у таких тестов есть важное ограничение: имитированные клиент и реле используют *один и тот же код*. Если, например, мы ошибёмся в порядке обновления дайджеста, обе стороны допустят одинаковую ошибку, и тест всё равно пройдёт. Окончательная проверка — общение с независимой реализацией.

Шаг 4: расширить реальную цепь. Устанавливаем первый переход к `test003r` через ntor-v3, как в лабораторной работе 3, затем расширяемся до `test004r` и выходного узла `test006e`. Модуль `labs/ru/код/torlab/laboratory.py` читает идентичности и ключи ntor реле из файлов лабораторной сети:

```
for r, est in ((middle, []), (exit_, [(1, b"")])):
    st, hd = ntor3_client_start(r.id_ed, r.ntor, encode_extensions(est))
    body = extend2_body("127.0.0.1", r.port, r.id_rsa, r.id_ed,
                        HTYPE_NTOR_V3, hd)
    msg = RelayMessage(RelayCommand.EXTEND2, 0, body)
    c.send(Cell(CIRC, CellCommand.RELAY_EARLY, circ.crypto.encrypt(msg)))
    i, response = circ.crypto.decrypt(c.receive().body)
    sm, k = ntor3_client_finish(st, hdata_created2(response.data))
    circ.hops.append(Hop.from_keys(CircuitKeys.from_stream(k)))
```

Сообщение `EXTEND2` имеет StreamID 0, поскольку это управляющее сообщение, и передаётся в **`RELAY_EARLY`**. Для выходного реле мы также запрашиваем управление перегрузкой, как делает tor. Выполнение даёт:

```
переход 1: 003r
  <- CircID=0x80000100 RELAY тело=509 байт
  от перехода 1 EXTENDED2 67 байт
переход 2: 004r  расширения реле: []
  <- CircID=0x80000100 RELAY тело=509 байт
  от перехода 2 EXTENDED2 70 байт
переход 3: 006e  расширения реле: [(2, b'\x1f')]
```

Теперь цепь содержит три перехода. Каждая строка «от перехода» подтверждает, что наш код правильно зашифровал данные для нужного реле и что реле правильно зашифровало ответ. Ошибка хотя бы в одном ключе, байте дайджеста или порядке слоёв заставила бы реле уничтожить цепь. Длины `EXTENDED2` согласуются с разд. 9.7: два байта HLEN, 32 для Y, 32 для $AUTH$ и сообщение SM, равное одному байту без расширений для промежуточного реле и четырём байтам с одним однобайтовым расширением для выходного реле.

Шаг 5: наконец заполнение. Продолжаем слушать канал после расширения цепи:

```
  4.1 s  <- CircID=0x00000000 PADDING тело=509 байт
  7.1 s  <- CircID=0x00000000 PADDING тело=509 байт
 12.8 s  <- CircID=0x00000000 PADDING тело=509 байт
 21.5 s  <- CircID=0x00000000 PADDING тело=509 байт
 28.3 s  <- CircID=0x00000000 PADDING тело=509 байт
```

Именно это мы предсказали в главах 7 и 9: теперь, когда по каналу проходит многошаговая цепь, входное реле считает его «используемым» и при бездействии отправляет **`PADDING`** через случайные интервалы. Наблюдавшиеся интервалы от трёх до 8,7 секунды укладываются в предусмотренное спецификацией окно 1,5–9,5 секунды.

Итоги

- В tor1 каждый переход имеет поток AES-128-CTR и накопительный дайджест SHA-1 для каждого направления; оба сохраняют состояние всю жизнь цепи.
- В прямом направлении клиент накладывает слои от последнего перехода к первому, а каждое реле снимает один. В обратном направлении каждое реле добавляет слой, а клиент снимает все.
- Клиент может адресовать любой переход (*leaky pipe*).
- Ячейка распознаётся, когда Recognized равен нулю и накопительный дайджест совпадает; ложное срабатывание Recognized не должно менять состояние дайджеста.
- AES-CTR пластичен, а целостность проверяется только в пункте назначения: это делает возможным маркирование.
- CGO делает ячейки непластичными, связывает их теги, обновляет ключи после каждого сообщения и использует 128-битную аутентификацию; схема согласуется отдельно с каждым переходом посредством ntor-v3.

Упражнения

Упражнение 10.1 ⋆ Клиент с трёхпереходной цепью хочет отправить сообщение второму переходу. Какие ключи он использует для шифрования и в каком порядке? Что видит третий переход?

Упражнение 10.2 ⋆ После отправки клиентом 1000 ячеек по цепи в какой позиции своей гаммы шифрования AES-CTR находится первый переход? Почему тот же ключ нельзя использовать для второй цепи?

Упражнение 10.3 ⋆⋆ Объясните, что произошло бы, если реле обновляло накопительный дайджест даже тогда, когда Recognized равен нулю, но дайджест не совпадает. В среднем через сколько ячеек возникла бы проблема?

Упражнение 10.4 ⋆⋆ Сотрудничающее входное реле (guard) хочет пометить прямую ячейку так, чтобы сотрудничающее выходное реле её распознало. Какие байты ячейки было бы удобно изменить с учётом структуры открытого сообщения `DATA`? Что увидит честное выходное реле?

Упражнение 10.5 ⋆⋆ Повторите сценарий предыдущего упражнения с CGO. Что увидит сотрудничающее выходное реле? Что произойдёт с последующими ячейками цепи?

Упражнение 10.6 ⋆⋆ В шаге 5 лабораторной работы вычислите интервалы между наблюдавшимися ячейками **`PADDING`**. Согласно спецификации, задержка выбирается из распределения максимума двух равномерных случайных величин. Какова ожидаемая задержка на интервале от 1,5 до 9,5 секунды? Согласуется ли она с наблюдениями?

Упражнение 10.7 ⋆⋆⋆ Расширьте лабораторную работу, отправив сообщение `DROP` (команда RELAY 10, заполнение на всём протяжении цепи) второму переходу. Ответит ли реле? Как проверить, что оно распознало сообщение и не уничтожило цепь?

Упражнение 10.8 ⋆⋆⋆ В лаборатории намеренно инвертируйте один бит зашифрованной ячейки **`RELAY_EARLY`**, содержащей `EXTEND2`. Какое реле обнаружит изменение и как отреагирует? Сначала предскажите результат, затем проверьте его.

Дополнительная литература

- *Tor Protocol Specification*, главы «Relay cells» и «Routing relay cells» [Thee].
- Предложение 359, *Counter Galois Onion, Updated*, со ссылками на работы Дегабриэле, Меллони, Мюнха и Стама.
- Предложения 202, 261, 295 и 308: многолетняя история подходов к предотвращению маркирования (tagging).
- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], раздел 4 (цепи, leaky pipes и проверки целостности).

Глава 11

Потоки, SENDME и управление перегрузкой

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; предложения 289, 324

Цели главы

К концу этой главы вы сможете:

- открывать, использовать и закрывать TCP-поток через цепь с помощью BEGIN, CONNECTED, DATA и END;
- объяснять, почему разрешение имён выполняется на выходном реле и как работает RESOLVE;
- описывать исходное управление потоком Tor, включая окна цепи и потока и сообщения SENDME;
- понимать, почему SENDME аутентифицируются и что они доказывают;
- описывать управление перегрузкой из предложения 324 и роли XON и XOFF;
- загружать файл через собственноручно построенную трёхпереходную цепь и измерять окна реального выходного реле (лабораторная работа 5).

Цепь из гл. 10 — зашифрованный канал до выходного реле, но пока по нему не передаётся ничего полезного. Теперь задействуем его для TCP-соединений с Интернетом, которые Tor называет **потоками**. Затем рассмотрим проблему, которую должна решать каждая сеть и которая в Tor принимает особую форму: как не дать отправителю, передающему быстрее, чем сеть или получатель способны принимать, заполнить очереди реле.

11.1. Потоки

Поток — это TCP-соединение клиента, например с веб-сервером, переносимое через цепь. Одна цепь может одновременно нести множество потоков. Они различаются по **StreamID** в сообщениях RELAY (разд. 8.6), который выбирает клиент и который должен быть ненулевым. Жизненный цикл потока показан на рис. 11.1.

11.1.1. Открытие потока: BEGIN и CONNECTED

Чтобы открыть поток, клиент отправляет выходному реле сообщение `BEGIN` (команда RELAY 1) с новым StreamID. Тело задаёт пункт назначения [tor-spec § Opening streams and transmitting data]:

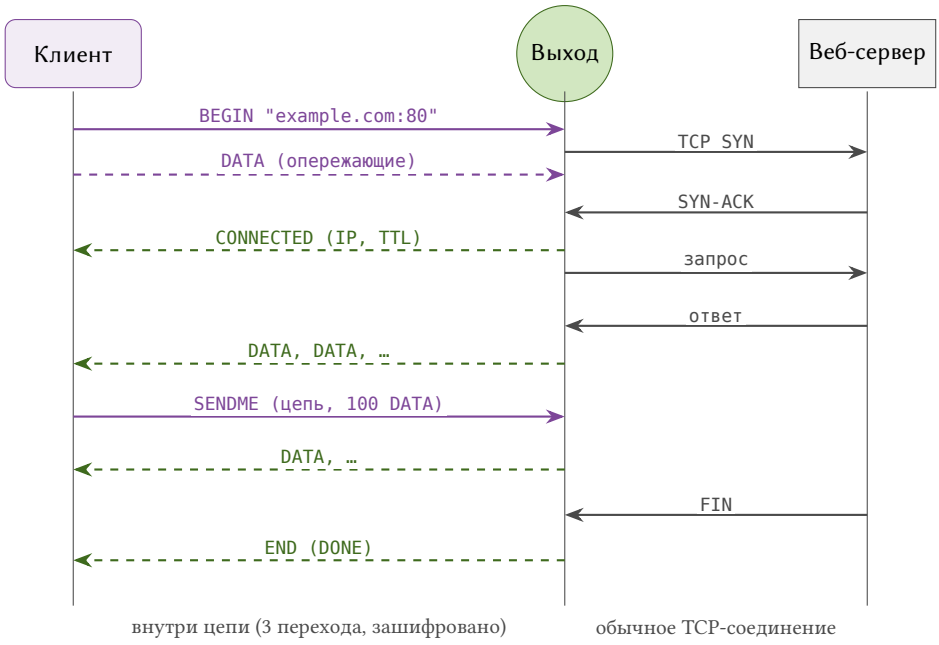


Рис. 11.1: Жизненный цикл потока к веб-серверу. Сообщения между клиентом и выходным реле идут через цепь под послойным шифрованием; между выходным реле и сервером находится обычное TCP-соединение, незашифрованное, если приложение не использует TLS. Клиент может отправить данные до получения CONNECTED (*опережающие данные (optimistic data)*).

Поле	Длина	Содержимое
ADDRPORT	переменная	`address:port`, завершённая нулевым байтом
FLAGS	4 (необязательно)	предпочтения IPv4 и IPv6

Адресом может быть DNS-имя, IPv4-адрес или IPv6-адрес в квадратных скобках. Два небольших правила показывают приоритеты дизайна: строка должна отправляться **в нижнем регистре**, чтобы различия в капитализации не помогали отличать клиентов, а FLAGS **должно быть опущено**, если равно нулю. Биты FLAGS указывают, принимает ли клиент IPv6 (бит 1), отказывается ли от IPv4 (бит 2) и предпочитает ли IPv6 (бит 3).

Получив `BEGIN`, выходное реле:

1. проверяет, что не является первым переходом цепи: канал, по которому пришла цепь, должен быть открыт аутентифицированным реле из консенсуса. Tor не должен работать как одношаговый прокси, что сделало бы выходные реле более привлекательными целями; такая попытка отклоняется **`DESTROY`**;

2. применяет свою *политику выхода (exit policy)*, список адресов и портов, к которым разрешены соединения (гл. 22);

3. при необходимости разрешает имя и открывает TCP-соединение.

Если всё успешно, реле отвечает `CONNECTED` (команда RELAY 4). Тело обычно содержит достигнутый IPv4-адрес (4 байта) и число секунд, в течение которых клиент может его кэшировать (TTL, 4 байта). Формат IPv6 немного отличается; разрешено и пустое тело. Клиент *может* игнорировать адрес; если он его кэширует, то не должен повторно использовать адрес в других цепях. Нечестное выходное реле может солгать или вернуть уникальный адрес, чтобы впоследствии узнавать клиента.

11.1.2. Передача данных: DATA

Прикладные данные передаются сообщениями `DATA` (команда RELAY 2), каждое до 498 байт в традиционном формате. Выходное реле копирует их содержимое в TCP-соединение и обратно. Клиент может отправлять `DATA` сразу после `BEGIN`, не ожидая `CONNECTED`: выходное реле ставит эти сообщения в очередь и пересылает после открытия соединения. Такие *опережающие данные (optimistic data)* экономят полный цикл прохождения туда и обратно по цепи, который в Tor легко может занимать несколько сотен миллисекунд.

11.1.3. Закрытие потока: END

Сторона, закрывающая соединение или обнаружившая ошибку, отправляет `END` (команда RELAY 3) с однобайтовой причиной [tor-spec § Closing streams]. Наиболее частые причины: `DONE` (6, обычное закрытие), `CONNECTREFUSED` (3), `RESOLVEFAILED` (2), `TIMEOUT` (7) и `EXITPOLICY` (4); после последней следуют отклонённый адрес и TTL. Для созданных им потоков клиент должен, как правило, отправлять только `MISC` (1): как и с **`DESTROY`**, нет причин сообщать другой стороне больше необходимого.

11.1.4. DNS на выходном реле

В `BEGIN` клиент может указать *имя*, а не адрес: разрешение происходит на выходном реле. Это не просто удобство, а решение безопасности. Если бы клиент разрешал имена сам, он обращался бы к собственному DNS-резолверу вне Tor для каждого сайта, который собирается посетить. Наблюдатель таких запросов, начиная с провайдера подключения, узнал бы пункты назначения. Это знакомая «утечка DNS (DNS leak)», демонстрируемая в упражнении 19.3 и наблюдаемая с точки зрения tor в лабораторной работе 18 (разд. 28.3).

Если клиенту нужен только адрес без открытия соединения, он использует `RESOLVE` (команда RELAY 11) с нуль-терминированным именем. Выходное реле отвечает `RESOLVED` (12), содержащим один или несколько ответов в формате тип, длина, значение и TTL: тип `0x04` для IPv4, `0x06` для IPv6, `0x00` для имени при обратном поиске и `0xF0`/`0xF1` для временной и постоянной ошибок [tor-spec § Remote hostname lookup]. На этом механизме основан DNS-порт, который клиент tor может предоставлять локальным приложениям (опция `DNSPort`).

11.1.5. Потоки каталогов: BEGIN_DIR

`BEGIN_DIR` (команда RELAY 13) открывает поток не в Интернет, а к собственному сервису каталогов реле. Так клиенты загружают консенсусы и дескрипторы (гл. 12), не открывая незашифрованных соединений. Тело пустое, а политика выхода не применяется, поскольку поток не выходит из процесса tor.

11.2. Исходное управление потоком

11.2.1. Проблема

Представим выходное реле с быстрым интернет-соединением и медленного клиента. Если выходное реле читает с сервера и без ограничений отправляет всё в цепь, данные накапливаются в очередях промежуточных реле, расходуя память и замедляя другие цепи. TCP решает эту проблему на каждой отдельной линии, но цепь — последовательность разных TCP-соединений: обратное давление TCP (backpressure) между промежуточным реле и клиентом не доходит до выходного реле. Нужен механизм *от конца до конца (end-to-end)*.

11.2.2. Окна и SENDME

Исходный механизм Tor использует **окна**, измеряемые в ячейках `DATA` [tor-spec § Flow control]:

- На уровне **цепи** отправитель имеет *окно отправки (package window)*, начинающееся с 1000 ячеек или значения параметра консенсуса `circwindow`. Каждая отправленная `DATA` уменьшает его; при нуле отправка останавливается. Получатель ведёт симметричное *окно доставки (deliver window)* и на каждые 100 полученных ячеек отправляет `SENDME` (команда RELAY 5) со StreamID 0, восстанавливая 100 ячеек в окне отправителя.

- На уровне **потока** окна начинаются с 500 ячеек и увеличиваются на 50 для каждого `SENDME` со StreamID этого потока.

Поскольку клиент может обмениваться данными с любым переходом (разд. 10.1), он поддерживает пару окон для каждого перехода. Эти окна считают только ячейки, созданные и потреблённые узлом, но не пересылаемые.

Следовательно, в любой момент в полёте может находиться не более 1000 ячеек на цепь и 500 на поток: примерно 500 KB и 250 KB. Это ограничивает использование памяти реле, но одновременно пропускную способность. Цепь с RTT в полсекунды не может превысить 1000 ячеек за полсекунды, то есть примерно 1 MB/s, независимо от доступной полосы. Если сеть перегружена, даже 1000 ячеек в полёте слишком много: очереди и задержка растут.

11.2.3. Аутентифицированные SENDME

Вредоносный клиент мог бы отправлять `SENDME`, ничего не получив. Выходное реле продолжало бы отправлять данные, которые накапливались бы в сети, позволяя клиенту исчерпывать память реле или усиливать трафик с малой собственной стоимостью. Начиная с версии 0.4.1.1-alpha сообщения `SENDME` уровня цепи **аутентифицируются** (Предложение 289). Их тело содержит версию (1), длину (20) и **накопительный дайджест** ячейки `DATA`, вызвавшей `SENDME`. Получатель SENDME знает этот digest, потому что вычислял его при отправке ячейки, и может сравнить значения. Несовпадение закрывает цепь.

Дайджест зависит от содержимого всех полученных ячеек, но клиент мог бы попытаться предсказать его, если заранее знает их содержимое, например при загрузке известного файла. Поэтому хотя бы одна ячейка в каждой группе из 100 должна содержать случайные байты: это одна из причин случайного заполнения сообщений RELAY (разд. 8.6).

Подробнее: Какой именно дайджест?

Спецификация говорит о «дайджест ячейки перед `SENDME`». В коде tor это полный 20-байтовый дайджест SHA-1 обратного потока после последней распознанной ячейки, сохранённый в поле `sendme_digest` функцией `tor1_relay_digest_matches_v0` (`src/core/crypto/relay_crypto_tor1.c`). В CGO вместо него используется тег последней ячейки (разд. 10.4).

11.3. Управление перегрузкой

Фиксированные окна имеют две противоположные слабости: они слишком малы для быстрых цепей и слишком велики для перегруженных. Предложение 324 Майка Перри ввела настоящее **управление перегрузкой** Tor, аналогичное TCP и принятое начиная с tor 0.4.7.7 (апрель 2022 года).

11.3.1. Сигнал: время туда и обратно (RTT)

Как выходное реле распознаёт перегрузку цепи? Используя SENDME как часы. Выходное реле записывает время отправки ячейки DATA, которая должна вызвать SENDME; после получения SENDME вычисляется **время туда и обратно** (RTT) цепи. Если RTT растёт относительно наблюдавшегося минимума, очереди вдоль цепи заполняются и отправителю следует замедлиться. Отправитель регулирует *окно перегрузки* (*cwnd*), стараясь удерживать его близко к произведению пропускной способности на задержку цепи: количеству данных, которое цепь способна переносить без создания очередей.

11.3.2. Алгоритмы

Предложение описывает три алгоритма, вдохновлённые TCP: Tor Westwood, Tor Vegas и Tor NOLA. После оценки в симуляторе Shadow (гл. 19) и в живой сети tor по умолчанию использует **Vegas** (константа CC_ALG_DFLT в src/core/or/congestion_control_common.c). Как TCP Vegas, он оценивает число «стоящих в очереди» ячеек, сравнивая текущее окно с окном, необходимым при минимальном наблюдавшемся RTT, затем увеличивает или уменьшает окно, удерживая оценку в заданных пределах.

11.3.3. Параметры: почему 31?

Управление перегрузкой согласуется отдельно для каждого перехода посредством ntor-v3 (разд. 9.7). Клиент запрашивает его расширением CC_FIELD_REQUEST, а реле отвечает CC_FIELD_RESPONSE с sendme_inc: числом ячеек DATA, после которого требуется SENDME. В гл. 9 выходное реле лабораторной сети вернул 31. Число не случайно: код tor определяет его как число ячеек, помещающихся в TLS-запись максимального размера,

$$\texttt{TLS_RECORD_MAX_CELLS} = \left\lfloor \frac{16384}{514} \right\rfloor = 31,$$

а начальное окно равно $4 \times 31 = 124$ ячейкам. Более частые SENDME, раз в 31 ячейку вместо 100, дают более частые измерения RTT и более быструю реакцию.

11.3.4. Больше никаких окон потока: XON и XOFF

При управлении перегрузкой сообщения SENDME уровня потока исчезают. Окна потока создавали дилемму: маленькие ограничивают скорость каждой загрузки; большие позволяют вредоносному клиенту заставить выходное реле буферизовать целое окно данных для каждого из множества открытых потоков. Предложение заменяет их двумя сообщениями:

XOFF (команда RELAY 43): «прекратить отправку по этому потоку», посылается, когда очередь к приложению превышает порог;

XON (команда RELAY 44): «возобновить», с 4-байтовым полем скорости опустошения очереди, чтобы другая сторона могла адаптироваться. Поле называется kbps_ewma, а Предложение 324 не определяет его единицу. Код tor вычисляет и интерпретирует его в единицах **1000 байт в секунду**, а не 1000 бит (src/core/or/congestion_control_flow.c): имя вводит в заблуждение, и буквальная интерпретация дала бы восьмикратную ошибку.

Сообщения SENDME уровня цепи, напротив, отправляются сразу после расшифрования и распознавания ячеек, до доставки приложению. Они измеряют сеть, а не приложение.

Внимание: Управление перегрузкой и анонимность

Любой механизм, реагирующий на трафик, может стать побочным каналом. Наблюдение за поведением окна способно раскрывать информацию о цепи. Предложение 324 посвящает таким рискам и мерам защиты отдельный раздел, включая ограничение частоты XON и XOFF. Это та же предосторожность, из-за которой в гл. 8 рекомендуется всегда отправлять причину NONE в **DESTROY**: любая наблюдаемая вариация, зависящая от внутреннего состояния, выдаёт информацию. Атаки по признакам трафика рассматриваются в гл. 26.

11.4. Лабораторная работа: загрузка файла через Tor

Лабораторная работа 5: Потоки и управление потоком

Цель. Написать `labs/ru/код/torlab/circuit.py`, объединяющий предыдущие лабораторные работы в класс `Circuit`, и `labs/ru/код/torlab/stream.py`, открывающий потоки и передающий данные. Загрузить файл с локального веб-сервера по трёхпереходной цепи и измерить окна выходном реле.
Предварительные условия. Лабораторные работы 1–4; работающая лабораторная сеть; локальный веб-сервер (`python3 -m http.server 8080`).
Оценочное время. 3 часа.

Шаг 1: цепь одной строкой. Функция `build` открывает канал к входному реле, создаёт первый переход и расширяет цепь, запрашивая управление перегрузкой только у последнего перехода, как делает tor:

```
circ = build([relay("003r"), relay("004r"), relay("006e")])
print(circ.exit_extensions)             # [(2, b'\x1f')]: sendme_inc = 31
```

Шаг 2: открытие потока.

```
def begin_body(address, port, flag=0):
    body = f"{address.lower()}:{port}".encode() + b"\x00"
    return body + (struct.pack(">I", flag) if flag else b"")

s = Stream(circ, 1)
s.open("127.0.0.1", 8080)       # BEGIN, затем ожидание CONNECTED или END
```

Выходное реле отвечает `CONNECTED` с 8-байтовым телом `7f000001 00000708`: адрес 127.0.0.1 и TTL `0x708` = 1800 секунд.

Шаг 3: SENDME. Цепь считает каждую ячейку `DATA`, полученную от выходного реле, и отправляет аутентифицированный `SENDME` через каждые `sendme_inc` ячеек:

```
def count_received_data(self):
    est = dict(self.exit_extensions)
    sendme_inc = est[2][0] if 2 in est else 100
    self.data_to_ack += 1
    if self.data_to_ack == sendme_inc:
        self.data_to_ack = 0
        digest = self.crypto.hops[-1].backward.digest.digest()
        body = struct.pack(">BH", 1, 20) + digest
        self.send(RelayMessage(RelayCommand.SENDME, 0, body))
```

Внимание: Поучительная ошибка

В первой версии лабораторной работы счётчик был атрибутом *потока*. Первый поток, страница из двух ячеек, работал; второй, файл 1 MB, прерывался выходным реле ячейкой **DESTROY**. Причина: сообщения SENDME уровня цепи считают DATA из *всех* потоков цепи. Выходное реле уже учло две ячейки первого потока и ожидало SENDME после 29-й ячейки второго. Мы отправляли его после 31-й с дайджестом, отличавшимся от ожидаемого, и выходное реле сделало именно то, что требует Предложение 289: закрыло цепь. Перенос счётчика в класс `Circuit` исправил проблему.

Шаг 4: загрузка. Отправляем HTTP-запрос и читаем до END:

```
s.send(b"GET /grande.bin HTTP/1.0\r\nHost: 127.0.0.1\r\n\r\n")
response = s.receive_all()
```

Для случайного файла размером 1 048 576 байт в одном из наших запусков:

```
stream 1: 209 байт за 0.05 с, DATA-ячеек 2, circuit SENDME пока 0,
          прочее ['END DONE']
stream 2: 1048781 байт
    за 0.18 с, DATA-ячеек 2120, circuit SENDME пока 68, прочее ['END DONE']
```

SHA-256-отпечаток полученного файла совпадает с исходным. 68 сообщений SENDME — ровно $\lfloor (2 + 2120)/31 \rfloor$: один счётчик на цепь. Ячеек 2120, а не минимальные $\lceil 1048781/498 \rceil = 2106$, поскольку выходное реле не ждёт накопления 498 байт перед отправкой ячейки, а упаковывает данные по мере чтения с сервера.

Шаг 5: наблюдение за окнами. Теперь проведём эксперимент, невозможный для обычного клиента: загрузим тот же файл, *никогда не отправляя* SENDME, и посчитаем, сколько ячеек выходное реле отправит до остановки. Повторим с управлением перегрузкой и без него:

```
circ = build(path, cc=cc)
circ.count_received_data = lambda: None      # SENDME не отправляются
```

```
управление перегрузкой=True:  выходное реле останавливается после 124 ячеек DATA
управление перегрузкой=False: выходное реле останавливается после 500 ячеек DATA
```

С управлением перегрузкой выходное реле останавливается на 124 ячейках: это начальное окно перегрузки, 4×31. Без него выходное реле останавливается на 500, а не 1000: *окно потока* (500) исчерпывается раньше окна цепи (1000), поскольку при исходном управлении потоком клиент также должен отправлять сообщения SENDME уровня потока. В обоих случаях выходное реле просто ждёт: данные остаются в его собственном буфере, а не в сети.

Итоги

- Поток открывается BEGIN (`address:port` в нижнем регистре), подтверждается CONNECTED, переносит DATA и закрывается END; клиент может отправлять данные до CONNECTED.
- Разрешение имён происходит на выходном реле, чтобы не раскрывать пункты назначения локальному DNS-резолверу; RESOLVE и RESOLVED позволяют разрешать имена без соединения.
- Исходное управление потоком использует окна 1000 ячеек на цепь и 500 на поток, с SENDME каждые 100 и 50 ячеек соответственно.

- Сообщения `SENDME` уровня цепи аутентифицируются накопительным дайджестом вызвавшей их ячейки (Предложение 289).
- При управлении перегрузкой (Предложение 324, алгоритм по умолчанию Vegas) окно адаптируется к RTT, измеренному через `SENDME`; `sendme_inc` равен 31, числу ячеек в одном TLS-записи, а начальное окно равно 124.
- XON и XOFF заменяют окна потока.

Упражнения

Упражнение 11.1 ⋆ Запишите в шестнадцатеричном виде тело сообщения `BEGIN` к `Example.COM` на порт 443 для клиента, который также принимает IPv6.

Упражнение 11.2 ⋆ Интерпретируйте тело `CONNECTED c0000201 0000012c`. Что клиент должен сделать с этой информацией?

Упражнение 11.3 ⋆ Оцените время, сэкономленное благодаря *опережающей передаче данных (optimistic data)* для цепи с RTT 600 ms и страницы, требующей одного HTTP-запроса. Почему выходное реле должно отбросить поставленные в очередь данные, если соединение не открылось?

Упражнение 11.4 ⋆⋆ Какова максимальная скорость одного потока при исходном управлении потоком в цепи с RTT 400 ms? А цепи с несколькими потоками? Сравните эти величины с начальным окном управления перегрузкой.

Упражнение 11.5 ⋆⋆ Опишите атаку на исчерпание памяти выходного реле, основанную на неаутентифицированных `SENDME`. Почему случайное заполнение в ячейках `DATA` необходимо для работы аутентификации?

Упражнение 11.6 ⋆⋆ Расширьте `stream.py` функцией `resolve(name)`, использующей `RESOLVE` и интерпретирующей `RESOLVED`. Проверьте её в лабораторной сети с `localhost`. Как выходное реле отвечает на запрос несуществующего имени?

Упражнение 11.7 ⋆⋆⋆ Попробуйте открыть поток на порт, который выходное реле лабораторной сети не принимает, после изменения политики выхода (`torrc` для `test006e`, затем `chutney hup`). Какую причину сообщает `END`? Интерпретируйте дополнительные поля.

Упражнение 11.8 ⋆⋆⋆ Измените лабораторную работу так, чтобы на клиенте измерять время между отправкой каждого `SENDME` и получением следующей ячейки `DATA`. Что именно вы измеряете? Сравните значения при наличии и отсутствии второй параллельной загрузки по той же цепи.

Дополнительная литература

- *Tor Protocol Specification*, разделы «Opening streams and transmitting data», «Closing streams», «Remote hostname lookup» и «Flow control» [Thee].
- Предложение 289, *Authenticating sendme cells to mitigate bandwidth attacks*.
- Предложение 324, *RTT-based Congestion Control for Tor*: мотивация, алгоритмы, параметры и анализ безопасности.

Глава 12

Система каталогов

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; метод консенсуса 36

Цели главы

К концу этой главы вы сможете:

- объяснять, зачем Tor нужна система каталогов и какие риски она должна предотвращать;
- различать её документы: дескрипторы, микродескрипторы, голоса, консенсусы и сертификаты центров каталогов;
- описывать, как центры каталогов каждый час голосуют и объединяют свои голоса;
- понимать критерии назначения флагов;
- читать консенсус построчно и проверять его подписи;
- загружать и проверять консенсус лабораторной сети с помощью собственного кода (лабораторная работа 6).

В предыдущих лабораторных работах мы всегда заранее знали, к каким реле подключаться, какие ключи они используют и какие порты слушают: эту информацию мы читали из файлов лабораторной сети. У реального клиента такой роскоши нет. Он должен узнать, какие реле существуют, где они находятся, какие ключи используют и насколько им можно доверять. Этим занимается **система каталогов**, которой посвящена эта глава.

12.1. Проблема

Система каталогов Tor должна удовлетворять нескольким частично конкурирующим требованиям:

Корректность Информация о реле (адреса и ключи) должна быть подлинной. Противник, убедивший клиента, что определённое реле имеет другой ключ, мог бы перехватить его трафик.

Единое представление Все клиенты должны видеть *один и тот же* список реле. Если бы противник мог выдавать каждому клиенту свой список, возможно содержащий только несколько контролируемых им реле, клиентов можно было бы узнавать по используемым ими реле. Это атака *разделения (partitioning)*, также называемая *эпистемической*.

Устойчивость к злоупотреблениям Добавить реле в сеть может любой. Система должна ограничивать влияние того, кто добавляет множество реле (*Sybil*-атака), и уметь исключать реле, ведущие себя некорректно.

Масштабируемость Миллионы клиентов загружают каталожную информацию несколько раз в день. Её распространение должно быть экономичным.

Решение Tor, сформировавшееся за три версии протокола каталогов, — небольшой набор доверенных серверов, **центры каталогов (directory authorities)**. Каждый час они голосуют и совместно подписывают единый документ: **консенсус** [dir-spec § Outline].

Историческая справка: Три поколения

В первой версии протокола каждый центр каталогов публиковал собственный список реле, поэтому нечестный центр каталогов мог вводить клиентов в заблуждение. Во второй клиенты загружали списки всех центров каталогов и сами объединяли представления, многократно увеличивая расход полосы. Третья версия, используемая более пятнадцати лет, передаёт объединение центрам каталогов: они создают один документ, подписанный всеми, а клиентам остаётся только проверить его. Тогда же центры каталогов перестали подписывать документы тем же ключом, который использовали как реле и который приходилось хранить в памяти в открытом виде.

12.2. Участники

Центр каталогов (Directory authority) Серверы, которыми управляют доверенные участники сообщества Tor. В публичной сети их девять, плюс отдельный центр для мостов (гл. 16). Их список встроен в tor (`src/app/config/auth_dirs.inc`), поэтому каждый клиент начинает с одного и того же набора. Каждый центр имеет долгосрочный *ключ идентичности*, хранящийся офлайн, который сертифицирует *ключ подписи* со сроком действия от 3 до 12 месяцев. Последний подписывает голоса и консенсусы.

Кэш каталогов (Directory cache) Реле, которые загружают консенсусы и дескрипторы у центров каталогов и повторно распространяют их клиентам, уменьшая нагрузку на центры каталогов. Они имеют флаг `V2Dir` и обслуживают потоки `BEGIN_DIR` (гл. 11).

Резервный сервер каталогов (Fallback directory) Список примерно из двухсот стабильных реле, также встроенный в tor (`fallback_dirs.inc`; 200 записей в tor 0.4.9.13). Новый клиент запрашивает первый консенсус у одного из них, а не перегружает центры каталогов.

Реле Реле периодически публикуют собственные дескрипторы центрам каталогов.

Клиент Клиенты загружают консенсус и микродескрипторы, проверяют их и используют для выбора маршрутов (гл. 13).

На рис. 12.1 показано движение этих документов.

12.3. Документы

Все документы каталогов представляют собой текст в формате, который спецификация называет *netdoc*: последовательность строк (*элементов (items)*), каждая с ключевым словом и аргументами, иногда с последующим объектом в base64 между строками `-----BEGIN ........` и `-----END ........`. Этот формат легко читать человеку и разбирать программой, что мы увидим в лабораторной работе.

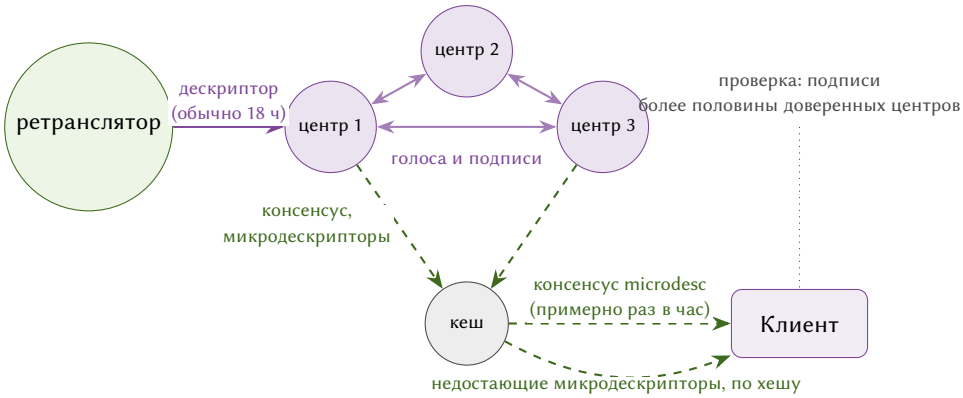


Рис. 12.1: Поток документов системы каталогов. Реле публикуют дескрипторы центрам каталогов; центры каталогов обмениваются голосами, вычисляют консенсус на основе полученных голосов и обмениваются подписями; кэши каталогов загружают консенсусы и микродескрипторы и обслуживают ими клиентов.

Дескриптор реле (*server descriptor*) Создаётся и подписывается реле: адреса и порты, ключи (идентичность, ключ onion ntor, сертификаты Ed25519), заявленная пропускная способность, полная политика выхода, семейство, платформа и версия. Он повторно публикуется не реже чем каждые 18 часов (`FORCE_REGENERATE_DESCRIPTOR_INTERVAL` в коде tor) или раньше при существенных изменениях.

Документ extra-info Статистика, не нужная для работы протокола, например трафик мостов по странам, публикуется отдельно, чтобы не раздувать дескрипторы. Эти данные питают Tor Metrics (гл. 24).

Микродескриптор Подмножество дескриптора, действительно нужное клиенту и редко меняющееся: ключ onion ntor, семейство, сводка политики выхода и идентичность Ed25519. Его вычисляют центры каталогов; каждый микродескриптор идентифицируется дайджестом SHA-256 своего текста.

Голос Оценка сети одного центра каталогов: флаги, пропускная способность и версия каждого реле.

Консенсус Результат объединения голосов, подписанный центрами каталогов. Он имеет два *варианта (flavor)*: `ns`, содержащий полный дайджест дескриптора для каждого реле, и `microdesc`, содержащий дайджест микродескриптора. Современные клиенты используют только более компактный второй вариант.

Сертификат ключа центра каталогов Связывает ключ подписи центра каталогов с его идентичностью.

Подробнее: Зачем микродескрипторы?

Полный дескриптор имеет размер несколько килобайт и часто меняется, например при изменении заявленной полосы. Если бы клиент несколько раз в день загружал все дескрипторы тысяч реле, это потребовало бы значительной полосы, особенно на мобильных соединениях. Микродескриптор содержит лишь редко меняющуюся информацию. Часто меняющиеся сведения, такие как пропускная способность и флаги, находятся в консенсусе, который клиент всё равно загружает. Поскольку клиент запрашивает микродескрипторы по дайджесту, а не по идентичности реле, кэш каталогов не может выдать

одному клиенту иной микродескриптор, чем всем остальным.

12.4. Голосование

12.4.1. Расписание

У каждого консенсуса три временные метки [dir-spec § Voting timeline]:

`valid-after` момент, когда он становится действительным;

`fresh-until` момент, когда он перестаёт быть самым свежим; обычно совпадает с `valid-after` следующего консенсуса;

`valid-until` момент, после которого клиент больше не может его использовать.

В публичной сети интервал голосования равен одному часу, а консенсус действует три интервала (опции `V3AuthVotingInterval` и `V3AuthNIntervalsValid`). Поэтому в любой момент одновременно существует несколько действительных консенсусов, и задержка центрам каталогов не блокирует клиентов. До начала каждого интервала центры каталогов имеют пять минут на обмен голосами и пять минут на обмен подписями (`V3AuthVoteDelay` и `V3AuthDistDelay`). В лабораторной сети всё ускорено: голосование каждые 20 секунд, по 4 секунды на каждую фазу (разд. 19.5).

12.4.2. Объединение голосов

Каждый центр каталогов собирает голоса остальных и вычисляет консенсус детерминированным алгоритмом, чтобы все центры каталогов получили одинаковый документ и могли его подписать [dir-spec § Computing a consensus from a set of votes]. Основные правила:

- реле появляется в консенсусе только если его знает достаточное число центров каталогов и, в современных методах консенсуса (24 и новее; лаборатория использует 36), только если оно имеет одновременно `Running` и `Valid`;
- реле получает флаг, если его назначает более половины центров, голосующих по этому флагу;
- пропускная способность (`w`) — нижняя медиана проголосованных значений, кроме случая, когда как минимум три центра каталогов *измерили* пропускную способность: тогда используется медиана этих измерений;
- сетевые параметры (`params`) включаются, если за них голосует большинство всех центров каталогов либо как минимум три центра каталогов (`i > total_authorities/2 || i >= 3` в коде); выбирается нижняя медиана;
- временные метки — медианы предложенных значений.

Поскольку центры каталогов могут работать на разных версиях tor, документ объявляет используемый *метод консенсуса*: число, идентифицирующее точный набор правил, выбранное как наивысший метод, поддерживаемый достаточным числом центров каталогов. Консенсус лабораторной сети использует метод 36.

Флаг	Критерий (кратко)
`Running`	центр каталогов установил соединение с реле на всех его ORPorts в последние 45 минут
`Valid`	версия tor не известна как дефектная; реле не исключено центрами каталогов
`Fast`	пропускная способность находится среди самых быстрых семи восьмых либо не ниже 100 KB/s
`Stable`	взвешенное по времени среднее время между отказами не ниже медианы либо не меньше 7 дней
`Guard`	`Fast`, `Stable`, *familiar* (в сети несколько недель), хорошая доступность, достаточная пропускная способность, `V2Dir`
`Exit`	разрешает выходной трафик как минимум к одному /8-блоку одновременно на портах 80 и 443
`HSDir`	`Stable` и `Fast`, работает не менее 96 часов: может хранить дескрипторы onion-сервисов
`V2Dir`	обслуживает каталожную информацию (кэш каталогов)
`Authority`	является центром каталогов
`BadExit`	выходное реле, которое не следует использовать для выхода, например из-за изменения трафика
`MiddleOnly`	следует использовать только как промежуточное реле (введён в 0.4.7.2-alpha)
`StaleDesc`	дескриптор опубликован более 18 часов назад
`Sybil`	превышает максимальное число реле на один IP-адрес (обычно 2)

Таблица 12.1: Флаги консенсуса и краткие критерии их назначения центрами каталогов.

12.4.3. Флаги

Флаги кратко выражают оценку реле каждого центра каталогов [dir-spec § Assigning flags in a vote]. Спецификация предупреждает, что точные критерии могут меняться и клиенты не должны от них зависеть. Табл. 12.1 сводит современные критерии.

`BadExit` и `MiddleOnly` позволяют центрам каталогов реагировать на подозрительные реле, не удаляя их полностью. Флаг `Sybil` ограничивает влияние того, кто запускает множество реле с одного адреса. В лабораторной сети, где все реле работают на 127.0.0.1, ограничение отключено параметром `AuthDirMaxServersPerAddr=0`; это будет видно в строке `params`.

12.4.4. Пропускная способность

Какова пропускная способность реле? Значению, заявленному в его дескрипторе, доверять нельзя: вредоносное реле могло бы объявить огромную пропускную способность, чтобы его выбирали чаще. Некоторые центры каталогов, называемые *центрами измерения пропускной способности (bandwidth authorities)*, поэтому измеряют пропускную способность реле, пропуская через них трафик специальным инструментом (сейчас *sbws*, гл. 24), и включают результат в голоса как `Measured`. Если доступно не менее трёх измерений, консенсус использует их медиану. Иначе строка `w` содержит `Unmeasured=1` и ограниченное значение. Consensus bandwidth — не просто скорость, а *вес*: он определяет вероятность выбора реле (гл. 13).

12.5. Анатомия консенсуса

Ниже приведён сокращённый консенсус варианта microdesc лабораторной сети, загруженный в лабораторной работе 6. Фрагменты взяты из сохранённой в репозитории копии (`labs/ru/код/torlab/test/data/consensus-microdesc.txt`), используемой тестами лаборатории. Поэтому их можно изучать построчно без запуска сети. Копия была получена 23 сентября 2026 года, когда лабораторные реле ещё работали на tor 0.4.9.12. Именно эта версия находится в строке `v` записи реле, а не референсная версия книги, на которой сеть работает сегодня.

Заголовок.

```
network-status-version 3 microdesc
vote-status consensus
consensus-method 36
valid-after 2026-09-23 15:37:20
fresh-until 2026-09-23 15:37:40
valid-until 2026-09-23 15:38:00
voting-delay 4 4
known-flags Authority Exit Fast Guard HSDir ... Running Stable ... Valid
params AuthDirMaxServersPerAddr=0 cc_alg=2
```

Строка `params` содержит *параметры консенсуса*, с помощью которых центры каталогов меняют поведение всей сети без обновления программного обеспечения. Здесь, помимо ограничения на один адрес, задаётся алгоритм управления перегрузкой (`cc_alg=2`, Vegas, разд. 11.3). В публичной сети параметров несколько десятков.

Запись реле.

```
r test006e ml1aPg9UFy4VCx85YcpsRs/9shc 2038-01-01 00:00:00 127.0.0.1 5106 0
m FfrgO+s4VYS6WKBq3pjfMDs+uZ2uVsvgG88JilbhIOo
s Exit Fast Guard HSDir Running Stable V2Dir Valid
v Tor 0.4.9.12
pr Conflux=1 Cons=1-2 Desc=1-4 ... Link=3-5 ... Relay=2-6
w Bandwidth=4096
```

`r` псевдоним реле, дайджест RSA-идентичности (base64), поле даты, больше не имеющее смысла в консенсусе, IPv4-адрес, ORPort и DirPort;

`m` дайджест микродескриптора, который следует загрузить;

`s` флаги;

`v`, `pr` версия tor и поддерживаемые *подпротоколы*: здесь клиент, например, узнаёт, что реле поддерживает CGO (`Relay=6`) или версии протокола соединения 3–5;

`w` вес пропускной способности в килобайтах в секунду.

В варианте `ns` каждая запись также содержит строку `p`, кратко описывающую политику выхода по портам. В варианте microdesc это описание находится в микродескрипторе и не повторяется в консенсусе.

Нижняя часть и подписи.

```
directory-footer
bandwidth-weights Wbd=821 Wbe=0 Wbg=5000 Wbm=10000 Wdb=10000 ...
directory-signature sha256 3776EF73A9214587AEFB3D732FA4A443CFF491BA
                           79E102D226590BC5D1D1EBBAD8A16122EC0F9801
-----BEGIN SIGNATURE-----
...
-----END SIGNATURE-----
```

Значения `bandwidth-weights` — коэффициенты, которыми клиенты балансируют использование реле в разных позициях цепи (гл. 13). Далее следуют подписи, по одной от каждого центра каталогов: алгоритм хеширования, дайджест идентичности центра каталогов, дайджест его ключа подписи и подпись.

12.6. Проверка консенсуса

Клиент принимает консенсус только при наличии действительных подписей **более чем половины** известных ему центров каталогов. В коде tor порог равен `n_v3_authorities/2 + 1` (`src/feature/nodelist/networkstatus.c`): 5 подписей из 9 в публичной сети или 2 из 3 в лаборатории. Для проверки подписи клиенту нужны:

1. **сертификат** центра каталогов, который клиент загружает (`/tor/keys/all`) и проверяет по заранее известной идентичности центра каталогов;

2. **подписанный текст**: документ от строки `network-status-version` до пробела включительно после первого ключевого слова `directory-signature`;

3. схема подписи: RSA с дополнением PKCS#1 v1.5, применённым непосредственно к дайджесту (SHA-1 или SHA-256), *без* идентификатора алгоритма, обычно ожидаемого PKCS#1 [dir-spec § Signing documents].

Подробнее: До пробела или до перевода строки?

Общий раздел спецификации о подписанных документах говорит, что подпись охватывает текст до *перевода строки* после строки подписи. Раздел, относящийся именно к консенсусу, вместо этого указывает *пробел* после `directory-signature`, чтобы все центры каталогов подписывали одинаковые байты, хотя строки их подписей различаются. В лабораторной работе 6 мы проверили обе трактовки на реальном консенсусе лабораторной сети. Все три подписи были действительны при правиле пробела; ни одна — при правиле перевода строки. При сомнении экспериментальная проверка против независимой реализации остаётся окончательным арбитром.

Внимание: Список центров каталогов --- корень доверия

Вся система опирается на список центров каталогов, встроенный в программное обеспечение. Пользователь может его изменить, но спецификация не рекомендует этого делать: клиент с другим набором центров каталогов увидит отличный от остальных консенсус и станет различимым. Сами центры каталогов являются целями. Если большинство будет скомпрометировано, они смогут подписать консенсус с вредоносными реле. Поэтому центров каталогов управляются разными людьми в разных странах, а ключи идентичности хранятся офлайн.

12.7. Как клиент остаётся актуальным

Клиент не загружает новый консенсус ровно каждый час: если бы все клиенты делали это одновременно, кэши каталогов оказались бы перегружены. Вместо этого он выбирает случайный момент между тремя четвертями первого интервала после потери консенсусом свежести и семью восьмыми оставшегося времени до истечения [dir-spec § Client operation]. Спецификация приводит пример: для консенсуса, действующего с 1:00, свежего до 2:00 и действительного до 4:00, клиент начнёт искать новый в случайный момент между 2:45 и 3:50.

Для экономии полосы пропускания клиент может запросить только *различия* с уже имеющимся консенсусом (*consensus diff*), идентифицируя его дайджестом SHA3-256. Затем он загружает только отсутствующие микродескрипторы, запрашивая их по дайджесту (`/tor/micro/d/`, после чего дайджесты разделяются `-`).

12.8. Лабораторная работа: загрузка и проверка консенсуса

Лабораторная работа 6: Консенсус лабораторной сети

Цель. Написать `labs/ru/код/torlab/directory.py` для загрузки консенсуса, сертификатов и микродескрипторов у центра каталогов через цепь, их разбора и проверки подписей.
Предварительные условия. Лабораторные работы 1–5; работающая лабораторная сеть.
Оценочное время. 3 часа.

Шаг 1: загрузка через BEGIN_DIR. Как tor, используем одношаговую цепь к серверу каталогов без управления перегрузкой и открываем потоки `BEGIN_DIR`. Внутри потока используется HTTP/1.0:

```
def download(circ, path, stream_id):
    s = Stream(circ, stream_id)
    circ.send(RelayMessage(RelayCommand.BEGIN_DIR, stream_id, b""))
    _, msg = s._next()                      # CONNECTED, пустое тело
    s.send(f"GET {path} HTTP/1.0\r\n\r\n".encode())
    header, _, body = s.receive_all().partition(b"\r\n\r\n")
    return body

circ = build([relay("000a")], cc=False)
text = download(circ, "/tor/status-vote/current/consensus-microdesc", 1)
```

Шаг 2: формат netdoc. Функция примерно из двадцати строк, `items`, делит документ на элементы и собирает объекты `-----BEGIN`/`-----END`. На её основе строятся два парсера: `parse_consensus`, создающий запись каждого реле с флагами, пропускной способностью, политикой выхода и дайджестом микродескриптора, и `parse_certificates`.

Шаг 3: проверка подписей.

```
def signed_text(text):
    start = text.index("network-status-version ")
    pos = text.index("\ndirectory-signature ") + len("\ndirectory-signature ")
    return text[start:pos].encode()

def verify_signature(consensus, signature, cert):
    h = hashlib.sha256 if signature.algorithm == "sha256" else hashlib.sha1
    expected = h(signed_text(consensus.text)).digest()
    recovered = cert.signing_key.recover_data_from_signature(
        signature.signature, padding.PKCS1v15(), None)
    return recovered == expected
```

Метод `recover_data_from_signature` с алгоритмом `None` возвращает дайджест, встроенный в подпись, не ожидая идентификатора алгоритма: именно так устроена схема Tor.

Шаг 4: результаты. В одном из наших запусков в лабораторной сети (даты, имена и количества зависят от экземпляра сети):

```
консенсус: 6035 байт; 2026-09-23 15:37:20 -> 2026-09-23 15:38:00
метод 36; подписи 3; сертификаты 3
params: {'AuthDirMaxServersPerAddr': 0, 'cc_alg': 2}
  test006e 127.0.0.1:5106 w=4096 flags=['Exit', 'Fast', 'Guard', ...]
  test000a 127.0.0.1:5100 w=20   flags=['Authority', 'Fast', 'Guard', ...]
```

```
  ...
действительных подписей: 3 (необходимо: 2)
микродескрипторов: 8; соответствуют строкам m: True
```

В исходной записи лабораторного опыта консенсус охватывает 40 секунд, то есть два 20-секундных интервала, подписан всеми тремя центрами каталогов, а дайджест SHA-256 каждого загруженного микродескриптора совпадает со строкой `m` в консенсусе. Цепочка доверия проходит от ключей центров каталогов до ключа ntor каждого реле.

Шаг 5: отрицательные проверки. Проверка, которая никогда не завершается ошибкой, ничего не проверяет. Измените одну цифру консенсуса (`Bandwidth=4096` на `4097`), и число действительных подписей упадёт до нуля; при наличии только одного известного центра каталогов — до единицы. Файл `test/test_directory.py` повторяет эти проверки по сохранённой копии консенсуса, поэтому тесты работают и без сети.

Шаг 6: настоящий консенсус. Тот же код работает с публичной сетью. Загрузите через CollecTor консенсус и соответствующие сертификаты центров каталогов за тот же период и проверьте:

```
c = parse_consensus(open("consensus-microdesc-2026-09-23-15").read())
cert = parse_certificates(open("certs").read())
valid, required, _ = verify_consensus(c, cert, 9)
print(valid, required)          # 9 5
```

В приведённом примере действительны все девять подписей, что выше порога в пять. При первой попытке мешает одна деталь: файлы CollecTor начинаются строкой-аннотацией `@type network-status-microdesc-consensus-3 1.0`, которая не входит в подписанный документ. Если включить её, дайджест изменится и *все* подписи станут недействительными. Это выглядит как криптографическая ошибка, но на деле является ошибкой форматирования. Поэтому функция `signed_text` начинает с `network-status-version`, и отдельный тест проверяет это поведение.

Итоги

- Система каталогов должна давать всем клиентам одинаковое подлинное представление сети, противостоять Sybil-атакам и масштабироваться до миллионов пользователей.
- Девять центров каталогов плюс один для мостов, перечисленные в коде, голосуют каждый час и подписывают единый консенсус, действительный три часа.
- Реле публикуют дескрипторы; клиенты используют консенсус варианта microdesc и микродескрипторы, загружаемые по дайджесту.
- Флаг назначается, если за него голосует более половины центров каталогов, ответственных за этот флаг; пропускная способность, измеренная как минимум тремя центрами каталогов, имеет приоритет над заявленной.
- Клиент принимает консенсус с действительными подписями более половины центров каталогов; подпись охватывает текст до пробела включительно после `directory-signature`.
- Параметры консенсуса изменяют поведение всей сети без обновления программного обеспечения.

Упражнения

Упражнение 12.1 ⋆ Консенсус имеет `valid-after` 10:00, `fresh-until` 11:00 и `valid-until` 13:00. В каком интервале клиент будет искать замену? Что произойдёт, если к 13:00 нового консенсуса не будет?

Упражнение 12.2 ⋆ Реле имеет пропускную способность 500 KB/s, находится в сети два дня и использует политику выхода `accept *:443, reject *:*`. Какие флаги оно может получить? Почему не `Exit`?

Упражнение 12.3 ⋆⋆ Опишите атаку разделения (partitioning), в которой вредоносный кэш каталогов отправляет конкретному клиенту старый, но ещё действительный консенсус. Что ограничивает атаку? Почему запрос микродескрипторов по дайджесту помогает?

Упражнение 12.4 ⋆⋆ Три центра каталогов измеряют пропускную способность реле: 5000, 5200 и 90000. Какое значение попадёт в консенсус? Что изменится, если третий центр каталогов, вступивший в сговор с реле, будет единственной, кто его измерял?

Упражнение 12.5 ⋆⋆ Найдите файл `v3-status-votes` в каталоге одного центра каталогов лабораторной сети. Сравните её голос с консенсусом: какие строки отличаются и почему?

Упражнение 12.6 ⋆⋆ Выберите реле из консенсуса, загруженного в лабораторной работе. Вручную вычислите дайджест его микродескриптора с помощью `sha256sum` и сравните со строкой `m`. С какой строки начинается хешируемый текст и где заканчивается?

Упражнение 12.7 ⋆⋆⋆ Остановите один из трёх центров каталогов лабораторной сети и через минуту снова загрузите консенсус. Сколько подписей он содержит? Принимает ли его ваша проверка? Повторите с двумя остановленными центрами каталогов и объясните наблюдаемое после истечения консенсуса.

Упражнение 12.8 ⋆ ⋆ ⋆ Также загрузите консенсус `ns` (`/tor/status-vote/current/consensus`) и проверьте его подписи. Какие алгоритмы хеширования он использует? Какая дополнительная информация находится в строке `r` по сравнению с вариантом microdesc?

Дополнительная литература

- *Tor directory protocol, version 3* (dir-spec) [Thee]: форматы, голосование, назначение флагов и работа клиентов и кэшей каталогов.
- Документ протокола общей случайности (shared randomness) (srv-spec), с помощью которого центры каталогов ежедневно создают общее случайное значение; оно понадобится для onion-сервисов (гл. 14).
- Tor Metrics и CollecTor, `https://metrics.torproject.org/`: исторические архивы консенсусов, голосов и дескрипторов.

Глава 13

Выбор маршрута, входные узлы и vanguards

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; публичный консенсус от 23.09.2026, 15:00 UTC

Цели главы

К концу этой главы вы сможете:

- описывать порядок и ограничения, которые tor использует при выборе реле для цепи;
- объяснять, как пропускная способность в консенсусе и *bandwidth-weights* распределяют нагрузку между позициями цепи;
- объяснять, зачем нужны входные узлы (guards), и прослеживать их эволюцию от трёх узлов к одному, а затем к двум;
- описывать алгоритм входных узлов: выборку, отфильтрованные, подтверждённые и основные узлы;
- объяснять, что такое vanguards и зачем они нужны onion-сервисам;
- моделировать выбор маршрута по реальному консенсусу публичной сети (лабораторная работа 7).

Система каталогов (гл. 12) предоставляет клиенту список реле. Остаётся вопрос, который сильнее почти любого другого влияет на защиту пользователя: *какие именно* реле следует использовать? Если противник контролирует одновременно первое и последнее реле цепи, он может коррелировать входящий и исходящий трафик и определить, кто с кем общается (гл. 26). Поэтому выбор маршрута — во многом искусство сделать такое событие как можно более редким, не жертвуя производительностью.

13.1. Порядок и ограничения

Для трёхпереходной цепи tor сначала выбирает **выходной узел (exit)**, затем входной (guard) и лишь после этого промежуточный (middle) [path-spec § Path selection and constraints]. Выходной узел выбирается первым, потому что именно к нему предъявляется больше всего ограничений: политика выхода должна разрешать запрошенный пункт назначения. Если клиент знает только имя назначения, но не его адрес, клиент выбирает выходной узел, который *может* принять соединение: его политика не должна отклонять нужный порт до разрешающего правила.

За несколькими исключениями все маршруты удовлетворяют следующим ограничениям:

- не использовать реле без флага `Fast`;
- одно реле не может появляться дважды;
- не более одного реле из каждой IPv4-сети /16 (или IPv6 /32): два реле в одном дата-центре могут находиться под контролем одного оператора;
- не более одного реле из каждого **семейства**: операторы нескольких реле должны объявлять их как одно семейство, а tor избегает помещать два из них в одну цепь;
- первое реле является входным узлом (guard) (разд. 13.3);
- выходные узлы с флагом `BadExit` не используются.

Соединения, которые предположительно будут жить долго (tor относит к ним некоторые порты, например SSH 22 и IRC 6667), дополнительно требуют реле `Stable`. Onion-сервисы добавляют ограничение, не зависящее от порта: цепи введения на стороне сервиса и маршруты встречи строятся как цепи `Stable`, а их конечные точки не должны иметь `MiddleOnly` [path-spec § Path selection and constraints]. Упомянутые «несколько исключений» в основном относятся к цепям onion-сервисов с vanguards (разд. 13.4), где некоторые ограничения намеренно ослабляются, например входной узел может занимать промежуточную позицию. Запрет таких комбинаций сообщил бы противнику, какие реле *не* были выбраны, и тем самым раскрыл бы сведения о выборе входных узлов.

Подробнее: Два способа объявить семейство

Традиционно оператор объявляет семейство, перечисляя все остальные свои реле в дескрипторе каждого реле. Два реле считаются принадлежащими одному семейству только если перечисляют друг друга. При большом числе реле списки становятся огромными, а одна ошибка конфигурации может нарушить связи. Новый механизм *happy families* даёт каждому семейству ключ. Каждое реле доказывает принадлежность сертификатом, подписанным этим ключом (`family-cert` в дескрипторе, `family-ids` в микродескрипторе). Параметры консенсуса `use-family-lists` и `use-family-ids` определяют, какой механизм применяют клиенты.

13.2. Взвешивание по пропускной способности

13.2.1. Зачем нужны веса

Если бы клиенты выбирали реле равномерно случайно, реле на 1 MB/s получало бы столько же трафика, сколько реле на 1 GB/s. Первое было бы перегружено, второе недоиспользовано. Поэтому Tor выбирает каждое реле с вероятностью, **пропорциональной его пропускной способности** в консенсусе (строка `w`, разд. 12.5), распределяя нагрузку по доступной мощности.

13.2.2. Позиции цепи неравноценны

Есть осложнение. Реле с флагом `Exit` сравнительно мало, а их исходящая пропускная способность — самый дефицитный ресурс сети. Использовать выходной узел в качестве входного или промежуточного означало бы расходовать ресурсы, необходимые для выхода из сети. В меньшей степени то же относится к входным узлам. Поэтому авторитетные серверы каталогов рассчитывают для каждого консенсуса **bandwidth-weights**: коэффициенты, умножающие пропускную способность реле в зависимости от его *muna* (только входной, только выходной,

оба типа или ни один из них) и *позиции*, для которой его выбирают [path-spec § Weighting node selection].

Каждый вес называется `W`*pt*: *p* — позиция (`g` для входного, `m` для промежуточного, `e` для выходного, `b` для запросов каталога), а *t* — тип (`g`, `e`, `d` для совмещения входной и выходной ролей, `m` при отсутствии обеих). Значения выражаются в десятитысячных долях: `Wmg=3805` означает, что при выборе входного узла в промежуточную позицию учитывается только 38,05 % его пропускной способности.

13.2.3. Веса реальной сети

Следующие веса взяты из публичного консенсуса от 23 сентября 2026 года, 15:00 UTC, загруженного из CollecTor в лабораторной работе 7:

Позиция \ тип	guard (`g`)	middle (`m`)	exit (`e`)	guard+exit (`d`)
guard (`Wg`)	6195	6195	—	0
middle (`Wm`)	3805	10000	0	0
exit (`We`)	10000	10000	10000	10000

Следствия очевидны. Выходные узлы и реле, имеющие одновременно роли входного и выходного узла, имеют **нулевой** вес во входной и промежуточной позициях. В этой сети пропускная способность выходных узлов (33 % общей) настолько ценна, что резервируется для выходной позиции. Входных узлов больше: на них приходится 70 % пропускной способности; их вес равен 62 % на входной позиции и 38 % на промежуточной. Как покажет лаборатория, в 60 % цепей промежуточный узел сам имеет флаг `Guard`.

Формулы, по которым управляющие каталогами вычисляют эти веса, приведены в спецификации каталогов. Они различают несколько случаев в зависимости от того, пропускная способность какого типа наиболее дефицитна, и по возможности стремятся выровнять доступную ёмкость трёх позиций.

13.3. Входные узлы (guards)

13.3.1. Какую проблему они решают

Предположим, противник контролирует долю c пропускной способности возможных первых и выходных реле. Если клиент выбирает новые входной и выходной узлы случайно для каждой цепи, вероятность, что оба принадлежат противнику, составляет приблизительно c^2 на цепь. После C цепей вероятность хотя бы одной компрометации равна $1 - (1 - c^2)^C$ и стремится к 1 [guard-spec § Introduction and motivation]. Пользователь, запускающий Tor каждый день, строит тысячи цепей; рано или поздно противник почти наверняка увидит часть его трафика.

Идея **входных узлов (guards)**, называемых в исследовательской литературе *helper nodes*, состоит в том, чтобы выбрать первое реле *один раз* и долго его использовать. Если входной узел честен, ни одна цепь не скомпрометирована таким способом; если он принадлежит противнику, компрометированы многие. Вероятность компрометации *хотя бы один раз* снижается с почти 1 примерно до c.

Число цепей	Без фиксированного узла	С фиксированным входным узлом
1	0.0025	0.0025
100	0.221	0.050
1000	0.918	0.050
10 000	1.000	0.050

Таблица, рассчитанная в лабораторной работе 7 для $c = 5\,\%$, показывает компромисс. Входные узлы не исключают компрометацию; они превращают событие, которое со случайными первыми реле со временем стало бы почти неизбежным, в редкое событие, затрагивающее небольшую долю пользователей вместо почти всех. Модель намеренно упрощена и, например, не учитывает смену выбранного guard со временем.

13.3.2. Три, один, два

Входные узлы появились в 2006 году после того, как Øverlier и Syverson показали, как локализовать onion-сервис, многократно заставляя его подключаться к контролируемым противником реле [ØS06]. Много лет клиенты использовали три входных узла, меняя их каждые два-три месяца. Последующие исследования показали, что три входных узла и частая ротация увеличивают экспозицию. Предложение 236 (2014) предложила один входной узел, сохраняемый девять-десять месяцев. Но один входной узел делает связность более хрупкой и облегчает принудительную замену. Предложение 291 (2018) перешла к **двум** входным узлам. Встроенное значение по умолчанию у tor по-прежнему равен одному (`DFLT_N_PRIMARY_GUARDS_TO_USE` в `src/feature/client/entrynodes.h`), но публичный консенсус содержит `guard-n-primary-guards-to-use=2`, переопределяя его для всех клиентов. Это наглядный пример управления сетью через консенсус.

13.3.3. Алгоритм

Современный алгоритм, описанный в Предложении 271 и спецификации входных узлов, управляет несколькими наборами узлов [guard-spec § The algorithm]:

Выборка входных узлов Набор случайно выбранных с учётом пропускной способности входных узлов, сохраняемый на диске. Он растёт только при необходимости до максимума 60 реле или 20 % числа входных узлов в консенсусе. Ограничение выборки мешает противнику заставить клиента последовательно попробовать каждый входной узел сети, пока тот не достигнет узла противника.

Отфильтрованные входные узлы Узлы из выборки, всё ещё присутствующие в консенсусе и пригодные при текущей конфигурации, например при наличии сетевого экрана, разрешающего только некоторые порты.

Подтверждённые входные узлы Узлы, реально использованные для пользовательского трафика, в порядке предпочтения.

Основные входные узлы Обычно три узла из отфильтрованного списка: сначала подтверждённые и отфильтрованные, затем при необходимости ещё не подтверждённые, но допустимые. Клиент использует первый доступный основной входной узел или первые два согласно параметру консенсуса. Если они недоступны, клиент временно пробует другие узлы.

Срок пребывания входного узла в выборке зависит от его состояния. Входной узел, который **никогда не подтверждался** (`CONFIRMED_ON_DATE` не задан), удаляется через `GUARD_LIFETIME`, то есть 120 дней после выборки. **Подтверждённый** входной узел, реально используемый клиентом, удаляется через `GUARD_CONFIRMED_MIN_LIFETIME` = 60 дней после подтверждения. Он также удаляется, если отсутствует в консенсусе 20 дней. Сохраняемые временные метки (`ADDED_ON_DATE` и `CONFIRMED_ON_DATE`) случайно сдвигаются назад максимум на `GUARD_LIFETIME`/10 = 12 дней. Поэтому файл состояния не раскрывает точный момент начала использования Tor, а фактический срок службы подтверждённого входного

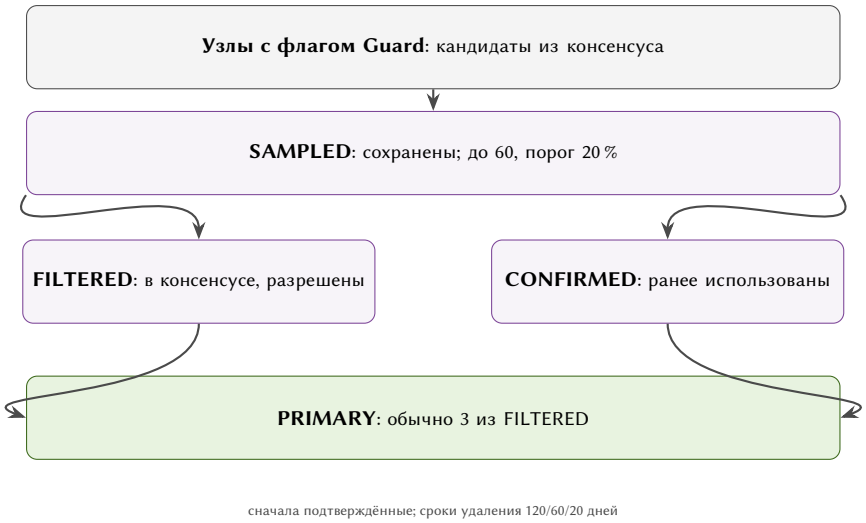


Рис. 13.1: Связи между списками сторожевых узлов Tor: постоянная выборка формируется из узлов консенсуса, а FILTERED и CONFIRMED — разные, потенциально пересекающиеся подмножества выборки. PRIMARY выбирают только из FILTERED, отдавая приоритет подтверждённым, а затем остальным допустимым узлам. Ограничение выборки снижает риск контакта с новыми узлами.

узла становится 48–60 дней. Таким образом, примерно раз в два месяца сменяется именно *подтверждённый* входной узел, а не раз в четыре. Эта деталь важна для симуляции в гл. 29.

Внимание: Новый входной узел --- это сигнал

Tor тщательно различает *недоступный* входной узел и *неудачную* цепь. Если бы клиент менял входные узлы при каждом отказе цепи, противник мог бы намеренно ломать цепи через честные входные узлы, пока клиент не выберет враждебный. Именно эту проблему исправило обновление безопасности tor 0.4.9.13 от 23 сентября 2026 года: теперь клиент относит отказ к входному узлу только когда не может установить соединение, а не при сбоях цепей, потоков или запросов каталога.

13.4. Vanguards

Входные узлы защищают *первый* переход. Onion-сервисы (гл. 14) сталкиваются с дополнительной проблемой: вредоносный клиент может заставлять сервис строить сколько угодно цепей к выбранным атакующим точкам встречи. Если промежуточный узел каждой цепи выбирается заново случайно, рано или поздно он окажется у противника, который увидит входной узел сервиса. Следующим шагом может стать атака на этот входной узел или получение его данных. **Vanguards** расширяют идею входных узлов на последующие переходы, также фиксируя реле второго уровня, а в некоторых конфигурациях и третьего.

Vanguards-lite (Предложение 333, tor 0.4.7.1-alpha) Включён по умолчанию у клиентов и onion-сервисов. Использует четыре реле второго слоя, не сохраняемые на диск, каждое со случайным сроком службы. Здесь код и спецификация расходятся. tor 0.4.9.13 использует диапазон 1–12 дней, в среднем около недели, заданный `MIN_SECOND_GUARD_LIFETIME` и `MAX_SECOND_GUARD_LIFETIME` в `src/feature/client/entrynodes.c`. Параметры консенсуса `guard-hs-l2-lifetime-min` и `-max` могут переопределить значения, но ни одного из них не было в консенсусе 23 сентября 2026 года. Спецификация vanguards,

напротив, всё ещё указывает 1–22 дня, в среднем две недели. Режим рассчитан на сервисы и пользователей, активных не дольше нескольких недель.

Full vanguards Реле фиксируются на двух уровнях, втором и третьем переходах, с более длительными сроками; предназначено для долго работающих onion-сервисов высокого риска. В tor реализуется через дополнение *vanguards*, задающее `HSLayer2Nodes` и `HSLayer3Nodes`. Arti реализует режим непосредственно, имея `Full` наряду с `Lite`.

В лабораторной сети (разд. 19.7.3) мы видели цепи с назначением `HS_VANGUARDS`: заранее построенные цепи с vanguards, готовые к расширению до точки введения или встречи.

13.5. Лабораторная работа: исследование публичной сети

Лабораторная работа 7: Выбор маршрута с учётом пропускной способности

Цель. Написать `labs/ru/код/torlab/path_selection.py` и использовать его для анализа реального консенсуса публичной сети.
Предварительные условия. Лабораторная работа 6 (`directory.py`). Публичный консенсус, загруженный из CollecTor, публичного архива Tor Project (`https://collector.torproject.org/`). Эти данные публикуются специально для исследований; их загрузка не затрагивает пользователей сети.
Оценочное время. 2 часа.

Шаг 1: вес реле.

```
def _relay_type(r):  # g = только Guard, e = только Exit, d = оба, m = ни одного
    g, e = "Guard" in r.flag, "Exit" in r.flag and "BadExit" not in r.flag
    return {(1, 1): "d", (1, 0): "g", (0, 1): "e", (0, 0): "m"}[(g, e)]

def weight(r, position, weights):
    key = f"W{position[0]}{_relay_type(r)}          # например "Wmg"
    return (r.bandwidth or 0) * weights.get(key, 10000) / 10000
```

Шаг 2: выбор маршрута. Класс `Selector` один раз вычисляет веса для каждой позиции. Ограничения применяются *отклонением*: выбирается реле, и если оно нарушает ограничение, например совпадает с уже выбранным или находится в том же /16, выбирается другое.

```
def path(self, rng, guard=None):
    e = self.choose("exit", [], rng)
    g = guard or self.choose("guard", [e], rng)
    m = self.choose("middle", [e, g], rng)
    return g, m, e
```

Первая версия, которая для каждого выбора фильтровала более девяти тысяч реле и заново считала веса и сети, строила 2000 маршрутов за 310 секунд. Эта версия строит 20 000 за 7 секунд. В статистическом моделировании производительность не является мелкой деталью.

Шаг 3: публичная сеть в числах. Для консенсуса от 23 сентября 2026 года, 15:00 UTC:

```
реле: 9358
пропускная способность с Guard: 70.1%    пропускная способность с Exit: 33.1%
guard : подходят 5306; первые 10 = 1.7%; первые 100 = 12.6%
middle: подходят 8961; первые 10 = 2.8%; первые 100 = 13.7%
```

```
exit  : подходят 3126; первые 10 = 2.3%; первые 100 = 13.7%
20000 маршрутов: gge 30.9%  ggd 29.4%  gme 20.4%  gmd 19.3%
```

Число *подходящих* реле меньше числа с соответствующим флагом: гл. 6 сообщает 3177 реле `Exit`, а здесь их 3126. Функция `eligible` также требует `Running`, `Valid` и `Fast`, исключая `BadExit` и `MiddleOnly`. Разница состоит из 48 выходных реле без `Fast` и 3 с `BadExit`. Последняя строка — выборка с фиксированным зерном генератора 7. Без фиксированного зерна генератора значения при каждом запуске меняются на несколько десятых процентного пункта, и два эксперимента нельзя корректно сравнивать. Буквенные коды обозначают типы guard, middle и exit в маршруте. Например, `gme` означает guard, middle без обоих флагов и выходной узел без флага Guard. Как и предсказывают веса, ни один маршрут не использует выходной узел на первой или второй позиции; в 60 % случаев промежуточный узел имеет флаг `Guard`. Концентрация невелика: десять наиболее часто выбираемых выходных узлов вместе дают 2,3 % выборов, а первые сто — 13,7 %.

Шаг 4: входные узлы и противники. Класс `Adversary` вычисляет таблицу из разд. 13.3. Измените долю противника: остаётся ли эффект входных узлов очевидным при $c = 1\,\%$ или $c = 20\,\%$?

Итоги

- tor сначала выбирает выходной, затем входной и промежуточный узлы, избегая повторения одного реле и реле из одной подсети /16 или одного семейства.
- Каждое реле выбирается с вероятностью, пропорциональной пропускной способности, умноженной на вес, зависящий от типа реле и позиции цепи; в рассмотренной сети выходные узлы используются только на выходной позиции.
- Входные узлы превращают компрометацию, которая со случайными первыми реле со временем становится почти неизбежной, в редкое событие; сейчас клиенты используют два входных узла.
- Алгоритм выбора входных узлов поддерживает ограниченную выборку, подтверждённые и основные узлы и осторожно связывает отказы с конкретным входным узлом.
- Vanguards фиксируют также второй слой onion-сервисов; vanguards-lite включён по умолчанию.
- Параметры консенсуса могут переопределять значения программы по умолчанию.

Упражнения

Упражнение 13.1 ⋆ Почему tor выбирает выходной узел первым? Что могло бы пойти не так, если сначала выбрать входной узел?

Упражнение 13.2 ⋆ Реле имеет флаги `Guard` и `Exit` и пропускная способность в консенсусе 20000. Каков его вес в каждой из трёх позиций цепи при весах реальной сети, приведённых в этой главе?

Упражнение 13.3 ⋆⋆ Повторите расчёт таблицы для входных узлов для $c = 1\,\%$ и $c = 20\,\%$. После какого числа цепей вероятность компрометации без фиксированных входных узлов превышает 50 % в каждом случае?

Упражнение 13.4 ⋆⋆ Противник хочет повысить вероятность выбора его реле в качестве выходных узлов. Какие ресурсы ему потребуется вложить и какие механизмы — измерение пропускной способности, семейства, ограничения на один IP-адрес и веса — усложняют задачу?

Упражнение 13.5 ⋆⋆ Почему выборка входных узлов ограничена 60 реле или 20 % их общего числа? Опишите атаку, которую предотвращает это ограничение.

Упражнение 13.6 ⋆ ⋆ ⋆ Расширьте `Selector`, добавив ограничение по семействам по строкам `family` и `family-ids` микродескрипторов. Загрузите из CollecTor микродескрипторы, соответствующие консенсусу, и измерьте, сколько выборов отклоняется из-за принадлежности к семействам.

Упражнение 13.7 ⋆ ⋆ ⋆ Сгруппируйте реле публичного консенсуса по заявленным семействам и вычислите долю вероятности выбора выходного узла для десяти крупнейших семейств. Сравните с долей десяти индивидуально наиболее часто выбираемых выходных узлов. Что можно заключить о рисках операторов, управляющих многими реле?

Дополнительная литература

- *Tor Path Specification* и *Tor Guard Specification* [Thee].
- L. Øverlier и P. Syverson, *Locating Hidden Servers* [ØS06]: атака, послужившая причиной введения входных узлов.
- Предложения 236, 271, 291 и 333: переход от одного входного узла к двум, современный алгоритм и vanguards-lite.
- Спецификация vanguards (vanguards-spec), включая таблицы временных шкал Sybil-атак.

Глава 14

Onion-сервисы v3

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; rend-spec v3

Цели главы

К концу этой главы вы сможете:

- объяснять, что предоставляет onion-сервис и чем он отличается от сайта, доступ к которому осуществляется через exit-реле;
- получать открытый ключ сервиса из его адреса `.onion` и выполнять обратное преобразование;
- описывать ослеплённые ключи, временные периоды и способ, которым клиент определяет ответственные HSDir;
- разбирать многоуровневую структуру дескриптора;
- пошагово прослеживать протокол введения и встречи (introduction и rendezvous);
- загружать, проверять и расшифровывать дескриптор onion-сервиса в лабораторной сети (Лабораторная работа 8).

До сих пор мы использовали Tor, чтобы скрывать *того, кто* подключается к серверу. Сам сервер при этом оставался публичным: его IP-адрес был известен, а exit-реле подключалось к нему напрямую через публичную сеть и знало адрес назначения. Содержимое передавалось открытым текстом только в том случае, если приложение не использовало сквозное шифрование, например HTTPS. **Onion-сервисы** меняют эту схему. Они позволяют предоставлять сервис, например веб-сайт, не раскрывая его сетевое местоположение. Клиент и сервис встречаются на промежуточной точке, каждый через собственную Tor-цепь, и ни одна сторона не узнаёт адрес другой.

14.1. Что предоставляет onion-сервис

Анонимность сервиса IP-адрес сервера никогда не раскрывается: сервис взаимодействует с сетью только через Tor-цепи, которые сам инициирует.

Самоподтверждаемые адреса Адрес `.onion` *кодирует* открытый ключ сервиса вместе с контрольной суммой и байтом версии. Любой, кто подключается по этому адресу, криптографически убеждается, что взаимодействует с владельцем соответствующего закрытого ключа, без участия центров сертификации.

Сквозное шифрование Трафик между клиентом и сервисом шифруется ключами, известными только этим двум сторонам; exit-реле, способного наблюдать незашифрованный трафик, нет.

Доступность Сервису не требуется принимать входящие соединения. Он может работать за NAT или межсетевым экраном, разрешающим только исходящие соединения.

Цена этого подхода — длина маршрута. В базовой модели клиент достигает точки встречи по трёхузловой цепи, а сервис приходит к ней по другой трёхузловой цепи: всего шесть реле вместо трёх при подключении к обычному веб-сайту. При включённом по умолчанию vanguards-lite (разд. 13.4) маршрут перестаёт быть симметричным. Сервис добавляет фиксированное реле второго уровня, поэтому его цепь на стороне rendezvous содержит четыре перехода ($\texttt{S} \to \texttt{G} \to \texttt{L2} \to \texttt{M} \to \texttt{Rend}$), тогда как у клиента их три.

Историческая справка: От v2 к v3

Hidden services существуют с 2004 года. Первая широко развёрнутая версия, v2, использовала 16-символьные адреса, полученные из 1024-битного RSA-ключа путём усечения SHA-1-дайджеста до 80 бит. Кроме того, она позволяла HSDir собирать адреса сервисов, дескрипторы которых они хранили. Версия 3 (Предложение 224), доступная начиная с tor 0.3.2.1-alpha (2017), ввела ключи Ed25519, 56-символьные адреса, содержащие весь открытый ключ, зашифрованные дескрипторы и ослеплённые ключи. Сервисы v2 были полностью отключены в октябре 2021 года в tor 0.4.5.11 и 0.4.6.8. Изменилось и название: сегодня используется термин *onion-сервис*.

14.2. Адрес

Адрес v3 представляет собой base32-кодирование открытого идентификационного ключа сервиса, короткой контрольной суммы и номера версии [rend-spec § Encoding onion addresses]:

$$\texttt{address} = \text{base32}(PUBKEY \,\|\, CHECKSUM \,\|\, VERSION) \,\|\, \texttt{".onion"}$$

Здесь $PUBKEY$ — 32-байтовый ключ Ed25519, $VERSION$ — байт `0x03`, а $CHECKSUM$ состоит из первых 2 байт значения $\text{SHA3-256}(\texttt{".onion checksum"} \,\|\, PUBKEY \,\|\, VERSION)$. Получившиеся 35 байт кодируются ровно в 56 символов base32. Контрольная сумма не защищает от атакующего, поскольку он может вычислить её так же, как и любой другой, но позволяет обнаруживать ошибки ввода.

В одном из запусков лабораторной сети onion-сервис имел следующий адрес. Это адрес примера, сохранённого в `labs/ru/код/torlab/test/data/` и используемого лабораторными тестами. Поэтому все выводы в этом разделе относятся к этому примеру и могут быть воспроизведены без доступа к сети. Когда читатель запускает собственную сеть, она генерирует новые ключи и, следовательно, другой адрес (разд. 19.6).

```
wdjtp7qktdjkkgjjbbx2axvg2gri5p4tuhlc4bq23n7o3uwu4rveppqd.onion
```

Функция `key_from_address` из Лабораторной работы 8 извлекает из этого адреса открытый ключ и проверяет его контрольную сумму.

Внимание: Адрес нельзя надёжно узнавать по памяти

Случайный 56-символьный адрес трудно запомнить и непросто проверить беглым взглядом. Пользователи копируют его из источника, которому доверяют, поэтому адрес, отличающийся всего несколькими символами, может остаться незамеченным. Это основной вектор *фишинга* против onion-сервисов. *Адреса с заданным префиксом (vanity addresses)*, создаваемые перебором миллионов ключей до тех пор, пока начальные символы адреса не образуют слово, помогают лишь частично: атакующий тоже может сгенерировать адрес с тем же префиксом. Мы вернёмся к этой проблеме в гл. 21.

14.3. Ослеплённые ключи

14.3.1. Проблема

Сервис должен где-то публиковать информацию о том, как к нему подключиться: *дескриптор*. Если бы дескриптор публиковался непосредственно под идентификационным ключом сервиса, любой оператор HSDir видел бы адреса всех сервисов, чьи дескрипторы он хранит. Он мог бы перечислять такие сервисы или связывать дескрипторы одного и того же сервиса во времени. Именно так было в v2.

14.3.2. Решение

В v3 сервис подписывает дескрипторы с помощью **ослеплённого ключа**, отдельного для каждого *временного периода* [rend-spec § Appendix A: Signature scheme with key blinding]. Имея идентификационный ключ $A = aB$ (где B — базовая точка Ed25519), сервис вычисляет множитель

$$h = \text{SHA3-256}(\texttt{"Derive temporary signing key\0"} \,\|\, A \,\|\, B \,\|\, N),$$
$$N = \texttt{"key-blind"} \,\|\, period \,\|\, period_length$$

(при этом некоторые биты принудительно устанавливаются так же, как для ключей Ed25519), после чего ослеплённый ключ определяется как

$$A' = h \cdot A.$$

У этой конструкции есть важные свойства:

- любой, кто знает адрес и, следовательно, A, может вычислить A' для каждого периода и понять, где искать дескриптор;
- тот, кто видит только A', например HSDir, не может восстановить A или определить, что два ослеплённых ключа из разных периодов принадлежат одному сервису;
- сервис может подписывать данные от имени A' с помощью закрытого ключа $a' = h \cdot a$, и такая подпись проверяется относительно A' как обычная подпись Ed25519.

Необычная деталь хорошо показывает, насколько близко спецификация следует реализации: константная строка содержит завершающий нулевой байт, потому что tor вычисляет её длину через `sizeof`, а не `strlen`. Комментарий в исходном коде (`src/feature/hs/hs_common.c`) прямо признаёт, что задокументировать этот «сюрприз» в спецификации оказалось предпочтительнее, чем менять код.

14.3.3. Креденциалы и ключи, хранимые вне сети

Из идентификационного ключа выводится *креденциал (credential)*: SHA3-256(`"credential"` $\| A$). Для каждого периода из него, в свою очередь, получается *производный креденциал (subcredential)*: SHA3-256(`"subcredential"` $\| credential \| A'$). Производный креденциал используется при шифровании дескриптора и в протоколе введения: он подтверждает знание адреса, которым HSDir не обладает.

Иерархия ключей также спроектирована так, чтобы идентификационный ключ мог оставаться *офлайн*. Ослеплённый ключ каждого периода сертифицирует краткоживущий ключ подписи дескриптора (сертификат Ed25519 типа 8, табл. 7.1). Теоретически сервер может заранее получать ключи подписи и сертификаты для будущих периодов, вообще не располагая ключом, соответствующим его адресу. Не все реализации поддерживают такой режим.

14.4. Определение местоположения дескриптора

14.4.1. Временные периоды

Время делится на периоды длиной в один день в публичной сети (параметр консенсуса `hsdir-interval`) [rend-spec § Dividing time into periods]. Периоды начинаются не в полночь, а в 12:00 UTC. Они смещены на 12 часов относительно протокола совместной генерации случайных значений авторитетными серверами каталогов, который каждый день в полночь публикует в консенсусе новое **совместное случайное значение** (SRV). Номер периода вычисляется из количества минут, прошедших с 1 января 1970 года, за вычетом смещения, делённого на длину интервала. Спецификация приводит пример: в 11:15:01 UTC 13 апреля 2016 года номер временного периода равен 16903; этот период начался в полдень предыдущего дня. Лабораторная работа 8 проверяет именно этот пример.

В тестовых сетях, где голосование проводится каждые 20 секунд, ждать целый день было бы неудобно. Поэтому tor сокращает период до одного полного раунда протокола SRV, то есть до 24 голосований (функция `get_time_period_length`). В лабораторной сети один период длится 8 минут и смещён на 4 минуты.

14.4.2. Кольцо HSDir

Какой HSDir хранит дескриптор? Каждое реле с флагом `HSDir` занимает позицию в 256-битном *кольце* [rend-spec § Where to publish a hidden service descriptor]:

$$relay_index = \text{SHA3-256}(\texttt{"node-idx"} \,\|\, Ed25519\ identity \,\|\, SRV \,\|\, period \,\|\, period_length)$$

Для каждой из двух *реплик* сервис вычисляет индекс:

$$service_index(r) = \text{SHA3-256}(\texttt{"store-at-idx"} \,\|\, A' \,\|\, r \,\|\, period_length \,\|\, period).$$

Сервис загружает свой дескриптор на первые четыре HSDir, следующие за каждым индексом по кольцу (параметр `hsdir_spread_store`); клиент случайным образом выбирает один из первых трёх (`hsdir_spread_fetch`). Включение SRV не позволяет атакующему задолго заранее предсказать, какие реле будут отвечать за конкретный сервис, и заранее разместить вредоносные реле в нужных позициях для цензуры сервиса или наблюдения за запросами дескриптора. Сервис одновременно публикует два дескриптора для двух последовательных периодов, чтобы оставаться доступным для клиентов, чьи консенсусы немного опережают или отстают. Какие именно два периода используются, зависит от точки в цикле. До или после ротации SRV это либо предыдущий и текущий периоды, либо текущий и следующий [rend-spec § Generating and publishing hidden service descriptors].

Подробнее: Какой SRV следует использовать?

Поскольку периоды и SRV смещены друг относительно друга на половину суток, клиент должен аккуратно выбирать SRV. В половине суток между началом периода и следующим SRV используется текущий SRV; в другой половине — предыдущий. Если в консенсусе отсутствует нужный SRV, например сразу после запуска сети, tor вычисляет запасное значение — *резервное значение SRV (disaster SRV)*: SHA3-256(`"shared-random-disaster"` $\|$ *period_length* $\|$ *period*). Выбор реализован в `hs_get_current_srv` и `hs_get_previous_srv` (`src/feature/hs/hs_common.c`). Если значение отсутствует в консенсусе, tor без дополнительного предупреждения использует резервное значение SRV. Это происходит в только что созданной лабораторной сети и длится недолго: в журнале авторитетного сервера нашей сети первый SRV появляется через 115 секунд после запуска; до этого в консенсусе нет строки `shared-rand-current-`

`value`. Чтобы увидеть использование резервного значения, исследуйте первые несколько минут после создания сети. После некоторого времени работы сети, как в разд. 14.9, используются настоящие SRV.

14.4.3. Только анонимные запросы

Клиент загружает дескриптор HTTP-запросом к `/tor/hs/3/`, за которым следует ослеплённый ключ в base64 без завершающих символов `=`, через поток `BEGIN_DIR` к HSDir. Есть одно условие: запрос должен поступать по **анонимной** цепи. HSDir отклоняет запрос, пришедший по цепи, открытой клиентом напрямую, то есть по цепи из одного перехода. Мы обнаружили это в лаборатории:

```
HTTP/1.0 503 Rejecting single hop HS v3 descriptor request
```

Проверка находится в `connection_dir_is_anonymous` (`src/feature/dircommon/directory.c`). Она не позволяет HSDir связать запрос дескриптора с IP-адресом запрашивающего клиента.

14.5. Дескриптор

Дескриптор состоит из трёх уровней [rend-spec § Generating and publishing hidden service descriptors], показанных на рис. 14.1.

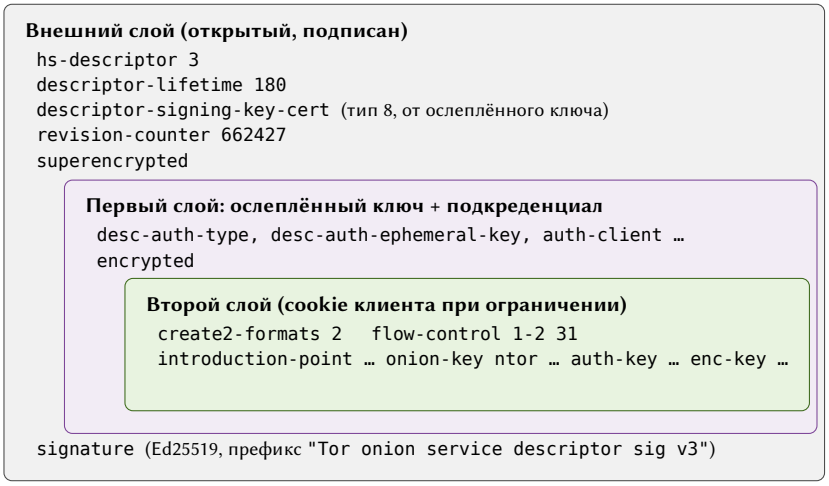


Рис. 14.1: Структура дескриптора onion-сервиса v3. Подписанная внешняя оболочка читается HSDir; первый зашифрованный слой раскрывается при знании идентичности сервиса. При ограниченном обнаружении разрешённый клиент использует ключ X25519 для извлечения cookie дескриптора, необходимого для расшифрования второго слоя.

Внешний уровень передаётся открытым текстом и подписывается. Он содержит версию, срок действия (`descriptor-lifetime`, 180 минут в лабораторной сети), сертификат ключа подписи, выданный ослеплённым ключом, счётчик ревизий, зашифрованный блок `superencrypted` и подпись. Подпись — Ed25519, созданная ключом подписи дескриптора по тексту с префиксом `"Tor onion service descriptor sig v3"`. Префикс не позволяет подписи, созданной для дескриптора, оказаться допустимой для другого типа данных. Счётчик ревизий должен увеличиваться с каждой новой версией, чтобы HSDir не принимал дескрипторы старее уже сохранённых.

Первый зашифрованный уровень шифруется ключом, выведенным из ослеплённого ключа и производного креденциала. HSDir, не знающий адреса, не может прочитать этот уровень. В нём находятся поля *ограниченного обнаружения (restricted discovery)* (разд. 14.7) и второй уровень. Чтобы не раскрывать сам факт включения ограниченного обнаружения, эти поля присутствуют всегда и заполняются случайными данными, когда они не нужны.

Второй уровень содержит сведения, необходимые для подключения: поддерживаемые форматы рукопожатия, параметры управления потоком, возможные параметры защиты от отказа в обслуживании (DoS) и, для каждой **точки введения** — спецификаторы соединения для доступа к ней (в base64), ключ ntor, сертифицированный ключ аутентификации и ключ шифрования. По умолчанию сервис использует три точки введения.

Оба уровня используют одну и ту же схему. Случайная 16-байтовая соль, секретные данные и константа, специфичная для уровня, подаются на SHAKE-256, чтобы вывести ключ AES-256, вектор инициализации и ключ аутентификации. Текст шифруется в режиме счётчика и аутентифицируется с помощью SHA3-256. Открытый текст первого уровня дополняется до длины, кратной 10 000 байт, а строки `auth-client` — до количества, кратного 16, чтобы размер дескриптора раскрывал как можно меньше информации о числе точек введения и авторизованных клиентов.

14.6. Введение и встреча (rendezvous)

Получив дескриптор, клиент может подключиться. Протокол, показанный на рис. 14.2, использует три «специальные» роли: точку введения, выбранную сервисом, точку встречи, выбранную клиентом, и серверы HSDir.

1. **Сервис подготавливает точки введения.** Он выбирает несколько реле, строит цепь к каждому и отправляет `ESTABLISH_INTRO` (команда реле 32), содержащую отдельный ключ аутентификации и подпись для каждой точки. Реле отвечает `INTRO_ESTABLISHED` (38); после этого оно пересылает сервису запросы, содержащие соответствующий ключ.

2. **Сервис публикует дескриптор** на ответственных HSDir, как описано выше.

3. **Клиент подготавливает точку встречи.** Он случайным образом выбирает реле в качестве точки встречи, строит к нему цепь и отправляет `ESTABLISH_RENDEZVOUS` (33) со случайным 20-байтовым *идентификатором (cookie)*. Реле отвечает `RENDEZVOUS_ESTABLISHED` (39).

4. **Клиент представляет себя сервису.** Он строит цепь к точке введения и отправляет `INTRODUCE1` (34). Открытая часть указывает ключ аутентификации точки введения. Зашифрованная часть, доступная только сервису, содержит идентификатор cookie, спецификаторы соединения и ключ ntor точки встречи, а также первую часть рукопожатия. Точка введения отправляет клиенту `INTRODUCE_ACK` (40) и пересылает сообщение сервису как `INTRODUCE2` (35).

5. **Сервис достигает точки встречи.** Он расшифровывает `INTRODUCE2`, проверяет, не дублирует ли сообщение уже обработанный запрос (*кеш защиты от повторов (replay cache)*), строит цепь к указанной точке встречи и отправляет `RENDEZVOUS1` (36) с идентификатором cookie и второй частью рукопожатия.

6. **Точка встречи соединяет цепи.** Она находит клиентскую цепь по тому же идентификатору cookie и пересылает вторую часть рукопожатия в сообщении `RENDEZVOUS2` (37), после чего начинает копировать ячейки между двумя цепями.

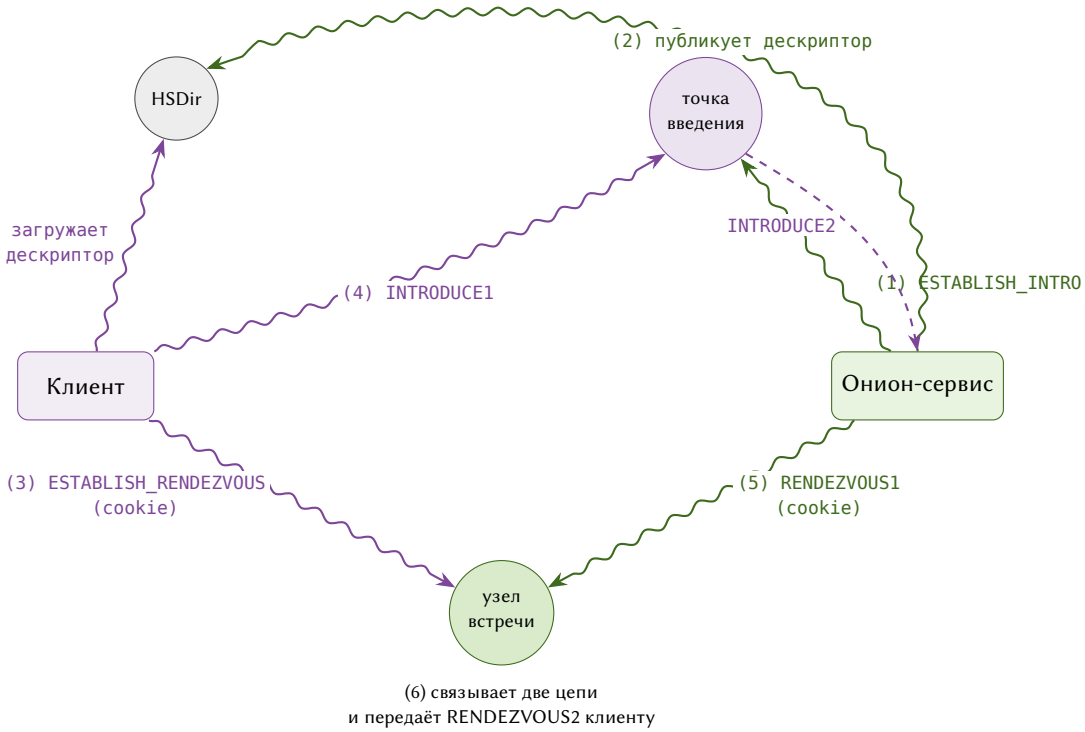


Рис. 14.2: Протокол встречи (rendezvous). Каждая стрелка представляет многоузловую цепь; прямого соединения между клиентом, сервисом и промежуточными точками нет. Числа показывают порядок шагов, описанных в тексте.

Рукопожатие между клиентом и сервисом, называемое *hs-ntor*, является вариантом ntor (разд. 9.6), в который включены производный креденциал и ключ аутентификации точки введения. Только сторона, знающая адрес, может сформировать `INTRODUCE1`, которое сервис примет. Выведенные ключи добавляют ещё один слой шифрования, подобно *виртуальному переходу*, добавленному к клиентской цепи, и используют SHA3-256 для накопительного дайджеста вместо SHA-1 (разд. 10.2). Ни точка введения, ни точка встречи не могут читать трафик.

Через объединённую цепь клиент открывает потоки командой `BEGIN`, как при работе через exit-реле (гл. 11). Сервис перенаправляет эти потоки на локальные порты, настроенные директивой `HiddenServicePort`.

Подробнее: Зачем нужно столько реле?

Точка введения знает сервис только как «ключ аутентификации некоторой цепи»; она не знает, какому адресу `.onion` служит. Точка встречи видит случайный идентификатор cookie и две цепи; она не знает, какой сервис соединяет с каким клиентом. Каждая роль видит лишь фрагмент картины, и ни одна не знает IP-адрес клиента или сервиса, которые остаются скрыты за соответствующими входными узлами (guards).

14.7. Ограниченное обнаружение (restricted discovery)

Сервис может быть предназначен только для определённых клиентов, например если это административная панель. При использовании **ограниченного обнаружения (restricted discovery)** (ранее *авторизации клиентов*) каждый авторизованный клиент имеет пару ключей X25519, а сервис знает её открытую часть. В строках `auth-client` первого уровня сервис отдельно шифрует для каждого клиента ключ второго уровня дескриптора. Тот, кто знает адрес, но не располагает авторизованным ключом, не может прочитать список точек введения и, следовательно, даже попытаться подключиться. В tor это настраивается добавлением открытых ключей клиентов в каталог `authorized_clients` сервиса.

14.8. Защитные механизмы

Onion-сервисы являются естественными целями атак: сервис не может просто блокировать IP-адреса атакующих клиентов, поскольку не видит их. Основные защитные механизмы, подробнее рассматриваемые в других разделах, таковы:

- **vanguards**, защищающие guard-реле сервиса от атак, основанных на цепях, создаваемых по запросу (разд. 13.4);
- **кеш защиты от повторов (replay cache)**, не позволяющий атакующему заставлять сервис повторно выполнять одну и ту же работу;
- параметры защиты от DoS, которые сервис может задавать на точках введения через расширение `ESTABLISH_INTRO`, ограничивая количество сообщений `INTRODUCE2` в секунду;
- **доказательство выполнения работы (proof of work)**, при котором атакуемый сервис требует от клиентов решить небольшую вычислительную задачу перед обслуживанием (гл. 15).

14.9. Лабораторная работа: поиск и чтение дескриптора

Лабораторная работа 8: Дескриптор onion-сервиса лабораторной сети

Цель. Реализовать `labs/ru/код/torlab/onion.py`: обработку адресов, ослеплённого ключа, кольца HSDir, загрузку, проверку и расшифрование дескриптора.
Предварительные условия. Лабораторные работы 1–6; лабораторная сеть должна быть запущена.
Оценочное время. 4 часа.

Шаг 1: ослеплённый ключ. Распространённые криптографические библиотеки поддерживают подписи Ed25519, но не умножение точки на скаляр, необходимое для вычисления $A' = h \cdot A$. Поэтому модуль содержит небольшую реализацию арифметики Ed25519 — около тридцати строк, основанных на формулах RFC 8032. Она достаточна для этой задачи, но непригодна для общего использования: работает медленно и не защищает от атак по времени выполнения.

```
def blinding_factor(A, period, duration_minutes):
    n = b"key-blind" + struct.pack(">QQ", period, duration_minutes)
    h = bytearray(hashlib.sha3_256(
        b"Derive temporary signing key\x00" + A + BASE_STR + n).digest())
    h[0] &= 248; h[31] &= 63; h[31] |= 64
    return int.from_bytes(h, "little")

def blinded_key(A, period, duration_minutes):
    return _encode_point(_multiply(blinding_factor(A, period, duration_minutes),
                                   _decode_point(A)))
```

Шаг 2: период и SRV. Функция `time_period_info` воспроизводит вычисление tor на основе метки времени `valid-after` и интервала голосования из консенсуса, определяя, нужен текущий или предыдущий SRV. Если в консенсусе нет требуемого значения, используется резервное значение SRV.

Шаг 3: трассировка ошибки. При первом запуске наш код не нашёл дескриптор ни на одном из восьми HSDir сети. Поскольку мы проверили их все, ошибка выбора HSDir не могла объяснить проблему. Оставались две возможности: неверный ослеплённый ключ или неверный запрос. Чтобы разделить их, мы заставили настоящий клиент сети загрузить дескриптор через управляющий порт (`HSFETCH`, затем `GETINFO hs/client/desc/id/...`). Из полученного сертификата мы прочитали ослеплённый ключ, реально использованный сервисом: он совпал с нашим. Следовательно, проблема была в запросе, и исходный код tor показал причину: HSDir отклоняют запросы по односкачковым цепям. После построения трёхузловой цепи к HSDir мы получили дескриптор. Это универсальный метод: когда эксперимент не работает, ищите промежуточную точку, в которой ваш результат можно сравнить с эталонной реализацией.

Шаг 4: проверка и расшифрование. В примере `path` обозначает список из трёх объектов `RelayInfo`, полученных из лабораторной сети, с HSDir последним; `A` — ключ идентичности, `Ap` — ослеплённый ключ для этого периода. Создание объектов здесь не показано.

```
d = download_descriptor(Ap, path)
desc = parse_descriptor(d, A, Ap)
```

`parse_descriptor` проверяет, что сертификат ключа подписи был выдан ожидаемым ослеплённым ключом, проверяет подпись дескриптора и расшифровывает оба уровня. Сокращённый результат предыдущей лабораторной сессии:

```
срок 180 ревизия 662427 точек введения 3
create2-formats 2
flow-control 1-2 31
introduction-point AwAGfwAAARPyAhQXE5GoH+wp3MoxW9M1QV6XSxb0fgM...
onion-key ntor yRURxDb2eeEmF/p5M/bmHA+giwVyxjApKYyNY1KkCDo=
auth-key
-----BEGIN ED25519 CERT-----
...
```

Строка `flow-control 1-2 31` сообщает клиентам, что сервис поддерживает управление перегрузкой с тем же значением `sendme_inc`, равным 31, которое встречалось в гл. 11.

Шаг 5: проверка согласованности. В лабораторной сети восемь HSDir, а сервис загружает дескриптор на $2 \times 4 = 8$ из них, то есть на все. Поэтому само обнаружение дескриптора ещё не доказывает правильность вычисления кольца. Однако журнал клиента (`info.log`) фиксирует HSDir, к которому он обращался: `test002a` в 16:07. Это одно из трёх реле, возвращённых нашим кодом для данной реплики в тот момент. Одно наблюдение не является доказательством, но согласуется с расчётом. В упражнении 14.6 предлагается более строгий тест.

Итоги

- Onion-сервис предоставляет сервис, не раскрывая его IP-адрес; адрес `.onion` содержит и аутентифицирует открытый ключ сервиса.
- Дескрипторы подписываются ослеплёнными ключами, специфичными для периода; связать такой ключ с сервисом может только сторона, знающая адрес.
- Позиции дескриптора в кольце HSDir зависят от ослеплённого ключа, периода и совместного случайного значения; клиенты загружают дескрипторы через анонимные запросы.
- Дескриптор содержит подписанный внешний уровень и два зашифрованных уровня; второй перечисляет точки введения.
- Клиент и сервис встречаются в точке встречи, выбранной клиентом, после запроса через точку введения. Рукопожатие hs-ntor добавляет слой сквозного шифрования.
- Ограниченное обнаружение, vanguards, кеш повторных запросов, ограничение частоты запросов и доказательство выполнения работы защищают сервисы.

Упражнения

Упражнение 14.1 ⋆ Сколько байт кодирует адрес v3 и почему в результате получается ровно 56 символов base32? Какова вероятность того, что случайная ошибка ввода пройдёт проверку контрольной суммы?

Упражнение 14.2 ⋆ Вычислите номер временного периода публичной сети для 09:00 UTC 1 января 2026 года. Когда начался этот период?

Упражнение 14.3 ⋆⋆ Атакующий хочет заблокировать доступ к onion-сервису, добившись контроля над ответственными HSDir. Что ему необходимо предсказать и насколько заранее это возможно? Какую роль играет SRV?

Упражнение 14.4 ⋆⋆ Перечислите, что точка введения, точка встречи и HSDir знают о клиенте и сервисе. Какая из этих ролей была бы наиболее интересна атакующему, пытающемуся оценить популярность сервиса?

Упражнение 14.5 ⋆⋆ Почему HSDir отклоняет запросы дескриптора, поступающие по односкачковым цепям? Что в противном случае он мог бы узнать?

Упражнение 14.6 ⋆⋆⋆ Проверьте вычисление кольца более строго. Добавьте реле в лабораторную сеть (упражнение 19.7), пока в ней не станет больше восьми HSDir, а затем убедитесь, что дескриптор доступен *только* на HSDir, выбранных вашим кодом.

Упражнение 14.7 ⋆⋆⋆ Декодируйте спецификаторы соединения точки введения из лабораторного дескриптора (значение base64 в строке `introduction-point`). Какому реле лабораторной сети она соответствует? Что содержит сертификат в строке `auth-key`?

Дополнительная литература

- *Tor Rendezvous Specification - Version 3* (rend-spec) [Thee]: адреса, ослеплённые ключи, дескрипторы, introduction и rendezvous, включая тестовые векторы hs-ntor.
- Предложение 224: архитектура сервисов v3.
- Спецификация совместной генерации случайных значений (srv-spec): как авторитетные серверы каталогов формируют SRV.

Глава 15

Защита от DoS и управление ресурсами

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`; tor 0.4.9.13; dos-spec, hspow-spec

Цели главы

К концу этой главы вы сможете:

- объяснять, почему атака типа «отказ в обслуживании» против Tor одновременно угрожает анонимности;
- описывать, как tor защищает память реле, ресурсы процессора и пропускную способность;
- понимать ограничения числа соединений и цепей по IP-адресам и роль параметров консенсуса;
- объяснять, как onion-сервисы защищаются от массовых запросов введения;
- описывать механизм доказательства выполнения работы (PoW) версии 1 (Equi-X и Blake2b) и управление вычислительной сложностью;
- вызывать и наблюдать защитные механизмы реле и onion-сервиса в лабораторной сети (лабораторная работа 9).

Открытая анонимная сеть, поддерживаемая добровольцами, естественным образом становится целью тех, кто хочет сделать её непригодной к использованию. В этой главе рассматриваются механизмы защиты Tor от *отказа в обслуживании* (denial of service, DoS): попыток исчерпать ресурсы реле или onion-сервиса, чтобы ими не могли пользоваться другие. Конкретные инциденты последних лет, включая волну атак на onion-сервисы в 2022–2023 годах, рассматриваются в гл. 27. Здесь мы сосредоточимся на механизмах.

15.1. Почему DoS — проблема анонимности

В обычной сети DoS — «лишь» проблема доступности. В Tor это также оружие против анонимности [dos-spec § Overview]:

- Противник, способный по требованию вывести реле из строя, может заставить клиентов, использующих его как входной узел (guard), выбрать другой, рассчитывая, что новым входным узлом станет реле противника. Jansen, Tschorsch, Johnson и Scheuermann показали, как противник с небольшими ресурсами может исчерпать память реле и использовать эту возможность для деанонимизации onion-сервисов: *атака Sniper Attack* [Jan+14].

- Замедление или недоступность честных реле перенаправляет трафик к враждебным реле, увеличивая долю наблюдаемого ими трафика.
- Атака, выборочно вызывающая отказы цепей, может служить сигналом, например для определения цепей, проходящих через конкретное реле.

Поэтому защита должна не просто «поддерживать работу реле», но и не создавать наблюдаемого поведения, которое способен использовать противник.

15.2. Физические ресурсы

15.2.1. Память

Память — наиболее чувствительный ресурс и защищена наиболее зрелыми механизмами [dos-spec § Memory exhaustion]. В Linux процесс, запросивший слишком много памяти, обычно не получает ошибку сразу. Система обещает выделить память и может позднее завершить процесс, когда тот действительно попытается её использовать. Поэтому ждать ошибки выделения памяти бессмысленно. tor устанавливает собственный лимит, `MaxMemInQueues`, задаваемый оператором или вычисляемый как доля системной памяти. В него учитывается всё, что потребляет память: ячейки в очередях цепей, буферы соединений, состояние сжатия, кэшированные дескрипторы onion-сервисов и DNS-запросы.

При превышении лимита tor освобождает память в два этапа. Сначала из кэшей, потребляющих более 20 % общего объёма, удаляются самые старые записи, пока каждый такой кэш не опустится ниже 10 %. Если использование всё ещё выше 90 % лимита, tor закрывает цепи и соединения в порядке возраста наиболее старых поставленных в очередь данных, начиная с тех, где данные стоят в очереди дольше всего. Цель — затронуть участников, которые накапливают данные, не читая их, что характерно для подобных атак.

15.2.2. Процессор

Нагрузка на процессор — одно из слабых мест реализации на C, обрабатывающей сетевые события в одном главном потоке. Наиболее дорогие операции, криптографические рукопожатия при создании цепей (гл. 9), делегируются пулу рабочих потоков и ставятся в очередь. Но при слишком большом числе рукопожатий очередь растёт, и легитимные цепи завершаются по тайм-ауту. Tor не измеряет загрузку CPU напрямую. Он защищает CPU косвенно, ограничивая операции, которые его расходуют, включая создание цепей и обработку запросов введения для onion-сервисов.

15.2.3. Пропускная способность

Операторы могут ограничивать пропускную способность с помощью параметров вроде `RelayBandwidthRate`, но такие лимиты прежде всего не позволяют выйти за условия канала связи и сами по себе не являются защитой. От аномального расхода пропускной способности Tor защищается механизмами управления потоком и перегрузкой (гл. 11) и справедливым планированием ячеек между цепями.

15.3. Ограничения по IP-адресам

Самая эффективная защита от простых атак с массовой отправкой запросов проста. Входной узел (guard) видит IP-адрес подключающегося клиента и может ограничить объём запросов от каждого адреса. Это единственное место сети, где применим такой лимит, поскольку последующие переходы намеренно не знают источник трафика. Tor применяет к клиентам,

но не к аутентифицированным реле, три семейства ограничений, настраиваемых опциями `DoS…`:

Соединения (`DoSConnection…`) Максимальное число одновременных TLS-соединений и максимальная скорость новых соединений с одного адреса. Превысившие пороги соединения закрываются.

Цепи (`DoSCircuitCreation…`) *корзина токенов (token bucket)* для каждого адреса: каждая новая цепь расходует один токен. Токены пополняются с фиксированной скоростью до предельного запаса (*burst*). Адрес, исчерпавший токены *и* удерживающий как минимум заданное число соединений, помечается; последующие запросы цепей отклоняются на определённый период.

Одношаговые запросы к пункту встречи (`DoSRefuseSingleHopClientRendezvous`) Отклоняет запросы к пунктам встречи по цепям из одного перехода, например от шлюзов Tor2web, позволяющих обращаться к onion-сервисам без анонимности клиента. Такие запросы представляют дешёвый способ генерировать нагрузку.

15.3.1. Код и консенсус

В коде tor эти механизмы **отключены** по умолчанию (`DOS_CC_ENABLED_DEFAULT 0` в `src/core/or/dos.h`). Общесетевые настройки поступают из параметров консенсуса. Публичный консенсус от 23 сентября 2026 года содержит:

Параметр	Консенсус	Значение в коде по умолчанию
`DoSCircuitCreationEnabled`	1	0
`DoSCircuitCreationMinConnections`	2	3
`DoSCircuitCreationRate`	2	3
`DoSCircuitCreationBurst`	60	90
`DoSConnectionEnabled`	1	0
`DoSConnectionMaxConcurrentCount`	50	100
`DoSRefuseSingleHopClientRendezvous`	1	—

Сетевые значения строже значений по умолчанию в коде: они отражают годы атак и последующих корректировок. Возможность менять их через консенсус за считанные часы без обновления тысяч реле — один из наиболее ценных инструментов Tor Project во время атаки.

Внимание: Недостатки ограничений по IP-адресам

Противник со множеством IP-адресов, например ботнет, может обходить ограничения по IP-адресам. С другой стороны, слишком строгие лимиты затрагивают легитимных пользователей, делящих один адрес, например за NAT мобильного оператора или университета. Значения всегда являются компромиссом.

15.4. Onion-сервисы под атакой

Onion-сервисы особенно уязвимы. Каждый запрос введения обходится клиенту дёшево: один `INTRODUCE1` (разд. 14.6). Для сервиса тот же запрос дорог: необходимо расшифровать сообщение, выполнить рукопожатие и построить многоузловую цепь к указанному пункту встречи. Возникает **усиление нагрузки**: имея сравнительно небольшие ресурсы, противник заставляет сервис и все реле его цепей выполнять значительно больше работы. Сервис не может блокировать IP-адреса атакующих, поскольку никогда их не видит.

15.4.1. Ограничения в пунктах введения

Первая защита — ограничение частоты запросов в пунктах введения. С помощью `HiddenServiceEnableIntroDoSDefense` и связанных `…RatePerSec` и `…BurstPerSec` сервис через расширение `DOS_PARAMS` в `ESTABLISH_INTRO` сообщает каждому пункту введения, сколько сообщений `INTRODUCE2` в секунду готов принимать. Это защищает сервис, но не отличает атакующего от легитимного пользователя. При атаке могут быть отклонены запросы и атакующего, и законного пользователя.

15.4.2. Доказательство выполнения работы

Более новый механизм, предложенный в Предложении 327 и доступный с tor 0.4.8.4 (август 2023 года), — **доказательство выполнения работы** (proof of work, PoW) [hspow-spec § Motivation]. Клиенты должны доказать, что выполнили определённый объём вычислений перед обслуживанием; запросы, подтверждающие большее вычислительное усилие, получают приоритет. Несколько секунд вычислений приемлемы для легитимного пользователя. Для атакующего, посылающего тысячи запросов в секунду, умножение этой стоимости на каждый запрос может стать запретительным.

Механизм состоит из четырёх частей [hspow-spec § Common protocol]:

1. **Параметры дескриптора.** Во втором слое дескриптора (разд. 14.5) сервис публикует строку `pow-params` со схемой (`v1`), случайным 32-байтовым *семенем*, *рекомендуемым усилием* и сроком действия семени. Семя обновляется каждые 105–120 минут.

2. **Клиент решает задачу.** Выбрав усилие E, клиент ищет решение Equi-X для задачи

   $$\texttt{"Tor hs intro v1\0"} \parallel A' \parallel \mathit{seed} \parallel \mathit{nonce} \parallel E,$$

   где A' — ослеплённый ключ сервиса. Одного решения Equi-X недостаточно. Клиент также вычисляет 32-битное значение хеша Blake2b R от задачи и решения; решение принимается только если $R \cdot E \leq 2^{32} - 1$. Иначе меняется одноразовое число (nonce) и попытка повторяется.

3. **Доказательство в сообщении введения.** Клиент включает одноразовое число nonce, усилие, первые 4 байта семени и решение в расширение зашифрованной части `INTRODUCE1`, доступной только сервису.

4. **Очередь приоритетов.** Сервис проверяет доказательство — значительно быстрее, чем его находят, — и помещает запрос в очередь, упорядоченную по усилию. Сначала обслуживаются запросы с наибольшим усилием; запросы без доказательства имеют нулевое усилие. При заполнении очереди сервис сразу отбрасывает половину с наименьшим усилием.

Подробнее: Почему Equi-X?

Equi-X, разработанный tevador для этой задачи, — *асимметричная* задача: поиск решения требует памяти и значительных вычислений, а проверка очень дешева. Он основан на HashX — семействе хеш-функций, реализации которых меняются с каждым семенем и задействуют возможности параллельной работы современных процессоров. Это усложняет ускорение с помощью специализированного оборудования и ограничивает преимущество атакующего над пользователем обычного компьютера. Существуют две реализации: C-версия в tor и Rust-версия для Arti, совместимость которых поддерживают автоматические тесты. C-код распространяется под LGPL, поэтому для включения tor

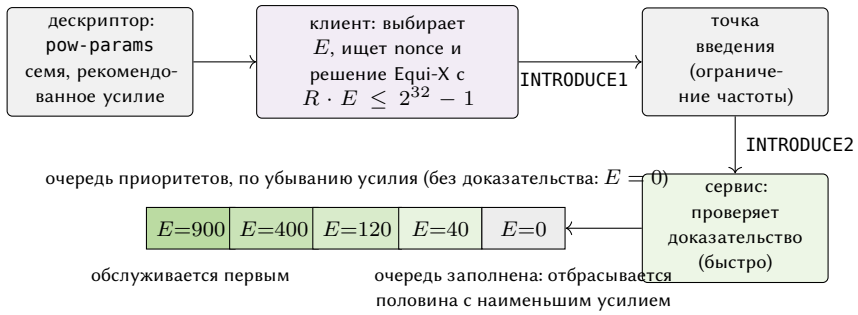


Рис. 15.1: Доказательство выполнения работы для onion-сервиса. Клиент читает семя и рекомендуемое усилие из дескриптора, ищет решение, проходящее проверку, и отправляет его в зашифрованной части `INTRODUCE1`. Сервис проверяет решение и упорядочивает запросы по усилию.

должен собираться с `--enable-gpl`. Сборка без этой опции не поддерживает PoW: `tor --list-modules` показывает `pow: no`, а не `pow: yes`, который мы увидим в гл. 17. Это не типичный случай: и лабораторный образ, и пакет Debian `tor` собраны с этой опцией и содержат модуль.

15.4.3. Линейная шкала усилия

Зачем нужна проверка Blake2b? Стоимость поиска отдельного решения Equi-X приблизительно постоянна. Случайное решение удовлетворяет $R \cdot E \leq 2^{32} - 1$ с вероятностью примерно $1/E$, поэтому клиент в среднем должен попробовать E решений. Стоимость растёт *линейно* с E, что позволяет плавно её регулировать. Многие PoW-схемы вместо этого требуют определённое число нулевых старших битов хеша, удваивая стоимость с каждым дополнительным битом: такая шкала слишком груба для управления с обратной связью. Кроме того, усилие объявляется *до* поиска решения и входит в задачу. Клиент не может сначала случайно получить особенно удачный результат, а затем заявить большее усилие, чем реально затрачен.

15.4.4. Управление усилием

Рекомендуемое усилие не является постоянным. Сервис начинает с нуля, то есть защита неактивна. Каждые пять минут он обновляет рекомендацию по состоянию очереди: увеличивает значение, если очередь оставалась полной или пришлось отбрасывать запросы с рекомендуемым уровнем усилия, и уменьшает иначе. Логика управления вдохновлена управлением перегрузкой TCP: аддитивное увеличение и мультипликативное уменьшение. Клиенты, со своей стороны, повышают усилие после истечения времени ожидания подключения. Получается система обратной связи из двух контуров, где более настойчивый пользователь может предложить большее усилие, чтобы продвинуться в очереди.

15.5. Лабораторная работа: вызов защитных механизмов

Лабораторная работа 9: Ограничения против DoS и доказательство выполнения работы в лабораторной сети

Цель. Включить на реле ограничения создания цепей и превысить их; включить доказательство выполнения работы для onion-сервиса и прочитать его параметры; изучить проверку уровня усилия через Blake2b с помощью `labs/ru/код/torlab/pow.py`.
Предварительные условия. Лабораторные работы 1–8; работающая лабораторная сеть с образом, собранным с `--enable-gpl`.
Оценочное время. 2 часа.

Внимание: Только в собственной сети

Эта лабораторная работа намеренно создаёт аномальный трафик. Выполняйте её исключительно в частной сети книги. Повторение тех же действий против реальных реле или сервисов является атакой, причиняющей вред другим людям, и во многих юрисдикциях незаконно (гл. 25).

Шаг 1: включить ограничения на реле. Добавляем очень низкие лимиты в `torrc` реле `test003r`, чтобы они быстро сработали, и отправляем сигнал `HUP` для перезагрузки конфигурации:

```
DoSCircuitCreationEnabled 1
DoSCircuitCreationMinConnections 2
DoSCircuitCreationRate 1
DoSCircuitCreationBurst 10
DoSCircuitCreationDefenseType 2
DoSCircuitCreationDefenseTimePeriod 60 seconds
```

Тип защиты 2 означает «отклонять новые цепи».

Шаг 2: превысить лимиты. Открываем один или два канала к реле и быстро запрашиваем 30 цепей через **`CREATE_FAST`**. Чтобы каждый запуск имел отдельные счётчики, соединения исходят с разных адресов обратной петли (127.0.0.2, 127.0.0.3), используя новый параметр `source_address` функции `open_channel`:

```
источник 127.0.0.2, 2 канала: 9 успешно, 21 DESTROY(5); первый отказ: 10
источник 127.0.0.3, 1 канал: 30 успешно
источник 127.0.0.1, 2 канала: 9 успешно, 21 DESTROY(5); первый отказ: 10
```

Следуют три наблюдения:

- Реле отклоняет запросы с помощью ячейки **`DESTROY`** с кодом причины 5, `RESOURCELIMIT`.
- При *burst* 10 успешно создаются девять цепей, а не десять. Код (`cc_has_exhausted_circuits` в `src/core/or/dos.c`) помечает клиента сразу, когда запас токенов исчерпан, поэтому десятый запрос уже отклоняется.
- При одном канале ограничение не срабатывает: защита требует не менее `MinConnections` одновременных соединений. Это не наказывает обычного клиента, который обычно держит только одно соединение со своим входным узлом (guard).

Мы предполагали, что соединения с 127.0.0.1, адреса лабораторных реле, будут исключением. Эксперимент опроверг предположение, а `dos.c` подтверждает отсутствие такого исключения для цепей. В конце восстанавливаем исходный `torrc`.

Шаг 3: доказательство выполнения работы onion-сервиса. После строк `HiddenService…` в `torrc` узла `test009h` добавляем:

```
HiddenServicePoWDefensesEnabled 1
```

После сигнала `HUP` сервис публикует новый дескриптор. Его загрузка и расшифрование с помощью лабораторной работы 8 показывает во втором слое:

```
pow-params v1 tx2Mtmzw5GHgWjcDoivTvWzwQPNZONwMwSMddOc+ZVQ= 0 2026-09-23T18:20:53
```

Строка содержит схему `v1`, семя в кодировке base64, рекомендуемое усилие **ноль** (очередь пока не создавала нагрузки) и срок действия семени. Срок истекает через час и пятьдесят минут после публикации, в предусмотренном диапазоне 105–120 минут.

Шаг 4: линейный рост усилия. Модуль `pow.py` реализует проверку Blake2b и имитирует решения Equi-X случайными байтами, имеющими для этой проверки то же распределение:

```
def check_value(service_id, seed, nonce, effort, solution):
    challenge = P_STRING + service_id + seed + nonce + struct.pack(">I", effort)
    h = hashlib.blake2b(challenge + solution, digest_size=4).digest()
    return int.from_bytes(h, "big")

def valid_effort(r, effort):
    return r * effort <= UINT32_MAX
```

Среднее число попыток до успеха в 400 испытаниях для каждого значения усилия:

```
усилие     1: 1.0
усилие    10: 9.7
усилие   100: 102.3
усилие  1000: 1068.8
```

Число попыток растёт пропорционально усилию, как и предсказано. Ожидаемое значение равно усилию, поскольку каждая попытка успешна с вероятностью $1/effort$, а число попыток подчиняется геометрическому распределению. Стандартное отклонение сопоставимо со средним значением. Поэтому среднее по 400 испытаниям имеет типичное относительное отклонение $1/\sqrt{400} = 5\,\%$. Два запуска могут различаться на десять процентов без каких-либо изменений; при повторении мы действительно получили 969,9 вместо 1068,8 в листинге. Это полезное напоминание: выборочное среднее следует рассматривать с учётом статистической неопределённости. Ещё одна деталь: `blake2b` с `digest_size=4` не равен первым четырём байтам 64-байтового результата Blake2b, поскольку длина выхода влияет на начальное состояние функции. Спецификация отдельно подчёркивает эту легко пропускаемую тонкость.

Итоги

- В Tor DoS-атаки одновременно являются атаками на анонимность: принудительное отключение или отказ реле может перенаправлять трафик к противнику.
- Память защищает внутренний лимит `MaxMemInQueues`; при его превышении tor очищает кэши и закрывает цепи, накапливающие данные.

- Входные узлы применяют к клиентам ограничения соединений и цепей по IP-адресам с помощью корзин токенов. Отключённые по умолчанию в коде механизмы включаются и регулируются консенсусом.
- Onion-сервисы сталкиваются с амплификацией: инициировать запрос дёшево, обслуживать его дорого.
- Доказательство выполнения работы v1 сочетает Equi-X с проверкой Blake2b, обеспечивающей линейный рост усилия. Сервис отдаёт приоритет запросам с большим усилием и регулирует рекомендуемое усилие по состоянию очереди.

Упражнения

Упражнение 15.1 ⋆ При `DoSCircuitCreationRate` 2 и `Burst` 60, как в публичном консенсусе, сколько цепей может создать клиент за одну минуту, начиная с полного запаса токенов? А за один час? Считайте, что защита уже активирована, то есть адрес достиг `DoSCircuitCreationMinConnections` одновременных соединений. Ниже этого порога ограничения нет.

Упражнение 15.2 ⋆ Почему ограничения по IP-адресам применяются только к клиентам и только на первом переходе? Что произойдёт, если промежуточный узел попытается ограничивать цепи «по адресу»?

Упражнение 15.3 ⋆⋆ Оцените отношение работы клиента к работе сети для одного запроса введения: сколько цепей, рукопожатий и ячеек порождает ответ сервиса?

Упражнение 15.4 ⋆⋆ Сравните доказательство выполнения работы в Tor со схемой, требующей k старших нулевых битов хеша. Какие уровни усилия допускает каждая схема? Почему линейная шкала лучше подходит для управления с обратной связью?

Упражнение 15.5 ⋆⋆ Клиент объявляет усилие $E = 50$, но случайно сразу находит решение с очень маленьким R, которое удовлетворяло бы и $E = 5000$. Может ли он отправить это решение, заявив усилие $E = 5000$? Почему?

Упражнение 15.6 ⋆ ⋆ ⋆ В лабораторной сети установите `MaxMemInQueues` в минимально допустимое значение на промежуточном реле и постройте через него цепь, загружая большой файл *без чтения* входящих данных и без отправки SENDME. Что происходит с накопившимися ячейками? Найдите сообщения системы управления памятью в журнале реле.

Упражнение 15.7 ⋆ ⋆ ⋆ Напишите небольшой симулятор очереди приоритетов onion-сервиса. Запросы поступают со случайным усилием, многие с нулевым и некоторые с более высокими значениями; сервис обрабатывает определённое число запросов в секунду и при превышении порога очереди отбрасывает половину с наименьшим усилием. Измерьте среднее время ожидания в зависимости от усилия.

Дополнительная литература

- *Tor denial-of-service mitigations* (dos-spec) и *Onion service proof-of-work* (hspow-spec) [Thee].
- R. Jansen, F. Tschorsch, A. Johnson, B. Scheuermann, *The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network* [Jan+14].
- Репозиторий Equi-X tevador и журнал разработки, на которые ссылается спецификация.

Глава 16

Цензура, мосты и подключаемые транспорты

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; pt-spec v1; obfs4proxy 0.0.14 (Debian); документация Tor Project

Цели главы

К концу этой главы вы сможете:

- описывать способы, которыми цензор может блокировать Tor;
- объяснять, что такое мосты и почему их распространение представляет сложную задачу;
- описывать протокол взаимодействия tor с подключаемым транспортом;
- различать стратегии obfs4, meek, Snowflake, WebTunnel и Conjure;
- эксплуатировать obfs4-мост в лабораторной сети и наблюдать, что способен видеть цензор (лабораторная работа 10).

Всё рассмотренное до сих пор предполагает, что клиент способен достичь сети. Во многих странах это не так: соединения с Tor блокируются, иногда только в периоды политической напряжённости, иногда постоянно. Эта глава завершает часть II темой **обхода цензуры**, одним из наиболее активно развивавшихся направлений Tor Project в последние годы.

16.1. Как блокируют Tor

Цензор, обычно государство, контролирующее провайдеров интернет-доступа страны, располагает несколькими инструментами:

Блокировка адресов Список реле публичен: это консенсус (гл. 12). Цензор может просто загрузить его и заблокировать все IP-адреса реле либо только авторитетных серверов каталогов и реле, у которых клиенты получают консенсусы.

Блокировка сайтов Блокировка `torproject.org` не позволяет людям загрузить Tor Browser, даже если сама сеть Tor остаётся доступной.

Распознавание протокола *Глубокий анализ пакетов* (deep packet inspection, DPI) позволяет распознать соединение Tor по особенностям рукопожатия TLS, размерам пакетов или временным характеристикам, даже когда адрес назначения ещё неизвестен.

Активное зондирование Если DPI предполагает, что сервер может быть нераскрытым реле, цензор способен подключиться к нему и попытаться говорить по протоколу Tor. Если сервер отвечает, адрес блокируется.

Списки разрешённых адресов и протоколов В крайнем случае цензор разрешает только явно одобренные протоколы или назначения и блокирует всё остальное.

Для каждой техники существует соответствующая контрмера, а противостояние цензоров и разработчиков продолжается. Такие проекты, как OONI, измеряют, где и как блокируется Tor (гл. 24).

16.2. Мосты

Ответ на блокировку адресов — **мост**: реле, отсутствующее в публичном консенсусе. Мост действует как входной узел (guard) (разд. 13.3). Клиент использует его первым переходом, после чего цепь обычным образом продолжается по сети. Поскольку адрес моста не публикуется, цензор не может заблокировать его просто по списку. Мосты публикуют дескрипторы на **авторитетном сервере мостов (bridge authority)**, который проверяет достижимость и ведёт непубличный список. В опубликованной статистике мостов используется хеш *от хеша* идентификатора, чтобы не раскрывать его.

Клиент настраивает мост с помощью *строки конфигурации моста (bridge line)*. В лабораторной сети файл `torrc` клиента `test011bc` содержит:

```
UseBridges 1
Bridge obfs4 127.0.0.1:9909 2557828B596B33AB02075FE7EAD86F2DC3CEAFD5 cert=4sKDQih...
    iat-mode=0
ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy
```

Строка задаёт тип транспорта, адрес, отпечаток идентичности моста, используемый для проверки собеседника (гл. 7), и параметры транспорта. Значение `cert=4sKDQih...` *сокращено* для печатного примера и непригодно для непосредственного копирования: следует использовать полный сертификат и отпечаток, созданные собственной инстанцией моста.

16.2.1. Проблема распространения

Мост полезен, пока цензор не знает его адрес. Но для пользы этот адрес нужно сообщать пользователям, а цензор может притвориться пользователем. Идеального решения нет: распределитель должен предоставить достаточно мостов большому числу легитимных пользователей, раскрыв как можно меньше цензору. Tor Project распространяет мосты несколькими каналами, каждый со своей защитой от автоматизированного сбора [Theg]:

- веб-сайт `https://bridges.torproject.org/`;
- электронная почта: `bridges@torproject.org`;
- бот Telegram (`@GetBridgesBot`);
- непосредственно через Tor Browser, который обнаруживает блокировку и запрашивает подходящие мосты для страны пользователя (*Connection Assist*, на основе канала *moat*);
- частные каналы организаций, лично знающих своих пользователей.

Многие годы распределением мостов по каналам занимался BridgeDB. Его заменил **rdsys**; миграция завершилась в октябре 2024 года [The24b]. Операторы мостов могут выбрать канал опцией `BridgeDistribution`.

16.2.2. Lox

Lox [TG23] подходит к распространению через систему *репутации*. Новый пользователь получает мост и анонимный криптографический сертификат (credential). Если мост со временем остаётся незаблокированным, уровень сертификата растёт вплоть до 4; начиная со второго уровня пользователь также может приглашать других. Если мост блокируется, можно получить новый, но репутация теряется. Идея состоит в том, что цензор, блокирующий каждый полученный мост, не способен повышать уровень, а честный пользователь способен. Такие удостоверения используют криптографические механизмы, которая не позволяет даже самой системе узнать, кто кого пригласил, защищая *социальный граф* пользователей. Lox тестируется в альфа-версиях Tor Browser.

16.3. Подключаемые транспорты

Скрыть адрес недостаточно против распознавания протокола: сам трафик должен выглядеть иначе. Вместо изменения tor каждый раз, когда цензор учится распознавать новый признак, Tor Project определил интерфейс внешних программ, преобразующих трафик: **подключаемых транспортов** (pluggable transports, PT) [pt-spec § Architecture Overview].

16.3.1. Архитектура

PT — отдельный процесс, который запускает tor, поэтому он называется *управляемым (managed)*. На стороне клиента PT предоставляет tor локальный SOCKS-прокси. tor передаёт ему соединение к мосту, а PT преобразует трафик перед отправкой. На стороне моста PT слушает публичный порт, выполняет обратное преобразование и передаёт трафик локальному ORPort tor. Дополнительный небольшой протокол, *Extended ORPort*, может передавать tor настоящий адрес клиента для статистики.

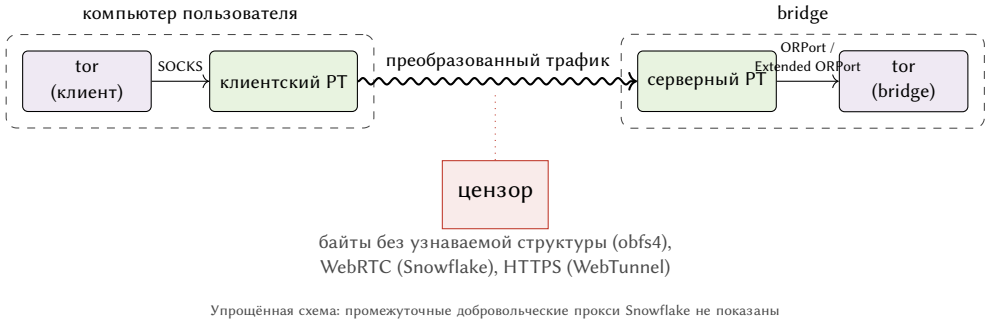


Рис. 16.1: Упрощённая архитектура подключаемого транспорта. На стороне клиента tor взаимодействует с PT через локальный SOCKS-прокси; на мосту PT передаёт восстановленный трафик tor через ORPort или, если он настроен, через Extended ORPort. Промежуточные добровольческие прокси Snowflake здесь опущены. На контролируемом участке цензор видит преобразованный трафик, однако возможность его распознавания не исключена.

16.3.2. Протокол между tor и PT

tor настраивает PT переменными окружения, а PT отвечает строками в стандартный вывод [pt-spec § Specification]. Это можно проверить, вручную запустив `obfs4proxy` в лабораторном контейнере точно так, как это сделал бы tor. В клиентском режиме:

```
$ env TOR_PT_MANAGED_TRANSPORT_VER=1 TOR_PT_STATE_LOCATION=/tmp/pt \
```

```
      TOR_PT_EXIT_ON_STDIN_CLOSE=1 TOR_PT_CLIENT_TRANSPORTS=obfs4 obfs4proxy
VERSION 1
CMETHOD obfs4 socks5 127.0.0.1:40057
CMETHODS DONE
```

PT объявляет версию протокола и сообщает, что предоставляет транспорт `obfs4` как прокси SOCKS5 на выбранном им локальном порту. В серверном режиме с переменными, задающими публичный адрес прослушивания и адрес назначения ORPort:

```
$ env ... TOR_PT_SERVER_TRANSPORTS=obfs4 \
      TOR_PT_SERVER_BINDADDR=obfs4-127.0.0.1:19999 \
      TOR_PT_ORPORT=127.0.0.1:5104 obfs4proxy
VERSION 1
SMETHOD obfs4 127.0.0.1:19999 ARGS:cert=5
    W6vcbWbFGGZgabwm4C3WipezwnAfadoBhqfNVpY72xnkjB/Szczbxg8fLWJJ1uRlyjPSg,iat-mode=0
SMETHODS DONE
```

Параметры после `ARGS:` нужны клиенту и включаются в строку конфигурации моста. PT также записывает их в `obfs4_bridgeline.txt` своего каталога состояния. Переменная `TOR_PT_EXIT_ON_STDIN_CLOSE` велит PT завершиться, когда tor закрывает входной поток, чтобы транспорт не пережил запустивший его процесс. (Из-за недосмотра пример спецификации утверждает, что PT пишет сообщения в «stdin»; на практике, как мы только что увидели, он пишет их в стандартный вывод.)

Команда сервера `env ...` приведена *схематически*: многоточие обозначает другие необходимые переменные окружения и не должно вводиться буквально. Порт SOCKS5 в ответе клиентского транспорта также взят из эталонного запуска и не является постоянным.

16.3.3. Lyrebird

Транспортная программа, используемая Tor Browser, — **lyrebird**, происходящая от более раннего `obfs4proxy`. Она реализует obfs4, meek, Snowflake и WebTunnel [Theg]. В лаборатории используется `obfs4proxy` 0.0.14 из пакетов Debian, который по-прежнему совместим с obfs4.

16.4. Стратегии

Современные транспорты следуют двум противоположным стратегиям: *не быть похожими ни на что распознаваемое* либо *быть похожими на что-то разрешённое*.

16.4.1. obfs4: не быть похожим ни на что распознаваемое

obfs4, разработанный Yawning Angel, превращает трафик в последовательность байтов, неотличимых от случайных данных: без узнаваемого заголовка, без рукопожатия TLS и с переменными размерами пакетов. Его эффективность основана на трёх решениях:

- Обмен ключами представляет собой вариант ntor (разд. 9.6), где открытые ключи Curve25519 кодируются с помощью Elligator2, техники, делающей точку кривой неотличимой от случайной строки.

- Клиент должен знать параметр `cert` моста, содержащий идентификатор и открытый ключ. Без него наблюдатель не может построить корректное рукопожатие, а мост не отвечает. Это противодействует **активному зондированию**: любопытный цензор, подключившийся к мосту, не получает распознаваемого ответа.

- Параметр `iat-mode` (*inter-arrival time*) ценой производительности может добавлять случайные задержки и фрагментацию, дополнительно скрывая временные характеристики.

Ограничение obfs4 является одновременно его сильной стороной: цензор со списком разрешённых протоколов или блокировкой «всего, что выглядит случайным» способен его остановить. Так происходит в некоторых странах.

16.4.2. meek: за крупным сервисом

meek [Fif+15] использует *маскировку домена (domain fronting)*. Клиент открывает HTTPS-соединение к крупной сети доставки контента, указывая безобидное доменное имя в видимой части соединения, а настоящий пункт назначения, сервер-шлюз в Tor, сообщает только внутри зашифрованных данных. Блокировка meek потребовала бы заблокировать весь сервис с большим сопутствующим ущербом. Однако техника зависит от разрешения провайдеров, и многие в последние годы сделали её невозможной. Кроме того, meek медленный и дорогой, поскольку весь трафик проходит через серверы провайдера.

16.4.3. Snowflake: множество эфемерных прокси

Snowflake [Boc+24] использует тысячи добровольцев, запускающих небольшие прокси, например расширение браузера или программу на сервере. Клиент связывается с *посредником (broker)* через трудно блокируемый канал, например domain fronting, получает адрес прокси и подключается к нему через WebRTC, протокол видеозвонков в браузерах. Прокси пересылает трафик на мост Snowflake Tor Project. Прокси много, и они постоянно меняются. Заблокировать все их адреса непрактично, а блокировка WebRTC означает блокировку видеозвонков.

16.4.4. WebTunnel: выглядеть как веб-сайт

WebTunnel, объявленный Tor Project в марте 2024 года, заворачивает трафик в HTTPS-соединение с поведением, похожим на WebSocket [The24a]. Мост может находиться за обычным сайтом, деля с ним домен, адрес и порт. Обратный прокси передаёт мосту только запросы к секретному пути, а всем остальным обслуживает обычный сайт. WebTunnel рассчитан на цензоров со списками разрешённых протоколов, против которых obfs4 не работает: он имеет «форму» разрешённого протокола.

16.4.5. Conjure: скрытие в адресном пространстве

Conjure [Fro+19] — система *преломление сетевого трафика (refraction networking)*. Вместо моста по известному адресу она использует станции, установленные у сотрудничающих провайдеров интернет-доступа. Эти станции наблюдают трафик к диапазонам неиспользуемых адресов и перенаправляют его в Tor. Клиент подключается к «фантомному» адресу, выбранному из миллионов вариантов. Чтобы заблокировать Conjure, цензору пришлось бы блокировать целые части Интернета. По документации Tor Project, Conjure успешно тестируется, но всё ещё требует стабилизации и доступен только в альфа-версиях Tor Browser и некоторых приложениях [Theg].

Внимание: Ни один транспорт не работает везде

У каждого транспорта есть слабости, а его эффективность зависит от страны и меняется со временем. Например, на форуме Tor Project в апреле 2025 года пользователи сообщали, что obfs4, meek и Snowflake не работали в Китае, а WebTunnel подключался, но быстро блокировался [Tor25]. Поэтому Tor Browser предлагает несколько транспортов и функцию

выбора подходящего для страны пользователя.

16.5. Лабораторная работа: obfs4-мост глазами цензора

Лабораторная работа 10: Obfs4-мост в лабораторной сети

Цель. Запустить сеть с obfs4-мостом и использующим его клиентом; захватить и сравнить трафик обычного Tor-соединения и obfs4-соединения.
Предварительные условия. Лабораторная сеть (гл. 19). Образ содержит `obfs4proxy` и `snowflake-client` из пакетов Debian.
Оценочное время. 1–2 часа.

Шаг 1: сеть с мостом. Файл `labs/ru/лаборатория/chutney/networks/book-bridge` добавляет к сети книги авторитетный сервер мостов (`test003ba`), obfs4-мост (`test009br`) и использующий его клиент (`test011bc`):

```
$ make -C labs/ru/лаборатория clean
$ make -C labs/ru/лаборатория start NETWORK=book-bridge
```

Журнал клиента из предыдущего эталонного запуска содержит:

```
[notice] new bridge descriptor 'test009br' (fresh): $2557828B...~test009br
[warn] Only one bridge (transport: 'obfs4') is configured. You should
       have at least two for conflux, for any transport that is not 'snowflake'.
[notice] Bootstrapped 100% (done): Done
```

Клиент получил дескриптор моста через сам мост и завершил загрузку конфигурации. Предупреждение напоминает, что Conflux (гл. 31) использует две параллельные цепи и не может делать это с единственным мостом. Веб-запрос через SOCKS-порт этого клиента (9011) работает так же, как через обычный клиент.

Шаг 2: что видит цензор. Внутри контейнера захватываем трафик `tcpdump` на obfs4-порту моста (9909) и обычном ORPort (5104), пока клиент использует мост и мы открываем канал к `test004r` лабораторной работой 1. Затем анализируем захваченные пакеты с помощью `tshark`. Вот результат одного из наших сеансов:

```
== ORPort 5104 (TLS), первые байты от клиента ==
1533  16030105f8010005f403033e1234a0401520d1c52b...
== obfs4 9909, первые байты от клиента ==
1957  b566475349099523c53005e1004a392160f15d473e...
== пакеты, распознанные как TLS ==
порт 5104: 83
порт 9909: 0
== размеры сегментов obfs4 (клиент -> мост) ==
1957 1677 1888 1891 1716 1716 3436 1395 1716 1748 1500 1194 ...
```

На ORPort первые байты однозначны: `16 03 01` начинает запись рукопожатия TLS, а Wireshark распознаёт весь разговор как TLS. Цензор видит TLS-соединение, которое можно исследовать на признаки Tor; адрес реле также опубликован в консенсусе. На obfs4-мост байты не имеют узнаваемой структуры, ни один пакет не определяется как TLS, а размеры сегментов не повторяют 514-байтовые ячейки. Цензор видит лишь поток кажущихся случайными данных к неизвестному адресу.

Шаг 3: зондирование моста. Пытаемся подключиться к порту 9909 моста через лабораторную работу 1, как мог бы сделать цензор, подозревающий реле:

```
TimeoutError: The handshake operation timed out
```

Мост не отвечает *вообще*: ни сообщения об ошибке, ни одного байта, пока клиент не сдаётся через 15 секунд. Для obfs4 наш TLS ClientHello — некорректное рукопожатие, а реакция на некорректное рукопожатие — молчание. Цензор, зондирующий адрес, не узнаёт ничего, даже того, что там слушает сервис иного протокола. Без параметра `cert` он всё равно не способен сформировать корректное рукопожатие obfs4.

Итоги

- Цензор может блокировать Tor по адресам, распознавать протокол, зондировать подозрительные серверы или разрешать только явно одобренный трафик.
- Мосты — непубличные реле. Их распространение, сегодня управляемое rdsys и исследуемое посредством механизма репутации Lox, балансирует доступ пользователей и обнаружение цензорами.
- Подключаемые транспорты — внешние процессы, преобразующие трафик; tor настраивает их переменными окружения и читает ответы (`CMETHOD`, `SMETHOD`).
- obfs4 делает трафик похожим на случайный и противостоит активному зондированию; meek, Snowflake, WebTunnel и Conjure скрывают его за сервисами, протоколами или адресами, блокировка которых дорого обходится цензору.
- Ни один транспорт не работает везде и навсегда.

Упражнения

Упражнение 16.1 ⋆ Чем мост отличается от входного узла? Почему клиент, использующий мосты, не использует дополнительно публичный входной узел?

Упражнение 16.2 ⋆ Классифицируйте obfs4, meek, Snowflake, WebTunnel и Conjure по стратегии («не быть похожим ни на что распознаваемое» или «быть похожим на что-то разрешённое») и укажите, что пришлось бы блокировать цензору, чтобы остановить каждый из них.

Упражнение 16.3 ⋆⋆ Опишите, как цензор может обнаружить obfs4-мост, наблюдая только трафик и не прибегая к активному зондированию. Какие статистические признаки он может использовать и как `iat-mode` пытается им противодействовать?

Упражнение 16.4 ⋆⋆ Почему в Lox цензор не способен повышать уровень репутации? Что он мог бы сделать, чтобы обойти механизм, и почему это дорого?

Упражнение 16.5 ⋆⋆ Вычислите энтропию Шеннона в битах на байт для первых 10 KB, захваченных из obfs4-соединения и обычного TLS-соединения в лаборатории. Что вы ожидаете и почему разница может оказаться небольшой? Что же тогда делает TLS распознаваемым?

Упражнение 16.6 ⋆⋆⋆ Попробуйте запустить лабораторную работу 1 против obfs4-порта своего моста. Какая ошибка возникает? Затем используйте `obfs4proxy` как клиент с неправильным параметром `cert`: что видно в журнале моста?

Дополнительная литература

- *Pluggable Transport Specification (Version 1)* (pt-spec) [Thee].
- L. Tulloch и I. Goldberg, *Lox: Protecting the Social Graph in Bridge Distribution* [TG23].
- D. Fifield et al., *Blocking-resistant communication through domain fronting* [Fif+15].
- C. Bocovich et al., *Snowflake, a censorship circumvention system using temporary WebRTC proxies* [Boc+24].
- S. Frolov et al., *Conjure: Summoning Proxies from Unused Address Space* [Fro+19].
- Запись блога Tor Project о запуске WebTunnel [The24a] и страницы поддержки мостов [Theg].

Часть III

Tor на практике

Глава 17

Установка и torrc

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; руководство `tor(1)` для той же версии

Цели главы

К концу этой главы вы сможете:

- выбирать подходящий способ установки Tor в зависимости от задачи;
- читать и писать файл `torrc` и знать, где искать документацию по каждой опции;
- настраивать tor как клиент, включая SOCKS-порты, изоляцию потоков и защищённый порт управления;
- проверять конфигурацию до её использования и диагностировать проблемы по журналам.

С этой главы начинается практическая часть книги. В предыдущих главах мы разбирали, как работает протокол; теперь научимся использовать и настраивать программу, которая его реализует. Все упомянутые здесь параметры описаны в руководстве tor (`man tor`), которое является авторитетным источником: значения по умолчанию могут меняться между версиями.

17.1. Какой вариант Tor устанавливать?

Tor Browser Для просмотра веб-страниц это почти всегда правильный выбор. Он включает предварительно настроенный tor и браузер, модифицированный так, чтобы не раскрывать сведения, способные свести на нет защиту сети (гл. 23). Напротив, использование tor с обычным браузером оставляет множество возможностей для идентификации пользователя.

Пакет `tor` Подходит для использования Tor с другими приложениями, а также для запуска реле или onion-сервиса. Дистрибутивы Linux обычно включают этот пакет, хотя не всегда последней версии. Tor Project поддерживает собственный репозиторий пакетов для Debian и Ubuntu и распространяет *Tor Expert Bundle*, содержащий tor и подключаемые транспорты, для Windows и macOS.

Сборка из исходного кода Как в лабораторной среде этой книги (разд. 19.3); полезна для изучения tor или получения строго определённой версии.

Arti Клиент на Rust (гл. 30), который становится всё более зрелым, но настраивается иначе.

Какой бы способ вы ни выбрали, загружайте программу только с официального сайта и проверяйте её подпись. Версия tor, модифицированная атакующим, свела бы его защитные свойства к нулю.

17.2. Файл torrc

tor читает конфигурацию из текстового файла, обычно `/etc/tor/torrc`, если работает как системная служба. Формат прост: одна опция на строку, за которой следуют её значения. Строки, начинающиеся с `#`, являются комментариями. Некоторые опции могут встречаться несколько раз, например несколько SOCKS-портов. Параметры также можно передавать в командной строке с двумя ведущими дефисами:

```
$ tor --SocksPort 9150 --DataDirectory /tmp/tor-test
```

Руководство описывает сотни опций; команда `tor --list-torrc-options` выводит их все. Дистрибутивы могут задавать второй файл значений по умолчанию через `--defaults-torrc`. Файл, указанный параметром `-f`, имеет приоритет.

17.2.1. Проверка перед использованием

Три команды позволяют избежать многих неожиданностей:

```
$ tor --verify-config -f /etc/tor/torrc     # конфигурация корректна?
Configuration was valid
$ tor --dump-config short -f /etc/tor/torrc # что отличается от значений по умолчанию?
$ tor --list-modules                        # какие модули собраны?
relay: yes
dirauth: yes
dircache: yes
pow: yes
```

Как мы видели в гл. 15, последняя команда, например, показывает, доступно ли доказательство выполнения работы (PoW) для onion-сервисов.

17.3. Конфигурация клиента

Клиенту требуется очень мало настроек. Фактически минимальный конфигурационный файл пуст: по умолчанию tor открывает SOCKS-порт 9050 на localhost. Ниже приведён более явный пример с комментариями для системной службы tor, у которой есть права на каталоги `/var/lib/tor` и `/var/log/tor`; он не рассчитан на запуск от произвольного непривилегированного пользователя:

```
# Где tor хранит состояние, ключи и кеш
DataDirectory /var/lib/tor

# Порт SOCKS для приложений (только localhost!)
SocksPort 127.0.0.1:9050

# Порт управления с cookie-аутентификацией (см. главу 18)
ControlPort 127.0.0.1:9051
CookieAuthentication 1

# Журнал с уровнем notice
Log notice file /var/log/tor/notice.log
```

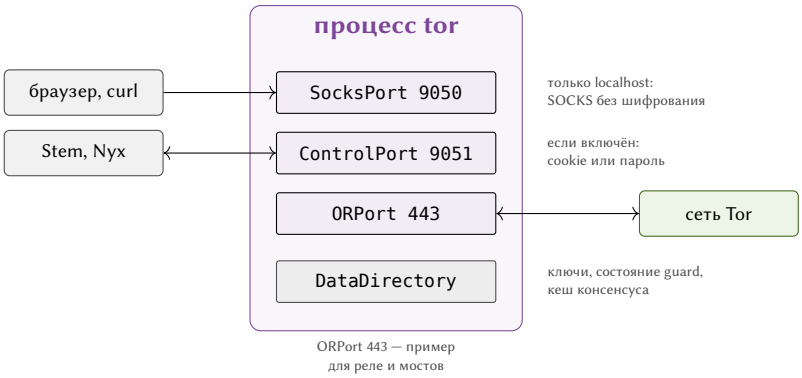


Рис. 17.1: Пример портов и каталога процесса tor. SOCKS на порту 9050 обслуживает локальные приложения; ControlPort 9051 не включён по умолчанию и при включении требует аутентификации. Эти порты нельзя открывать для внешней сети. DataDirectory — каталог, а не порт. ORPort 443 — лишь пример настройки реле или моста, а не обязательный номер порта.

Внимание: Не публикуйте SOCKS-порт

Руководство формулирует это однозначно: протокол SOCKS не обеспечивает ни шифрования, ни аутентификации. `SocksPort`, слушающий адрес, доступный из сети, превращает компьютер в открытый прокси и раскрывает запросы приложений в незашифрованном виде в локальной сети.

17.3.1. Изоляция потоков

Несколько потоков могут использовать одну и ту же цепь. Это эффективно, но позволяет выходному реле (exit) связать разные действия одного пользователя. Если личная электронная почта и псевдонимный форум используют одну цепь, выходное реле может определить, что эти действия исходят от одного и того же человека. Поэтому tor поддерживает **изоляцию потоков**: правила, определяющие, какие потоки могут совместно использовать цепь. Эти правила задаются как параметры `SocksPort`:

`IsolateClientAddr` Потоки от разных клиентских адресов не могут использовать одни и те же цепи; включено по умолчанию.

`IsolateSOCKSAuth` Потоки с разными SOCKS-учётными данными не могут совместно использовать цепи; включено по умолчанию.

`IsolateDestAddr`, `IsolateDestPort` Изоляция по адресу или порту назначения.

`IsolateClientProtocol` Изоляция по протоколу клиента (SOCKS4, SOCKS5, DNS…).

Разные SOCKS-порты по умолчанию также изолированы друг от друга: каждому прослушивающему порту назначается отдельная *группа сеансов (session group)*. Опция `SessionGroup` позволяет изменить это поведение и разрешить портам совместно использовать цепи. Наиболее распространённый механизм — изоляция по SOCKS-аутентификации: Tor Browser отправляет разные SOCKS-учётные данные для каждого посещаемого сайта, получая отдельную цепь для каждого сайта. Мы увидим это в гл. 18.

Цепь не используется бесконечно. После истечения `MaxCircuitDirtiness` — по умолчанию 10 минут, отсчитываемых от первого использования — цепь перестаёт принимать новые потоки, и tor строит новую.

17.3.2. Выбор реле: соблюдайте осторожность

Опции `EntryNodes`, `ExitNodes` и `ExcludeNodes` позволяют принудительно выбирать или исключать реле по отпечатку, псевдониму, адресу или коду страны. `StrictNodes` делает требования `ExcludeNodes` обязательными даже в тех случаях, когда это нарушает работоспособность. Эти инструменты полезны для экспериментов и особых сценариев. Однако использовать их только для того, чтобы «чувствовать себя безопаснее», обычно вредно: сужение выбора реле уменьшает разнообразие маршрутов, делает пользователя более заметным и подрывает защиту, обеспечиваемую взвешенным выбором маршрута и входных реле (guard) (гл. 13).

17.3.3. Другие порты

`DNSPort` tor принимает DNS-запросы и разрешает имена через сеть (разд. 11.1.4).

`TransPort` Принимает *прозрачные* TCP-соединения, перенаправленные системным межсетевым экраном. Такие системы, как Whonix, используют этот механизм для маршрутизации трафика приложений через Tor (гл. 23).

`AutomapHostsOnResolve` Назначает виртуальные адреса именам `.onion`, запрошенным через DNS, что позволяет приложениям без поддержки SOCKS обращаться к onion-сервисам через `TransPort`.

17.4. Порт управления

`ControlPort` позволяет другим программам управлять работающим tor: читать его состояние, запрашивать новые цепи или динамически создавать onion-сервисы. Это чрезвычайно мощный интерфейс, поэтому он должен быть защищён. Доступны два метода аутентификации:

Файл cookie При `CookieAuthentication 1` tor записывает в каталог данных файл, содержащий 32 случайных байта. Любой, кто может прочитать этот файл, может пройти аутентификацию. Следовательно, безопасность зависит от прав доступа к файлу.

Пароль При `HashedControlPassword` tor хранит хэш пароля, созданный командой `tor --hash-password`:

```
$ tor --hash-password test
16:458528FE0C0AF36260B5071446823FB789212185D7EE7F615BC120D55D
```

Показанный хеш соответствует демонстрационному паролю `test` и вычислен для фиксированной соли. При настоящем вызове команда выбирает случайную соль, поэтому результат обычно будет другим даже для того же пароля. Нельзя повторно использовать пароль или хеш из примера для защиты реального порта.

Как и порт SOCKS, порт управления никогда не должен быть доступен из сети. Протокол управления рассматривается в гл. 18.

17.5. Журналы

tor записывает события с несколькими уровнями детализации: `err`, `warn`, `notice`, `info` и `debug`. Для обычной эксплуатации достаточно уровня `notice`. Уровни `info` и `debug` полезны при исследовании протокола, как в лабораторных работах, но создают большой объём

данных. Опция `SafeLogging`, включённая по умолчанию, заменяет адреса и назначения в журналах на `[scrubbed]`. Забытый файл журнала не должен превращаться в историю действий пользователя. В лабораторной сети эта защита намеренно отключена (`SafeLogging 0`), чтобы можно было исследовать все детали.

Наиболее полезны при запуске сообщения *bootstrap*. Они показывают этапы подключения к сети и заканчиваются строкой `Bootstrapped 100% (done): Done`. Если процесс останавливается на определённом проценте, соответствующий этап подсказывает направление диагностики. Например, зависание при подключении к первому реле обычно указывает на цензуру или межсетевой экран и говорит в пользу использования мостов (гл. 16).

Итоги

- Для просмотра веб-страниц используйте Tor Browser; пакет `tor` предназначен для других приложений, реле и onion-сервисов.
- Файл `torrc` содержит одну опцию на строку. Руководство является авторитетным источником, а `--verify-config` проверяет конфигурацию.
- Порты SOCKS и управления должны оставаться на localhost. Защищайте порт управления файлом cookie или паролем.
- Изоляция потоков не позволяет несвязанным действиям совместно использовать одну цепь. Tor Browser достигает этого с помощью разных SOCKS-учётных данных для каждого сайта.
- Принудительный выбор или исключение реле обычно снижает безопасность; `SafeLogging` защищает содержимое журналов.

Упражнения

Упражнение 17.1 ⋆ В лабораторном контейнере создайте `torrc` с опечаткой в имени одной из опций и проверьте его с помощью `--verify-config`. Какое сообщение вы получите? Затем выполните `--dump-config short` для файла `torrc` клиента лабораторной сети.

Упражнение 17.2 ⋆⋆ Пользователь подключает личный почтовый клиент под своей реальной идентичностью и псевдонимный форум через один и тот же `SocksPort`. Что может вывести из этого выходное реле? Предложите две конфигурации, позволяющие избежать проблемы.

Упражнение 17.3 ⋆⋆ Почему `ExitNodes {it}`, то есть использование только итальянских выходных реле, снизило бы анонимность итальянского пользователя, посещающего итальянский веб-сайт? Обоснуйте ответ в терминах модели угроз из гл. 4.

Дополнительная литература

- Руководство tor, `man tor`, и `doc/man/tor.1.txt` в дереве исходного кода.
- Страницы поддержки Tor Project по установке и репозиториям пакетов: `https://support.torproject.org/`.

Глава 18

Порт управления и Stem

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; control-spec; Stem 1.8.2

Цели главы

К концу этой главы вы сможете:

- взаимодействовать с tor через порт управления, в том числе вручную;
- использовать основные команды: `GETINFO`, `SETCONF`, `SIGNAL`, `ADD_ONION` и подписку на события;
- писать программы на Python с библиотекой Stem;
- наблюдать изоляцию потоков и влияние новой идентичности на работающую систему (Лабораторная работа 11).

Tor Browser показывает цепь, используемую текущей страницей; такие инструменты, как Nyx, отображают пропускную способность и соединения реле; системы измерения сети строят цепи по запросу. Все эти приложения используют один интерфейс: **протокол управления** tor, доступный через `ControlPort` (разд. 17.4).

18.1. Протокол управления

Протокол текстовый и напоминает почтовые протоколы [control-spec § Message format]. Клиент отправляет по одной команде в строке, а tor отвечает строками, начинающимися с трёхзначного кода. Код `250` означает успех. Дефис после кода (`250-`) указывает, что ответ продолжается, а пробел (`250 `) — что это последняя строка; символ `+` вводит многострочный блок, завершаемый строкой, содержащей только точку. Коды `5xx` обозначают ошибки, а `650` — асинхронные события.

Попробуем это на клиенте лабораторной сети, открыв простое TCP-соединение с портом 8008:

```
> PROTOCOLINFO 1
250-PROTOCOLINFO 1
250-AUTH METHODS=COOKIE,SAFECOOKIE COOKIEFILE="/lab/net/.../008c/control_auth_cookie"
250-VERSION Tor="0.4.9.13"
250 OK
> AUTHENTICATE 52977550adebb73908610d7fa3355aba3fbdbdaef01d5ef86523d3232b87e5fa
250 OK
> GETINFO status/bootstrap-phase
250-status/bootstrap-phase=NOTICE BOOTSTRAP PROGRESS=100 TAG=done SUMMARY="Done"
250 OK
```

Это запись предыдущего демонстрационного запуска: пути, версия и cookie аутентификации зависят от конкретного экземпляра. **Никогда не публикуйте реальный cookie порта управления**: его обладатель может пройти аутентификацию. `PROTOCOLINFO` входит в число немногих команд, разрешённых до аутентификации (остальные — `AUTHCHALLENGE`, `AUTHENTICATE` и `QUIT`). Она сообщает поддерживаемые методы аутентификации и путь к cookie-файлу. При аутентификации `COOKIE` клиент отправляет содержимое файла в шестнадцатеричном виде. `SAFECOOKIE` позволяет не передавать сам cookie: клиент и tor обмениваются случайными значениями и доказывают знание cookie с помощью HMAC, не позволяя программе, выдающей себя за tor, украсть его.

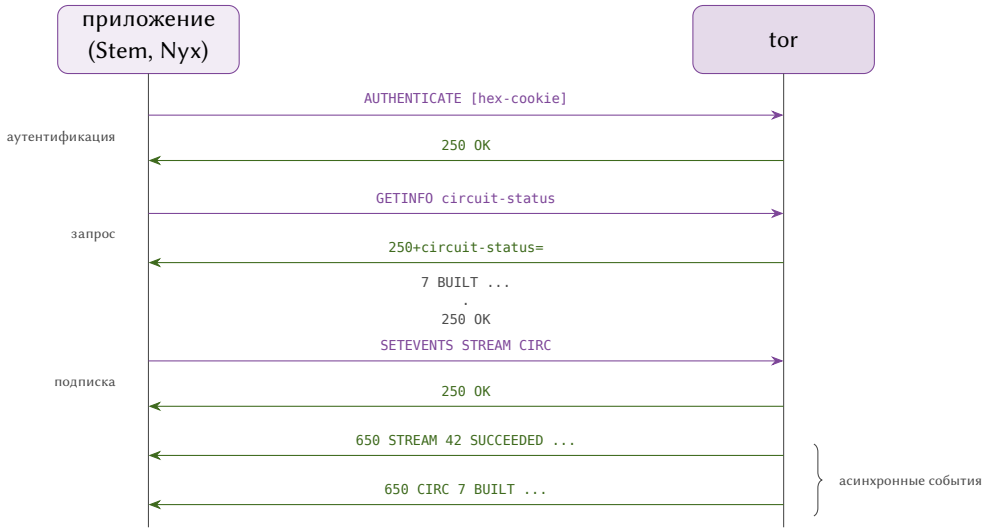


Рис. 18.1: Упрощённый сеанс протокола управления: после аутентификации COOKIE (в схеме указан заполнитель) запрос GETINFO circuit-status может вернуть многострочные данные, завершающиеся отдельной точкой и 250 OK. Подписка SETEVENTS подтверждается 250 OK; асинхронные события 650 могут перемежаться с ответами на другие команды. Идентификаторы условные.

18.1.1. Цепи с точки зрения tor

`GETINFO circuit-status` перечисляет клиентские цепи. Сокращённая запись:

```
3 BUILT $50160F8F...~test004r,$12415FEB...~test005r,$9449F36E...~test007e
  BUILD_FLAGS=NEED_CAPACITY,NEED_UPTIME PURPOSE=CONFLUX_LINKED
  CONFLUX_ID=DE224696F58F72950D70DDA2D7981726 CONFLUX_RTT=42538
```

Каждое реле представлено отпечатком идентичности RSA и псевдонимом. Цепи Conflux появляются парами с одинаковым `CONFLUX_ID`, причём две «ветви» начинаются с разных входных реле (guard). tor измеряет время прохождения туда и обратно (`CONFLUX_RTT`, в микросекундах), чтобы решить, по какой ветви передавать данные (гл. 31). Одна деталь в коде tor (`src/feature/control/control_fmt.c`) заслуживает внимания: `CONFLUX_ID` раскрывает только *половину* секретного идентификатора пары. Тот, кто знал бы идентификатор целиком, мог бы с позиции выходного реле добавлять ветви к паре и внедрять данные.

18.1.2. Основные команды

Команда	Назначение
`GETINFO`	читает информацию: версию, состояние начальной загрузки, цепи, потоки, консенсус, дескрипторы…
`GETCONF`, `SETCONF`	читает и изменяет текущую конфигурацию
`SIGNAL`	отправляет сигналы: `NEWNYM` (новая идентичность), `RELOAD`, `SHUTDOWN`…
`SETEVENTS`	подписывает на события: `CIRC`, `STREAM`, `BW`, `HS_DESC`…
`EXTENDCIRCUIT`	строит цепь по маршруту, выбранному контроллером
`ATTACHSTREAM`	назначает поток цепи (вместе с `__LeaveStreamsUnattached`)
`ADD_ONION`, `DEL_ONION`	создаёт и удаляет onion-сервисы без редактирования `torrc`
`HSFETCH`	загружает дескриптор onion-сервиса (использовалась в разд. 14.9)

Внимание: Контроль над портом означает контроль над tor

Через порт управления можно указать tor, какие реле использовать, заставить его строить одношаговые цепи и раскрыть назначения всех потоков. Вредоносная программа с доступом к порту управления способна разрушить анонимность пользователя. Поэтому такие системы, как Tails и Whonix, помещают между приложениями и портом управления фильтр (onion-grater), разрешающий только строго необходимые команды.

18.2. Stem

Stem — Python-библиотека Tor Project для протокола управления. Она скрывает детали формата сообщений, разбирает ответы и события и предоставляет удобные объекты. Последняя версия, 1.8.2, вышла в июне 2023 года. Библиотека работает с актуальными версиями tor и используется многими инструментами, но новых релизов давно не было. Для новой функциональности иногда приходится отправлять команды напрямую (`Controller.msg`). У Arti другой интерфейс — Arti RPC (гл. 30).

Минимальный пример, уже использованный в разд. 19.7.3:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                  # найти cookie или запросить пароль
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.purpose, [name for _, name in circ.path])
```

Для получения событий зарегистрируем функцию. Следующий пример — *самостоятельная программа*: слушатель регистрируется при открытом контроллере, после чего программа ждёт десять секунд для получения событий. Время ожидания здесь только демонстрационное.

```
import time
from stem.control import Controller, EventType

def on_circuit(event):
    print(event.id, event.status, event.purpose)

with Controller.from_port(port=8008) as c:
    c.authenticate()
    c.add_event_listener(on_circuit, EventType.CIRC)
    time.sleep(10)
```

Stem вызывает эту функцию из своего потока обработки событий. Такой поток только один, и все зарегистрированные обработчики вызываются последовательно. Поэтому медленный обработчик событий задерживает последующие события. Если обработка требует значительных вычислений, передайте её отдельному потоку.

18.3. Лабораторная работа: изоляция потоков в реальном времени

Лабораторная работа 11: Изоляция потоков и новая идентичность

Цель. Использовать Stem на клиенте лабораторной сети, чтобы проверить, какие запросы совместно используют цепь в зависимости от SOCKS-учётных данных и что меняется после `SIGNAL NEWNYM`.
Предварительные условия. Лабораторная сеть должна быть запущена, а локальный веб-сервер — работать на порту 8080. Запустите сценарий `labs/ru/лаборатория/isolation.py` внутри контейнера, где доступны Stem и `curl`.
Оценочное время. 1 час.

Шаг 1: регистрация потоков. Подпишемся на события `STREAM` и при каждом успешном потоке запишем цепь, к которой он был присоединён. Фильтрация по назначению обязательна: tor также открывает потоки для собственных задач, например загрузки документов каталога.

```
assigned = []
def on_stream(ev):
    if ev.status == StreamStatus.SUCCEEDED and ev.target == DESTINATION:
        assigned.append((ev.id, ev.circ_id, ev.target))

c.add_event_listener(on_stream, EventType.STREAM)
```

Фрагмент взят из `labs/ru/лаборатория/isolation.py`: полный сценарий также задаёт `DESTINATION`, импортирует `StreamStatus` и держит контроллер `c` открытым. Отдельно этот фрагмент запускать нельзя.

Шаг 2: запросы с разными учётными данными. Выполним четыре запроса через `curl`: два с SOCKS-учётными данными `site-a.example:x1`, один с `site-b.example:x2` и один без учётных данных. Tor Browser использует домен основного сайта как имя пользователя SOCKS для каждой вкладки:

```
$ curl --proxy-user site-a.example:x1 --socks5-hostname 127.0.0.1:9008 http
    ://127.0.0.1:8080/
```

Шаг 3: результаты. Ниже приведён вывод предыдущего эталонного запуска; идентификаторы Conflux, маршруты и задержка NEWNYM не являются постоянными значениями.

```
учётные
    данные site-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
учётные
    данные site-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
учётные
    данные site-b.example  -> conflux 31B1E3E0  ['test003r', 'test004r', 'test007e']
учётные
    данные -               -> conflux 35FD16D0  ['test003r', 'test004r', 'test007e']
(NEWNYM задержан на 6 с)
```

```
после NEWNYM, site-a.example -> conflux F7011ABE
```

Два запроса к «сайту A» используют один и тот же туннель, хотя выполняются в разное время. Запрос к «сайту B» и запрос без аутентификации используют другие туннели. Выведенный маршрут — одна из двух ветвей туннеля; в данном запуске они совпадают, поскольку в лабораторной сети мало реле. Далее объясняется, почему следует анализировать туннель, а не отдельную цепь.

Подробнее: Одной цепи недостаточно: туннели Conflux

Если повторить лабораторную работу много раз, два запроса с одинаковыми учётными данными иногда оказываются на цепях с разными номерами. Это не нарушение изоляции. Запрос полного списка цепей tor показывает, что почти все клиентские цепи имеют назначение `CONFLUX_LINKED` и появляются *парами* с одинаковым `CONFLUX_ID`. Это две «ветви» одного туннеля (разд. 31.4), и событие `STREAM` может указать любую из них. Следовательно, изоляция разделяет *туннели*, а не отдельные ветви. Когда это возможно, сценарий книги группирует их по `CONFLUX_ID`.
Нужно соблюдать ещё две меры предосторожности, обе можно проверить по журналам:

- `SIGNAL NEWNYM` исполняется не чаще одного раза в десять секунд (`MAX_SIGNEWNYM_RATE` в `src/core/mainloop/mainloop.c`). Более ранний запрос откладывается, и tor записывает «Rate limiting NEWNYM request: delaying by N second(s)». Если не дождаться выполнения, NEWNYM из предыдущего эксперимента может сработать уже в середине следующего.
- События `STREAM` также сообщают о потоках, которые tor открывает для собственных нужд, например для загрузки документов каталога. Фильтруйте их по назначению.

Даже при соблюдении этих мер в небольшой сети под высокой нагрузкой tor может выбрать только что построенный туннель для второго запроса. Поэтому автоматическая проверка книги повторяет эксперимент.

Внимание: Ошибка в лабораторной работе

В первом запуске все четыре запроса использовали одну цепь, что, казалось, показывало отсутствие изоляции. Проблема была в нашем сценарии: мы поместили `--proxy-user` сразу после опции `-o` команды `curl`, и программа восприняла его как имя выходного файла. Учётные данные вообще не отправлялись. Прежде чем делать вывод, что система ведёт себя не так, как предписывает спецификация, убедитесь, что эксперимент действительно делает то, что вы предполагаете.

Итоги

- Протокол управления — текстовый: однострочные команды, трёхзначные коды ответов и асинхронные события с кодом 650.
- Используйте файл cookie, предпочтительно с методом `SAFECOOKIE`, или пароль. Тот, кто контролирует порт, контролирует tor.
- Stem предоставляет Python-интерфейс к командам и событиям. Библиотека стабильна, но новых релизов не было с 2023 года.

- Изоляция по SOCKS-учётным данным выделяет каждому сайту отдельный туннель, который при Conflux состоит из двух цепей. `NEWNYM` переключает на новые туннели не чаще одного раза в десять секунд.

Упражнения

Упражнение 18.1 ⋆ Интерпретируйте ответ `552 Unrecognized key "pippo"` на команду `GETINFO pippo`. Чем код `5xx` отличается от события `650`?

Упражнение 18.2 ⋆⋆ С помощью Stem создайте временный onion-сервис функцией `create_ephemeral_hidden_service` на клиенте лабораторной сети, перенаправив порт 80 на локальный сервер порта 8080. Подключитесь к нему через `curl`. Где хранятся его ключи?

Упражнение 18.3 ⋆⋆ Напишите программу, которая каждую секунду отображает входящую и исходящую пропускную способность клиента по событиям `BW`, пока через лабораторную сеть загружается большой файл.

Упражнение 18.4 ⋆ ⋆ ⋆ Используя `__LeaveStreamsUnattached 1`, `EXTENDCIRCUIT` и `ATTACHSTREAM`, направьте запрос по выбранному вами маршруту, например через `test004r`, `test005r`, `test006e`. Почему такая возможность опасна для анонимности при реальном использовании?

Дополнительная литература

- *Tor's control protocol* (control-spec) [Thee]: все команды и события.
- Документация Stem: `https://stem.torproject.org/`.

Глава 19

Приватная сеть Tor: Chutney и Shadow

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; Chutney commit `6cc1588`; Shadow 3.3.0

Цели главы

К концу этой главы вы сможете:

- объяснять, почему эксперименты с Tor следует проводить в приватной сети, а не в публичной;
- с помощью двух отдельных команд собирать и запускать лабораторную сеть, используемую во всей книге;
- определять параметры конфигурации, отличающие тестовую сеть от реальной;
- ориентироваться в файлах состояния узла, включая ключи, документы консенсуса и журналы;
- исследовать клиентские цепи с помощью Stem и передавать через сеть обычный веб-трафик и трафик onion-сервисов;
- понимать, когда вместо эмулируемой сети Chutney следует использовать симулятор вроде Shadow.

Эта глава находится в Части III, однако прочитать её полезно заранее: каждая лабораторная работа книги, начиная с Лабораторной работы 1 в гл. 7, выполняется в сети, которую мы строим здесь. Читатели, последовательно изучающие Часть II, могут сначала прочитать разд. 19.1 и 19.3, а к остальному материалу вернуться позднее.

19.1. Почему нужна приватная сеть?

Публичная сеть Tor состоит из тысяч реле, управляемых добровольцами, и ежедневно используется миллионами людей, многие из которых полагаются на неё ради собственной безопасности. Эксперименты в этой сети создают две проблемы: этическую и практическую.

Этическая проблема. Любой эксперимент, который измеряет, изменяет или даже просто наблюдает трафик реальной сети, затрагивает пользователей, не дававших на это согласия и потенциально способных пострадать. Даже на первый взгляд безобидный эксперимент, например реле, записывающее больше информации, чем необходимо, может создать данные, которые при раскрытии позволят деанонимизировать человека. Поэтому Tor Project поддерживает *Tor Research Safety Board*, рассматривающий экспериментальные планы исследователей, и рекомендует общий принцип: если на исследовательский вопрос можно ответить,

не вовлекая реальных пользователей, следует поступать именно так. К этим принципам мы вернёмся в гл. 29.

Практическая проблема. Мы почти не контролируем публичную сеть: невозможно заранее знать, какие реле будут выбраны, цепи меняются, консенсус обновляется каждый час, а эксперимент нельзя повторить в абсолютно одинаковых условиях. В приватной сети, напротив, можно выбирать число узлов, читать подробные журналы, останавливать и перезапускать всю сеть за несколько минут и повторять эксперименты, пока результаты не станут понятны.

Внимание: Правило лабораторных работ

Каждое практическое упражнение этой книги, которое отправляет вручную сформированные ячейки, изменяет поведение узла или перехватывает трафик, должно выполняться **исключительно** в приватной сети. Учебный код `torlab` не обеспечивает анонимность и никогда не должен использоваться в публичной сети.

19.2. Эмуляция или симуляция?

Существует два способа построить лабораторную сеть Tor, и оба используются Tor Project.

Эмуляция На одном компьютере запускается множество настоящих процессов `tor`, взаимодействующих через интерфейс обратной петли (loopback). Время реальное, код настоящий, а сетевое взаимодействие обеспечивает операционная система. Именно так работает **Chutney**. Chutney — инструмент Tor Project для интеграционного тестирования tor и Arti.

Симуляция Реальные программы запускаются внутри дискретно-событийного симулятора, который перехватывает системные вызовы и предоставляет виртуальную сеть и виртуальные часы. Так работает **Shadow** [JH12], разработанный для масштабных исследований Tor.

	Chutney (эмуляция)	Shadow (симуляция)
Типичный масштаб	От нескольких узлов до нескольких десятков	От сотен до тысяч узлов
Время	Реальное	Виртуальное и детерминированное
Сеть	Loopback без задержек и потерь	Топология с настраиваемыми задержками и пропускной способностью
Воспроизводимость	Ограничена планировщиком операционной системы	Полная при одинаковом случайном зерне
Типичное применение	Разработка, тестирование, обучение	Исследования производительности и атак
В этой книге	Все лабораторные работы	разд. 19.9 и Часть V

Таблица 19.1: Сравнение Chutney и Shadow.

Для изучения протокола естественным выбором является Chutney. Он лёгкий, запускается за несколько минут и позволяет напрямую взаимодействовать с работающими узлами. Поэтому основная часть главы посвящена Chutney, а Shadow вводится в конце.

19.3. Лабораторная среда книги

Чтобы исключить различия между операционными системами, вся лабораторная среда упакована в Docker-образ, описанный в `labs/ru/лаборатория/Dockerfile` репозитория. Образ:

- собирает tor **из официального исходного кода** в эталонной для книги версии 0.4.9.13, используя `--enable-gpl` для включения модуля доказательства выполнения работы (PoW) для onion-сервисов (гл. 15);
- устанавливает Chutney на зафиксированной ревизии, а также Stem и инструменты перехвата трафика (`tcpdump`, `tshark`), используемые в гл. 20.

Фиксация версий принципиальна: книга, описывающая сообщения протокола байт за байтом, должна гарантировать, что читатель увидит те же байты.

Подробнее: Когда выходит новая версия tor

Результаты, приведённые в этой главе и в Части II, получены в лабораторной сети с tor 0.4.9.13 — эталонной версией книги. Некоторые из них были записаны на предыдущей версии 0.4.9.12, выпущенной всего несколькими днями ранее, а затем воспроизведены: скрипт `scripts/verify_labs.py`, который заново выполняет все лабораторные работы в живой сети и проверяет их результаты, даёт одинаковые итоги на обеих версиях (сейчас 53 проверки плюс 14 для лабораторной работы с мостом). Скрипт выполняет лабораторные работы исходного итальянского издания из `labs/it`, переводом кода которых является код этого издания; в листингах имена и сообщения соответствуют коду этого издания. Меняются только элементы, зависящие от конкретного запуска: ключи, отпечатки, адреса `.onion`, временные метки и размеры сертификатов, которые различаются даже между реле. Тот же скрипт позволяет вам проверить на используемой версии, что описанное в книге по-прежнему верно.

19.3.1. Требования

Потребуется Linux с Docker и примерно 1 GB свободного дискового пространства. Docker Desktop работает в macOS и Windows, но используемый здесь режим сети хоста (host networking) ведёт себя иначе; в таком случае выполняйте эксперименты внутри контейнера (разд. 19.7.4). На хосте сеть использует порты, начинающиеся с 5100, 7100, 8000 и 9000. Убедитесь, что они свободны.

19.3.2. Сборка образа и запуск сети

Из **корня репозитория** (не из каталога `labs/ru/лаборатория`):

```
$ make -C labs/ru/лаборатория image        # собрать tor и подготовить Chutney (один раз)
$ make -C labs/ru/лаборатория start        # настроить и запустить сеть; дождаться первоначальной загрузки
```

Сборка образа занимает несколько минут. На машине, использованной при написании этой главы, запуск сети занял примерно две с половиной минуты и завершился следующим сокращённым отчётом:

```
  test006e:
    bootstrap progress: 100 (SUCCESS), done, Done
    desc publishing: 100 (SUCCESS), all nodes, DESC MD MD_CONS NS_CONS
  test008c:
```

```
    bootstrap progress: 100 (SUCCESS), done, Done
  test009h:
    bootstrap progress: 100 (SUCCESS), done, Done
    oniondesc publishing: 200 (ONIONDESC_PUBLISHED), hidden service v3
```

Каждый узел завершил первоначальную загрузку, каждое реле видит свой дескриптор опубликованным в консенсусе, а onion-сервис опубликовал собственный дескриптор. Затем команда `make -C labs/ru/лаборатория verify` запускает тест Chutney, передавая данные как через выходное реле, так и через onion-сервис, и проверяет целостность:

```
  not-done:0 successes:4 failures:0
Transmission: Success
```

`make -C labs/ru/лаборатория status` показывает состояние узлов. `make -C labs/ru/лаборатория shell` открывает оболочку внутри контейнера, `make -C labs/ru/лаборатория stop` останавливает сеть, а `make -C labs/ru/лаборатория clean` также удаляет созданные данные.

19.4. Топология сети

Сеть книги описывается коротким Python-файлом `labs/ru/лаборатория/chutney/networks/book`, написанным на языке конфигурации Chutney:

```
Authority = Node(tag="a", authority=1, relay=1)
Middle    = Node(tag="r", relay=1)
Exit      = Node(tag="e", relay=1, exit=1)
Client    = Node(tag="c", client=1, launch_phase=2)
Onion     = Node(tag="h", hs=1, launch_phase=2)

NODES = (Authority.getN(3) + Middle.getN(3) + Exit.getN(2)
         + Client.getN(1) + Onion.getN(1))
ConfigureNodes(NODES)
```

Каждый `Node` — шаблон, а `getN` создаёт несколько экземпляров. Клиент и onion-сервис запускаются во второй фазе (`launch_phase=2`), после того как авторитетные серверы сформируют консенсус. Chutney нумерует узлы по порядку и формирует имя каждого из номера и тега: `test000a`, …, `test009h`. Номера узлов также определяют назначение портов, как показано на рис. 19.1.

Узел	Роль	ORPort	ControlPort	Дополнительно
`test000a–002a`	авторитетный сервер каталогов	5100–5102	8000–8002	DirPort 7100–7102
`test003r–005r`	промежуточное реле	5103–5105	8003–8005	
`test006e–007e`	выходное реле	5106–5107	8006–8007	
`test008c`	клиент	—	8008	SocksPort 9008
`test009h`	onion-сервис	—	8009	виртуальный порт 5858

Подробнее: Почему три авторитетных сервера каталогов?

Консенсус должен быть подписан большинством авторитетных серверов каталогов (гл. 12). С одним таким сервером мы не смогли бы наблюдать голосование; с тремя можно увидеть весь процесс и в упражнении проверить, что происходит после остановки одного из них. В публичной сети их девять плюс отдельный авторитетный сервер мостов. Список встроен непосредственно в tor (`src/app/config/auth_dirs.inc`).

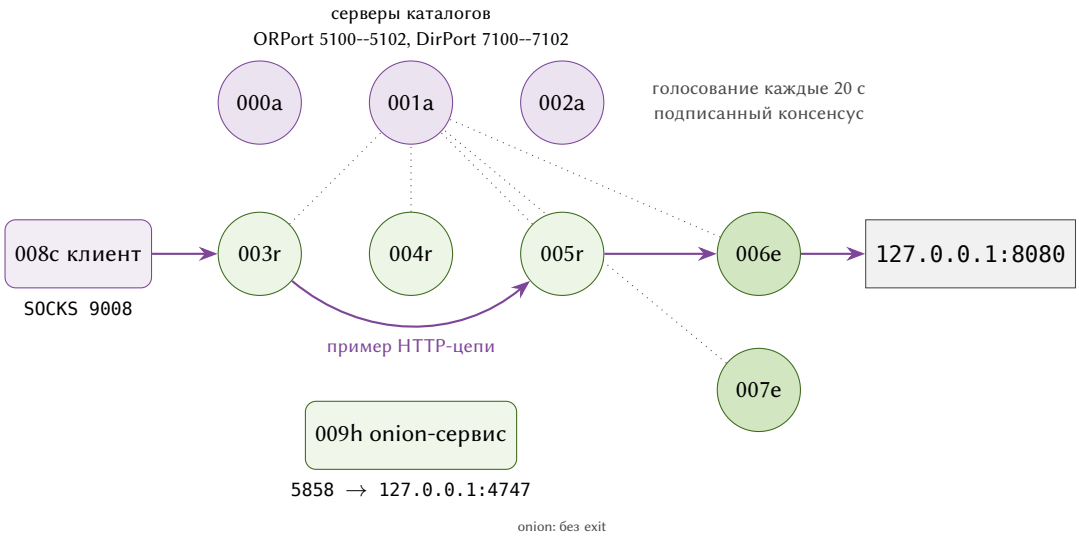


Рис. 19.1: Топология Chutney в лаборатории на `127.0.0.1`: три авторитетных сервера каталогов (также реле), три промежуточных реле, два выходных реле, клиент и onion-сервис. ORPort $5100 + n$ используется только у серверов каталогов и реле, включая выходные; ControlPort $8000 + n$ имеется у каждого узла. Серверы каталогов также имеют DirPort $7100 + n$, клиент — SocksPort 9008, а onion-сервис — виртуальный порт 5858 с перенаправлением на `127.0.0.1:4747`. Пример HTTP-цепи через выходное реле не относится к onion-сервису: тот не использует exit. Loopback не является отдельной сетью контейнера.

19.5. Тестовая сеть — не реальная сеть

Chutney генерирует отдельный `torrc` для каждого узла. Эти файлы показывают, что именно необходимо изменить для работы миниатюрной сети Tor и, следовательно, какие предположения о публичной сети здесь неприменимы. Ниже приведены наиболее важные параметры из torrc одного авторитетного сервера (`net/nodes/000a/torrc`):

```
TestingTorNetwork 1
AuthoritativeDirectory 1
V3AuthoritativeDirectory 1
AssumeReachable 1
V3AuthVotingInterval 20 seconds
V3AuthVoteDelay 4
V3AuthDistDelay 4
TestingDirAuthVoteGuard *
TestingDirAuthVoteHSDir *
```

`TestingTorNetwork` Включает тестовый режим. Его можно использовать только вместе с явным списком авторитетных серверов каталогов (`DirAuthority`), что не позволяет тестовым узлам случайно присоединиться к публичной сети. Кроме того, этот режим изменяет многие значения по умолчанию ради ускорения работы.

`V3AuthVotingInterval` В реальной сети авторитетные серверы голосуют за новый консенсус каждый час. Здесь голосование проходит каждые 20 секунд, что видно и в самом консенсусе:

```
valid-after 2026-09-23 14:37:40
fresh-until 2026-09-23 14:38:00
valid-until 2026-09-23 14:38:20
```

`AssumeReachable` В обычной сети реле сначала проверяет, доступен ли он извне, и лишь затем публикует себя. На интерфейсе обратной петли такая проверка бессмысленна, поэтому пропускается.

`TestingDirAuthVoteGuard *` В реальной сети реле получает флаг Guard только после недель стабильной работы и при достаточной пропускной способности. Здесь авторитетные серверы присваивают этот флаг всем реле. Поэтому **каждое реле нашего консенсуса может быть входным узлом (guard)**, чего в публичной сети нет.

Файлы torrc выходных реле также содержат:

```
ExitPolicyRejectPrivate 0
ExitPolicy accept private:*
```

По умолчанию выходные реле отклоняют соединения к приватным адресам и адресам обратной петли, чтобы не открывать доступ к собственной локальной сети. В этой лаборатории `127.0.0.1` — единственный доступный «Интернет», поэтому политика должна быть обратной.

Внимание: Не копируйте эти параметры в другие среды

Опции `Testing…`, `AssumeReachable` и особенно `ExitPolicy accept private:*` предназначены исключительно для тестовых сетей. На публичном реле такие настройки были бы неверными или опасными для сети хоста.

19.6. Внутри каталога узла

У каждого узла есть собственный каталог данных в `labs/ru/лаборатория/net/nodes/NNNx/`. Среди прочего, каталог авторитетного сервера содержит:

Файл или каталог	Содержимое
`torrc`	конфигурация, сгенерированная Chutney
`keys/`	идентификационные и onion-ключи узла (гл. 5 и 6)
`fingerprint`, `fingerprint-ed25519`	отпечатки идентичности RSA и Ed25519
`cached-consensus`, `cached-microdesc-consensus`	оба варианта консенсуса (гл. 12)
`cached-microdescs.new`	загруженные микродескрипторы
`v3-status-votes`	голоса авторитетных серверов (только для них)
`sr-state`	состояние протокола общей случайности (только для авторитетных серверов)
`notice.log`, `info.log`	журналы уровней *notice* и *info*
`control_auth_cookie`	cookie для аутентификации через порт управления

Каталог onion-сервиса дополнительно содержит `hidden_service/` с его ключами и файл `hostname` с адресом `.onion`, например:

```
rpns2r36zn5u46wctlgnvtwjotxaf7iflsc3k5eas3lqoxeesv4zwyad.onion
```

Адрес меняется при каждой новой конфигурации, поскольку генерируется новый ключ (гл. 14).

Консенсус — обычный человекочитаемый текстовый файл. Если извлечь строки `r` (идентификаторы реле и адреса) и `s` (флаги), получим, например, такие два реле:

```
r test000a H+9DDHx4rEo6I6Jdn8gIyKyi3jk ... 127.0.0.1 5100 7100
s Authority Fast Guard HSDir Running Stable V2Dir Valid
r test006e dEPHoFFHamQyy6pRzlsLB9/ilzU ... 127.0.0.1 5106 0
s Exit Fast Guard HSDir Running Stable V2Dir Valid
```

Как и ожидалось, все реле имеют флаг Guard, а Exit — только два выходных реле. Поля подробно разбираются в гл. 12.

19.7. Использование сети

19.7.1. Доступ через приватную сеть

Оба HTTP-примера выполняйте *внутри лабораторного контейнера*, в каталоге `/lab`, открыв его из отдельного терминала командой `make -C labs/ru/лаборатория shell`. Пустая строка в параметре curl `--noproxy` принуждает использовать SOCKS-прокси даже для `127.0.0.1`, если задана переменная `NO_PROXY`. Сервер на порту 4747 использует тот же `index.html`, который создан в первом примере.

Клиент `test008c` предоставляет SOCKS на порту 9008. Его может использовать любая программа с поддержкой SOCKS5. Запустим небольшой локальный веб-сервер и обратимся к нему через Tor:

```
$ echo "Привет из приватной сети" > index.html
$ python3 -m http.server 8080 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 http://127.0.0.1:8080/index.html
Привет из приватной сети
```

Запрос прошёл по цепи из трёх реле, после чего одно из выходных реле подключилось к серверу. Использование `--socks5-hostname` вместо `--socks5` заставляет exit-реле разрешать имена назначения вместо локальной системы пользователя. В реальной сети это принципиально для предотвращения утечки посещаемых адресов локальному DNS-резолверу (гл. 11).

19.7.2. Доступ к onion-сервису

Chutney настраивает onion-сервис так:

```
HiddenServicePort 5858 127.0.0.1:4747
```

Это сопоставляет виртуальный порт 5858 адреса `.onion` с локальным портом 4747. Остаётся лишь запустить сервер:

```
$ python3 -m http.server 4747 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    http://$(cat net/nodes/009h/hidden_service/hostname):5858/index.html
Привет из приватной сети
```

На этот раз выходное реле не участвует. Клиент и сервис встречаются в пункте встречи (rendezvous) внутри сети (гл. 14).

19.7.3. Наблюдение за цепями с помощью Stem

Порт управления позволяет исследовать работающий узел. С помощью Stem (гл. 18), включённого в образ, выведем цепи клиента:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
```

```
    c.authenticate()                    # cookie используется автоматически
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.status, circ.purpose,
              [name for _, name in circ.path])
```

Фрагмент вывода после `make -C labs/ru/лаборатория verify`:

```
0.4.9.13 (git-3c575400909efe65)
10 BUILT GENERAL ['test004r', 'test005r', 'test007e']
124 BUILT CONFLUX_LINKED ['test006e', 'test004r', 'test007e']
114 BUILT CONFLUX_UNLINKED ['test007e', 'test004r', 'test006e']
128 BUILT CIRCUIT_PADDING ['test004r', 'test001a', 'test007e', 'test006e']
163 BUILT HS_CLIENT_REND ['test004r', 'test001a', 'test006e']
166 BUILT HS_CLIENT_HSDIR ['test004r', 'test006e', 'test007e', 'test003r']
181 BUILT HS_VANGUARDS ['test004r', 'test007e', 'test006e']
```

Даже в столь небольшой сети список предваряет значительную часть книги:

- Почти все цепи начинаются с `test004r`: это **входное реле (guard)** клиента, выбранный один раз и затем многократно используемый (гл. 13). Исключение составляют ветви туннеля Conflux, начинающиеся со второго входного реле (гл. 18).
- Трёхузловые цепи `GENERAL` переносят обычный трафик; одношаговые цепи, не показанные здесь, используются для загрузки документов каталога.
- Цепи `CONFLUX_LINKED` появляются парами и совместно переносят один и тот же трафик: это механизм Conflux (Предложение 329, гл. 31).
- `HS_VANGUARDS`, `HS_CLIENT_HSDIR` и `HS_CLIENT_REND` обозначают цепи, участвующие в доступе к onion-сервису (гл. 14).
- Согласно спецификации протокола управления, `CIRCUIT_PADDING` обозначает цепь, сохраняемую после окончания реального использования, чтобы скрыть точный момент завершения активности. Поэтому её закрытие не обязательно показывает, когда завершилась пользовательская активность.

19.7.4. Работа внутри контейнера

Контейнер использует сетевой режим хоста (host networking), поэтому все порты узлов доступны непосредственно с хостовой системы. Если этот режим недоступен, например в Docker Desktop, `make -C labs/ru/лаборатория shell` открывает оболочку внутри контейнера, где установлены tor, Chutney, Stem, Python и инструменты перехвата; лабораторные файлы доступны в `/lab`.

19.8. Первый контакт с реле

Следующий фрагмент Python импортирует учебный пакет `torlab` из `labs/ru/код/`; контейнер Chutney монтирует только `labs/ru/лаборатория/` в `/lab`, поэтому пакет *не доступен в контейнере автоматически*. В Linux с режимом сети host сохраните фрагмент в файл и запустите из корня репозитория с `PYTHONPATH=labs/ru/код python3 script.py` либо явно предоставьте пакет контейнеру. Пример предназначен исключительно для приватной сети.

Вернёмся к гл. 8. Используя `labs/ru/код/torlab/cells.py` из Лабораторной работы 2, можно открыть TLS-соединение с ORPort лабораторного реле, отправить ячейку **`VERSIONS`** и декодировать ответ. Лабораторная работа 1 в гл. 7 содержит полный код с проверкой сертификатов. Здесь мы лишь наблюдаем начальный обмен:

```
import socket, ssl
from torlab.cells import *

ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE   # только приватная сеть; сертификаты TLS НЕ проверяются
s = ctx.wrap_socket(socket.create_connection(("127.0.0.1", 5103)))
s.sendall(encode(versions_cell([3, 4, 5]), 0))

buf, version, received = b"", 0, 0
while received < 4:
    chunk = s.recv(4096)
    if not chunk:
        raise ConnectionError("TLS channel closed early")
    buf += chunk
    while (r := decode(buf, version)) is not None:
        cell, buf = r
        received += 1
        print(describe(cell, version))
        if cell.command == CellCommand.VERSIONS:
            version = negotiate_version([3, 4, 5], cell.body)
s.close()
```

Предупреждение: эта краткая демонстрация намеренно отключает проверку сертификатов TLS через `ssl.CERT_NONE` и *не* проверяет подлинность реле по ячейке CERTS. Не выполняйте её на публичных реле. Полная процедура проверки приведена в лабораторной работе 1.

Следующий вывод взят из предыдущего эталонного запуска: идентификаторы цепей, отпечатки, onion-адреса, временные метки и размеры сертификатов зависят от конкретного эксперимента.

Реле `test003r` отвечает:

```
CircID=0x0000 VERSIONS тело=6 байт
CircID=0x00000000 CERTS тело=1451 байт
CircID=0x00000000 AUTH_CHALLENGE тело=36 байт
CircID=0x00000000 NETINFO тело=509 байт
```

Это ровно та последовательность, которая описана в разд. 8.8: сначала **`VERSIONS`** с двухбайтовым CircID; после согласования версии 5 последующие ячейки используют четырёхбайтовый CircID. **`CERTS`** и **`AUTH_CHALLENGE`** имеют переменную длину и сохраняют точные тела сообщений; **`NETINFO`** имеет фиксированную длину и несёт все 509 байт. Обратите внимание, что `version = ...` присваивается сразу после **`VERSIONS`**: декодирование следующих ячеек зависит от только что согласованной версии.

19.9. Shadow: сеть внутри симулятора

Chutney недостаточно для вопросов о поведении сети *в целом*. Как другой алгоритм управления перегрузкой повлиял бы на задержку? Какой объём трафика понадобился бы атакующему для определённой атаки? Для ответа нужны тысячи узлов, реалистичные задержки между континентами и, прежде всего, воспроизводимые эксперименты.

Shadow, первоначально разработанный Робом Янсеном и Николасом Хоппером [JH12], решает эти задачи. Современные версии запускают реальные программы, включая tor, как обычные Linux-процессы, одновременно перехватывая сетевые и связанные со временем

системные вызовы. Виртуальный граф задаёт пользовательские задержки и пропускную способность, а время продвигается только тогда, когда это решает симулятор. Поэтому один симулированный час может занимать больше или меньше одного реального часа в зависимости от вычислительной мощности, а повтор эксперимента с тем же случайным зерном даёт тот же результат.

Для построения реалистичных Tor-сетей в Shadow исследователи используют **tornettools**. На основе публичных данных Tor Metrics (гл. 24) он создаёт уменьшенную сеть с распределениями пропускной способности, флагов и ролей, соответствующими реальной сети.

Мы используем Shadow в управляемом эксперименте в гл. 29. Пока достаточно понимать, когда именно симулятор является подходящим инструментом.

Итоги

- Проводите эксперименты с Tor в приватных сетях. В публичной сети они могут затрагивать пользователей без их согласия и не обеспечивают воспроизводимость.
- Chutney эмулирует Tor-сеть настоящими процессами `tor` на интерфейсе обратной петли. Shadow запускает настоящее программное обеспечение внутри дискретно-событийного симулятора, подходящего для больших сетей и повторяемых экспериментов.
- Лабораторная среда книги (`labs/ru/лаборатория/`) собирает tor 0.4.9.13 из исходного кода. `make -C labs/ru/лаборатория start` запускает сеть из десяти узлов: три авторитетных сервера, три промежуточных реле, два выходных реле, клиент и onion-сервис.
- Узел n использует ORPort $5100 + n$, ControlPort $8000 + n$ и SocksPort $9000 + n$.
- Тестовые настройки (`TestingTorNetwork`, голосование каждые 20 секунд, назначение Guard всем реле и разрешение приватных адресов для выходных реле) позволяют миниатюрной сети работать, но отличают её от реальной. Результаты измерений требуют осторожности при экстраполяции.
- SocksPort 9008 передаёт прикладной трафик через сеть; Stem наблюдает цепи; `torlab` напрямую взаимодействует с реле.

Упражнения

Упражнение 19.1 ⋆ Не читая torrc, назовите ORPort и ControlPort узла `test006e`. Есть ли ORPort у `test009h`? Почему?

Упражнение 19.2 ⋆ Каждое реле приватной сети имеет флаг Guard. Какая опция вызывает это поведение и почему в реальной сети всё иначе? Как это влияет на лабораторные измерения выбора маршрута?

Упражнение 19.3 ⋆ Повторите запрос `curl` из разд. 19.7, используя `--socks5` вместо `--socks5-hostname` и `localhost` вместо `127.0.0.1`. Что изменится в данных, получаемых exit-реле? Почему это важно в публичной сети?

Упражнение 19.4 ⋆⋆ Несколько минут наблюдайте поле `valid-after` консенсуса клиента (`grep valid-after net/nodes/008c/cached-microdesc-consensus`). Как часто оно меняется? Вычислите, сколько консенсусов создаётся за сутки в лабораторной и реальной сетях.

Упражнение 19.5 ⋆⋆ Остановите одну из трёх authorities (`make -C labs/ru/лаборатория shell`, затем `kill` соответствующего процесса, используя его файл `pid`). Продолжает ли

сеть создавать консенсусы? Что происходит после остановки двух authorities? Объясните результат; основная причина рассмотрена в гл. 12.

Упражнение 19.6 ⋆⋆ Измените сценарий Stem из разд. 19.7.3, чтобы он выводил ORPort всех реле в каждой трёхузловой цепи, получая их из консенсуса через `get_network_status`.

Упражнение 19.7 ⋆⋆ Создайте `labs/ru/лаборатория/chutney/networks/book-large` с пятью авторитетными серверами, десятью промежуточными и четырьмя выходными реле, затем запустите его, установив `NETWORK=book-large` в команде `make`. Как время первоначальной загрузки соотносится со временем запуска сети книги?

Упражнение 19.8 ⋆⋆⋆ Найдите в `info.log` клиента сообщения, относящиеся к построению цепи, начиная с выбора узлов и заканчивая завершением. Сопоставьте каждое сообщение с ячейкой на рис. 8.5.

Упражнение 19.9 ⋆ ⋆ ⋆ Внутри контейнера перехватите трафик к порту 5103 командой `tcpdump -i lo` во время запуска сценария из разд. 19.8. Видны ли ячейки? Почему нет? Что всё ещё остаётся наблюдаемым: размеры, временные интервалы? Глава 20 показывает, как исследовать глубже.

Дополнительная литература

- Chutney, `https://gitlab.torproject.org/tpo/core/chutney`: README и каталог `networks/` содержат множество примеров сетей, включая конфигурации с мостами и Arti.
- R. Jansen, N. Hopper, *Shadow: Running Tor in a Box for Accurate and Efficient Experimentation* [JH12].
- Документация Shadow и tornettools: `https://shadow.github.io/`.
- Tor Research Safety Board, `https://research.torproject.org/safetyboard/`: рекомендации по этическим исследованиям Tor.

Глава 20

Наблюдение за трафиком: Wireshark и журналы

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; tshark (Debian trixie); лабораторная сеть

Цели главы

К концу этой главы вы сможете:

- перехватывать трафик Tor с помощью `tcpdump` и анализировать его через `tshark` или Wireshark;
- отличать то, что видит внешний наблюдатель, от того, что остаётся скрытым;
- расшифровывать TLS на *собственном* канале с помощью сессионных ключей и исследовать его ячейки (Лабораторная работа 12);
- пошагово прослеживать протокол по журналам tor.

В предыдущих главах мы исследовали протокол «изнутри», создавая код, который разговаривает на нём. Здесь мы наблюдаем его «снаружи», как это делал бы тот, кто следит за сетью. У такого упражнения две противоположные цели: понять, что атакующий может вывести из трафика, и получить независимый способ проверки собственной реализации.

Внимание: Только собственный трафик и только в лабораторной среде

Перехват чужого трафика без разрешения незаконен в Италии, как и в большинстве стран (гл. 25), и противоречит этике исследований анонимности (разд. 19.1). Все перехваты в этой главе относятся либо к приватной сети книги, либо к соединениям, которые мы открываем сами.

20.1. Что может видеть наблюдатель

Тот, кто наблюдает соединение между клиентом и его входным реле (guard), например интернет-провайдер, может видеть:

- IP-адреса и порты: клиент подключается к адресу, который, если это не мост (bridge), присутствует в публичном консенсусе и поэтому распознаётся как реле Tor;
- рукопожатие TLS: сертификаты передаются в зашифрованном виде при TLS 1.3, но некоторые свойства начального сообщения клиента остаются видимыми;
- размеры и временные характеристики записи TLS.

Наблюдатель не видит содержимое ячеек, цепи или адреса назначения. Однако именно размеры и временные характеристики лежат в основе атак *website fingerprinting* и корреляции (гл. 26). Зашифрованное содержимое не означает, что трафик вообще ничего не раскрывает.

В разд. 16.5 мы сравнивали обычный Tor-трафик, распознаваемый как TLS, с трафиком obfs4, не имеющим узнаваемой структуры. Здесь мы пойдём ещё дальше и заглянем внутрь TLS.

20.2. Инструменты

`tcpdump` Перехватывает пакеты на сетевом интерфейсе и сохраняет их в файл `pcap`. Например, внутри лабораторного контейнера можно перехватить трафик к ORPort узла `test004r`:

```
$ docker exec -u 0 torbook-network tcpdump -i lo \
    -w /lab/net/tls.pcap "tcp port 5104"
```

Wireshark и `tshark` Анализируют захваченные пакеты: первый через графический интерфейс, второй из командной строки. Они распознают TLS, но официального анализатора протокола Tor нет. Даже после расшифрования ячейки Tor выглядят как необработанные байты. В роли анализатора будет выступать наш модуль `cells.py`.

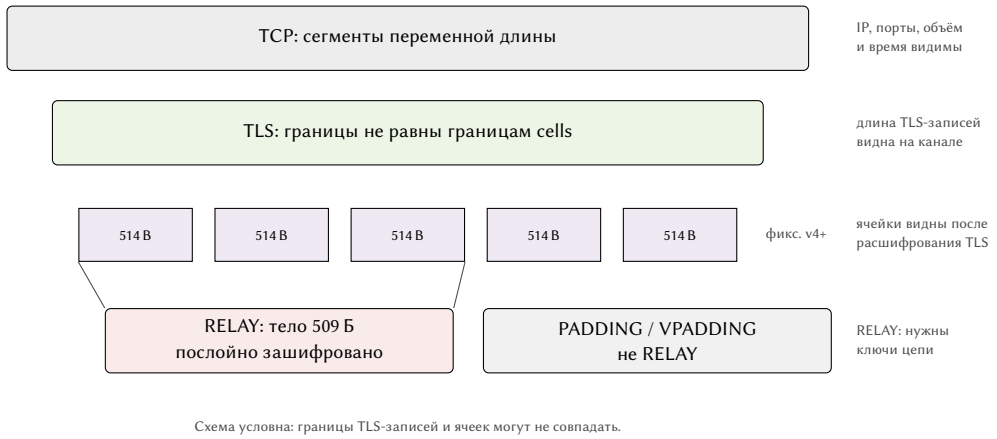


Рис. 20.1: Схема канала Tor: сегменты TCP передают записи TLS, границы которых не обязаны совпадать с границами ячеек. Наблюдатель видит IP-адреса, порты, длины и время; после расшифрования TLS можно выделить ячейки. При версии протокола канала v4+ фиксированная ячейка занимает 514 байт (заголовок 5, тело 509), но существуют также ячейки переменной длины. Послойно шифруются тела ячеек `RELAY`, а ячейки заполнения не являются сообщениями RELAY.

20.2.1. Расшифрование собственного TLS

TLS использует сессионные ключи, которыми внешний наблюдатель не располагает. Однако сторона, *участвующая* в соединении, может экспортировать эти ключи. Браузеры и приложения Python могут записывать ключи сессии в файл, если включено их журналирование. Наш модуль `channel.py` явно устанавливает `SSLContext.keylog_filename`, когда определена переменная `SSLKEYLOGFILE`. Wireshark и `tshark` используют этот файл для расшифрования перехвата. Скрипт `channel.py` из Лабораторной работы 1 следует этому соглашению: установите переменную перед запуском. Файл содержит строки вида:

```
# TLS secrets log file, generated by OpenSSL / Python
SERVER_HANDSHAKE_TRAFFIC_SECRET 4878224d3f9fb9aedb96d... 0da89dff6feb...
```

Они содержат случайное клиентское значение и секреты каждой фазы TLS 1.3 для соответствующей сессии.

Подробнее: Почему это не атака на Tor

Расшифрование TLS на собственном канале не раскрывает ничего, чего клиент уже не знал. Даже при наличии сессионных TLS-ключей ячейки **RELAY** остаются послойно зашифрованными ключами отдельных переходов (гл. 10). Для их чтения потребовались бы ключи цепи, которыми располагают только клиент и соответствующие реле. TLS — внешняя оболочка многослойного шифрования, а не единственный уровень защиты.

20.3. Лабораторная работа: ячейки внутри TLS

Лабораторная работа 12: Перехват, расшифрование и чтение канала

Цель. Перехватить канал, открытый нашим клиентом к реле, расшифровать его с помощью сессионных ключей и восстановить последовательность ячеек через `labs/ru/код/torlab/traffic.py`.
Предварительные условия. Лабораторные работы 1–3; лабораторная сеть должна быть запущена; в контейнере должны быть установлены `tcpdump` и `tshark`.
Оценочное время. 1–2 часа.

Для лабораторной работы 12 запустите приватную сеть и откройте два терминала из корня репозитория. Следующую команду захвата выполняйте *на хосте*: она запускает `tcpdump` от root *внутри лабораторного контейнера*, поскольку обычная оболочка Makefile работает с правами непривилегированного пользователя. Пока во втором терминале работает клиент, оставьте захват активным, затем остановите его сочетанием Ctrl+C. Файл сохраняется в общем томе `labs/ru/лаборатория/net/`.

Шаг 1: перехват. Запустите перехват внутри контейнера. Затем с хоста откройте канал к `test004r` и постройте одноузловую цепь, одновременно экспортируя сессионные ключи:

```
$ PYTHONPATH=labs/ru/код SSLKEYLOGFILE=labs/ru/лаборатория/net/tls-keys.log python3 -c
    "
from torlab.laboratory import relay
from torlab.circuit import build
build([relay('004r')], cc=False)"
```

Команду Python выполняйте из *корня репозитория* в Linux с сетевым режимом хоста. `PYTHONPATH` делает пакет `torlab` из `labs/ru/код/` доступным для импорта; `labs/ru/код/torlab/channel.py` явно считывает `SSLKEYLOGFILE` и включает экспорт секретов TLS. Файл содержит секретные ключевые данные: не публикуйте его и удалите после завершения опыта.

Шаг 2: определение нашего соединения. Другие лабораторные реле тоже подключаются к тому же порту, но их сессионные ключи отличаются. Попросим `tshark` проследить каждый поток TCP после попытки расшифрования и сохраним тот, который успешно расшифровывается:

```
$ docker exec torbook-network tshark -r /lab/net/tls.pcap \
    -o tls.keylog_file:/lab/net/tls-keys.log \
    -q -z follow,tls,raw,2
```

Номер потока TCP 2 относится лишь к предыдущей демонстрационной записи. Найдите возможные номера потоков с помощью `tshark` и `-T fields -e tcp.stream`, затем выберите расшифровываемый поток. Приведённый ниже вывод исторический: размеры сертификатов и идентификаторы цепей меняются при новом запуске. Вывод содержит расшифрованные данные обоих направлений в шестнадцатеричном виде; одно из направлений отображается с отступом табуляцией.

Шаг 3: исследование ячеек. Функция `cells_from_follow` восстанавливает оба потока и передаёт их в `decode` из Лабораторной работы 2. Каждое направление имеет собственное согласование версии: первой ячейкой является **`VERSIONS`** с двухбайтовым CircID. Четырёхбайтовые CircID используются только после согласования. Результат:

```
клиент -> реле  CircID=0x0000 VERSIONS тело=4 байт
реле   -> клиент CircID=0x0000 VERSIONS тело=6 байт
реле   -> клиент CircID=0x00000000 CERTS тело=1443 байт
реле   -> клиент CircID=0x00000000 AUTH_CHALLENGE тело=36 байт
реле   -> клиент CircID=0x00000000 NETINFO тело=509 байт
клиент -> реле  CircID=0x00000000 NETINFO тело=509 байт
клиент -> реле  CircID=0xc244f1f6 CREATE2 тело=509 байт
реле   -> клиент CircID=0xc244f1f6 CREATED2 тело=509 байт
```

При внешнем наблюдении это в точности последовательность из рис. 8.5 и гл. 7. Ячейка клиента **`VERSIONS`** имеет тело длиной четыре байта, поскольку `channel.py` предлагает только версии 4 и 5. У Circuit ID `0xc244f1f6` старший бит равен 1, поскольку канал открыл клиент (разд. 8.4).

Внимание: Две ошибки в нашей первой версии

Первоначальный анализ содержал две ошибки, которые обнаружились при сравнении вывода с ожидаемой последовательностью. Мы использовали одну согласованную версию протокола для обоих направлений и поэтому неправильно интерпретировали ячейку **`VERSIONS`** от реле. Кроме того, мы назначили блоки неверным направлениям, потому что предположили, какое именно направление `tshark` выводит с отступом. Правильное правило простое: первый блок всегда является клиентской ячейкой **`VERSIONS`**.

20.4. Журналы tor

Перехват пакетов показывает, *что* передаётся по сети. Журналы tor объясняют, *почему*. На уровне `info` tor записывает этапы построения цепей, выбор входных реле, загрузку и публикацию дескрипторов. На уровне `debug` записи доходят до отдельных ячеек. Мы уже использовали журналы, чтобы проверить, к какому HSDir обращался клиент (разд. 14.9) и когда onion-сервис ротировал свои дескрипторы.

Несколько практических рекомендаций:

- Уровень `debug` создаёт огромные объёмы текста. Включайте его только для конкретного домена, например `Log [circ]debug file circ.log`, чтобы ограничить подробные сообщения событиями цепей.

- В реальной сети `SafeLogging` маскирует адреса. Журналы клиента никогда не должны содержать посещаемые им назначения.
- Журналы публичного реле могут содержать персональные данные других людей с юридическими последствиями, рассматриваемыми в гл. 25; сохраняйте только строго необходимую информацию.

Итоги

- Наблюдатель соединения видит адреса, рукопожатие TLS, размеры пакетов и временные характеристики, но не ячейки Tor и не адреса назначения.
- `SSLKEYLOGFILE` позволяет расшифровать TLS на собственном канале и исследовать его ячейки; ячейки **`RELAY`** при этом всё равно остаются послойно зашифрованными.
- Анализ перехваченного трафика предоставляет независимый способ проверить реализацию.
- Журналы tor уровней `info` и `debug` объясняют решения программы; при реальном использовании объём логирования следует минимизировать и защищать.

Упражнения

Упражнение 20.1 ⋆ В перехвате загрузки через Tor не все записи TLS имеют длину, кратную 514 байтам. Почему, если ячейки имеют фиксированный размер?

Упражнение 20.2 ⋆⋆ Расширьте Лабораторную работу 12, расшифровав ячейки **`RELAY`** собственной цепи с помощью ключей из `circuit.py`. Какие сообщения реле видны при построении трёхузловой цепи?

Упражнение 20.3 ⋆⋆ Перехватывайте канал лабораторного клиента в течение одной минуты без создания прикладного трафика. Подсчитайте записи TLS в каждом направлении и сопоставьте наблюдения с заполнением канала (channel padding) из гл. 7.

Упражнение 20.4 ⋆⋆⋆ Напишите небольшой анализатор Wireshark на Lua, который распознаёт ячейки Tor в расшифрованных данных и отображает их CircID и команду.

Дополнительная литература

- Документация Wireshark по расшифрованию TLS с использованием файлов журнала сессионных ключей.
- Раздел `Log` руководства tor, описывающий домены журналирования и уровни детализации.

Глава 21

Публикация onion-сервиса

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13; документация Onionbalance

Цели главы

К концу этой главы вы сможете:

- настраивать onion-сервис и защищать его ключи;
- избегать ошибок конфигурации, раскрывающих сервер;
- предлагать посетителям обычного веб-сайта его версию `.onion` через Onion-Location;
- ограничивать доступ авторизованными клиентами с помощью ограниченного обнаружения (restricted discovery, Лабораторная работа 13);
- оценивать адреса с узнаваемым префиксом (vanity), одношаговые сервисы и балансировку нагрузки через Onionbalance.

В гл. 14 мы разобрали, как работают onion-сервисы. Эта глава — практическое продолжение: публикация сервиса, предотвращение простых ошибок, способных нарушить его анонимность, и выбор инструментов для типичных эксплуатационных задач.

21.1. Первый onion-сервис

Достаточно двух строк в `torrc`:

```
HiddenServiceDir /var/lib/tor/my_service/
HiddenServicePort 80 127.0.0.1:8080
```

Путь `HiddenServiceDir` должен быть доступен процессу tor для чтения и записи; в лаборатории это путь *внутри контейнера Chutney*. В первом примере создаётся *общедоступный* onion-сервис, которому файл `alice.auth` не нужен. Авторизация клиентов и файлы `authorized_clients/*.auth` рассматриваются далее в этой главе. Первая задаёт каталог, в котором tor хранит ключи сервиса. Вторая сопоставляет *виртуальный* порт 80 его адреса `.onion` с локальным сервером на порту 8080. После перезапуска tor создаёт в этом каталоге следующие файлы:

`hostname` Адрес `.onion` (разд. 14.2).

`hs_ed25519_public_key`, `hs_ed25519_secret_key` Пара идентификационных ключей сервиса. **Тот, кто владеет секретным ключом, контролирует адрес**: он может публиковать дескрипторы и выдавать себя за сервис. Защищайте этот ключ так же, как закрытый ключ сертификата, и храните безопасную резервную копию. Если ключ потерян, адрес утрачен навсегда.

Каталог должен быть доступен только пользователю, от имени которого работает tor; в противном случае tor откажется его использовать. Один процесс tor может размещать несколько сервисов, каждый в своём каталоге. В лаборатории мы добавили второй сервис к `test009h` ещё двумя строками.

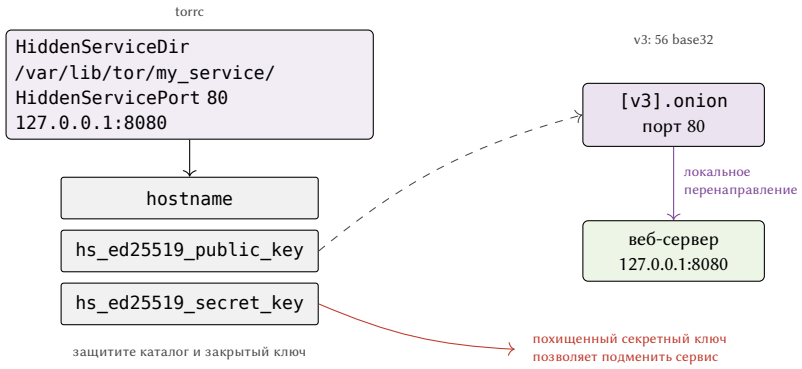


Рис. 21.1: Директивы `HiddenServiceDir` и `HiddenServicePort` создают общедоступный onion-сервис v3. Tor генерирует в каталоге `hostname` и ключи Ed25519; адрес содержит 56 символов base32, полученных из открытого ключа, контрольной суммы и байта версии (в схеме указан заполнитель). Виртуальный порт 80 перенаправляется на `127.0.0.1:8080`. Кража закрытого ключа позволяет подменить сервис; для этого публичного примера файл `alice.auth` не нужен.

21.1.1. Как не раскрыть сервер

Протокол скрывает адрес сервера, но небрежная конфигурация может его выдать. Типичные ошибки:

- **Одновременная внешняя доступность сервиса.** Если веб-сервер слушает все интерфейсы, а не только `127.0.0.1`, сканирование Интернета может обнаружить его и сопоставить содержимое с onion-сервисом. Привязывайте сервер к localhost или, предпочтительно, используйте сокет Unix (`HiddenServicePort 80 unix:/path/socket`).
- **Утечки в ответах.** Страницы ошибок могут раскрывать имя сервера, HTTP-заголовки — внутренние адреса, TLS-сертификаты могут быть выпущены для публичного домена, а страницы состояния веб-сервера — оставлены доступными.
- **Внешние ресурсы.** Страницы, загружающие изображения или скрипты с обычных веб-сайтов, отправляют эти запросы за пределы onion-сервиса. В Tor Browser они всё равно проходят через Tor, поэтому IP-адрес посетителя не раскрывается, но сквозное шифрование onion-сервиса больше не покрывает эти ресурсы, и выходное реле может наблюдать метаданные соединения и незашифрованный HTTP-трафик (содержимое HTTPS по-прежнему защищено). Серверные приложения, самостоятельно обращающиеся к внешним сервисам за обновлениями или аналитикой, могут раскрыть адрес сервера.
- **Часы и временные характеристики.** Сервер, отображающий локальное время, связанное с конкретным часовым поясом, или становящийся недоступным одновременно с известным публичным сервером во время обслуживания, оставляет потенциально коррелируемые следы.

В гл. 28 мы рассмотрим реальные случаи, где подобные ошибки помогли определить местоположение сервисов.

21.2. Onion-Location

Многие публичные сайты, включая крупные новостные организации, социальные сети и Tor Project, предлагают версию `.onion`. Обычный веб-сайт может сообщить посетителю Tor Browser о такой версии через HTTP-заголовок:

```
Onion-Location: http://<address>.onion/path
```

Эквивалентный элемент `meta` можно использовать в самой странице. Tor Browser после этого показывает кнопку в адресной строке, предлагающую перейти на onion-версию. Посетитель избегает выходное реле и получает соединение, аутентифицированное самим адресом; сайт принимает Tor-трафик без выхода через выходное реле, которые иногда блокируются или считаются подозрительными.

21.3. Ограниченное обнаружение

Личный сервис, например SSH-доступ к домашнему серверу или административная панель, не обязан быть доступным всем, кто знает его адрес. Ограниченное обнаружение (restricted discovery, разд. 14.7) шифрует второй уровень дескриптора так, чтобы расшифровать его могли только авторизованные клиенты. Конфигурация использует пары ключей X25519 [Thee]:

На стороне сервиса Поместите по одному файлу `.auth` на клиента в каталог `authorized_clients/` сервиса. Каждый файл содержит строку `descriptor:x25519:`, за которой следует открытый ключ клиента в base32.

На стороне клиента Опция `ClientOnionAuthDir` указывает каталог, содержащий по одному файлу `.auth_private` на сервис. В каждом файле указывается адрес без `.onion`, затем `:descriptor:x25519:` и закрытый ключ клиента в base32.

Tor Browser позволяет ввести закрытый ключ при посещении сервиса, который его требует.

Предупреждение о безопасности лаборатории: скрипт `labs/ru/лаборатория/restricted_discovery.py` во время начальной очистки удаляет существующие каталоги `restricted_service/` и `onion_auth/`. Запускайте его *только* в выделенной одноразовой сети Chutney, убедившись, что в каталогах нет нужных данных. Не используйте его для настоящих onion-сервисов. При ручном выполнении сначала создайте файлы авторизации и только затем перезагрузите затронутые узлы.

Лабораторная работа 13: Onion-сервис с ограниченным обнаружением

Цель. Настроить второй onion-сервис на `test009h`, доступный только авторизованному клиенту, и проверить доступ с ключом и без него.
Предварительные условия. Лабораторная сеть должна быть запущена; локальный веб-сервер должен работать на порту 8080.
Оценочное время. 1 час.

Шаг 1: пара ключей клиента. Сгенерируем её с помощью библиотеки `cryptography`: Следующий самостоятельный пример Python печатает строку с открытым ключом авторизации. Закрытый ключ `priv` необходимо надёжно сохранить отдельно для создания файла `.auth_private`; никому его не передавайте.

```
1 import base64
2 from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
```

```
from cryptography.hazmat.primitives.serialization import (
    Encoding, PrivateFormat, PublicFormat, NoEncryption
)

sk = X25519PrivateKey.generate()
priv = sk.private_bytes(Encoding.Raw, PrivateFormat.Raw, NoEncryption())
pub = sk.public_key().public_bytes(Encoding.Raw, PublicFormat.Raw)
b32 = lambda b: base64.b32encode(b).decode().rstrip("=")
print("descriptor:x25519:" + b32(pub))
```

Значения base32 должны записываться без завершающих символов заполнения =. Фрагмент печатает только открытую строку авторизации и *не сохраняет* закрытый ключ. При отдельном запуске сценария переменная `priv` исчезнет после завершения процесса. Для ручного выполнения заранее сохраните ключ в защищённый файл с ограниченными правами либо используйте автоматизированную лабораторную работу: она создаёт клиентский файл авторизации и удаляет временные ключи по окончании. Не следует смешивать ручной и автоматический сценарии.

Шаг 2: сервис. Создайте `restricted_service/authorized_clients/alice.auth` с открытым ключом. Добавьте в `torrc` узла `test009h`:

```
HiddenServiceDir /lab/net/nodes/009h/restricted_service
HiddenServicePort 80 127.0.0.1:8080
```

Затем отправьте `HUP`. tor сгенерирует адрес и опубликует дескрипторы.

Следующая команда считывает настоящий onion-адрес этого экземпляра из файла `hostname`, который tor создаёт вместе с сервисом: у каждой сети свой адрес. Ошибка SOCKS и сообщения журнала ниже взяты из предыдущей эталонной сессии и могут отличаться при новом запуске.

Шаг 3: без ключа. На клиенте сети:

```
$ ONION_ADDR=$(cat labs/ru/лаборатория/net/nodes/009h/restricted_service/hostname)
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    "http://${ONION_ADDR}/index.html"
(curl: SOCKS proxy error 97)
```

Журнал клиента сообщает:

```
[notice] Fail to decrypt descriptor for requested onion address.
         It is likely requiring client authorization.
```

Клиент нашёл и загрузил дескриптор, но не может расшифровать его второй уровень. Он даже не знает точек введения (introduction points) и потому не способен попытаться установить соединение.

Внимание: Пустой каталог делает сервис недоступным

Если `authorized_clients/` существует, но не содержит ни одного файла, сервис в исходном эксперименте попытался опубликовать дескриптор, который серверы HSDir отклонили:

```
[warn] Uploading hidden service descriptor: http status 400 ("Invalid HS
       descriptor. Rejected.") response from dirserver 127.0.0.1:5106.
```

Соответствующая спецификация требует как минимум одну строку `auth-client` в дополнительно зашифрованном слое (superencrypted) [rend-spec § Hidden service descriptors:

encryption format]. Если ограниченное обнаружение активировано, но авторизованных клиентов нет, наблюдаемая конфигурация не создаёт ни одной такой строки, поэтому дескриптор оказывается недопустимым. Это эксплуатационная ловушка при отзыве последнего клиента или создании каталога до добавления ключей. Прерванный лабораторный сценарий также может оставить тестовую конфигурацию в незавершённом состоянии.

Шаг 4: с ключом. Запишите файл `.auth_private` в каталог, указанный через `ClientOnionAuthDir`, перезагрузите конфигурацию клиента и повторите запрос. Теперь страница будет доставлена. Скрипт `labs/ru/лаборатория/restricted_discovery.py` автоматизирует эксперимент, очистку и восстановление `torrc`; его фактический запуск не проводился в рамках редакционной проверки.

21.4. Другие требования

21.4.1. Адреса с узнаваемым префиксом

Адрес, начинающийся с узнаваемого слова, может помочь пользователю визуально его распознавать. Для этого специализированная программа вроде *mkp224o* генерирует случайные ключи, пока адрес не начнётся с нужного префикса. Каждый дополнительный символ base32 увеличивает ожидаемое число попыток в 32 раза. Префикс из шести или семи символов можно получить за разумное время на обычном компьютере, но каждый следующий символ требует во много раз больше работы. Как обсуждалось в разд. 14.2, атакующий тоже может создать адрес с тем же префиксом. Префикс помогает распознаванию, но не аутентифицирует сервис. Обращайтесь с ключами, созданными сторонними программами, так же осторожно, как с любыми другими секретными ключами.

21.4.2. Одношаговые сервисы

Публичный веб-сайт, которому не требуется скрывать собственное местоположение, например onion-версия известной газеты, может пожертвовать анонимностью сервера ради скорости. При включённых `HiddenServiceSingleHopMode` и `HiddenServiceNonAnonymousMode` сервис строит одношаговые цепи к точкам введения (introduction points) и встречи (rendezvous points). Клиент остаётся анонимным как обычно. Публикация дескрипторов всё равно использует трёхузловые цепи: как мы выяснили в разд. 14.9, HSDir отклоняют прямые запросы, и это прямо отмечено в руководстве tor. Слово «NonAnonymous» в названии опции выбрано намеренно: экземпляр tor, настроенный таким образом, нельзя одновременно использовать как анонимный клиент. Кроме того, такое решение трудно обратить. После работы сервиса в неанонимном режиме его IP-адрес может быть связан с адресом `.onion`. Простого отключения опции недостаточно для восстановления анонимности. Необходимы новый адрес, новые ключи, новый `HiddenServiceDir` и чистый экземпляр tor.

21.4.3. Onionbalance

Один процесс tor одновременно является узким местом производительности и единой точкой отказа. Onionbalance [Theb] позволяет нескольким серверам обслуживать один адрес. Каждый сервер запускает собственный onion-сервис со своими точками введения. Onionbalance хранит ключ основного адреса и публикует дескриптор, объединяющий точки введения всех серверов. Клиенты распределяются между ними прозрачно. Tor Project использует Onionbalance для собственных сайтов.

Итоги

- Onion-сервису достаточно двух строк конфигурации. Его секретный ключ *контролирует* адрес и должен быть защищён и резервно сохранён.
- Сервер должен слушать только localhost или сокет Unix и не должен раскрывать сведения о себе через ответы или внешние соединения.
- Onion-Location сообщает посетителям Tor Browser о наличии onion-версии сайта.
- Ограниченное обнаружение использует ключи X25519: без авторизации клиент не может даже расшифровать дескриптор.
- Адреса с узнаваемым префиксом, одношаговые сервисы и Onionbalance решают задачи узнаваемости, скорости и масштабирования.

Упражнения

Упражнение 21.1 ⋆ Сколько попыток в среднем требуется для генерации шестисимвольного префикса base32? А восьмисимвольного? Если компьютер проверяет десять миллионов ключей в секунду, сколько времени займёт каждый вариант?

Упражнение 21.2 ⋆⋆ Onion-сервис показывает на страницах ошибок строку `Apache/2.4.62 (Debian) Server at 192.0.2.10 Port 80`. Какую информацию она раскрывает? Как избежать этой утечки?

Упражнение 21.3 ⋆⋆ Как в Лабораторной работе 13 отозвать доступ Алисы? Что произойдёт с клиентами, которые уже загрузили дескриптор?

Упражнение 21.4 ⋆⋆⋆ Измените лабораторный сервис так, чтобы соединения перенаправлялись в Unix socket, а не на локальный TCP-порт. Какие дополнительные преимущества это даёт помимо отсутствия открытого порта?

Дополнительная литература

- Разделы руководства tor для `HiddenServiceDir`, `HiddenServicePort` и client authorization.
- Документация Tor Project по onion-сервисам и Onionbalance [Theb].

Глава 22

Эксплуатация реле или моста

Последняя проверка: 23 сентября 2026 г. — tor 0.4.9.13 (руководство); лабораторная сеть

Цели главы

К концу этой главы вы сможете:

- выбирать между промежуточным реле, выходным реле и мостом с учётом доступных ресурсов и приемлемых рисков;
- писать базовую конфигурацию для каждого варианта, включая ограничения пропускной способности и хранение ключей вне сети;
- читать и составлять политику выхода и проверять её влияние в лабораторной сети (Лабораторная работа 14);
- правильно объявлять семейство реле.

Сеть Tor существует благодаря тысячам людей и организаций, предоставляющих пропускную способность и компьютеры. В предыдущих главах мы рассматривали реле с точки зрения клиента; теперь перейдём на другую сторону. Эта глава не является руководством по системному администрированию — Tor Project поддерживает актуальное руководство оператора, — а объясняет, что означают основные эксплуатационные решения на уровне протокола.

22.1. Какой тип узла выбрать?

Промежуточное реле (которое со временем может также стать входным реле, guard): пересылает ячейки между другими реле, но никогда не открывает соединения к внешним интернет-ресурсам. Это наименее требовательная роль. Его адрес присутствует в консенсусе, но веб-сайты не видят трафик, исходящий от него.

Выходное реле Открывает соединения к назначениям от имени клиентов. Эта роль особенно ценна, поскольку выходных реле мало относительно спроса (разд. 13.2), но именно она создаёт наибольшую внешнюю нагрузку и риск: получатели вредоносного или нежелательного трафика видят IP-адрес выходного реле как кажущийся источник.

Мост Это реле, не включённое в публичный список и используемое как первый переход людьми, которые не могут подключиться к публичной сети (разд. 16.2). Даже скромное домашнее соединение может быть полезно в сочетании с подключаемым транспортом, например obfs4 или WebTunnel.

Внимание: Перед началом

Эксплуатация выходного реле имеет юридические и операционные последствия, включая сообщения о злоупотреблениях и запросы от органов власти; они рассматриваются в гл. 25. Разумно начать с промежуточного реле или моста, уведомить провайдера связи и не размещать реле на том же интернет-подключении, которое используется для чувствительной деятельности через Tor в роли клиента.

22.2. Базовая конфигурация

Для промежуточного реле достаточно нескольких строк `torrc`:

```
Nickname        MyRelay
ContactInfo     operator@example.org
ORPort          443
ExitRelay       0
SocksPort       0
RelayBandwidthRate  4 MBytes
RelayBandwidthBurst 8 MBytes
```

`ContactInfo` позволяет другим связаться с оператором при проблемах с реле. Руководство напоминает, что дескрипторы архивируются и публикуются, поэтому этот адрес может оставаться публично доступным неопределённо долго. `ExitRelay 0` явно фиксирует поведение, которое уже является значением по умолчанию в современных версиях: при значении `auto` tor становится выходным реле только тогда, когда оператор задаёт один из параметров, разрешающих выходной трафик. `RelayBandwidthRate` и `RelayBandwidthBurst` ограничивают среднюю и пиковую пропускную способность. В отличие от них, `AccountingMax` ограничивает общий объём трафика за расчётный период, например месяц. После достижения лимита реле перестаёт принимать новый трафик до следующего периода.

Практическое замечание. При значении `ExitRelay auto` к включению режима выходного реле могут привести не только явные правила `ExitPolicy`, но и настройки `ReducedExitPolicy` или `IPv6Exit`. Напротив, `ExitRelay 0` из примера полностью запрещает выходной трафик. Перед запуском публичного реле проверяйте эффективную конфигурацию.

22.2.1. Жизненный цикл нового реле

Только что запущенное реле публикует свой дескриптор авторитетным серверам каталога, которые добавляют его в консенсус и начинают измерять (разд. 12.4). Сначала оно получает мало трафика: клиенты выбирают реле пропорционально весам их пропускной способности, которые остаются низкими, пока измерения не стабилизируются. Флаги присваиваются со временем (разд. 12.4.3). Для `Stable` требуется длительная непрерывная работа, а для `Guard` дополнительно необходимы достаточный возраст и высокая доступность. После получения флага Guard объём трафика может сначала даже уменьшиться: клиенты, уже выбравшие свои входные реле, не начинают использовать новый узел как промежуточный с той же частотой. Затем трафик постепенно растёт по мере того, как новые клиенты выбирают его (разд. 13.3).

Два вида сбоев не эквивалентны. Частые перезапуски без смены идентичности сохраняют измерения пропускной способности, но уменьшают непрерывную доступность, необходимую для `Stable` и `Guard`; эти флаги приходится заслуживать заново. Смена же *идентификационных ключей* создаёт в представлении authorities совершенно новое реле, начинающий с нуля даже по измерениям пропускной способности.

22.2.2. Ключи

Идентичность реле определяется его основным ключом Ed25519 (разд. 7.2). По умолчанию tor хранит его на диске реле. При `OfflineMasterKey 1` основной ключ остаётся на другом компьютере; на сервер передаются только временные ключи подписи, сертифицированные с помощью `tor --keygen`. Если сервер будет скомпрометирован, атакующий получит истекающие временные ключи, а не долгосрочную идентичность реле. Этот подход требует дисциплины: временные ключи нужно обновлять до истечения срока действия, иначе relay перестанет работать.

22.3. Политика выхода

Политика выхода — упорядоченный список правил `accept` и `reject`, каждое из которых задаёт адрес или сеть и порт либо диапазон портов. Tor обрабатывает правила **по порядку**, применяя первое совпавшее. Правила оператора располагаются перед правилами по умолчанию, перечисленными в руководстве. По умолчанию блокируются порты электронной почты (25), NNTP (119), NetBIOS и SMB (135–139, 445), а также некоторых приложений обмена файлами; остальной трафик разрешается. Отдельно tor по умолчанию отклоняет приватные назначения, включая `127.0.0.0/8`, `10.0.0.0/8` и `192.168.0.0/16`, а также собственный публичный адрес реле. Это защищает внутреннюю сеть оператора (`ExitPolicyRejectPrivate`). В лабораторной сети эта защита отключена, чтобы разрешить трафик к `127.0.0.1`.

Порядок защитных правил. Если включён `ExitPolicyRejectPrivate` (по умолчанию), запреты на приватные сети и локальные адреса добавляются *перед* правилами оператора. В частной сети Chutney эта защита намеренно отключена; такую настройку нельзя переносить на публичное выходное реле.

Выходное реле, ограниченное веб-трафиком, может использовать:

```
ExitRelay  1
ExitPolicy accept *:80
ExitPolicy accept *:443
ExitPolicy reject *:*
```

Tor Project также рекомендует `ReducedExitPolicy` как распространённый компромисс: она разрешает набор широко используемых сервисных портов и исключает порты, чаще вызывающие жалобы на злоупотребления.

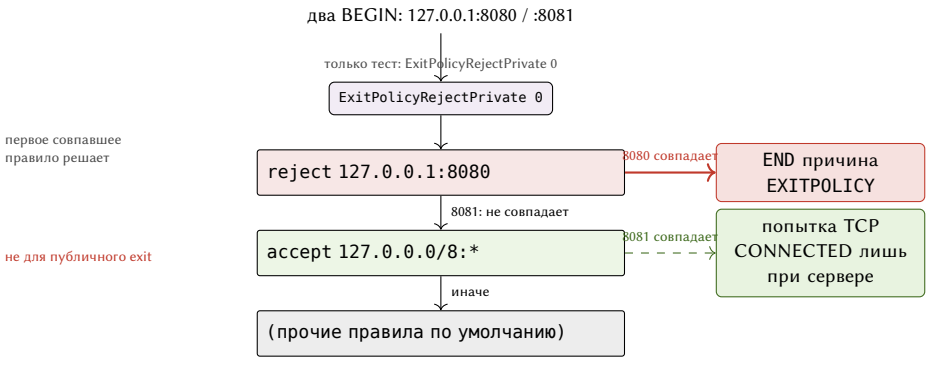


Рис. 22.1: В лаборатории Chutney `ExitPolicyRejectPrivate 0` включается только для тестов с приватными адресами; обычно Tor отвергает их до правил оператора. `BEGIN` к `127.0.0.1:8080` совпадает с `reject` и приводит к `END EXITPOLICY`; порт 8081 проходит это правило и совпадает с `accept`. Разрешение позволяет попытку TCP-соединения, но `CONNECTED` требует работающего сервера. Если поменять правила местами, запрет не подействует.

Что получает клиент. Дескриптор реле содержит полную политику, а консенсус и микродескрипторы — только сводку по портам, например `p accept 80,443`. Клиент использует её, чтобы решить, способно ли выходное реле обслужить запрос (разд. 13.1). Поскольку адреса в этой сводке отсутствуют, о запрете, зависящем от адреса, клиент иногда узнаёт только после попытки соединения, как показывает лабораторная работа.

22.4. Лабораторная работа: проверка политики выхода

Лабораторная работа 14: Правило выхода и его последствия

Цель. Добавить правило к лабораторному выходному реле и с помощью `labs/ru/код/torlab` наблюдать ответы на разрешённые и запрещённые назначения.
Предварительные условия. Лабораторные работы 3 и 4; лабораторная сеть должна быть запущена.
Оценочное время. 1 час.

Шаг 1: правило. В `torrc` узла `test006e` вставьте правило, запрещающее один порт, перед существующими правилами, разрешающими `127.0.0.0/8`:

```
ExitPolicy reject 127.0.0.1:8080
ExitPolicy accept 127.0.0.0/8:*
```

Листинг показывает порядок двух соответствующих правил: в существующем `torrc` достаточно вставить новую строку `reject` перед первой строкой `ExitPolicy`, не дублируя имеющийся `accept`. Порядок имеет значение. Если поместить `reject` после `accept`, запрет не сработает, поскольку побеждает первое совпавшее правило. Именно такая ошибка была зафиксирована в предыдущем эталонном эксперименте. После перезапуска выходного реле запустите два небольших HTTP-сервера на портах контейнера 8080 и 8081.

Шаг 2: три запроса. Используя `build` и `stream.py`, постройте цепь, заканчивающуюся на `006e`, и отправьте три сообщения `BEGIN` к портам 8080, 8081 и 9999, где ничего не слушает. Ответы в одном из наших сеансов (коды, адрес и TTL могут отличаться в зависимости от версии и среды):

```
127.0.0.1:8080 -> END  reason=EXITPOLICY address=127.0.0.1 TTL=1800
127.0.0.1:8081 -> CONNECTED
127.0.0.1:9999 -> END  reason=MISC
```

Шаг 3: интерпретация ответов. Отказ по политике приводит к причине `EXITPOLICY`. Ячейка `END` также содержит разрешённый адрес и TTL. Клиент может запомнить, что данное выходное реле не обслуживает это назначение, и выбрать другой, не дожидаясь нового дескриптора. В отличие от этого, порт без слушающего сервера возвращает общий `MISC`, а не `CONNECTREFUSED`, который спецификация перечисляет для отказа в соединении [torspec § Closing streams].

Подробнее: Почему `MISC`, а не `CONNECTREFUSED`?

Ответ находится в `info`-журналах выходного реле. Прослеживание этой ситуации — хороший пример исследования расхождения между спецификацией и реализацией:

```
[info] connection_handle_write_impl(): in-progress connect failed.
       Removing. (Connection refused)
```

```
[info] errno_to_stream_end_reason(): Didn't recognize errno 115 (Operation
       now in progress); telling the client that we are ending a stream
       for 'misc' reason.
```

Выходное реле *знает*, что соединение было отклонено, но выводит причину из другого значения. Соединения к назначениям неблокирующие: `connect()` сначала возвращает `EINPROGRESS` (errno 115), а реальный результат становится доступен позже через событие записи. В этот момент `connection.c` получает настоящую ошибку через `getsockopt(SO_ERROR)`, записывает «Connection refused» и вызывает `connection_edge_end_errno()`. Однако эта функция повторно вычисляет причину через `errno_to_stream_end_reason(tor_socket_errno(…))`. В Unix `tor_socket_errno` определён как макрос `(errno)` (`src/lib/net/socket.h`), то есть обращается к глобальной переменной, где всё ещё остаётся `EINPROGRESS`. Значение 115 отсутствует в списке `src/core/or/reasons.c`, поэтому ветвь по умолчанию возвращает `MISC`.
Правильная информация существует, но не используется. Две ветви кода в этом участке тоже различаются: ветвь соединений между реле передаёт полученную ошибку дальше, а ветвь потоков её игнорирует. Согласно имеющимся отчёту и патчу в `docs/it/segnalazioni/`, в предыдущем испытании на частной сети с tor 0.4.9.13 передача настоящего кода ошибки изменила третий ответ на `CONNECTREFUSED`. Это небольшой дефект без последствий для анонимности: клиент всё равно знает, что поток завершился ошибкой, но теряет полезную информацию. `CONNECTREFUSED` показал бы, что выходное реле достигло назначения и повтор через другой exit бессмысленен. Анализ и патч находятся в `docs/it/segnalazioni/` для Tor Project. Этот пример также показывает, почему спецификации нужно проверять по реальному поведению.

Реальный локализованный скрипт — `labs/ru/лаборатория/exit_policy.py`. В Linux с запущенной выделенной сетью Chutney его можно запустить из корня репозитория командой `PYTHONPATH=labs/ru/код python3 labs/ru/лаборатория/exit_policy.py`. Он выполняет три проверки и пытается восстановить исходный `torrc` узла `006e` в блоке `finally`. Используйте только одноразовую лабораторную сеть: скрипт временно меняет настройки выходного реле и открывает локальные HTTP-серверы. Обычная цель `make -C labs/ru/лаборатория verify` проверяет только Chutney и не запускает скрипт. Его запуск и фактическое восстановление не проверялись в ходе аудита. Важная деталь: цепь должна содержать три перехода. Выходное реле отказывается пересылать трафик от одношаговой цепи и вернёт `END` с причиной `TORPROTOCOL` (разд. 15.3).

22.5. Семейства реле

Операторы, управляющие несколькими реле, должны объявлять **семейство**. Клиенты никогда не помещают два реле из одного семейства в одну цепь (разд. 13.1), поскольку общий оператор мог бы коррелировать трафик. Исторический механизм — `MyFamily`: каждое реле перечисляет отпечатки всех остальных, и принадлежность к семейству учитывается только при взаимных объявлениях. Поэтому при десяти реле приходится поддерживать десять совпадающих списков, каждый из которых нужно менять при добавлении нового узла.

Механизм, введённый в tor 0.4.9, вместо этого использует **общий ключ семейства**. Оператор создаёт его командой `tor --keygen-family MyFamilyKey`, копирует файл `MyFamilyKey.secret_family_key` в каталог `KeyDirectory` каждого реле, сохраняя суффикс `.secret_family_key`, и добавляет в каждый `torrc` полученную строку `FamilyId`. Общий секретный ключ нужно защищать и хранить в безопасной резервной копии; при утечке его необходимо заменить на всех реле семейства. Каждое реле подписывает этим ключом

сертификат принадлежности к семейству. Проверяющим больше не нужно сравнивать списки. На переходном этапе рекомендуется настраивать оба механизма.

Подробнее: Почему объявление семейства полезно

Ничто технически не заставляет оператора объявлять свои реле как семейство, и атакующий может намеренно этого не делать. Однако авторитетные серверы каталога и добровольцы, наблюдающие сеть, ищут общие характеристики: одни и те же сети, версии, синхронные времена запуска, контактные данные. Необъявленное семейство может привлечь внимание, и его реле могут быть исключены из консенсуса. Декларация — часть модели доверия, на которой держится сеть.

Не включайте мосты в публичные списки `MyFamily`: это может раскрыть их связь с опубликованными реле. Руководство Tor прямо рекомендует не использовать эту опцию на мостах. Кроме того, `tor --keygen-family MyFamilyKey` может перезаписать существующие файлы с таким именем: выберите новый базовый идентификатор и защитите созданный секретный файл.

22.6. Запуск моста

Мост с obfs4 можно настроить, как в Лабораторной работе 10 (разд. 16.5):

```
BridgeRelay 1
ORPort 9001
ServerTransportPlugin obfs4 exec /usr/bin/obfs4proxy
ServerTransportListenAddr obfs4 0.0.0.0:443
ExtORPort auto
BridgeDistribution any
```

`BridgeDistribution` сообщает системе распределения, сейчас rdsys, каким способом оператор хочет распространять мост. Значение `any` оставляет выбор системе, а `none` полностью исключает его из распространения, что удобно для передачи моста доверенным людям. Доступные каналы, включая веб-сайт, электронную почту и Telegram, перечислены на странице информации о мостах. Мост отсутствует в публичном консенсусе, но его дескриптор всё равно отправляется авторитетному серверу мостов, который проверяет доступность.

Подключаемый транспорт должен прослушивать порт, отличный от ORPort, и по возможности привычный порт вроде 443. Именно этот порт используют клиенты и видит цензор (разд. 16.4).

Итоги

- Промежуточное реле — наименее требовательный способ помочь сети; выходное реле особенно ценно, но несёт больше внешних рисков; мост помогает пользователям в условиях цензуры.
- Существенно всего несколько параметров: идентичность, контактные данные, порты и лимиты пропускной способности. Основной ключ идентичности можно хранить вне сети.
- Политика выхода — упорядоченный список, где побеждает первое совпавшее правило; приватные назначения по умолчанию отклоняются.
- Отказ по политике приходит клиенту как сообщение `END` с причиной `EXITPOLICY`, адресом и TTL.

- Операторы нескольких реле должны объявлять их как семейство с помощью `MyFamily` или более нового механизма общего ключа семейства.

Упражнения

Упражнение 22.1 ⋆ Для policy `accept *:80`, `reject 192.0.2.0/24:*`, `reject *:*` может ли клиент подключиться через этот exit к `192.0.2.7:80`? Перепишите policy так, чтобы это было невозможно.

Упражнение 22.2 ⋆ Почему сводки только по портам в консенсусе недостаточно, чтобы определить, примет ли exit соединение? Что делает клиент после получения `END` с причиной `EXITPOLICY`?

Упражнение 22.3 ⋆⋆ Оператор управляет пятью relay через `MyFamily`, затем добавляет шестой, но обновляет конфигурацию только на новом узле. Что видят клиенты? Что меняется при использовании family key?

Упражнение 22.4 ⋆⋆ Проверьте объяснение из технического разбора: включите `info`-журналы на exit и повторите три запроса. Что изменилось бы, если `connection_edge_end_errno()` получала ошибку, возвращённую `getsockopt`? Какие ещё причины закрытия stream тогда могли бы сообщаться корректно?

Упражнение 22.5 ⋆⋆⋆ Для реле задано `AccountingMax 1 TBytes` в месяц и пропускная способность 10 MB/s. Через сколько дней работы на полной скорости закончится лимит? Как реле, отключённое половину месяца, влияет на сеть по сравнению с узлом, работающим весь месяц на половине пропускной способности?

Дополнительная литература

- Руководство Tor Project для операторов relay: `https://community.torproject.org/relay/`.
- Разделы руководства tor для `ExitPolicy`, `ReducedExitPolicy`, `FamilyId`, `OfflineMasterKey` и `BridgeDistribution`.
- Спецификация причин закрытия stream: [tor-spec § Closing streams].

Глава 23

Tor Browser, Tails и Whonix

Последняя проверка: 23 сентября 2026 г. — документ по архитектуре Tor Browser (2018); руководство Tor Browser; документация Tails и Whonix

Цели главы

К концу этой главы вы сможете:

- объяснять, почему недостаточно просто настроить обычный браузер на использование Tor;
- перечислять требования безопасности и приватности, лежащие в основе архитектуры Tor Browser;
- связывать изоляцию по сайтам в Tor Browser с изоляцией потоков tor;
- сравнивать, что защищают Tor Browser, Tails и Whonix и какой ценой достигается каждая модель.

Модель угроз 2004 года прямо указывала, что Tor не нормализует прикладные протоколы (разд. 4.2). Если браузер раскрывает вашу идентичность, луковая маршрутизация не сможет вас защитить. В этой главе рассматриваются инструменты, созданные именно для этого уровня: Tor Browser, который согласует поведение браузера с защитными свойствами сети, и две операционные системы — Tails и Whonix, распространяющие этот подход на весь компьютер.

23.1. Почему одного прокси недостаточно

Настройка обычного браузера на использование SocksPort tor во многих случаях скрывает IP-адрес, но не во всех и почти ничего не скрывает сверх этого:

- Некоторые компоненты могут подключаться в обход прокси: локальное DNS-разрешение, WebRTC, расширения или внешние программы, запускаемые браузером.
- Файлы cookie и другие данные состояния браузера связывают сегодняшние посещения со вчерашними, возможно выполненными без Tor.
- Сайт может собирать десятки характеристик браузера — размер окна, установленные шрифты, язык, часовой пояс, особенности отрисовки — и объединять их в **цифровой отпечаток**, отличающий одного пользователя от миллионов.
- Если все вкладки используют одну цепь, отслеживающий компонент, встроенный в несколько сайтов, может наблюдать запросы, приходящие одновременно с одного выходного реле.

Первая проблема угрожает адресу, остальные — *несвязываемости* действий, одной из центральных целей Tor.

23.2. Tor Browser

Tor Browser — модифицированная версия Firefox Extended Support Release от Mozilla, распространяемая Tor Project вместе с tor. Его архитектура описана в документе, который уже не актуален во всех деталях, но остаётся важным изложением основных принципов [Per+18]. Требования делятся на две группы.

Требования безопасности. Они определяют безопасное использование Tor; их нарушение может немедленно раскрыть пользователя:

Обязательное использование прокси Никакой контент не должен обходить tor.

Разделение состояния Информация не должна совместно использоваться с другими браузерами или сеансами вне Tor.

Отсутствие постоянных следов Ничто, раскрывающее историю просмотра, не должно оставаться на диске или в памяти после завершения сеанса, если пользователь явно не решил сохранить это.

Изоляция приложения Всё, что браузер оставляет в системе, должно удаляться вместе с ним.

Требования приватности. Они не позволяют связывать разные действия и организованы вокруг **сайта в адресной строке (источника, origin)**, то есть сайта, который посещает пользователь:

Идентификаторы Данные, сохранённые третьей стороной, например отслеживающим компонентом, встроенным в несколько сайтов, во время посещения одного сайта не должны становиться доступными той же стороне при её появлении на другом.

Цифровые отпечатки Характеристики браузера не должны позволять связывать действия на разных сайтах.

Долгосрочная несвязываемость Пользователь должен иметь возможность одной простой командой удалить всё состояние и получить новую идентичность.

Документ предлагает и полезный принцип проектирования: защитные механизмы не должны нарушать ожидаемое пользователем поведение браузера. Предшественник Tor Browser, расширение Torbutton, позволял включать и выключать Tor в одном браузере. Пользователи путали вкладки, открытые в разных режимах. Интерфейс, провоцирующий ошибки, сам становится слабостью безопасности.

23.2.1. Одна цепь на сайт

Изоляция по источнику сайта распространяется и на сеть. Tor Browser передаёт идентификатор сайта из адресной строки в SOCKS-учётных данных, отправляемых tor. По умолчанию tor помещает потоки с разными учётными данными (`IsolateSOCKSAuth`) в разные цепи. Этот механизм мы проверили в Лабораторной работе 11 (разд. 18.3). С точки зрения пользователя, как описано в руководстве [Thed], два сайта, содержащие один и тот же отслеживающий компонент, используют разные цепи, тогда как вкладки одного сайта разделяют одну.

На эту схему действуют две команды:

Новая цепь для этого сайта (New Circuit for This Site) Перезагружает текущий сайт через новую цепь, например когда выходное реле не может к нему подключиться. Состояние браузера не очищается, а цепи других сайтов не меняются.

Новая личность (New Identity) Закрывает все вкладки, удаляет файлы cookie, историю и другое состояние и получает новые цепи для всей активности, реализуя требование долгосрочной несвязываемости.

Ни одно из этих действий само по себе не меняет входное реле (guard), который остаётся стабильным месяцами по причинам, рассмотренным в разд. 13.3.

Изоляция по сайтам означает прежде всего *разделение разных сайтов*, а не гарантию единственной цепи для сайта на весь сеанс: цепи со временем заменяются. Команда «Новая цепь для этого сайта» перезагружает активную вкладку; другие вкладки того же сайта перейдут на новый маршрут после перезагрузки. Текущие соединения не обязательно разрываются. «Новая личность» также прекращает текущую активность и загрузки. Ни одна из команд не предназначена для принудительной смены входного реле: tor управляет им отдельно.

23.2.2. Защита от цифровых отпечатков

Стратегия Tor Browser против снятия цифровых отпечатков состоит не в том, чтобы сделать каждого пользователя уникальным, а в том, чтобы пользователи выглядели *одинаково*. Если все пользователи Tor Browser сообщают одинаковые значения, отпечаток говорит немногим больше, чем «это пользователь Tor Browser». Отсюда следуют несколько заметных решений. Окна открываются с округлёнными размерами, а при изменении размера вокруг содержимого страницы добавляются поля, чтобы сообщаемые размеры оставались стандартизированными кратными значениями (*letterboxing — добавление полей*). Сообщаемый часовой пояс — UTC. Многие интерфейсы, раскрывающие детали операционной системы, отключены или стандартизированы. Этим же объясняется рекомендация не устанавливать расширения и не менять настройки: каждое отличие уменьшает множество пользователей, среди которых можно скрыться, то есть набор анонимности, обсуждавшийся в гл. 1.

Цель защиты — *уменьшить число наблюдаемых отличий*, а не сделать все отпечатки буквально одинаковыми. Платформа, язык, уровень безопасности и другие свойства могут различать группы пользователей. Поэтому речь идёт о снижении уникальности, а не о гарантии полного совпадения.

23.2.3. Уровни безопасности

Многие уязвимости браузера используют JavaScript и сложные функции. Tor Browser предлагает три уровня [Thed]:

Standard Все функции включены; уровень по умолчанию.

Safer JavaScript отключён на сайтах без HTTPS; некоторые шрифты и математические символы отключены; аудио и видео запускаются только после клика.

Safest JavaScript отключён везде, а изображения и другой контент также могут быть ограничены. Статические сайты остаются доступными.

Выбор уровня — компромисс между безопасностью и удобством. Сам выбранный уровень может влиять на цифровой отпечаток, поскольку сайт способен косвенно определить его.

Подробнее: Воспроизводимые сборки

Пользователям, загружающим Tor Browser, нужна уверенность, что распространяемый бинарный файл соответствует опубликованному исходному коду. Tor Project создаёт *воспроизводимые сборки*: одинаковые исходники и среда сборки должны давать побитно идентичные файлы. Документ по архитектуре описывает работу, необходимую для устранения источников различий, включая даты, пути файлов и порядок элементов архивов [Per+18]. Независимая повторная сборка и сравнение результатов позволяют обнаружить вредоносные изменения, внесённые в среду сборки дистрибутива.

23.3. За пределами браузера

Tor Browser защищает собственный трафик, но не трафик других приложений на том же компьютере. Эти программы всё ещё могут подключаться напрямую, а ошибка или эксплуатация уязвимости браузера может раскрыть реальный адрес. Две операционные системы распространяют защиту на всю машину с помощью разных архитектур (рис. 23.1).

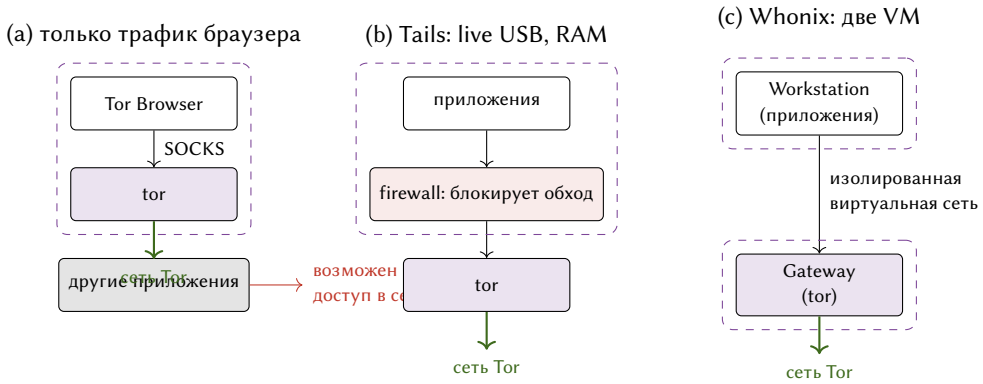


Рис. 23.1: Три модели маршрутизации и изоляции, не абсолютные гарантии анонимности. (a) Tor Browser пропускает собственный трафик через tor, но другие программы на хосте могут подключаться напрямую. (b) Tails обычно блокирует обход Tor межсетевым экраном; *Unsafe Browser* для captive portal — явное неанонимное исключение. Tails загружается с USB как live-система, работает преимущественно в RAM и поддерживает необязательное постоянное хранилище. (c) Whonix размещает Workstation с приложениями и Gateway с tor в двух VM, связанных изолированной виртуальной сетью; компрометация хоста или гипервизора может нарушить эту защиту.

23.3.1. Tails

Tails — основанная на Debian операционная система, загружаемая с USB-накопителя [Taia]. У неё два определяющих свойства:

Всё через Tor Приложения, пытающиеся выйти в Интернет без tor, блокируются.

Амнезия Система работает в памяти, не записывает данные на диск компьютера и забывает своё состояние после выключения. Данные, которые пользователь решил сохранить, помещаются в необязательное зашифрованное *постоянное хранилище* на USB-накопителе.

Tails включает Tor Browser и другие осторожно настроенные приложения. Проект присоединился к Tor Project в сентябре 2024 года [The24c]. Документация прямо указывает ограничения:

Tails не защищает от модифицированного аппаратного обеспечения, например аппаратного регистратора нажатий клавиш, или от USB-накопителя, подготовленного на уже заражённом компьютере.

Эти свойства относятся к обычному режиму работы Tails и *не дают абсолютной гарантии* отсутствия следов на любых носителях. Пользователь может самостоятельно подключить диски или создать постоянные файлы; вредоносное ПО, скомпрометированное оборудование и действия вне Tails тоже выходят за границы модели угроз.

23.3.2. Whonix

Whonix разделяет функции между двумя виртуальными машинами. **Gateway** запускает tor и является единственной виртуальной машиной, подключённой к внешней сети. **Workstation** запускает приложения и соединяется только с Gateway через внутреннюю виртуальную сеть. Поэтому весь трафик Workstation обязан проходить через tor благодаря архитектуре, а не только настройкам приложений. По сравнению с Tails это даёт более сильное разделение: даже атакующий, получивший административный контроль над Workstation, не может напрямую узнать реальный адрес, поскольку сама машина его не знает. Цена — зависимость от безопасности ПО виртуализации и отсутствие амнезии, если Whonix не запускается внутри системы, которая её обеспечивает.

Разделение шлюза и рабочей станции — важная граница безопасности, но *не абсолютная гарантия* при компрометации рабочей станции. Атакующий может раскрыть личность через данные приложений; взлом хоста, гипервизора или шлюза способен обойти сетевую изоляцию. Сохраняют значение корректная конфигурация и защита межсетевого экрана.

Фильтрация порта управления. Некоторые приложения Workstation, включая функцию New Identity в Tor Browser, должны взаимодействовать с ControlPort tor на шлюзе (Gateway). Однако протокол управления (разд. 18.1) также позволяет запросить публичный адрес tor через `GETINFO address` — именно то, чего Workstation знать не должна. Поэтому Whonix помещает между ними *onion-grater*, прокси, разрешающий только явно допустимые команды [Who]. Функция, безопасная в одном контексте, может стать утечкой в другом.

Внимание: Инструменты не заменяют осторожное поведение

Tor Browser, Tails и Whonix уменьшают число возможных технических ошибок, но не устраняют поведенческие: вход в личный аккаунт, открытие загруженных документов вне защищённой среды или раскрытие идентифицирующих сведений. Этот уровень рассматривается в гл. 28.

Итоги

- Обычный браузер за Tor может пропускать трафик мимо прокси и, что ещё важнее, связывать действия через общее состояние, цифровые отпечатки или цепи.
- Tor Browser следует явным требованиям архитектуры: обязательное использование прокси, разделение состояния, ограниченное долговременное хранение данных, несвязываемость между сайтами и доступная функция смены личности (New Identity).
- Изоляция по сайтам основана на SOCKS-учётных данных и механизме изоляции потоков tor.
- Tor Browser противодействует цифровым отпечаткам, стандартизируя наблюдаемые характеристики пользователей; уровни безопасности уменьшают поверхность атаки.

- Tails принудительно направляет весь трафик через Tor и избегает постоянных следов; Whonix изолирует приложения внутри машины, не знающей реального сетевого адреса.

Упражнения

Упражнение 23.1 ⋆ С точки зрения цепей и состояния браузера, чем отличается «Новая цепь для этого сайта» от «Новая личность»? Почему ни одна из команд не меняет входное реле (guard)?

Упражнение 23.2 ⋆ Пользователь устанавливает в Tor Browser расширение для блокировки рекламы, считая, что это улучшит приватность. Что он может выиграть и что потерять?

Методическое замечание. Лабораторная работа 11 имитирует разные учётные данные SOCKS, но не открывает три настоящие вкладки Tor Browser. Совпадающие учётные данные могут привести к повторному использованию цепи, однако это не гарантировано постоянно: цепи устаревают, возможны дополнительные правила изоляции.

Упражнение 23.3 ⋆⋆ Используя лабораторную сеть и скрипт Лабораторной работы 11, смоделируйте три вкладки: две посещают один сайт, третья — другой, причём имя сайта используется как учётные данные SOCKS. Через порт управления (ControlPort) проверьте, какие потоки используют одну цепь.

Упражнение 23.4 ⋆⋆ Для каждого сценария выберите подходящий инструмент из Tor Browser, Tails и Whonix и объясните решение: (a) человек просматривает заблокированные сайты с домашнего компьютера; (b) журналист обрабатывает конфиденциальные документы на чужих компьютерах; (c) исследователь запускает несколько долгоживущих программ, которые все должны использовать Tor.

Упражнение 23.5 ⋆⋆⋆ Выберите пять свойств браузера, доступных сайту через JavaScript, и оцените, сколько бит информации каждое раскрывает в обычном браузере. Как Tor Browser стандартизирует эти свойства и какие различия между пользователями всё ещё остаются?

Дополнительная литература

- M. Perry, E. Clark, S. Murdoch, G. Koppen, *The Design and Implementation of the Tor Browser* [Per+18].
- Руководство Tor Browser [Thed].
- Документация Tails [Taia] и Whonix, особенно страница onion-grater [Who].

Глава 24

Измерение сети: Tor Metrics и OONI

Последняя проверка: 23 сентября 2026 г. — данные Tor Metrics по 20 сентября 2026 г.; консенсус от 23 сентября 2026 г., 15:00; спецификации OONI ts-023 и ts-030

Цели главы

К концу этой главы вы сможете:

- описывать данные, публикуемые Tor Project, и источники этих данных;
- объяснять, как оценивается число пользователей в сети, которая специально не должна знать своих пользователей, и насколько неопределённы такие оценки;
- извлекать сетевую статистику из реального консенсуса (Лабораторная работа 15);
- объяснять, как OONI измеряет блокировку Tor с точки зрения пользователей.

Сеть анонимности создаёт для операторов необычную проблему: для улучшения сети её нужно измерять, но измерение означает сбор информации именно о том, что она призвана защищать. Подход Tor Project — измерять минимально необходимое, агрегировать и публиковать результаты и документировать каждый шаг так, чтобы расчёты мог воспроизвести любой исследователь [Thec]. В этой главе мы проследим путь данных от источников до графиков, а затем посмотрим на сеть снаружи через измерения OONI.

24.1. Источники данных

Почти всё, что публично известно о сети Tor, происходит из документов, уже встречавшихся нам:

Консенсус и дескрипторы (гл. 12): список ретрансляторов, флаги, весовые коэффициенты пропускной способности, версии и политики выхода.

Дополнительные дескрипторы (extra-info) Публикуются каждым ретранслятором вместе с основным дескриптором и содержат историю объёма переданных байтов и агрегированную статистику, включая число запросов к каталогу по странам.

Активные измерения OnionPerf периодически загружает файлы через Tor из выбранных точек по всему миру, записывая время передачи и ошибки.

CollecTor архивирует эти документы, начиная с ранних лет сети, и предоставляет их для загрузки. **Tor Metrics** строит на их основе графики и таблицы, сопровождая их CSV-данными и документированными методиками. **Relay Search** позволяет находить сведения об отдельных ретрансляторах. Консенсус, использованный в лабораторных работах этой книги (`fonti/collector`), был получен из CollecTor.

24.2. Подсчёт анонимных пользователей

Tor не регистрирует пользователей, и ни один ретранслятор не может определить, принадлежат ли два соединения одному человеку. Как тогда сеть оценивает, что ею пользуются примерно три миллиона клиентов? Метод косвенный (рис. 24.1). Каждый клиент периодически загружает консенсус, а запросы к каталогу можно подсчитывать, не идентифицируя запрашивающих. Ретрансляторы, обслуживающие запросы к каталогу, ежедневно считают успешные запросы, группируя их по стране, определённой по адресу источника. Они публикуют агрегированные значения в дескрипторах extra-info, округляя числа по странам вверх до кратных восьми, чтобы маленькие точные значения раскрывали меньше информации [dir-spec § Extra-info document format].

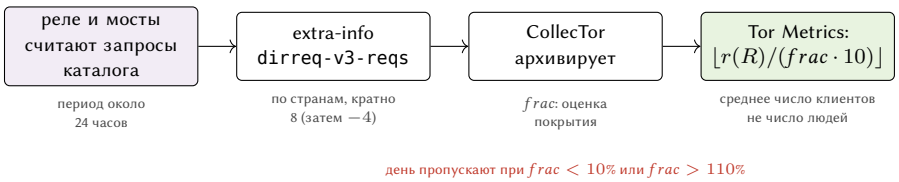


Рис. 24.1: От успешных запросов каталога к косвенной оценке *среднего числа подключённых клиентов*, а не отдельных людей. Реле каталогов и мосты дают отдельные агрегированные ряды в дескрипторах extra-info; количества запросов по странам в `dirreq-v3-reqs` округляются вверх до кратных 8. CollecTor архивирует данные; Tor Metrics вычитает 4 из допустимых значений, оценивает долю охвата $frac$ и применяет формулу $\lfloor r(R)/(frac \cdot 10) \rfloor$. Число 10 — предположение о среднем количестве запросов клиента в день. Дни при $frac < 10\%$ или $frac > 110\%$ исключаются.

Tor Metrics собирает эти значения и выполняет три шага [Thec]:

1. Компенсирует округление, вычитая половину интервала округления из каждого значения.

2. Оценивает **долю**, $frac$, всех запросов к каталогу сети, покрываемую ретрансляторами, предоставившими статистику, с учётом времени их работы и объёма обслуженных запросов.

3. Делит зарегистрированные запросы на эту долю, получая оценку общего числа запросов в сети, затем делит на 10:
$$r(N) = \left\lfloor \frac{r(R)}{frac \cdot 10} \right\rfloor .$$

Делитель 10 — предположение. Непрерывно работающий клиент делает примерно 15 запросов в сутки, но многие клиенты подключены лишь часть дня, поэтому Tor Project выбрал 10 как среднее значение. Оценки с долей охвата ниже 10 % или выше 110 % отбрасываются.

Внимание: Что означает «пользователи»

Результат оценивает число *в среднем подключённых клиентов*, а не отдельных людей. Клиент, работающий два часа в день, обычно выполняет меньше запросов к каталогу, чем клиент, работающий круглосуточно, но фиксированного соотношения между ними нет; несколько программ с Tor на одной машине считаются несколькими клиентами. Изменения поведения ПО, например менее частая загрузка консенсуса, меняют оценку даже при неизменном числе людей. Поэтому графики полезнее для анализа *изменений*, чем абсолютных значений.

Значения на эталонную дату книги. С 1 августа по 20 сентября 2026 года оценка ежедневно подключённых напрямую пользователей колебалась между 2,8 и 3,5 миллиона, при доле охвата от 57 % до 73 %. Среднее значение для Италии составляло примерно 60 000, но ежедневные результаты имеют широкие ожидаемые интервалы: например, 20 сентября — примерно от 34 000 до 111 000. Изменения внутри таких диапазонов сами по себе не доказывают наличие события. В тот же день оценка числа пользователей мостов составляла около 98 000 для obfs4, 60 000 для Snowflake и 35 000 для WebTunnel.

24.2.1. Выявление цензуры

Metrics использует те же временные ряды для выявления потенциальных событий цензуры. Для стран со значительным числом пользователей система сравнивает недельное изменение с типичными изменениями в других странах и выделяет даты, когда конкретная страна выходит за ожидаемый интервал [Thec]. Резкое падение числа прямых пользователей одновременно с ростом пользователей мостов — характерный признак блокировки. Это указание, а не доказательство. Для подтверждения нужны измерения внутри страны, и здесь используется OONI.

24.3. Производительность

OnionPerf из фиксированных точек измеряет время загрузки файлов разных размеров через Tor, ошибки и задержку цепи — интервал между отправкой HTTP-запроса и получением заголовков ответа [Thec]. 20 сентября 2026 года медианная задержка до публичного сервера составляла около 110–140 мс из немецкой точки измерения, 280 мс из США и более 500 мс из Гонконга. Географическое расстояние до основного сосредоточения сети в Европе и Северной Америке имеет значение. Некоторые точки измерения используют Conflux — систему многопутевой передачи, описанную в гл. 31, что позволяет проводить сравнения при сходных условиях.

24.4. Лабораторная работа: чтение реального консенсуса

Лабораторная работа 15: Получение сетевой статистики из консенсуса

Цель. Извлечь основные сетевые показатели из реального консенсуса CollecTor с помощью нового модуля `labs/ru/код/torlab/metrics.py`.
Предварительные условия. Лабораторная работа 6 (`directory.py`) и Лабораторная работа 7 (`path_selection.py`); работающая сеть не требуется.
Оценочное время. 1 час.

Модуль добавляет функцию `summary`, повторно используя анализатор консенсуса из Лабораторной работы 6, а также две функции оценки пользователей. При равных значениях счётчики дополнительно сортируются по имени. Без этого их порядок менялся бы между запусками из-за порядка обхода множеств в Python. Чтобы воспроизвести данные исторического консенсуса от 23 сентября 2026 года, 15:00, сначала подготовьте файл из корня репозитория командой `make -C labs/ru data` (для загрузки нужен Интернет). Применительно к этому консенсусу (вывод сокращён):

```
$ PYTHONPATH=labs/ru/код python3 -m torlab.metrics labs/ru/данные/collector/consensus-
    microdesc-2026-09-23-15
relay: 9358
флаги: {'Running': 9358, 'Valid': 9358, 'Fast': 8961, 'Stable': 8199,
```

```
       'V2Dir': 7674, 'Guard': 5306, 'HSDir': 4050, 'Exit': 3177,
       'BadExit': 32, 'MiddleOnly': 28, 'StaleDesc': 21, 'Authority': 10}
общая пропускная способность: 198927900
доля пропускной способности выходных узлов: 0.3308947161257923
доля пропускной способности сторожевых узлов: 0.7010271560701139
версии: [('Tor 0.4.9.12', 6027), ('Tor 0.4.9.11', 2520), ...]
сети /16: [('64.65', 771), ('192.42', 636), ('204.8', 598), ...]
```

Несколько наблюдений:

- 9 358 ретрансляторов согласуются с графиком Tor Metrics, который на 20 сентября показывает 9 416 ретрансляторов и 2 333 моста.
- Примерно треть ретрансляторов имеет флаг `Exit` и обеспечивает примерно треть измеренной пропускной способности; на сторожевые узлы приходится 70 %. Эти доли пересекаются, поскольку один ретранслятор может иметь оба флага. Дисбаланс компенсируется весовыми коэффициентами пропускной способности, обсуждаемыми в разд. 13.2.
- `BadExit` (32 ретранслятора) и `MiddleOnly` (28) отражают решения центров каталогов о подозрительных или неверно настроенных ретрансляторах.
- Десять ретрансляторов имеют флаг `Authority`, но в голосовании по консенсусу участвуют только девять центров каталогов (разд. 6.1). Десятый — `Serge`, центр каталогов мостов: флаг у него есть, но он не голосует.
- Большинство ретрансляторов использует современные версии 0.4.9.
- В нескольких сетях /16 размещены сотни ретрансляторов, обычно принадлежащих крупным операторам выходных узлов. Правило не более одного ретранслятора из одной сети /16 в цепи (разд. 13.1) ограничивает зависимость от одной сети.

Сумма весов пропускной способности в консенсусе представляет единицы пропускной способности, измеренные центрами каталогов, а не фактически переданный трафик. Для оценки трафика Metrics использует историю переданных байтов из дескрипторов extra-info.

24.5. OONI

Open Observatory of Network Interference (OONI) измеряет интернет-цензуру с точки зрения пользователей. Добровольцы по всему миру устанавливают OONI Probe, который периодически выполняет тесты и публикует открытые результаты. Некоторые тесты специально относятся к Tor [OON]:

`tor` Проверяет доступность выбранных центров каталогов и мостов. Тесты не ограничиваются установлением TCP-соединения и выполняют действия, специфичные для протокола, например рукопожатие obfs4.

`torsf` Пытается запустить tor через Snowflake, записывая, завершилась ли первоначальная загрузка сети успешно и сколько времени он занял.

Спецификации тестов содержат защитные меры, предотвращающие публикацию приватных адресов мостов: их включение в результаты фактически передало бы эти адреса цензорам. Metrics и OONI дополняют друг друга. Metrics может показать падение числа пользователей из некоторой страны; OONI позволяет проверить, действительно ли ретрансляторы или мосты были недоступны изнутри этой страны, и определить технику блокировки.

Подробнее: Измерять, не причиняя вреда

Каждая статистика, публикуемая ретранслятором, является компромиссом. Чем выше детализация, тем полезнее данные, но тем выше риск раскрытия информации. Поэтому значения округляются и агрегируются по дням и странам. Исследования сети следуют тем же принципам (разд. 19.1): собирать минимум необходимого, агрегировать до публикации и не хранить лишние данные.

Итоги

- Сетевые данные происходят из консенсусов, дескрипторов и документов extra-info, архивируемых CollecTor, а также из активных измерений OnionPerf. Tor Metrics анализирует их по документированным и воспроизводимым методикам.
- Число пользователей оценивается по запросам к каталогу: зарегистрированные запросы делятся на покрываемую долю, затем на 10. Это оценка среднего числа подключённых клиентов, особенно полезная для анализа изменений.
- Те же данные могут указывать на возможные события цензуры, требующие подтверждения локальными измерениями.
- Реальный консенсус позволяет получить число relay, флаги, доли пропускной способности, версии и концентрацию по сетям.
- OONI измеряет доступность центров каталогов и мостов и работу Snowflake изнутри стран.

Упражнения

Упражнение 24.1 ⋆ Если все клиенты начнут загружать консенсус вдвое реже, что произойдёт с оценкой числа пользователей при неизменном числе людей? Как интерпретатор графиков мог бы заметить такое изменение?

Упражнение 24.2 ⋆ За один день ретрансляторы, предоставившие статистику, зарегистрировали двадцать миллионов запросов, а оценённая доля охвата составила 60 %. Сколько пользователей оценит Metrics? Что произойдёт, если доля охвата составит 8 %?

Упражнение 24.3 ⋆⋆ Расширьте `summary`, чтобы вычислять долю пропускной способности ретрансляторов, имеющих одновременно `Guard` и `Exit`, а также общую взвешенную вероятность того, что выходной узел цепи принадлежит одной из пяти наиболее представленных сетей /16.

Упражнение 24.4 ⋆⋆ Загрузите из Metrics временные ряды прямых пользователей и пользователей мостов для страны с документированным событием блокировки Tor. Опишите, что происходит с двумя рядами в даты блокировки.

Упражнение 24.5 ⋆ ⋆ ⋆ Статистика по странам округляется вверх до кратных восьми. Опишите атаку, которую это округление затрудняет, и объясните, почему оно не защищает от атакующего, контролирующего множество ретрансляторов.

Дополнительная литература

- Страница Tor Project *Reproducible Metrics* [Thec], объясняющая методы построения отдельных графиков.
- Спецификации тестов OONI [OON] и OONI Explorer: `https://explorer.ooni.org/`.
- Формат extra-info descriptor: [dir-spec § Extra-info document format].

Глава 25

Правовые и этические аспекты в Италии и ЕС

Последняя проверка: 23 сентября 2026 г. — Регламент (ЕС) 2022/2065; Законодательный декрет 196/2003 (сводный текст, опубликованный Итальянским органом по защите персональных данных); рекомендации Tor Research Safety Board

Цели главы

К концу этой главы вы сможете:

- отличать законность инструмента от законности способа его использования;
- интерпретировать европейское правило «простая передача» (*mere conduit*) применительно к работе выходного ретранслятора Tor;
- определять нерешённые вопросы для операторов ретрансляторов в Италии, включая ответственность, хранение данных и изъятие оборудования;
- применять принципы безопасности Tor Research Safety Board к исследованиям Tor.

Внимание: Эта глава не является юридической консультацией

Право меняется, зависит от обстоятельств конкретного дела, а решений, непосредственно касающихся Tor, относительно немного. Эта глава очерчивает соответствующие вопросы и указывает нормы, рассмотренные по их официальному тексту, прямо обозначая неопределённость. Любой, кто принимает эксплуатационное решение, например запускает выходной ретранслятор, должен получить консультацию юриста, специализирующегося на технологическом праве.

25.1. Инструмент и способы его использования

В Италии, как и в других странах Европейского союза, нет закона, который в общем случае запрещал бы использование Tor или других инструментов, защищающих конфиденциальность коммуникаций. Напротив, тайна переписки и иных сообщений (статья 15 Конституции Италии) и свобода выражения мнения (статья 21) являются основными правами, как и уважение частной жизни и защита персональных данных по статьям 7 и 8 Хартии Европейского союза об основных правах.

Действия, совершаемые через Tor, подчиняются тем же законам, что действуют и без Tor. Несанкционированный доступ к компьютерной системе (статья 615-*ter* Уголовного кодекса Италии) не становится законным только потому, что проходит через три ретранслятора.

Различие между инструментом и его использованием является центральным для этой главы и важно также для тех, кто предоставляет инфраструктуру: операторы ретрансляторов предоставляют возможность передачи данных, подобно провайдерам связи, а не выбирают передаваемое содержимое.

25.2. Операторы ретрансляторов

25.2.1. Простая передача (mere conduit)

Европейский *Регламент о цифровых услугах* (Digital Services Act, DSA), применяемый с 17 февраля 2024 года, заменил правила ответственности посредников, первоначально установленные Директивой об электронной коммерции 2000 года, которую Италия реализовала Законодательным декретом 70/2003 [22]. Статья 4 устанавливает режим **простой передачи** (*mere conduit*): сервис, передающий информацию, предоставленную другими лицами, освобождается от ответственности за эту информацию, если он не инициирует передачу, не выбирает получателя и не выбирает либо не изменяет передаваемую информацию. Рис. 25.1 сопоставляет эти условия с работой выходного ретранслятора Tor на уровне протокола ячеек.

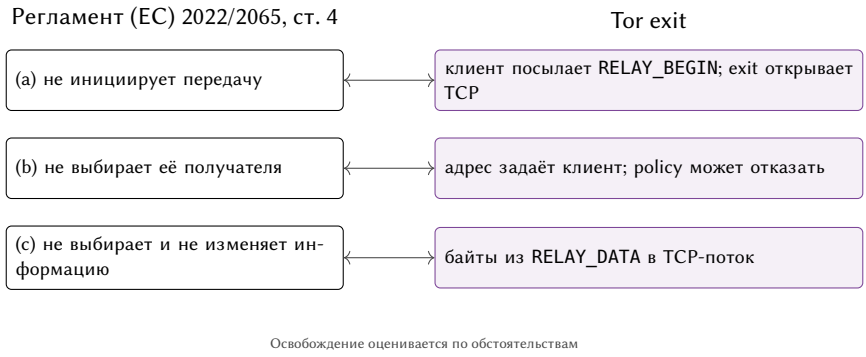


Рис. 25.1: Техническое сопоставление условий *mere conduit* из ст. 4 Регламента (ЕС) 2022/2065 и работы выхода Tor, а не автоматическое юридическое заключение. Клиент указывает адрес в `RELAY_BEGIN`, а выходной узел открывает TCP-соединение и может отклонить его согласно политике выхода. Прикладные байты из `RELAY_DATA` передаются в поток TCP, а не в неизменённых ячейках Tor. Применимость освобождения зависит от всех правовых условий и обстоятельств.

Важны ещё два положения. Статья 8 исключает *общую* обязанность контролировать передаваемую информацию или активно искать признаки противоправной деятельности: оператор в общем случае не обязан без разбора проверять трафик; промежуточный ретранслятор также не может прочесть зашифрованное содержимое прикладного уровня в открытом виде. Однако статья 4(3) сохраняет полномочия судебных и административных органов обязывать поставщиков услуг предотвращать или прекращать нарушения.

Открытый вопрос. Регламент применяется к «услугам информационного общества», которые право ЕС определяет как услуги, *обычно предоставляемые за вознаграждение*. Подпадает ли под это определение бесплатно работающий добровольческий ретранслятор? Существуют аргументы в пользу широкого толкования, а само освобождение отражает принцип, который суд мог бы учитывать: тот, кто лишь передаёт информацию, не должен отвечать за её содержимое. Однако правовое положение добровольческих ретрансляторов Tor в Италии не установлено здесь посредством прямо применимого судебного решения. Эта неопределённость отчасти объясняет, почему некоторые выходные ретрансляторы обслуживаются

ассоциациями или организациями, уже предоставляющими сетевые услуги.

25.2.2. Гражданская ответственность и уголовные расследования

Освобождение касается ответственности за передаваемую информацию. Оно не препятствует тому, чтобы уголовное расследование началось с IP-адреса выходного ретранслятора. С точки зрения потерпевшего правонарушение, совершённое через Tor, выглядит исходящим от выходного ретранслятора. Следователи могут первоначально провести обыск или изъять компьютеры оператора. Юридический FAQ Electronic Frontier Foundation для операторов ретрансляторов, составленный по законодательству США, но полезный как запись практического опыта, сообщает, что организации не известны случаи осуждения оператора исключительно за эксплуатацию ретранслятора, однако описывает расследования и изъятия, вызванные неправильным пониманием Tor [Ele]. Снизить практический риск могут несколько мер:

- Размещать выходной ретранслятор на отдельном сервере в дата-центре, а не дома или на личном рабочем компьютере.
- Явно указывать, что IP-адрес принадлежит выходному ретранслятору, например через информативную запись обратного DNS и информационную страницу, которую tor может отдавать через DirPort с помощью `DirPortFrontPage`.
- Заранее уведомить провайдера связи и согласовать порядок обработки сообщений о злоупотреблениях.
- Объяснять работу Tor в ответах на жалобы и сохранять переписку.

25.2.3. Хранение данных

Статья 132 Кодекса Италии о защите персональных данных требует от подпадающих под норму *провайдеров* хранить данные телематического трафика в течение двенадцати месяцев для целей расследования и преследования преступлений [03]; для телефонного трафика срок составляет двадцать четыре месяца. Пункт 5-*bis* сохраняет специальное правило: статья 24 Закона № 167 от 20 ноября 2017 года увеличивает срок до **семидесяти двух месяцев** для расследований преступлений, указанных в статьях 51(3-*quater*) и 407(2)(a) Уголовно-процессуального кодекса, включая терроризм и другие тяжкие преступления [17]. Соответствующая часть Кодекса о защите персональных данных применяется к услугам электронной связи, *доступным населению через публичные сети связи* (статья 121).

Является ли добровольческий оператор ретранслятора таким провайдером — ещё один нерешённый вопрос, зависящий в том числе от способа предоставления сервиса. Операторам, особенно организациям, уже предоставляющим услуги связи, следует до запуска выходного ретранслятора получить правовую оценку. С технической точки зрения при рекомендуемых настройках журналирования tor обычно не хранит историю адресов назначения пользователей. Выходной ретранслятор видит адреса назначения соединений, которые он пересылает, но не знает исходный IP-адрес клиента. Фактически доступные данные зависят от роли и конфигурации ретранслятора: нельзя предполагать, что выходной ретранслятор способен устанавливать личности отдельных пользователей Tor.

25.2.4. Журналы — это персональные данные

IP-адреса относятся к персональным данным. Общий регламент по защите данных (GDPR) требует обрабатывать только необходимые данные и хранить их не дольше необходимого

(статья 5, принцип минимизации данных). Для ретранслятора выбор, согласованный с безопасностью, — журналировать только на уровне `notice`, оставлять `SafeLogging` включённым и избегать захвата сетевых пакетов (разд. 20.4).

25.3. Исследователи

В истории исследований Tor были этически сомнительные эксперименты, некоторые из которых обсуждаются в гл. 27. Исследователи запускали ретрансляторы, которые наблюдали или изменяли трафик реальных пользователей, чтобы продемонстрировать атаки. Помимо этических вопросов, подобные эксперименты могут затрагивать нормы уголовного права Италии, включая положения о незаконном перехвате компьютерных коммуникаций (статья 617-*quater* Уголовного кодекса), а также правила защиты данных.

Tor Research Safety Board — группа исследователей, консультирующая по безопасности экспериментов и разработавшая соответствующие рекомендации [Tor]:

- По возможности используйте тестовую сеть и атакуйте только собственный трафик.
- Собирайте только необходимые данные и только такие данные, которые было бы безопасно опубликовать.
- Защищайте собранные данные и экспериментальные инструменты.
- Уменьшайте точность данных округлением или добавлением шума.
- Учитывайте сторонние наборы данных, которые при объединении с вашими могут раскрыть дополнительную информацию.
- Задайтесь вопросом, ожидали ли пользователи, что соответствующие данные останутся приватными.
- Убедитесь, что ожидаемая польза оправдывает риски.

Совет просит исследователей объяснить, что они хотят узнать, как намерены это сделать, какие риски создаёт эксперимент и почему польза оправдывает эти риски. Эта книга следует той же дисциплине: лабораторные работы, создающие реальный трафик Tor, используют приватную тестовую сеть (разд. 19.1), а анализ общедоступных документов, например в гл. 24, проводится автономно по открытым агрегированным данным.

Итоги

- Само по себе использование Tor законно; деятельность через Tor регулируется обычным правом.
- Статья 4 DSA освобождает подпадающие под неё сервисы, которые лишь передают чужую информацию, от ответственности при соблюдении определённых условий. Принципы работы выходного ретранслятора важны для оценки этих условий, но вопрос о том, подпадает ли добровольческий ретранслятор под Регламент, остаётся юридически открытым.
- Освобождение не исключает расследований и изъятия оборудования. Выделенные серверы, явное обозначение выходного ретранслятора и договорённости с провайдерами связи могут уменьшить практические проблемы.

- Требования к хранению данных применяются к подпадающим под норму общедоступным провайдерам связи; применимость к добровольческим ретрансляторам требует правовой оценки конкретного случая.
- Исследования Tor по возможности должны избегать экспериментов с реальными пользователями: используйте тестовые сети, минимизируйте данные, публикуйте только безопасные агрегаты и сопоставляйте пользу с рисками.

Упражнения

Упражнение 25.1 ⋆ Для каждого из трёх условий статьи 4 DSA опишите изменение в поведении выходного ретранслятора, которое нарушило бы соответствующее условие.

Упражнение 25.2 ⋆ На примере объясните различие между законностью инструмента и законностью его использования, применив его и к пользователю Tor, и к оператору ретранслятора.

Упражнение 25.3 ⋆⋆ Составьте ответ на сообщение о злоупотреблениях, полученное от провайдера связи выходного ретранслятора после жалобы сайта на повторяющиеся попытки доступа с вашего IP-адреса. Что можно объяснить и какие практические меры предпринять?

Упражнение 25.4 ⋆⋆ Исследователь предлагает подсчитывать доменные имена, наблюдаемые на выходном ретрансляторе, чтобы оценить, сколько пользователей Tor посещают итальянские новостные сайты. Оцените предложение по рекомендациям Tor Research Safety Board и предложите более безопасную альтернативу.

Упражнение 25.5 ⋆⋆⋆ Найдите судебные решения, касающиеся операторов ретрансляторов Tor, в судах Италии и других европейских стран. Для каждого кратко изложите юридический вопрос и результат, ссылаясь на первичные источники.

Дополнительная литература

- Регламент (ЕС) 2022/2065, в особенности статьи 4, 8 и 9 [22].
- Законодательный декрет 196/2003, статьи 121 и 132, в сводном тексте, опубликованном Итальянским органом по защите персональных данных [03].
- Electronic Frontier Foundation, *Legal FAQ for Tor Relay Operators* [Ele].
- Сайт Tor Research Safety Board [Tor].

Часть IV

Атаки и защита

Глава 26

Корреляция трафика, распознавание сайтов и атаки Sybil

Последняя проверка: 23 сентября 2026 г. — работы, цитируемые в этой главе; tor 0.4.9.13; лабораторная сеть

Цели главы

К концу этой главы вы сможете:

- классифицировать атаки против анонимности Tor по позиции наблюдения атакующего;
- объяснять сквозную корреляцию трафика и измерять её на собственном трафике в лабораторной сети (Лабораторная работа 16);
- описывать распознавание сайтов по характеру трафика и ограничения его экспериментальных оценок;
- распознавать Sybil-атаки и защитные механизмы сети — от сторожевых узлов до измерения пропускной способности.

Модель угроз из гл. 4 ясно показывала, что Tor не рассчитан на противника, наблюдающего оба конца коммуникации. Двадцать лет исследований количественно подтвердили это предупреждение и показали, что точки наблюдения многочисленнее и доступнее, чем может казаться сначала. В этой главе мы рассматриваем атаки с позиции защитника: что требуется атакующему, чего он добивается и какие уже изученные механизмы протокола усложняют задачу.

26.1. Позиция атакующего

Полезно классифицировать атаки по тому, что может наблюдать противник:

Один конец Соединение клиент–сторожевой узел, наблюдаемое провайдером связи, локальной сетью или цензором; либо соединение выходной узел–назначение. На линии клиент–сторожевой узел наблюдаются время, объём и направление зашифрованного трафика; между выходом и назначением шифрование прикладного уровня зависит от протокола. Распознавание сайтов может использовать метаданные на стороне клиента.

Оба конца Атакующий наблюдает и вход, и выход, либо управляя ретрансляторами в обеих позициях, либо контролируя сети на путях между ними. Это позволяет выполнять корреляцию.

Внутри сети Враждебные ретрансляторы, включая промежуточные узлы, могут измерять, задерживать, маркировать или отбрасывать ячейки.

Над сетью Базовая инфраструктура Интернета, включая автономные системы (AS) и точки обмена интернет-трафиком (IXP), через которые проходят соединения ретрансляторов.

26.2. Сквозная корреляция

Tor намеренно не задерживает и не перемешивает ячейки так, как это делают высоколатентные mix-сети, чтобы оставаться пригодным для интерактивного использования (разд. 2.2). Поэтому *форма* трафика — сколько байт проходит в каждом временном интервале — часто остаётся узнаваемой с двух концов, несмотря на задержки, буферизацию, мультиплексирование и посторонний трафик (рис. 26.1). Тот, кто наблюдает оба конца, может сравнивать формы и решать, соответствуют ли потоки друг другу, не взламывая шифрование.

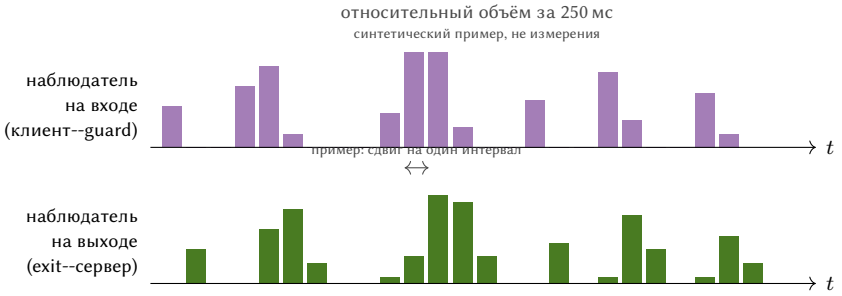


Рис. 26.1: Два *синтетических* ряда, а не реальные измерения объёма трафика в одном направлении на линиях клиент–guard и exit–сервер. Всплески похожи, но не идентичны; сдвиг на один интервал 250 мс выбран лишь для иллюстрации. Шифрование скрывает содержимое, но не всегда временные и объёмные признаки. Буферизация, накладные расходы, мультиплексирование и дополнительные потоки изменяют картину; статистическое сходство само по себе не доказывает совпадение.

Насколько легко сопоставлять потоки? В 2018 году DeepCorr обучил нейронную сеть распознавать пары Tor-потоков. При примерно 900 пакетах на поток он достиг 96 % точности против 4 % у более раннего статистического метода в тех же условиях [NBH18]. Для атакующего основная трудность обычно заключается не в сравнении потоков, а в занятии правильных точек наблюдения.

Насколько вероятно наблюдать оба конца? Johnson и коллеги месяцами симулировали типичных пользователей с реальным алгоритмом выбора маршрута Tor [Joh+13]. При вредоносных ретрансляторах умеренной пропускной способности 80 % пользователей во всех смоделированных типах были деанонимизированы хотя бы один раз в течение шести месяцев. Для одного AS пользователи в некоторых распространённых географических положениях приближались к 100 % в течение трёх месяцев. Эти исторические результаты симуляции не означают, что каждое соединение раскрыто; они показывают, что возможности накапливаются со временем. Именно такая логика стала одной из причин перехода к более долгоживущим сторожевым узлам и меньшему числу сторожевых узлов на клиента (разд. 13.3).

Интернет как противник. RAPTOR описал три способа, которыми AS может увеличить свои возможности [Sun+15]: использовать асимметричные интернет-маршруты, поскольку иногда достаточно наблюдать *одно* направление на каждом конце, включая подтверждения TCP; использовать естественные изменения маршрутизации со временем; а также перехват или перенаправление префиксов посредством BGP. Эти методы не меняют Tor. Защита включает мониторинг маршрутизации и потенциально выбор путей с учётом топологии.

Активные атаки. Атакующий, контролирующий один конец, может намеренно *впечатывать* отличимый трафиковый рисунок, задерживая или группируя пакеты, а затем искать его на другом конце. Атака маркировки, обсуждавшаяся на уровне шифрования в разд. 10.3, — крайняя форма этой идеи. Уже в 2005 году Murdoch и Danezis показали, что наблюдать реальные каналы не всегда обязательно. Вредоносный сервер навязывает ритм трафику к жертве; атакующий измеряет задержку на собственных цепях через ретрансляторы-кандидаты, и те, чья задержка меняется в том же ритме, могут находиться в цепи жертвы [MD05].

26.3. Лабораторная работа: корреляция собственного трафика

Лабораторная работа 16: Сквозная корреляция

Цель. Показать в лабораторной сети, что рисунок трафика сохраняется через цепь и позволяет определить, какой поток на выходной стороне соответствует какому клиенту.
Предварительные условия. Лабораторная работа 11; лабораторная сеть должна быть запущена. Все потоки создаются самим экспериментальным сценарием.
Оценочное время. 1 час.

Скрипт `labs/ru/лаборатория/correlation.py` запускает сервер на порту контейнера 8095 и шесть клиентов. Каждый клиент использует разные SOCKS-учётные данные и при включённой опции `IsolateSOCKSAuth` их потоки не разделяют одну цепь (разд. 18.3). В течение 20 секунд клиенты случайным образом чередуют серии блоков по 2 KB с паузами. Два «наблюдателя» записывают данные:

- Входной наблюдатель записывает моменты, когда каждый клиент отправляет данные. Это идеализирует картину наблюдателя на линии к сторожевому узлу. Там данные были бы упакованы в ячейки и TLS, но нужный для корреляции временной рисунок переживает оба слоя.
- Наблюдатель на выходной стороне, наш сервер, записывает моменты поступления байтов по каждому соединению, и получает четырёхбайтовый идентификатор для проверки результатов, который не участвует в выборе соответствия.

Оба агрегируют байты в интервалы по 250 мс. Для каждого клиента вычисляется коэффициент корреляции Пирсона со всеми шестью потоками на стороне exit. Наиболее коррелированный поток считается предполагаемым соответствием. Скрипт знает правильный ответ, передаваемый клиентом в первых четырёх байтах, но использует его только для оценки, а не для выбора соответствия.

```
$ docker exec -w /tmp torbook-network python3 /lab/correlation.py 1
клиент 0: +0.79 -0.11 +0.12 +0.17 +0.11 -0.01  -> поток exit 0 ВЕРНО
клиент 1: -0.11 +0.80 -0.02 +0.07 +0.18 +0.08  -> поток exit 1 ВЕРНО
клиент 4: +0.13 +0.02 +1.00 +0.21 +0.10 +0.21  -> поток exit 2 ВЕРНО
клиент 3: +0.21 +0.09 +0.33 +0.81 -0.07 +0.03  -> поток exit 3 ВЕРНО
клиент 2: +0.17 +0.05 +0.10 -0.09 +0.85 -0.03  -> поток exit 4 ВЕРНО
клиент 5: -0.00 +0.01 +0.23 +0.06 -0.13 +0.92  -> поток exit 5 ВЕРНО
```

```
правильных совпадений: 6/6
```

Столбцы обозначают выходные потоки в порядке установления соединений, который отличается от порядка клиентов. В этом иллюстративном выводе правильные коэффициенты лежат между 0,79 и 1,00; порядок потоков и значения могут меняться от запуска к запуску. Не нужны ни ключи расшифрования, ни расшифрованные ячейки — только объём данных и время. Лабораторная сеть мала и почти не нагружена, поэтому задержки минимальны. Исторические результаты DeepCorr показывают, что значительно более сложные классификаторы могут работать с несколькими сотнями пакетов в реальной сети при исследованных условиях. Это фрагмент эталонного запуска, а не точный прогноз: возможны и 5/6 совпадений. Лабораторная работа входит в `make verifica-lab`.

26.4. Распознавание сайтов по трафику

Атакующий, наблюдающий только линию клиент–сторожевой узел, не может выполнить сквозную корреляцию, но может попытаться *распознать* посещаемый сайт. Каждая веб-страница порождает характерную последовательность ячеек в обоих направлениях: количество, порядок и серии передач. Атакующий посещает целевые сайты через Tor, записывает их последовательности и обучает классификатор. Затем он сравнивает трафик жертвы с изученными паттернами.

Исследования атак много лет представляют собой соревнование между классификаторами и защитой:

- В 2011 году Panchenko и коллеги использовали объём, время и направление трафика, повысив распознавание в закрытом наборе из 775 сайтов с 3 % до 55 %. Они также продемонстрировали эффективную атаку в *открытом мире*, где жертва может посещать сайты вне обучающего набора атакующего [Pan+11].
- В 2018 году Deep Fingerprinting использовал свёрточную нейронную сеть и превысил 98 % точности на 95 незащищённых сайтах и 90 % против WTF-PAD — защиты на основе адаптивного заполнения, рассматривавшейся для Tor [Sir+18].

Подробнее: Ошибка базовой частоты

Очень высокие значения точности требуют осторожной интерпретации. Juarez и коллеги показали, что многие исследования предполагали выгодные условия: посещается одна страница за раз, у жертвы и атакующего одна версия Tor Browser, а обучающие данные собраны незадолго до атаки [Jua+14]. Ещё важнее то, что в открытом мире отслеживаемые сайты составляют крошечную долю всех посещений. Даже классификатор с низкой частотой ложных срабатываний может выдавать больше ложных тревог, чем правильных распознаваний, среди миллионов нерелевантных посещений. Это та же статистическая ловушка, что и при скрининговых тестах редких заболеваний; реальному атакующему приходится учитывать её.

Защита. Заполнение канала (гл. 7) затрудняет определение периодов бездействия, но не предназначен специально для защиты от распознавания страниц по трафику. tor также реализует более общий механизм *заполнения цепей*: конечные автоматы могут планировать фиктивные ячейки. На эталонную дату исходного текста развёрнутые машины помогают цепям представления и встречи onion-сервисов выглядеть как обычные цепи (гл. 14, Предложение 302). Эффективная широкая защита от распознавания сайтов по трафику с приемлемыми накладными расходами по пропускной способности остаётся открытой исследовательской задачей.

26.5. Sybil: размножение идентичностей ретрансляторов

Sybil-атака вводит в сеть множество идентичностей. В Tor атакующий добавляет многочисленные ретрансляторы в надежде чаще занимать позиции сторожевых и выходных узлов. Три защитных механизма повышают стоимость:

Измеренная пропускная способность Ретранслятор, завышающий заявленную пропускную способность, мог бы привлечь слишком много клиентов. Bauer и коллеги продемонстрировали такую атаку через дескрипторы в 2007 году [Bau+07]. Сегодня веса консенсуса основаны на измерениях, учитываемых центрами каталогов, а не на одних заявлениях (разд. 12.4); чтобы привлечь значительный трафик, нужно предоставить реальную пропускную способность.

Ограничения маршрута Два ретранслятора из одного семейства или одной IPv4-сети /16 не могут находиться в одной цепи (разд. 13.1). Честные операторы объявляют семейства; нечестных хотя бы ограничивают адреса.

Мониторинг сети Центры каталогов могут исключать ретрансляторы или присваивать `BadExit`. Добровольцы и исследователи изучают группы с общими характеристиками и наблюдают поведение выходных узлов. Например, в 2014 году сканер выходных узлов обнаружил 25 ретрансляторов, изменявших трафик или имевших опасную конфигурацию, включая перехват HTTPS, изменение DNS и *sslstrip* [WL14].

В гл. 27 рассматриваются вредоносные группы ретрансляторов, найденные в последующие годы, и показано, почему мониторинг остаётся постоянной задачей.

Sybil-атаки против onion-сервисов. В v2 позиции HSDir, ответственных за сервис, были предсказуемы. Атакующий мог генерировать идентичности ретрансляторов так, чтобы они занимали нужные позиции. Biryukov и коллеги в 2013 году использовали это для измерения популярности сервисов, создания недоступности и облегчения атак на местоположение [BPW13]. В v3 позиции зависят от ослеплённых ключей и общего случайного значения, меняющегося ежедневно (гл. 14), что сильно усложняет заранее рассчитанное размещение; после публикации случайного значения текущего периода ответственные HSDir всё ещё можно вычислить.

26.6. Обнаружение сторожевого узла

Сторожевой узел защищает клиента, но одновременно является привлекательной целью: его идентификация показывает атакующему, где наблюдать. Против onion-сервиса атакующий может провоцировать множество построений цепей, например создавая много соединений, пока вредоносный ретранслятор не окажется промежуточным узлом непосредственно рядом со сторожевым. Первая атака такого типа, опубликованная в 2006 году, сама стала одним из стимулов к использованию сторожевых узлов [ØS06]. Vanguards (разд. 13.4) добавляют слой промежуточных ретрансляторов, фиксируемых на некоторое время. Это затрудняет продвижение атакующего к настоящему сторожевому узлу, но не гарантирует невозможность его обнаружения.

Итоги

- Атаки различаются по точке наблюдения: один конец, оба конца, внутри сети или базовые интернет-маршруты.

- Корреляция сравнивает форму трафика на обоих концах без взлома шифрования. Главная задача атакующего — занять нужные позиции; возможности накапливаются со временем.
- Распознавание сайтов по трафику определяет страницы по зашифрованному трафику на одном конце. Оценки должны учитывать базовые частоты и реалистичные условия.
- Защита от Sybil включает измеренные веса, семейства, ограничения /16 и мониторинг выходных узлов и групп ретрансляторов. В v3 размещением HSDir сложнее манипулировать заранее, хотя узлы, ответственные за текущий период, можно вычислить.
- Сторожевые узлы и vanguards делают обнаружение первого перехода более медленным и дорогостоящим.

Упражнения

Упражнение 26.1 ⋆ Для каждой атаки в этой главе укажите минимальную позицию наблюдения атакующего и защиту Tor, которая усложняет атаку.

Упражнение 26.2 ⋆ Классификатор распознаёт отслеживаемый сайт с вероятностью 0,95 и ошибочно помечает нерелевантное посещение с вероятностью 0,001. Если только одно посещение из 100 000 действительно относится к отслеживаемому сайту, какая доля срабатываний будет правильной?

Упражнение 26.3 ⋆⋆ Повторите Лабораторную работу 16 с двухсекундными интервалами, затем с длительностью всего пять секунд и, наконец, заставив всех клиентов использовать одинаковый ритм трафика. Какое изменение сильнее всего ухудшает сопоставление и почему?

Упражнение 26.4 ⋆⋆ Измените клиентов Лабораторной работы 16 так, чтобы они отправляли блоки с постоянной скоростью, отправляя фиктивные блоки при отсутствии данных. Каковы дополнительные затраты пропускной способности и что происходит с корреляцией?

Упражнение 26.5 ⋆ ⋆ ⋆ Используя `path_selection.py` и реальный консенсус из Лабораторной работы 15, вычислите вероятность того, что атакующий, контролирующий 5 % пропускной способности сторожевых узлов и 5 % пропускной способности выходных узлов, займёт оба конца хотя бы одной клиентской цепи в течение шести месяцев. Сравните фиксированный сторожевой узел с выбором нового для каждой цепи. Задайте условное число цепей за шесть месяцев; лишь для учебной модели считайте выбор независимым и взвешенным по пропускной способности, пояснив пределы такого расчёта.

Дополнительная литература

- A. Johnson et al., *Users Get Routed* [Joh+13].
- Y. Sun et al., *RAPTOR: Routing Attacks on Privacy in Tor* [Sun+15].
- M. Juarez et al., *A Critical Evaluation of Website Fingerprinting Attacks* [Jua+14].
- *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`, для более поздних исследований.

Глава 27

Реальные атаки на сеть

Последняя проверка: 23 сентября 2026 г. — публикации Tor Project (2014, 2022, 2023); tor 0.4.9.13; лабораторная сеть

Цели главы

К концу этой главы вы сможете:

- восстановить атаку 2014 года с использованием ячеек **RELAY_EARLY**, объяснить добавленную в tor защиту и проверить её в работающей лаборатории (Лабораторная работа 17);
- описать инцидент KAX17 и последовавшую работу по наблюдению за ретрансляторами;
- связать DoS-атаки 2022–2023 годов с защитами, описанными в гл. 15;
- сформулировать общие выводы о защите открытой сети.

В предыдущей главе атаки рассматривались как исследовательские темы: модели, эксперименты и проценты. Здесь мы разберём три инцидента безопасности реальной сети по публичным сообщениям Tor Project. В случае KAX17 не установлены ни намерения оператора, ни факт конкретной вредоносной деятельности. Нас интересует прежде всего то, чему они учат о протоколе и об управлении открытой сетью, а не практические детали проведения атак.

27.1. 2014: сигнализация через типы ячеек

Что произошло. 4 июля 2014 года Tor Project обнаружил и исключил группу ретрансляторов, которые, по его мнению, пытались деанонимизировать людей, публиковавших дескрипторы onion-сервисов или запрашивавших сведения о них, тогда называвшиеся *hidden services*. Ретрансляторы присоединились к сети 30 января: около 115 быстрых ретрансляторов без функции выхода в двух сетях /16, вместе составлявших примерно 6,4 % пропускной способности сторожевых узлов. 30 июля Tor Project опубликовал предупреждение о безопасности с описанием атаки и контрмер [Din14].

Как работала атака. Атака сочетала Sybil-размещение (разд. 26.5) с активным подтверждением трафика (разд. 26.2). Ретрансляторы оставались стабильными достаточно долго, чтобы получить флаги `HSDir` и `Guard`. Когда вредоносный HSDir получал запрос дескриптора, он отвечал последовательностью ячеек, в которой чередование типов **RELAY** и **RELAY_EARLY** кодировало имя запрошенного сервиса (рис. 27.1). Если другой участник той же группы одновременно был сторожевым узлом этого клиента, он мог прочитать последовательность

и связать клиента с запросом дескриптора сервиса, но не доказать, что он действительно посетил этот сервис.

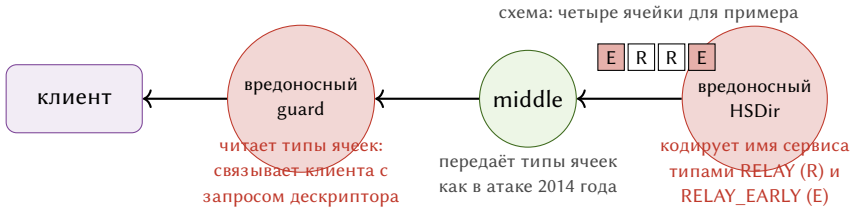


Рис. 27.1: Историческая атака 2014 года: вредоносный HSDir кодировал запрошенный сервис внешними командами ячеек RELAY и RELAY_EARLY, видимыми ретрансляторам в цепи, но защищёнными TLS на сетевых соединениях. Сговорившийся с ним guard мог связать клиента с запросом дескриптора, но не доказать посещение сервиса. Четыре ячейки показаны схематично и не кодируют полный onion-адрес.

Почему именно это поле? Ячейки **RELAY_EARLY** ограничивают длину цепи: клиенты обязаны использовать их для запросов расширения, а ретрансляторы принимают не более восьми на цепь ([tor-spec § Handling RELAY_EARLY cells]). Однако старые проверки применялись только к трафику от клиента в сеть. Обычно никто не отправлял такие ячейки в обратном направлении, и никто их там не проверял. В отличие от содержимого, защищённого послойным шифрованием (гл. 10), внешняя команда ячейки видна каждому ретранслятору. Сигнал в этом поле проходит транзитом без изменений. Предупреждение также отмечало, что сигнал кодировал имя сервиса в читаемом виде, поэтому ретранслятор, получивший такие ячейки, мог распознать сигнал. Постороннему наблюдателю с одними лишь записями сетевых пакетов пришлось бы сначала расшифровать TLS между узлами, чтобы впоследствии извлечь его.

Ответ сети. Последовали три шага: вредоносные ретрансляторы исключили; tor 0.2.4.23 и 0.2.5.6-alpha начали закрывать цепи при получении направленной к клиенту **RELAY_EARLY**; параллельно продолжалась работа по сокращению числа сторожевых узлов каждого клиента с трёх до одного, уменьшая число возможностей попасть в позицию наблюдения. Предупреждение подчёркивало, что *пассивное* подтверждение трафика по-прежнему остаётся нерешённой проблемой. Канал через тип ячейки сделал подтверждение проще, а не впервые возможным.

27.2. Лабораторная работа: лимит RELAY_EARLY

Лабораторная работа 17: Восемь разрешены; девять — нет

Цель. С помощью `labs/ru/код/torlab` проверить, как современный ретранслятор применяет лимит ячеек **RELAY_EARLY**.
Предварительные условия. Лабораторные работы 4 и 5; лабораторная сеть должна быть запущена.
Оценочное время. 30 минут.

Наш `circuit.py` уже отказывается отправлять более восьми ячеек **RELAY_EARLY**. Скрипт `labs/ru/лаборатория/relay_early.py` намеренно обходит эту защиту. Он строит одноузловую цепь к `test004r`, отправляет n ячеек **RELAY_EARLY**, содержащих сообщение `DROP`, которое ретранслятор отбрасывает без ответа, а затем запрашивает соединение с каталогом через `BEGIN_DIR`, чтобы проверить, остаётся ли цепь активной:

```
$ PYTHONPATH=labs/ru/код python3 labs/ru/лаборатория/relay_early.py 0 8 9
0 ячеек RELAY_EARLY -> CONNECTED
8 ячеек RELAY_EARLY -> CONNECTED
9 ячеек RELAY_EARLY -> circuit уничтожена, причина 1
```

Девятая ячейка закрывает цепь: relay отправляет **DESTROY** с причиной 1, PROTOCOL. Листинг получен в эталонном запуске: адреса, идентификаторы цепей и записи журнала различаются между запусками. Эталонный журнал ретранслятора:

```
[warn] Received too many RELAY_EARLY cells on circ 4045911407 from
       127.0.0.1:58048.  Closing circuit.
```

Проверка находится в `src/core/or/command.c`, до расшифрования ячейки. Несколькими строками выше расположен контроль, добавленный в 2014 году: направленная к клиенту **RELAY_EARLY** вызывает предупреждение и закрытие цепи. Комментарий в исходном коде объясняет, что когда-то такие ячейки возникали из-за старой ошибки tor, но ретрансляторы с этой ошибкой давно исчезли. Сегодня такая ячейка считается «неожиданной» и обрабатывается как атака. Лабораторная работа также входит в `make verifica-lab`.

27.3. 2020–2021: KAX17 и здоровье сети

Второй инцидент показывает менее заметную часть защиты: сначала противника нужно обнаружить. Tor Project рассказал об этом случае в апреле 2022 года в статье о своей команде по состоянию сети [The22].

В декабре 2020 года графики ретрансляторов показали необычный паттерн. Каждый месяц, точно в полночь первого числа, в сеть входили десятки ретрансляторов, а затем постепенно исчезали. Это напоминало ошибку в механизме перехода в спящий режим (`AccountingMax`, разд. 22.2). В сентябре 2021 года начали появляться группы ретрансляторов, которые, судя по признакам, принадлежали одному оператору, но не объявляли семейство и не указывали контактные данные. Центры каталогов исключали их, но появлялись замены. В конце октября анонимное сообщение помогло определить группу, позднее получившую название KAX17; её удалили. После этого прекратились и ежемесячные пики, и волны появления ретрансляторов, из-за чего Tor Project счёл правдоподобным, что за обоими явлениями стоял один оператор. Его личность и цели остаются неизвестными.

Статья необычно откровенно описывает запоздалую реакцию: инструменты мониторинга годами оставались без должного внимания; доверие между добровольцами, сотрудниками и центрами каталогов было слабым; организация не была структурирована для такой работы. Она также объясняет процедуру управления. Специалисты по мониторингу предлагают исключить ретранслятор, но решение большинством принимают центры каталогов, а не сотрудники Tor Project. Это распределённое доверие из гл. 12, применённое к эксплуатации сети.

Подробнее: Ретрансляторы без объявления семейства

Первым тревожным сигналом в случае KAX17 был не выявленная эксплуатация уязвимости, а нарушение социальной нормы: многочисленные ретрансляторы, очевидно управляемые одним оператором, без объявленной family (разд. 22.5). Более общий вывод состоит в том, что атакующему иногда не нужно ломать протокол, если он может обойти соглашения, на которых основан выбор маршрута.

27.4. 2022–2023: отказ в обслуживании

Начиная с лета 2022 года сеть месяцами испытывала различные атаки отказа в обслуживании. В феврале 2023 года Tor Project сообщил, что пользователи иногда не могли загружать страницы или обращаться к onion-сервисам, что тактика и цели атак менялись со временем и что личности атакующих невозможно было определить с уверенностью [The23b]. Onion-сервисы были особенно уязвимы: они не могли блокировать IP-адреса атакующих, потому что запросы приходили по анонимным цепям.

Долгосрочным ответом стала система доказательства выполнения работы (proof of work), появившаяся в августе 2023 года в tor 0.4.8 (разд. 15.4) [The23a]. В обычном состоянии защита неактивна; при атаке сервис просит клиентов решать вычислительные задачи возрастающей сложности и отдаёт приоритет тем, кто выполнил больше работы. Tor Project оценивал обычные затраты для нормального клиента в несколько миллисекунд, а при интенсивной атаке — потенциально примерно в минуту. Принцип тот же, что в Лабораторной работе 9: сделать дешёвую для атакующего атаку дорогой, не требуя от пользователя раскрывать личность.

27.5. Выводы

- **Каждое видимое поле может стать каналом.** Атака 2014 года не взламывала криптографию. Она использовала поле, которое relay могли читать и пересылать. Для каждого незашифрованного бита протокол анонимности должен задаваться вопросами: кто может его записать и кто может прочитать? Это одна из мотиваций более новой схемы шифрования ретрансляционных сообщений (разд. 10.4).

- **Sybil-атаки могут быть дешёвыми; мониторинг — нет.** Добавление ретрансляторов стоит сравнительно мало по сравнению с поддержкой инструментов, компетенций и доверия, необходимых для выявления скоординированного вредоносного поведения.

- **Защиты нужно оценивать во времени.** Использование одного сторожевого узла и увеличение срока его использования и появление vanguards уменьшают число возможностей атакующего с течением времени (разд. 26.2).

- **Прозрачность — часть защиты.** Во всех трёх случаях Tor Project публиковал и то, что было известно, и то, что оставалось неизвестным, позволяя пользователям, операторам и исследователям оценивать риск.

Итоги

- В 2014 году Sybil-группа, чьи ретрансляторы выполняли роли сторожевых узлов и HSDir, кодировала в типах ячеек имена onion-сервисов, дескрипторы которых запрашивали клиенты. Сейчас tor закрывает цепи при получении направленной к клиенту **`RELAY_EARLY`**, а переход от трёх сторожевых узлов к одному, а позднее к двум в 2018 году, сократил возможности выбора вредоносного сторожевого узла (разд. 13.3).

- Лимит в восемь ячеек **`RELAY_EARLY`** можно проверить в лабораторной сети: девятая уничтожает цепь.

- KAX17 был выявлен после месяцев косвенных признаков, включая ежемесячные пики и необъявленные семейства ретрансляторов, а также после анонимного сообщения. Решение об исключении ретрансляторов принимают центры каталогов.

- DoS-атаки 2022–2023 годов способствовали внедрению доказательства выполнения работы для onion-сервисов.

Упражнения

Упражнение 27.1 ⋆ Почему сигнал в зашифрованном содержимом ячейки, записанный HSDir, не мог быть прочитан сторожевым узлом, тогда как сигнал во внешнем типе ячейки мог?

Упражнение 27.2 ⋆ Объясните, как сокращение числа сторожевых узлов с трёх до одного уменьшает риск атаки вроде случая 2014 года и какой недостаток это создаёт.

Упражнение 27.3 ⋆⋆ Измените Лабораторную работу 17 так, чтобы показать: ячейка **RELAY_EARLY** учитывается в лимите, даже если она не содержит запрос на расширение цепи. Затем исследуйте код tor и определите, что происходит, когда **RELAY_EARLY** приходит в направлении клиента.

Упражнение 27.4 ⋆⋆ Используя данные CollecTor, какие признаки могли бы показать, что несколько ретрансляторов принадлежат одному оператору без объявленного семейства? Предложите три критерия и обсудите ложные срабатывания.

Упражнение 27.5 ⋆⋆⋆ Перечислите поля ячеек и ретрансляционных сообщений, видимые промежуточному узлу открытым текстом (гл. 8). Для каждого обсудите, может ли вредоносный ретранслятор использовать его как канал к другому ретранслятору в той же цепи, и укажите существующую или предлагаемую защиту.

Дополнительная литература

- Tor Project, *Tor security advisory: «relay early» traffic confirmation attack* [Din14].
- Tor Project, *Malicious relays and the health of the Tor network* [The22].
- Tor Project, *Introducing Proof-of-Work Defense for Onion Services* [The23a].

Глава 28

Ошибки операционной безопасности и деанонимизация: уроки

Последняя проверка: 23 сентября 2026 г. — рекомендации Tor Project; предупреждения Tails; tor 0.4.9.13; лабораторная сеть

Цели главы

К концу этой главы вы сможете:

- различать уровни, на которых может произойти утечка идентичности, и определять, какие инструменты защищают каждый из них;
- распознавать распространённые операционные ошибки — от утечек DNS до слишком малого множества анонимности;
- использовать сам tor, чтобы проверить, передаёт ли приложение ему разрешение имён (Лабораторная работа 18);
- анализировать реальный инцидент в терминах модели угроз.

Глава 26 завершалась неприятным наблюдением: атаки на протокол могут быть дорогими и требовать привилегированных точек наблюдения. Во многих описанных случаях утрата анонимности также связана с ошибками использования инструментов, которые один лишь сетевой протокол предотвратить не может. В этой главе рассматриваются такие ошибки — не для того, чтобы объяснять, как идентифицировать людей, а чтобы ясно провести границу между защитой Tor и ответственностью самого пользователя.

Внимание: Границы этой главы

Мы обсуждаем ошибки и защитные практики с точки зрения людей, стремящихся защитить себя. Процедуры идентификации людей не приводятся. Упомянутые инциденты являются публичными и реконструированы по указанным источникам.

28.1. Четыре уровня

Идентичность может утечь на четырёх уровнях (рис. 28.1). Первый — сеть — это уровень, который защищает tor: при обычном соединении через Tor назначение не видит исходный IP-адрес клиента, а отдельный ретранслятор цепи обычно не обладает полной картиной обоих концов. Это не исключает корреляционные атаки и другие риски. Второй — приложение — рассматривается Tor Browser, Tails и Whonix (гл. 23). Последние два — содержимое и поведение

— tor не защищает автоматически. Специальные инструменты способны снизить часть рисков, но важно и то, что пользователь пишет, загружает и делает.

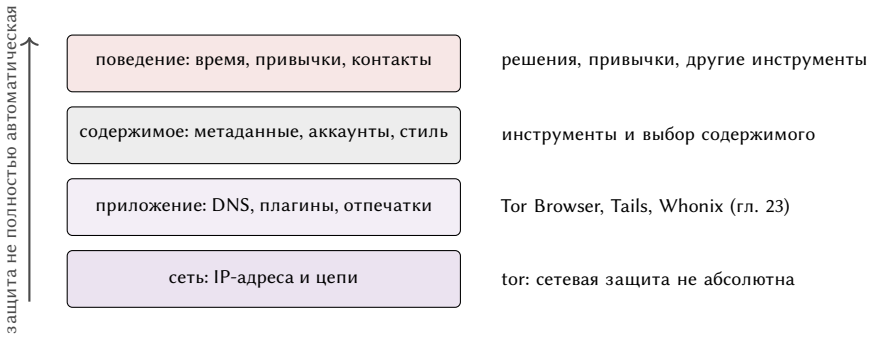


Рис. 28.1: Учебная схема четырёх уровней возможного раскрытия личности. Tor снижает часть сетевых рисков, приложения и инструменты обработки содержимого помогают, но раскрываемые сведения и поведение важны на каждом уровне. Ни один уровень сам по себе не гарантирует анонимности.

28.2. Ошибки приложений

Трафик вне Tor. tor защищает только приложения, настроенные на его использование [Thef]. Программа, выполняющая прямые соединения, даже ради обновлений или предпросмотра, может раскрыть реальный IP-адрес. Служба поддержки Tor Project выделяет два известных случая. BitTorrent-приложения могут игнорировать настройки прокси и дополнительно раскрывать адрес трекерам. Загруженные документы, открытые при активном сетевом соединении, создают другой риск: DOC- или PDF-файлы могут содержать удалённые ресурсы, которые загружаются приложениями, не использующими Tor.

Утечки DNS. Более тонкий случай связан с SOCKS. Приложение может передать tor *имя* назначения, позволяя выходному ретранслятору разрешить его (разд. 11.1.4), либо самостоятельно разрешить имя через системную DNS-службу и передать tor только IP-адрес. Во втором случае последующее соединение всё ещё идёт через Tor, но доменное имя могло уже раскрыться системе разрешения имён вне цепи Tor и оказаться связано с сетевым подключением клиента. Открытый DNS-запрос не обязателен: адрес мог быть получен из кеша, локального файла или по зашифрованному DNS. tor не может доказать, что утечка уже произошла, но способен предупредить о SOCKS-запросе только с IP-адресом, как показывает лабораторная работа.

28.3. Лабораторная работа: tor обнаруживает признак DNS leak

Лабораторная работа 18: SOCKS с hostname и без него

Цель. Наблюдать, как tor различает приложение, передающее ему разрешение имени, и приложение, выполняющее разрешение самостоятельно, а также как tor может отклонить второй вариант.
Предварительные условия. Лабораторная работа 11; лабораторная сеть должна быть запущена.
Оценочное время. 30 минут.

`curl` поддерживает оба варианта. При `--socks5` он разрешает имя локально и передаёт tor IP-адрес; при `--socks5-hostname` передаёт само имя. Скрипт `labs/ru/лаборатория/dns_leak.py` подключается к ControlPort клиента `008c` и включает `TestSocks`, заставляя tor записывать тип каждого SOCKS-запроса. Скрипт слушает сообщения журнала и проверяет оба варианта на `http://localhost:8080/`.

Эта лабораторная работа демонстрирует *механизм*, а не реальную DNS leak, наблюдаемую на линии. `localhost` разрешается локально без обращения к resolver, поэтому запрос не покидает компьютер. tor получает IP-адрес вместо имени хоста: это возможный признак предварительного локального разрешения, а не доказательство утечки DNS (программа могла сразу использовать IP-адрес). Если использовать обычный домен вместо `localhost`, аналогичное локальное разрешение может породить внешний DNS-запрос, который при определённой конфигурации будет открытым. Мы намеренно не генерируем такой трафик и остаёмся внутри приватной сети. Скрипт также принудительно использует SOCKS для `localhost` вопреки возможному исключению `NO_PROXY`. Наконец, он включает `SafeSocks`, отклоняющий запросы только с IP-адресом даже при отсутствии DNS-утечки:

```
--socks5           HTTP 200  tor: Your application (using socks5 to port
                   8080) is giving Tor only an IP address. Applications
                   that do DNS resolves themselves may leak information...
--socks5-hostname  HTTP 200  tor: Your application (using socks5 to port
                   8080) instructed Tor to take care of the DNS resolution
                   itself if necessary. This is good.
--socks5 SafeSocks HTTP 000
```

В иллюстративном выводе оба первых запроса успешны, но только второй передаёт разрешение имени tor; первый не доказывает DNS-утечку, поскольку используется `localhost`. В коде tor (`src/core/proto/proto_socks.c`) первое предупреждение записывается на уровне `warn`, с ограничением частоты записи, чтобы не переполнять журнал, и сопровождается событием `STATUS_CLIENT` с действием `DANGEROUS_SOCKS` на ControlPort. Поэтому управляющее приложение может предупредить пользователя. При включённом `SafeSocks` тот же запрос отклоняется, и `curl` не получает ответа. Лабораторная работа включена в `make verifica-lab`.

28.4. Ошибки содержимого

Метаданные. Многие файлы содержат информацию, которую авторы даже не замечают: фотографии могут раскрывать местоположение или модель камеры; документы — автора и даты создания. Tails включает инструмент удаления метаданных, а его предупреждения упоминают случаи, когда метаданные фотографии раскрывали, где находился фотограф [Taib].

Учётные записи и идентичность. Вход в личную учётную запись сообщает сайту, кто вы, независимо от видимого IP-адреса соединения [Thef]. Использование двух идентичностей на одном сайте в одном сеансе также может связать их через cookie, данные сеанса, идентификаторы приложений и другие признаки. При этом сайт не видит напрямую идентификатор цепи Tor. Tails рекомендует перезапускаться между действиями, которые должны оставаться разделёнными [Taib]. Требование несвязываемости Tor Browser (разд. 23.2) относится к разным сайтам, а не к разным идентичностям на одном сайте.

Стиль письма. Даже без метаданных особенности письма, тематика, повторяющиеся ошибки и время публикаций могут связать анонимный текст с подписанным. Ни один автоматический инструмент не способен устранить все такие признаки.

28.5. Поведенческие ошибки: слишком малое множество анонимности

В гл. 1 анонимность определялась через неразличимость внутри множества. Tor может уменьшить некоторые признаки, позволяющие различать сетевую активность, но не гарантирует, что в конкретном месте и в конкретное время Tor использует достаточно людей. Если пользователей мало, знание того, *кто использовал Tor*, может существенно сузить множество вариантов.

Инцидент. В декабре 2013 года один американский университет получил ложные угрозы взрыва по электронной почте незадолго до экзамена. Отправитель использовал временный почтовый сервис и Tor. Согласно письменному показанию под присягой сотрудника ФБР, приложенному к обвинению и описанному несколькими СМИ, университет определил, кто использовал Tor в его беспроводной сети в часы перед отправкой сообщений [WBU13; San13]. Студент позднее признался во время допроса. Для сужения круга подозреваемых не понадобилось взламывать шифрование Tor: локальные наблюдатели могли установить факт использования Tor, что само по себе не означало знания содержимого или назначения соединений. Но они могли видеть, *кто* использовал Tor в этот временной интервал, уменьшив круг подозреваемых до нескольких имён.

Инцидент иллюстрирует три принципа модели угроз (гл. 4):

- Tor без моста не скрывает сам факт использования: наблюдатели локальной сети могут распознать его [Taib]. Мост с подключаемым транспортом (гл. 16) способны уменьшить эту заметность.
- Корреляции не обязательно нужен глобальный противник, если множество кандидатов мало, а событие происходит в узком временном окне. Это базовая задача пересечения множеств.
- Анонимность любит компанию [DM06]. Чем больше людей с разными причинами используют Tor в одной среде, тем меньше информации несёт сам факт использования конкретным человеком. Служба поддержки Tor Project прямо поощряет более широкое распространение Tor [Thef].

28.6. Контрольный список

Список не исчерпывающий, но последовательно охватывает уровни:

1. Используйте Tor Browser или систему вроде Tails или Whonix, а не обычный браузер с вручную настроенным прокси.
2. Не устанавливайте расширения и не меняйте настройки так, чтобы браузер выделялся.
3. Для других приложений проверяйте, что они передают имена хостов в tor (`SafeSocks`, `TestSocks`).
4. Не открывайте загруженные документы при подключении к сети либо используйте для них изолированную среду.
5. Предпочитайте HTTPS и onion-сервисы, поскольку выходной ретранслятор может видеть незашифрованное содержимое.
6. Удаляйте метаданные из файлов перед публикацией.

7. Не объединяйте в одном сеансе идентичности, которые должны оставаться разделёнными.
8. Если сам факт использования Tor создаёт риск, рассмотрите подходящий замаскированный мост, не предполагая полной невидимости.
9. Помните, что расписание, привычки и стиль письма тоже передают информацию.

Итоги

- Идентичность может утечь на сетевом, прикладном, содержательном и поведенческом уровнях. tor напрямую защищает только первый.
- Приложения, обходящие Tor, загруженные документы, открытые онлайн, и DNS leaks — ошибки уровня приложения. tor может предупреждать о SOCKS-запросах, содержащих только IP-адрес, и отклонять их.
- Метаданные, учётные записи и стиль письма способны связать содержимое с человеком.
- Когда множество пользователей Tor мало, знания о том, кто и когда использовал Tor, может быть достаточно для существенного сужения кандидатов. Мосты и большое сообщество пользователей помогают снизить риск.

Упражнения

Упражнение 28.1 ⋆ Для каждой ошибки, рассмотренной здесь, определите её уровень на рис. 28.1 и инструмент либо привычку, помогающие её предотвратить.

Упражнение 28.2 ⋆ Объясните, почему DNS leak раскрывает информацию, даже если последующее соединение проходит через Tor. Кто видит эту информацию?

Упражнение 28.3 ⋆⋆ В Лабораторной работе 18 подпишитесь через Stem на события `STATUS_CLIENT` и выведите действие `DANGEROUS_SOCKS`, создаваемое запросом только с IP. Как графический интерфейс мог бы использовать это событие?

Упражнение 28.4 ⋆⋆ Повторно рассмотрите инцидент 2013 года, предположив, что студент использовал bridge obfs4 из той же сети. Что могли бы видеть наблюдатели локальной сети и какая другая информация всё ещё могла сузить множество кандидатов?

Упражнение 28.5 ⋆ ⋆ ⋆ Выберите реального или вымышленного человека с законной потребностью в анонимности, например журналистский источник, активиста или исследователя, и составьте операционный план: угрозы, соответствующие уровни, инструменты и привычки, остаточные риски.

Дополнительная литература

- Рекомендации службы поддержки Tor Project по анонимности [Thef].
- Предупреждения Tails, *Tails is safe but not magic* [Taib].
- R. Dingledine и N. Mathewson, *Anonymity Loves Company* [DM06].

Часть V

Исследования и будущее

Глава 29

Академические исследования Tor

Последняя проверка: 23 сентября 2026 г. — работы, цитируемые в этой главе; исследовательский портал Tor Project; консенсус от 23 сентября 2026 г.

Цели главы

К концу этой главы вы сможете:

- находить основные площадки публикации исследований Tor и ресурсы Tor Project;
- количественно измерять анонимность — от энтропии до времени первого компрометационного события;
- выбирать подходящий экспериментальный метод: аналитическое моделирование, симуляцию, эмуляцию или измерение;
- симулировать тысячи клиентов на протяжении месяцев, используя реальный консенсус (Лабораторная работа 19).

Tor с самого начала развивался под влиянием исследований. Onion routing появился в исследовательской лаборатории (гл. 3); архитектура Tor была представлена на академической конференции [DMS04]; многие механизмы, рассматриваемые в книге, от сторожевых узлов до управления перегрузкой, возникли как ответ на опубликованные исследования. Эта глава предназначена для читателей, которые хотят заниматься исследованиями сами: где публиковаться, как измерять анонимность, какие экспериментальные инструменты использовать и какие принципы безопасности соблюдать.

29.1. Где публикуются исследования Tor

Исследования сетей анонимности в основном публикуются на площадках двух типов:

Privacy Enhancing Technologies Symposium (PETS) и его журнал *Proceedings on Privacy Enhancing Technologies* (PoPETs) — специализированная площадка; ключевая работа по измерению анонимности, рассматриваемая далее, была представлена на семинаре PET в 2002 году, до появления журнала PoPETs [Día+02].

Крупные конференции по безопасности USENIX Security, ACM CCS, IEEE Symposium on Security and Privacy и NDSS. Многие статьи из гл. 26 были опубликованы именно там.

Anonymity Bibliography проекта Free Haven более двадцати лет собирает публикации в этой области и является естественной отправной точкой для обзора литературы.

Tor Project также поддерживает исследовательский портал с данными, инструментами, техническими отчётами, предлагаемыми темами исследований и информацией о Research

Safety Board. Рекомендации подчёркивают, что протокол и сеть постоянно меняются: измерения, выполненные без понимания внутренней работы, могут приводить к неверным выводам. Поэтому исследователям рекомендуется связываться с проектом заранее, чтобы узнать о параллельных экспериментах и планируемых изменениях.

29.2. Измерение анонимности

В гл. 1 анонимность определялась через неразличимость внутри множества. Однако с точки зрения атакующего не каждый участник обязательно одинаково вероятен. После наблюдения трафика противник может назначить некоторым пользователям гораздо более высокие вероятности, чем другим. Одного размера множества недостаточно.

Степень анонимности. В 2002 году Díaz и соавторы предложили измерять анонимность через энтропию распределения вероятностей, которое атакующий назначает возможным отправителям [Día+02]. Если p_i — вероятность, приписанная пользователю i как отправителю, энтропия равна

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i,$$

и достигает максимума, $H_M = \log_2 N$, когда все N пользователей равновероятны. **Степень анонимности** определяется как

$$d = \frac{H(X)}{H_M}, \qquad 0 \le d \le 1 :$$

при $N \ge 2$ она равна 1 для равномерного апостериорного распределения (это не обязательно означает отсутствие новых сведений), а 0 — когда вероятность одного отправителя равна 1. При $N = 1$ отношение не определено, поскольку $H_M = 0$.

Интуитивная интерпретация энтропии — $2^{H(X)}$: число равновероятных пользователей, создающее ту же неопределённость. Если среди 1 000 пользователей атакующий сузил подозреваемых до двух с вероятностью 0,5 для каждого, неопределённость равна множеству из двух: $H = 1$ бит.

Ограничения разового измерения. Энтропия описывает отдельное наблюдение, но экспозиция Tor накапливается во времени, как мы видели в разд. 26.2: каждая цепь создаёт новую возможность для атакующего. Поэтому Johnson и коллеги предложили *продольные* метрики, включая вероятность хотя бы одной деанонимизации за заданный период и медианное время до первого компрометационного события, вычисляемые путём симуляции поведения пользователей на реальных сетевых данных [Joh+13]. В лабораторной работе мы используем сходный подход.

29.3. Четыре метода

Аналитическое моделирование Математическая модель, например степень анонимности или модель Sybil-атак из разд. 26.5. Метод быстрый и общий, но зависит от предположений, которые реальные системы в той или иной степени нарушают.

Поведенческая симуляция Воспроизводится упрощённый компонент системы, например выбор маршрута, на основе реальных данных. TorPS, симулятор маршрутов Джонсона и соавторов [Joh+13], и наша Лабораторная работа 19 используют этот подход. Он быстрый, но не учитывает ничего, что не включено в модель.

Сетевая эмуляция и симуляция Реальное ПО запускается либо в приватной сети, например лаборатории Chutney, используемой в книге, либо внутри дискретно-событийного симулятора вроде Shadow [JH12], способного запускать тысячи ретрансляторов и клиентов на одном компьютере с виртуальным временем (разд. 19.9). Особенно полезно для исследований производительности: предлагаемые изменения протокола можно проверить в реалистичной сети до развёртывания.

Измерения в публичной сети Только прямое измерение показывает, что происходит в реальной сети. Оно же создаёт наибольший риск для реальных пользователей и требует особенно осторожного дизайна эксперимента (разд. 29.5).

Сильные исследования часто сочетают несколько методов: математический анализ формулирует гипотезу, симуляция проверяет её в масштабе, а осторожно спроектированное измерение оценивает реалистичность.

Подробнее: Измерение без наблюдения отдельных пользователей

PrivCount, разработанный Jansen и Johnson, показывает, как измерения могут сочетаться с приватностью [JJ16]. Участвующие ретрансляторы подсчитывают нужные события, но счётчики маскируются случайными значениями, разделяемыми с другими участниками. Добавляется шум с применением принципов *дифференциальной приватности*. Видимым становится только агрегированный, намеренно зашумлённый результат, что ограничивает выводы об отдельных пользователях. В 2016 году авторы с помощью PrivCount оценили примерно 710 000 одновременно подключённых пользователей, из которых 550 000 были активны в данный момент, и установили, что на трафик к веб-портам приходился 91% байтов данных, наблюдавшихся на участвующих выходных узлах. Это оценка исследования 2016 года, а не характеристика современной сети. Это развивает те же принципы приватности, что и округление статистики ретрансляторов (разд. 24.2).

29.4. Лабораторная работа: время до первого компрометационного события

Лабораторная работа 19: Месяцы цепей в симуляции

Цель. Используя реальный консенсус, оценить, сколько клиентов атакующий, контролирующий заданную долю пропускной способности, может деанонимизировать со временем при постоянных guard и без них.
Предварительные условия. Лабораторные работы 7 и 15; работающая сеть не требуется.
Оценочное время. 1–2 часа.

Модуль `labs/ru/код/torlab/simulation.py` повторно использует веса из Лабораторной работы 7. Атакующий добавляет сторожевой и выходной ретрансляторы, каждый из которых представляет долю q взвешенной пропускной способности в своей позиции цепи. Мы моделируем 2 000 клиентов в течение 180 дней, каждый строит по 20 цепей в день.

В модели постоянный сторожевой узел меняется по приближённому расписанию для *подтверждённого* сторожевого узла. Согласно guard-spec и коду `entrynodes.c`, подтверждённый сторожевой узел удаляется из выборки через `GUARD_CONFIRMED_MIN_LIFETIME` (60 дней) после подтверждения, а `CONFIRMED_ON_DATE` случайно сдвигается назад максимум на `GUARD_LIFETIME`/10 (12 дней). Поэтому в модели период смены узла составляет 48–60 дней. Параметр `GUARD_LIFETIME`, равный 120 дням, относится к сроку хранения неподтверждённых узлов в выборке. Настоящий клиент учитывает также списки узлов, доступность и замены:

модель не воспроизводит полный алгоритм. Ранняя версия этой лабораторной работы ошибочно моделировала ротацию через 120 дней и занижала результат для выбранной упрощённой модели. Для сравнения мы также моделируем клиентов, выбирающих новый guard для каждой цепи. Клиент считается скомпрометированным в первый день, когда одна из его цепей содержит и вредоносный guard, и вредоносный exit.

Модель намеренно проста: она игнорирует промежуточные ретрансляторы, семейства, сети /16, резервные сторожевые узлы и отказы. Ради скорости она не использует `random.choices`, который пересчитывал бы тысячи весов при каждом выборе. Вместо этого используется бинарный поиск по накопленным весам, позволяющий выполнить миллионы выборок без повторного расчёта всех весов; реальное время зависит от компьютера.

Для воспроизведения сначала загрузите исторический открытый консенсус (`make -C labs/ru data`): файл не хранится в Git. Симуляция детерминирована: при той же модели, консенсусе, доле и зерне результат всегда один и тот же:

```
$ PYTHONPATH=labs/ru/код python3 -m torlab.simulation labs/ru/данные/collector/
    consensus-microdesc-2026-09-23-15 0.05
противник: 5% пропускной способности сторожевых узлов и 5% выходных
    узлов; 2000 клиентов, 180 дней, 20 цепей/день
постоянный сторожевой узел    1 дн.:  2.6%    7 дн.:  4.4%   30 дн.:  4.4%   90 дн.:
    9.4%  180 дн.: 18.4%  медиана: None
новый сторожевой узел на цепь 1 дн.:  4.3%    7 дн.: 27.9%   30 дн.: 76.5%   90 дн.:
    98.9%  180 дн.:100.0%  медиана: 15
```

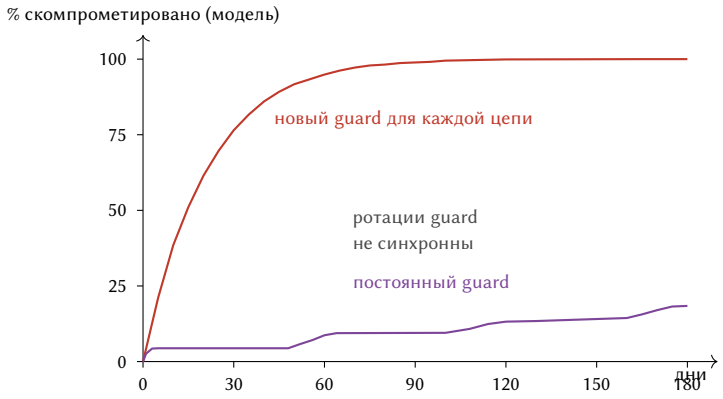


Рис. 29.1: Лабораторная работа 19, упрощённая симуляция: 2000 клиентов, по 20 цепей в день в течение 180 дней, консенсус от 23 сентября 2026 года; атакующий контролирует 5% взвешенной полосы сторожевых и 5% выходных узлов. Накопительная доля отражает клиентов, чья хотя бы одна цепь в модели содержала оба вредоносных конца. Ротации постоянных guard распределены по клиентам (модельные интервалы 48–60 дней), а не происходят в три общие даты. Это не наблюдения реальной деанонимизации в публичной сети.

Рис. 29.1 показывает обе кривые. Без постоянных guard половина клиентов оказывается скомпрометирована за пятнадцать дней, а почти все — за три месяца, потому что каждая новая цепь создаёт ещё одну независимую возможность. При постоянных guard экспозиция клиента в основном определяется моментом выбора guard. Около 4% клиентов, выбравших вредоносный guard, компрометируются рано; остальные не удовлетворяют этому конкретному условию компрометации, пока сохраняют честный сторожевой узел. По мере того как разные клиенты меняют guard в разные моменты времени (примерно каждые 48–60 дней для каждого клиента в модели), возникают новые возможности и доводит итог примерно до 18% за шесть месяцев. При доле вредоносной пропускной способности 1% соответствующий шестимесячный результат падает примерно до 4% против 31% без постоянных guard.

Эти результаты, основанные на эталонном снимке сентября 2026 года, иллюстрируют назначение сторожевых узлов. В этой упрощённой модели их главное преимущество проявляется в распределении риска между клиентами, а не в гарантированном снижении вероятности на каждой отдельной цепи. Для большинства пользователей смоделированный риск сначала близок к нулю, но для меньшей группы он высок. Поэтому время жизни guard — компромисс. Более медленная ротация дольше защищает пользователей с честным guard, но также продлевает экспозицию тех, кто выбрал вредоносный. Терпеливый атакующий, остающийся в сети месяцами, получает новые возможности при каждой ротации.

29.5. Безопасное проведение исследований

В гл. 25 были введены рекомендации Tor Research Safety Board [Tor]. При проектировании эксперимента сначала спросите: *действительно ли исследовательский вопрос требует публичной сети?* Многие вопросы производительности и атак можно изучить с помощью Shadow или приватной сети, получая воспроизводимые эксперименты и контроль переменных. Если измерение публичной сети действительно необходимо, следуйте уже обсуждавшимся принципам: воздействуйте только на собственный трафик, собирайте минимум данных, агрегируйте до публикации и заранее консультируйтесь с Board и Tor Project.

Недавняя история даёт противоположные примеры. В гл. 27 описан эксперимент или атака, затронувшая реальных пользователей через сигнал, доступный получающим ретрансляторам, но не непосредственно стороннему наблюдателю за зашифрованными TLS-соединениями. PrivCount, напротив, показывает, что подробные измерения сети возможны без раскрытия отдельных пользователей. Разница заключается не только в инструментах, но и в дизайне эксперимента.

Итоги

- Исследования Tor публикуются на PETS и крупных конференциях по безопасности. Tor Project предоставляет данные, инструменты и Research Safety Board.
- Энтропия измеряет анонимность для отдельного наблюдения; степень анонимности нормализует её между нулём и единицей. Во времени нужны метрики вроде вероятности компрометации за заданный интервал.
- Аналитические модели, поведенческие симуляции, сетевая эмуляция или симуляция и измерения публичной сети отвечают на разные вопросы и несут разные риски.
- В симуляции по эталонному консенсусу постоянные guard существенно уменьшают долю клиентов, компрометируемых со временем, концентрируя экспозицию у меньшего числа пользователей.

Упражнения

Упражнение 29.1 ⋆ Атакующий наблюдает систему из четырёх пользователей и назначает вероятности $1/2, 1/4, 1/8$ и $1/8$. Вычислите энтропию, степень анонимности и эквивалентное число равновероятных пользователей.

Упражнение 29.2 ⋆ Выберите и обоснуйте наиболее подходящий метод для каждого вопроса: (a) как изменилась бы задержка, если бы цепи использовали четыре перехода? (b) Сколько пользователей подключается к Tor из конкретной страны? (c) Как новый алгоритм выбора сторожевого узла изменил бы время до первого компрометационного события?

Упражнение 29.3 ⋆⋆ Запустите `simulate` из Лабораторной работы 19 с `guard_lifetime`, равным 30 и 180 дням вместо значения по умолчанию 60, и сравните долю скомпрометированных клиентов через 180 дней. Объясните результаты.

Упражнение 29.4 ⋆⋆ Повторите Лабораторную работу 19 с атакующим, контролирующим 10% пропускной способности выходных узлов, но не сторожевых, а затем с атакующим, контролирующим 10% пропускной способности сторожевых узлов, но не выходных. Чего способен добиться каждый и почему?

Упражнение 29.5 ⋆⋆⋆ Расширьте симулятор тремя функциями, которые текущая модель игнорирует: повторное использование клиентами цепей в течение десяти минут вместо постоянного создания новых, ограничение /16 между guard и exit и атакующий, контролирующий целую автономную систему крупной сети. Какие дополнения сильнее всего меняют результат?

Дополнительная литература

- C. Díaz, S. Seys, J. Claessens, B. Preneel, *Towards Measuring Anonymity* [Día+02].
- R. Jansen, A. Johnson, *Safely Measuring Tor* [JJ16].
- Исследовательский портал Tor Project: `https://research.torproject.org/`.
- *Anonymity Bibliography*: `https://www.freehaven.net/anonbib/`.

Глава 30

Arti: Tor на Rust

Последняя проверка: 23 сентября 2026 г. — Arti 2.6.0 (1 сентября 2026 г.); tor 0.4.9.13; лабораторная сеть

Цели главы

К концу этой главы вы сможете:

- объяснять, почему Tor Project переписывает tor на Rust, и оценивать прогресс проекта;
- понимать архитектуру Arti и находить в ней концепции, рассмотренные в Части II;
- интерпретировать таблицу поддерживаемых подпротоколов и понимать, какие функции всё ещё отсутствуют;
- настраивать Arti как клиент приватной сети книги (Лабораторная работа 20).

На протяжении всей книги мы опирались на *tor*, реализацию на C, разработка которой началась в 2002 году. Однако будущее клиентской части Tor — **Arti**, полная реализация с нуля на Rust. Tor Project обозначил Arti как предпочтительный клиент для разработки новой функциональности и объявил, что каждая новая клиентская функция будет получать реализацию в Arti. Эта глава рассматривает Arti глазами читателя, который в лабораторных работах только что написал небольшой Tor-клиент на Python. Многие архитектурные решения Arti — те же решения, с которыми столкнулись мы, только реализованные в промышленном масштабе.

30.1. Зачем переписывать tor?

Вводный документ проекта перечисляет четыре причины:

Безопасность Безопасный код Rust предотвращает многие ошибки работы с памятью, но небезопасный код, зависимости и логика приложения всё ещё требуют проверки. По оценке Tor Project, как минимум половина зарегистрированных уязвимостей безопасности tor была бы невозможна на Rust, а многие другие стали бы крайне маловероятными.

Скорость разработки Гарантии на уровне языка позволяют разрабатывать новый код увереннее и быстрее.

Гибкость tor начинался как прокси SOCKS, а возможности интеграции добавлялись позднее. Arti изначально проектировался как модульная библиотека, которую можно встраивать в другие приложения.

Сопровождаемость Двадцать лет разработки создали сложные взаимосвязи между компонентами кодовой базы tor. Переписывание даёт возможность яснее разделить ответственности.

Вехи. Первая публичная версия Arti, 0.0.1, появилась в октябре 2021 года. Версия 1.0.0, выпущенная 1 сентября 2022 года, объявила Arti стабильным клиентом. В августе 2023 года версия 1.1.7 включила сборку клиентской поддержки onion-сервисов по умолчанию, хотя пользователю всё ещё требовалось явно активировать функцию. Версия 2.0.0 вышла в феврале 2026 года. Эталонная версия этой главы — 2.6.0 от 1 сентября 2026 года; в ней управление перегрузкой (разд. 11.3) и шифрование CGO (разд. 10.4) включены постоянно.

Текущая область применения. Согласно документации проекта на эталонную дату, Arti — полноценный универсальный клиент с поддержкой мостов и подключаемых транспортов; он умеет и обращаться к onion-сервисам, и публиковать их. Он пока не может работать как ретранслятор, мост или центр каталогов. Эти возможности находятся в разработке, и значительная часть работы в последних релизах была сосредоточена на поддержке ретрансляторов и центров каталогов.

30.2. Архитектура

Arti состоит из десятков *крейтов* Rust — единиц компиляции языка: в версии 2.6.0 их более семидесяти. Соглашения об именовании различают роли. Пользовательские крейты начинаются с `arti`, специфичные для протокола Tor — с `tor-`, а крейты, полезные вне Tor, имеют более общие названия. Архитектуру определяет важный принцип: код, который напрямую читает или записывает сетевые данные, должен быть минимальным; большинство компонентов должно вести себя «так, как если бы сети не существовало», что упрощает тестирование.

Рис. 30.1 сопоставляет основные крейты с модулями `torlab`, выполняющими сходные задачи. Соответствие не случайно: обе архитектуры следуют уровням протокола, от низкоуровневых примитивов до высокоуровневого клиента.

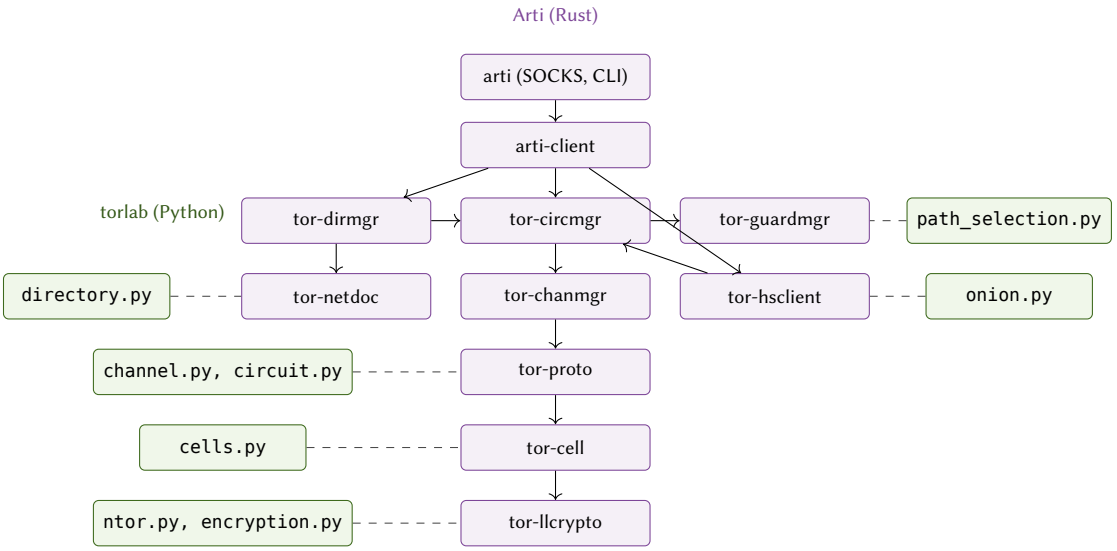


Рис. 30.1: Упрощённая функциональная схема крейтов Arti и учебных модулей `torlab`. Сплошные стрелки показывают архитектурные связи, но не точный граф зависимостей Cargo; пунктир указывает на сходные задачи, а не на зависимости между проектами или полную эквивалентность. Например, `onion.py` только расшифровывает дескрипторы и не заменяет клиент onion-сервисов; крейты `tor-bytes` и `tor-hsservice` опущены.

tor-llcrypto, tor-bytes Криптографические примитивы и разбор/сериализацию двоичных данных. Они предоставляют основу, которую наши `ntor.py` и `encryption.py` получали из модулей Python `cryptography` и `struct`.

tor-cell Ячейки и ретрансляционные сообщения: аналог `cells.py` в Arti (гл. 8).

tor-proto Каналы, цепи, послойное шифрование и потоки: всё, что мы строили с помощью `channel.py`, `circuit.py` и `stream.py`.

tor-chanmgr, tor-circmgr, tor-guardmgr Управление множеством каналов и цепей, выбор маршрутов и поддержка сторожевых узлов. Наш учебный клиент почти не реализовывал эти возможности, хотя они относятся к наиболее сложным частям настоящего клиента.

tor-netdoc, tor-dirmgr Чтение и обновление документов каталога, соответствующие `directory.py`.

tor-hsclient, tor-hsservice Обе стороны onion-сервисов; наш `onion.py` реализовывал только расшифрование дескриптора.

arti-client, arti Высокоуровневая библиотека для встраивания в другие приложения и командная программа, предоставляющая прокси SOCKS.

30.2.1. Сравнение: запись ячейки

В Лабораторной работе 2 мы написали функцию кодирования ячейки. Соответствующая функция Arti, `write_cell` в `crates/tor-cell/src/chancell/codec.rs`, выполняет те же шаги: записывает CircID, затем команду. Для ячейки переменной длины резервирует два байта под длину, кодирует тело, затем возвращается и записывает фактическую длину. Для ячейки фиксированной длины кодирует тело и проверяет, что оно не превышает 509 байт (`CELL_DATA_LEN`). Два отличия показательны:

- CircID *всегда* записывается четырьмя байтами. Arti не реализует версии протокола соединения с номером ниже 4, поэтому специальный случай двухбайтовых CircID ему не нужен (гл. 8). Только ячейка **`VERSIONS`**, предшествующая согласованию, обрабатывается отдельно.
- Нарушение ограничения длины приводит не к ошибке вызывающего кода, а к *внутренней* ошибке: слишком большая исходящая ячейка была бы дефектом самого Arti, а не ошибкой, вызванной данными, полученными от удалённого узла. Тип ошибки явно отличает баг реализации от недоверенного ввода.

Arti сознательно не поддерживает устаревшие протоколы. Его цель — совместимость с *рекомендуемыми сейчас* версиями, а не сохранение каждого исторического формата.

30.3. Поддерживаемые функции

Самое точное описание возможностей реализации Tor — поддерживаемые ею **подпротоколы** ([tor-spec § Subprotocol-based versioning]). Каждый ретранслятор и клиент объявляет поддерживаемые версии для каждой группы функций: `Link`, `Relay`, `FlowCtrl`, `HSIntro` и т.д. Документация совместимости Arti перечисляет, среди прочего, следующие клиентские возможности:

- `Link=4` и `Link=5`: четырёхбайтовые CircID и современное заполнение канала (гл. 7);

- `Relay=4` (ntor-v3, гл. 9), `Relay=5` (согласование подпротоколов) и `Relay=6` (шифрование CGO);
- `FlowCtrl=1` (аутентифицированные SENDME) и `FlowCtrl=2` (управление перегрузкой);
- `Desc=4`: семейства на основе ключей, описанные в разд. 22.5.

Внимание: Ошибка на единицу в таблице Arti

В указанном документе совместимости Arti эти два подпротокола управления потоком пронумерованы как `FlowCtrl=0` и `FlowCtrl=1`. Однако спецификация назначает аутентифицированным SENDME значение 1, а управление перегрузкой — 2 [tor-spec § Subprotocol-based versioning]; ретрансляторы в публичном консенсусе объявляют `FlowCtrl=1-2`. Это ошибка на единицу в документации, а не в реализации. Она также показывает полезную практику: когда источники расходятся, сверяйтесь со спецификацией и ищите независимое подтверждение в консенсусе.

На дату, принятую в главе за точку отсчёта в клиенте Arti отсутствуют две возможности C-клиента. `Conflux=1` обеспечивает многопутевую передачу: crate `tor-proto` поддерживает её, но Arti пока не строит цепи Conflux. `Padding=2` обозначает автоматы заполнения, предназначенные для маскировки построения цепей onion-сервисов (разд. 26.4). Хотя `tor-proto` содержит механизм заполнения на основе библиотеки *maybenot*, сами машины ещё не реализованы. Ни один подпротокол со стороны ретранслятора пока не помечен поддерживаемым: значительная часть кода для ретрансляторов уже существует, но функции будут объявлены поддерживаемыми только после появления работающего ретранслятора, который способен их предоставлять.

30.4. Лабораторная работа: Arti в приватной сети

Лабораторная работа 20: Второй клиент — на Rust

Цель. Скомпилировать Arti, настроить его как клиент приватной сети книги и использовать его прокси SOCKS вместо `test008c`.
Предварительные условия. Лабораторная сеть должна быть запущена; требуется инструментарий Rust (`cargo`).
Оценочное время. 1–2 часа, большая часть времени уходит на компиляцию.

Шаг 1: компиляция. Из корня репозитория книги получите эталонную версию Arti 2.6.0 в `labs/ru/vendor/arti` и соберите исполняемый файл `arti`:

```
$ mkdir -p labs/ru/vendor
$ git clone https://gitlab.torproject.org/tpo/core/arti.git labs/ru/vendor/arti
$ git -C labs/ru/vendor/arti checkout arti-v2.6.0
$ (cd labs/ru/vendor/arti && cargo build --release -p arti --locked)
$ labs/ru/vendor/arti/target/release/arti --version
Arti 2.6.0
```

Первая сборка занимает несколько минут, поскольку компилирует всё дерево крейтов и их зависимостей.

Шаг 2: конфигурация. Arti не читает `torrc`; он использует TOML. Для доступа к непубличной Tor-сети Arti должен знать центры каталогов и резервные каталожные кеши, из которых получит начальные документы каталога, подобно строкам `DirAuthority` в `torrc`,

созданных Chutney (разд. 19.6). Скрипт `labs/ru/лаборатория/arti_config.py` читает отпечаток RSA, ключ Ed25519, идентификатор сертификата v3 и порты каждой авторитетной ноды из локальных файлов. Он записывает значения в `labs/ru/лаборатория/net/arti/arti.toml`. Следующий сокращённый фрагмент демонстрационный, а не готовый TOML: идентификаторы генерируются локально, а многоточия обозначают пропущенные строки:

```
[tor_network]
fallback_caches = [
  { rsa_identity = "7B465200C885ED566C6CD4F46DBB0492076D203A",
    ed_identity = "JwhsA9VS01mRVnQseL9xsKZfstoBae7OcgL3jBvDzis",
    orports = ["127.0.0.1:5100"] },
  ...
]
[tor_network.authorities]
v3idents = ["390998046DEFF1E68AC346D7577319791B32C91A", ...]
```

Ещё три изменения конфигурации адаптируют Arti к сети, где всё работает на `127.0.0.1`. `allow_local_addrs` разрешает соединения с локальными адресами, которые клиенты публичной сети обычно отклоняют. Две невозможные длины сетевых префиксов — 33 бита для IPv4 и 129 для IPv6 — отключают ограничение совместного сетевого префикса, иначе блокирующий здесь каждую цепь. Наконец, `hsdir_interval` сокращает период onion-сервиса так же, как `TestingTorNetwork` в tor. Эти параметры повторяют конфигурации Arti, генерируемые самим Chutney.

Шаг 3: запуск. После создания конфигурации командой `python3 labs/ru/лаборатория/arti_config.py` из корня книги выполните `labs/ru/vendor/arti/target/release/arti proxy -p 9150 -c labs/ru/лаборатория/net/arti/arti.toml`. Время начальной загрузки зависит от машины; в одном из наших запусков:

```
INFO tor_dirmgr::bootstrap: 1: Looking for a consensus.
INFO tor_dirmgr::bootstrap: 1: Downloading certificates for consensus
     (we are missing 3/3).
INFO tor_dirmgr::bootstrap: 1: Downloading microdescriptors (we are missing 8).
INFO tor_dirmgr: Marked consensus usable.
WARN tor_netdir: Requested 60 relays, but only 8 were usable, ...  role=Guard
INFO arti::subcommands::proxy: Sufficiently bootstrapped; proxy now functional.
INFO tor_guardmgr::guard: We have found that guard [scrubbed] is usable.
```

Три детали показательны. Процедура начальной загрузки следует разд. 12.7 буквально по шагам: сначала консенсус, затем сертификаты трёх центров каталогов и наконец восемь микродескрипторов. Предупреждение `tor_netdir` отражает размер нашей крошечной сети: менеджер сторожевых узлов предпочёл бы выборку из 60 кандидатов, но доступны только восемь. Идентификатор сторожевого узла в этом примере заменён на `[scrubbed]`: Arti помечает чувствительные значения с помощью крейта `safelog` и скрывает их в логах, аналогично `SafeLogging` в tor. В других журналах обновлений может появляться `Applying a consensus diff` (в приведённом отрывке этой строки нет). При последующих обновлениях Arti может скачать только изменения, если доступна разница, согласованная с помощью `DirCache=2`; такой ответ не гарантируется при каждом обновлении.

Шаг 4: использование. Arti предоставляет прокси SOCKS на порту 9150. `curl` использует его так же, как раньше `test008c`:

```
$ curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n' \
       --noproxy "" --socks5-hostname 127.0.0.1:9150 http://127.0.0.1:8080/
200 in 0.047224s
```

Пустой аргумент `--noproxy` не позволяет переменной `NO_PROXY` обходить SOCKS для локального адреса. В предполагаемом тесте цепь проходит через ретрансляторы на C в приватной сети: клиент Rust и ретранслятор на C говорят на одном и том же байтовом протоколе, изученном в Части II. Последнее наблюдение связано с Лабораторной работой 18: SOCKS-запросы, содержащие только IP-адрес (`--socks5`), также успешно выполняются, а стандартные журналы Arti не показывают предупреждение, аналогичное tor. Однако документация крейта `arti-client` предупреждает пользователей API, что локальное разрешение имён может раскрыть назначение DNS-серверу. Сам по себе SOCKS-запрос с IP-адресом не доказывает утечку DNS (разд. 28.3).

Лабораторная работа входит в `make verifica-lab`, если Arti собран; иначе проверка пропускается с соответствующим сообщением.

Итоги

- Arti переписывает tor на Rust с механизмами безопасности памяти. Он модульный, проектировался как библиотека, стабилен как клиент с 2022 года и является предпочтительным клиентом для разработки новых функций.
- Его архитектура следует уровням протокола: примитивы и ячейки, каналы и цепи, затем управление каталогом, маршрутами и guard.
- Версии подпротоколов описывают возможности Arti. Клиент реализует ntor-v3, CGO, управление перегрузкой и семейства узлов на основе ключей, тогда как Conflux и автоматы заполнения на эталонную дату главы ещё не завершены. Поддержка роли ретранслятора всё ещё разрабатывается.
- Arti подключается к приватной сети через настроенные центры каталогов и резервные кеши, подобно узлу Chutney.

Упражнения

Упражнение 30.1 ⋆ Для каждой главы Части II определите крейт Arti, реализующий основные рассматриваемые там понятия.

Упражнение 30.2 ⋆ Каковы преимущества и недостатки решения Arti не реализовывать устаревшие версии протокола? Кто может оказаться исключён?

Упражнение 30.3 ⋆⋆ Прочитайте `decode_cell` в `crates/tor-cell/src/chancell/codec.rs` и сравните с `decode` из Лабораторной работы 2. Как распознаётся ячейка переменной длины и что происходит, если входных данных ещё недостаточно для полной ячейки?

Упражнение 30.4 ⋆⋆ Повторите эксперимент по изоляции потоков из Лабораторной работы 11, используя Arti как прокси SOCKS. Не позволяют ли разные учётные данные SOCKS объединять потоки в одну цепь в Arti? Отличайте изоляцию от ошибочного предположения, что каждой паре данных навсегда соответствует ровно одна цепь; найдите параметры настройки.

Упражнение 30.5 ⋆⋆⋆ Напишите небольшую программу на Rust с `arti-client`, которая получает страницу с сервера лабораторной сети, загружая конфигурацию Лабораторной работы 20 из файла.

Дополнительная литература

- Сайт проекта `https://arti.torproject.org/` и репозиторий исходного кода, особенно README, *обзор архитектуры* (`doc/dev/Architecture.md`) и руководство совместимости (`doc/Compatibility.md`).
- Журнал изменений Arti, документирующий изменения каждого ежемесячного релиза.
- Конфигурация Arti, генерируемая Chutney (`lib/chutney/arti/config.py`), на которой основана Лабораторная работа 20.

Глава 31

Открытые проблемы и текущие проекты

Последняя проверка: 23 сентября 2026 г. — предложения torspec до номера 369; tor 0.4.9.13; Arti 2.6.0

Цели главы

К концу этой главы вы сможете:

- прослеживать процесс изменения протокола Tor и интерпретировать статус proposal;
- объяснять, почему постквантовая криптография требует изменений форматов сообщений, а не только алгоритмов;
- описывать Conflux, Walking Onions и другие проекты, направленные на производительность и масштабируемость;
- определять конкретные способы участия в проекте.

Ничто из описанного в этой книге не является окончательным. Протокол 2026 года отличается от версии 2016 года, где не было ntor-v3, управления перегрузкой и CGO; к 2036 году он снова изменится. Эта заключительная глава смотрит вперёд. Для каждой открытой проблемы мы спрашиваем: что пока работает недостаточно хорошо, что предложено и насколько далеко продвинулась реализация. Всё в следующих разделах по необходимости имеет предварительный характер: статусы предложений соответствуют репозиторию спецификаций на эталонную дату главы.

31.1. Как меняется протокол

С 2007 года изменения протокола разрабатываются через нумерованные **предложения**, в некотором смысле похожие на RFC IETF. Участник обсуждает идею в рассылке разработчиков, оформляет её в требуемом формате и, когда возникает широкое согласие, что идею стоит рассматривать, редакторы добавляют документ в репозиторий под номером предложения. Далее предложение обсуждается и уточняется, пока не становится достаточно подробным для реализации. После реализации его нормативное содержание переносится в спецификации. В отличие от RFC, предложение может изменяться, сохраняя номер, и никогда не является окончательным документом для уже реализованной функции: эту роль выполняют сами спецификации (Предложение 001).

Статус предложения (рис. 31.1) — первое, что следует проверить читателю. В книге встречались предложения в разных состояниях. ntor-v3 (Предложение 332) имеет статус *Closed*: реализован и включён в спецификации. Управление перегрузкой (Предложение 324) и Conflux



Рис. 31.1: Статусы предложений Tor согласно Proposal 001 (упрощённая и неполная схема). Путь *Draft–Open–Accepted–Finished–Closed* лишь пример: *Finished* означает реализацию, а *Closed* также включение в спецификации. *Needs-Revision* и *Needs-Research* обозначают разные препятствия; *Dead* может вернуться в обсуждение, тогда как *Rejected* означает отказ от предложенного решения. *Informational* — итоговый информационный документ, не нормативная спецификация. Другие статусы, включая *Meta*, *Reserve* и *Obsolete*, не показаны.

(Предложение 329) — *Finished*: реализованы, но ещё не полностью перенесены в спецификации. CGO (Предложение 359) остаётся *Open*, хотя уже присутствует и в tor, и в Arti. План удаления TAP (Предложение 350) имеет статус *Accepted*, а Walking Onions (Предложение 300) — *Informational*.

31.2. Квантовые компьютеры

Проблема. Все механизмы открытого ключа Tor, включая X25519 для согласования ключей, Ed25519 для подписей и оставшиеся применения RSA, будут нарушены достаточно мощным квантовым компьютером (гл. 5). Никто не знает, появится ли он и когда. Однако для системы анонимности угроза может начаться раньше: атакующий способен записывать зашифрованный трафик сегодня и расшифровать его в будущем.

Два сценария. Предложение 355 от 2025 года различает две ситуации. В *переходной* среде достаточно защиты от *будущего* квантового компьютера: записанный сегодня трафик должен выдержать последующее расшифрование, тогда как традиционные подписи остаются приемлемыми, поскольку активная подделка подписи потребовала бы квантового компьютера уже сегодня. В системе *нового поколения* защита должна учитывать и квантовый компьютер, который *уже существует* и способен подделывать подписи или активно атаковать согласование ключей.

Цена. В 2024 году NIST стандартизировал ML-KEM, постквантовый механизм установления ключей, а также алгоритмы подписи ML-DSA и SLH-DSA [NIS24]. Proposal сравнивает их размеры с современными алгоритмами:

Алгоритм	Открытый ключ	Шифртекст или подпись
X25519	32 байта	32 байта
ML-KEM-768	1 184 байта	1 088 байт
Ed25519	32 байта	64 байта
ML-DSA-65	1 952 байта	3 309 байт

В гл. 8 мы видели, что relay message может содержать не более 498 байт. Один открытый ключ ML-KEM-768 потребовал бы трёх ячеек ещё до учёта остальных данных handshake. Поэтому постквантовая криптография — не просто замена алгоритма: она меняет размещение сообщений внутри ячеек. В нынешнем формате relay нельзя непосредственно передать сообщение такого размера: три ячейки — лишь нижняя граница по объёму, если новый формат допускает фрагментацию.

Текущее направление. Выводы предложения осторожны. В переходный период может быть достаточно добавить постквантовый механизм к ntor, как предлагали более ранние proposal. Для системы нового поколения предпочтительная конструкция использует два постквантовых механизма установления ключей, чтобы дополнительно обеспечить прямую секретность, если стоимость окажется приемлемой. Добавление постквантового ключа в микродескрипторы также частично съест экономию пропускной способности, достигнутую удалением TAP-ключей (Предложение 350) и внедрением семейств, основанных на ключах (разд. 22.5). Решения зависят от ещё необходимых измерений: как часто цепи строятся с нуля и сколько пропускной способности и процессорного времени ретрансляторов уходит на согласование ключей? Это остаётся открытой исследовательской задачей и возможностью внести вклад измерениями (гл. 29).

31.3. Более крупные сообщения — и более мелкие

Предложение 340, имеющий статус *Open* на эталонную дату, напрямую решает ограничение «одно ретрансляционное сообщение на ячейку». Оно отделяет ретрансляционные сообщения от ячеек: длинные сообщения смогут занимать несколько ячеек, а несколько коротких — совместно использовать одну. Мотивация включает и сообщения длиннее 498 байт, необходимые для постквантового согласования ключей, передачи UDP и Walking Onions, и очень короткие сообщения вроде `SENDME`, `XON` и `XOFF`. Сейчас каждое из них занимает целую ячейку; новый формат позволит скрывать их в пространстве заполнения других сообщений, делая невидимыми для наблюдателя. Формат проектируется для работы с обновлённым шифрованием ретрансляционных сообщений, чья схема CGO рассмотрена в разд. 10.4; координация изменений позволит избежать множества несовместимых форматов.

31.4. Производительность: несколько путей

Управление перегрузкой (разд. 11.3) сделал отдельные цепи эффективнее. Следующий шаг — **Conflux** (Предложение 329, *Finished*). Клиент строит две цепи к одному выходному узлу с разными сторожевыми и промежуточными ретрансляторами, связывает их и распределяет трафик с учётом измеренного времени прохождения сигнала туда и обратно по каждой цепи. Идея возникла в исследованиях AlSabah, Bauer, Elahi и Goldberg; версия Tor адаптирует её, используя данные управления перегрузкой и опыт многопутевого TCP. Мы наблюдали цепи Conflux в лабораторной сети (разд. 19.7.3) и через Stem (гл. 18). Предложение рекомендует, чтобы две ветви не имели общих ретрансляторов, кроме выходного узла, чтобы избегать общих узких мест. Это рекомендация, а не жёсткое правило, и на маленькой лабораторной сети она часто нарушается. tor использует Conflux с версии 0.4.8; Arti, как отмечено в разд. 30.3, на эталонную дату ещё не строит такие цепи. Точки OnionPerf, работающие с Conflux и без него, позволяют сравнивать эффект в реальной сети (разд. 24.3).

31.5. Масштабируемость: Walking Onions

Сейчас каждый клиент Tor знает *каждый* relay: он загружает консенсус и microdescriptors, поэтому трафик каталогов растёт вместе с числом ретрансляторов. Микродескрипторы и разностные документы консенсуса уменьшают стоимость, но не меняют фундаментальную форму. Начальная стоимость также сосредоточена на начальной загрузке: новый клиент должен скачать полный каталог до построения первой цепи (Предложение 300).

У очевидных решений есть известные недостатки. Если показывать каждому клиенту лишь часть сети, возникают *эпистемические* атаки: различия между представлениями сети могут идентифицировать клиентов. Если позволить каждому ретранслятору выбирать следующий, как в некоторых децентрализованных системах, вредоносные ретрансляторы получают возможность перехватывать маршруты. Walking Onions предлагает иной подход: центры каталогов публикуют подписанную структуру, связывающую каждый ретранслятор с числовым интервалом. Клиент случайно выбирает число, а предыдущий ретранслятор цепи предоставляет подписанное доказательство, указывающее ретранслятор, занимающий этот интервал, вместе с запросом расширения цепи. Клиент проверяет подпись, не получая полного списка. Исходное предложение имеет статус *Informational*; более подробное продолжение (Предложение 323) пытается закрыть оставшиеся пробелы. Более новые предложения, включая постквантовые, уже учитывают эту архитектуру.

31.6. Другие открытые направления

UDP Tor передаёт только TCP. Поэтому видеозвонки, игры и QUIC, транспорт под HTTP/3, не поддерживаются или требуют потенциально небезопасных обходных решений. Предложение 348 (*Open*) рассматривает варианты от туннелей к выходным узлам до непосредственной передачи датаграмм, одновременно учитывая новые риски: отпечатки приложений, сбор метаданных однорангового обмена и исчерпание портов.

Распознавание сайтов по трафику В конце разд. 26.4 осталась открытая проблема: универсальная защита от распознавания сайтов по трафику с приемлемыми накладными расходами пока не найдена. Автоматы заполнения, уже используемые tor для цепей onion-сервисов и разрабатываемые в Arti на основе библиотеки *maybenot*, создают основу для будущих средств защиты.

Корреляция Фундаментальная проблема гл. 26 остаётся нерешённой: сеть с низкой задержкой может сохранять коррелируемые временные и объёмные закономерности, несмотря на буферизацию и параллельный трафик. Известные защиты вроде заполнения с постоянной скоростью и дополнительных задержек дороги; исследования продолжаются в поиске более эффективных компромиссов и выбора маршрута с учётом интернет-топологии.

DoS Доказательство выполнения работы защищает onion-сервисы (гл. 15); автоматическая настройка механизма является темой Предложения 362.

Arti как ретранслятор Когда Arti сможет полноценно работать ретранслятором и центром каталогов, C-кодовая база со временем может быть выведена из эксплуатации. Это одна из долгосрочных задач Tor Project (гл. 30).

31.7. Как внести вклад

У читателей, дошедших до этого места, есть несколько способов помочь, и не все требуют программирования:

- **Предоставить пропускную способность**: запустить промежуточный ретранслятор или мост либо, соблюдая меры из гл. 25, выходной ретранслятор (гл. 22); альтернативно можно запустить прокси Snowflake даже из браузера.
- **Читать и обсуждать предложения**: многие остаются *Open*, потому что им нужны комментарии, анализ и измерения. Рассылка разработчиков публична.
- **Писать код**: tor и Arti разрабатываются в GitLab Tor Project; есть задачи с метками для новичков. Arti особенно важен для новых функций.
- **Проводить исследования**: использовать Shadow, Chutney и данные Metrics с соблюдением рекомендаций Research Safety Board (гл. 29).
- **Переводить и документировать**: добровольцы переводят Tor Browser, его руководство и сайт. Документация всегда полезна на разных языках, включая итальянский.
- **Использовать Tor**: большее сообщество может расширить множество анонимности, но одно лишь число пользователей не гарантирует неразличимости, если отличаются время, поведение и метаданные.

Итоги

- Протокол развивается через нумерованные предложения, статус которых движется от *Draft* к *Closed* по мере реализации изменений и включения их в спецификации.
- Постквантовая криптография важна ещё до появления мощных квантовых компьютеров; она требует сообщений больше одной ячейки и мотивирует предложение 340 о фрагментации и объединении сообщений.
- Conflux распределяет трафик по двум цепям; Walking Onions потенциально устраняет необходимость каждому клиенту знать всю сеть.
- UDP, защита от распознавания сайтов по трафику и корреляции, настройка DoS-защиты и переход Arti к роли ретранслятора остаются открытыми задачами.

Упражнения

Упражнение 31.1 ⋆ Выберите три proposal, упомянутые в этой книге. Для каждого определите текущий статус и его значение: развёрнута ли функция и описана ли она в спецификациях?

Упражнение 31.2 ⋆ Сколько ячеек требуется для передачи одного открытого ключа ML-KEM-768 и одного шифртекста ML-KEM-768, если одно ретрансляционное сообщение содержит максимум 498 байт, а отдельные сообщения не могут совместно использовать одну ячейку? Что изменится при использовании Предложения 340? Различите теоретическую нижнюю границу числа ячеек (без служебных данных) и фактическое число, зависящее от упаковки и фрагментации по предложению.

Упражнение 31.3 ⋆⋆ Проиллюстрируйте epistemic-атаку: если разные клиенты знают разные подмножества сети, как атакующий может использовать различия?

Упражнение 31.4 ⋆⋆ Используя Stem в лабораторной сети, найдите пары Conflux circuits с общим `CONFLUX_ID`. Проверьте, заканчиваются ли обе ветви на одном exit и избегают ли

остальных общих relay, как рекомендует Предложение 329. Что вы наблюдаете и как это объяснить?

Упражнение 31.5 ⋆⋆⋆ Выберите открытую проблему из этой главы и напишите короткий proposal в формате Tor Project: мотивация, дизайн, анализ безопасности, совместимость и план реализации.

Дополнительная литература

- Индекс proposal в репозитории `torspec`, организованный по статусам, и документ процесса proposal (Предложение 001).
- N. Mathewson, *Options for postquantum circuit extension handshakes* (Предложение 355).
- *Walking Onions: Scaling and Saving Bandwidth* (Предложение 300) и продолжение (Предложение 323).
- Блог Tor Project, где объявляются и объясняются изменения протокола.

Приложение A

Глоссарий терминов Tor

Этот глоссарий определяет технические термины, используемые во всём русском издании. Для каждой записи также указана глава, в которой соответствующее понятие вводится.

Термин	Значение	Гл.
анализ трафика (traffic analysis)	Получение информации из временных характеристик, объёма и адресации коммуникаций без чтения их содержимого.	1
анонимность (anonymity)	Состояние, при котором субъект остаётся неидентифицируемым внутри множества субъектов, называемого множеством анонимности.	1
Sybil-атака	Представление сети множества идентичностей, например многочисленных ретрансляторов, чтобы увеличить вероятность выбора атакующего.	26
центр каталогов (directory authority)	Один из небольшого числа серверов, голосующих за консенсус и подписывающих его.	12
мост (bridge)	Ретранслятор, отсутствующий в публичном консенсусе и используемый как точка входа пользователями, сталкивающимися с цензурой.	16
канал (channel)	TLS-соединение между двумя соседними узлами Tor, по которому передаются ячейки нескольких цепей.	7
ячейка (cell)	Единица передачи в канале: обычно 514 байт, а для некоторых команд — переменной длины.	8
ослеплённый ключ (blinded key)	Ключ, выводимый из идентификационного ключа onion-сервиса для заданного периода и скрывающий адрес сервиса от HSDir.	14
ключ семейства (family key)	Ключ, используемый ретрансляторами одного оператора для подтверждения принадлежности к одному семейству.	22
Circuit ID	Идентификатор цепи, локальный для одного канала; меняется на каждом переходе.	8
цепь (circuit)	Маршрут через несколько ретрансляторов с послойным шифрованием, строящийся по одному переходу за раз.	9
консенсус (consensus)	Подписанный центрами каталогов документ со списком ретрансляторов, их флагами и весами.	12
управление перегрузкой (congestion control)	Регулирование объёма данных цепи в полёте в зависимости от измеренного времени прохождения туда и обратно.	11

Термин	Значение	Гл.
управление потоком (flow control)	Предотвращение перегрузки получателя отправителем с помощью окон и сообщений SENDME.	11
корреляция трафика (traffic correlation)	Распознавание того, что два наблюдаемых потока в разных точках сети принадлежат одной коммуникации.	26
дескриптор (descriptor)	Документ, посредством которого ретранслятор или onion-сервис публикует ключи, адреса и свойства.	12
выходной ретранслятор	Последний ретранслятор цепи, открывающий соединение к назначению.	6
семейство ретрансляторов (family)	Группа ретрансляторов под общим управлением, которые не должны одновременно присутствовать в одной цепи.	13
флаг (flag)	Свойство, присваиваемое ретранслятору directory authorities, например `Guard`, `Exit` или `Stable`.	12
прямая секретность (forward secrecy)	Свойство, при котором компрометация долгосрочного ключа не позволяет расшифровать ранее записанный трафик.	5
guard, entry guard	Первый ретранслятор клиентских цепей, выбираемый на длительный срок и многократно используемый.	13
handshake	Обмен сообщениями, в ходе которого две стороны согласуют ключи, например ntor или ntor-v3.	9
hidden service directory (HSDir)	Ретранслятор, хранящий и предоставляющий дескрипторы onion-сервисов.	14
промежуточный ретранслятор (middle relay)	Ретранслятор в середине цепи, который не знает одновременно исходного клиента и назначения.	6
изоляция потоков (stream isolation)	Назначение разных потоков разным цепям, например на основе учётных данных SOCKS.	18
протокол связи (link protocol)	Версионированный протокол канала, согласуемый ячейкой **`VERSIONS`**.	7
ретрансляционное сообщение (relay message)	Содержимое ячейки **`RELAY`**, адресованное определённому переходу, например `BEGIN` или `DATA`.	8
microdescriptor	Компактный дескриптор, содержащий только сведения о ретрансляторе, необходимые клиенту.	12
модель угроз (threat model)	Явное описание противников и угроз, которым система должна противостоять.	4
несвязываемость (unlinkability)	Невозможность для наблюдателя определить, принадлежат ли два действия одному субъекту.	1
onion-сервис (ранее hidden service)	Сервис, доступный через Tor, чей адрес кодирует его открытый ключ; ранее назывался hidden service.	14
padding	Заполняющие данные в ячейках или канале, используемые для маскировки размеров и временных характеристик.	7
веса пропускной способности (bandwidth weights)	Коэффициенты консенсуса, балансирующие выбор ретрансляторов для разных позиций цепи.	13
подключаемый транспорт (pluggable transport, PT)	Отдельная программа, преобразующая Tor-трафик для противодействия распознаванию или блокировке, например obfs4, Snowflake или WebTunnel.	16
политика выхода (exit policy)	Упорядоченный набор правил, описывающий назначения, которые принимает выходной ретранслятор.	22

Термин	Значение	Гл.
доказательство выполнения работы (PoW)	Вычислительная задача, которую клиент решает, чтобы получить приоритет при доступе к onion-сервису во время атаки.	15
предложение (proposal)	Нумерованный документ, используемый для обсуждения и проектирования изменений протокола Tor.	31
псевдоним (pseudonym)	Идентификатор, отличный от реального имени человека и позволяющий связывать действия без раскрытия личности.	1
точка представления (introduction point)	Ретранслятор, выбранный onion-сервисом для получения начальных запросов на установление контакта.	14
точка встречи (rendezvous point)	Ретранслятор, выбранный клиентом, в котором соединяются цепи клиента и onion-сервиса.	14
ретранслятор (relay, onion router)	Узел сети Tor, пересылающий ячейки.	6
SENDME	Сообщение, подтверждающее получение данных цепи и повторно открывающее окно управления потоком.	11
подпротокол (subprotocol)	Версионированное семейство функций протокола, объявляемое ретрансляторами и клиентами, например `Link` или `Relay`.	30
поток (stream)	Поток прикладных данных, например TCP-соединение, передаваемый внутри цепи.	11
vanguard (защитный промежуточный ретранслятор)	Ретранслятор промежуточного уровня, фиксируемый на определённый период для дополнительной защиты сторожевого узла onion-сервиса.	13
распознавание сайтов по трафику (website fingerprinting)	Распознавание посещённой страницы только по размерам и временным характеристикам зашифрованного трафика.	26

Приложение B

Дополнительная криптография

Последняя проверка: 23 сентября 2026 г. — dir-spec и tor-spec (коммит torspec `8e1f245`); библиотека `cryptography`; лабораторная сеть

Глава 5 вводила криптографические основы Tor с минимальной математической подготовкой. В этом приложении подробнее рассматриваются три ранее лишь кратко упомянутые темы, каждая с воспроизводимыми примерами: ключи X25519 и принудительная фиксация отдельных битов, преобразование между формами Монтгомери и Эдвардса одной кривой с проверкой по реальному сертификату, а также изменяемость потокового шифртекста, лежащая в основе атаки маркировки («marking attack») из разд. 10.3.

B.1. Одна кривая, две формы

X25519 и Ed25519 используют *одну и ту же* эллиптическую кривую над полем целых по модулю $p = 2^{255} - 19$, записанную в двух формах:

Форма Монтгомери $v^2 = u^3 + 486662\,u^2 + u$. В X25519 открытый ключ содержит лишь координату u точки, записанную в 32 байтах с младшим байтом первым (little-endian). Эта форма допускает простой и быстрый алгоритм умножения точки на скаляр — «лестницу Монтгомери», время выполнения которого не зависит от секретного ключа.

Скрученная форма Эдвардса $-x^2 + y^2 = 1 + d\,x^2y^2$, где $d = -121665/121666$. В Ed25519 эта форма используется для цифровых подписей: открытый ключ кодирует координату y, а знак координаты x хранится в старшем бите последнего байта.

Две формы связаны явным преобразованием; оно используется в B.2.

B.1.1. Фиксация битов ключей X25519

Закрытый ключ X25519 задаётся 32 байтами, но перед использованием реализация фиксирует отдельные биты: обнуляет три младших бита, обнуляет старший бит и устанавливает предпоследний в единицу. Благодаря обнулению трёх младших битов скаляр становится кратным 8 — *кофактору* кривой, — поэтому компоненты малого порядка принятой точки не раскрывают сведения о скаляре. Фиксация старших битов также задаёт постоянное число шагов лестницы Монтгомери. Это можно проверить библиотекой, используемой в лабораторных работах:

```
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives import serialization as s
import os
def public_key(k):
```

```
    return X25519PrivateKey.from_private_bytes(k).public_key().public_bytes(
        s.Encoding.Raw, s.PublicFormat.Raw)
a = bytearray(os.urandom(32)); b = bytearray(a)
b[0] ^= 0x07                    # меняются только три младших бита
print(public_key(bytes(a)) == public_key(bytes(b)))      # True
```

Две последовательности байтов закрытого ключа, различающиеся только битами, которые перезаписываются при фиксации, дают один и тот же открытый ключ.

В.1.2. Точка на бесконечности

Во время рукопожатия ntor и клиент, и ретранслятор проверяют, что ни одна операция EXP() не возвращает точку на бесконечности. Спецификация отмечает, что для Curve25519 эта проверка достаточно заменяет валидацию принадлежности Y группе [tor-spec § The "ntor" handshake]. Без неё атакующий мог бы передать открытый ключ малого порядка и получить предсказуемый общий секрет, например строку из нулей. Библиотека `cryptography` выполняет эту проверку самостоятельно: вызов `exchange` с полностью нулевым 32-байтовым открытым ключом возбуждает исключение (`Error computing shared key`), а не возвращает нулевой секрет.

В.2. Из формы Монтгомери в форму Эдвардса: перекрёстный сертификат ntor

Каждый ретранслятор публикует в дескрипторе открытый ключ ntor (X25519) и доказывает владение соответствующим закрытым ключом с помощью поля `ntor-onion-key-crosscert`. Оно содержит сертификат Ed25519 типа `0a`, удостоверяющий основную идентичность Ed25519 ретранслятора, но *подписанный ключом ntor* [dir-spec § ntor-onion-key-crosscert]. Для этой подписи ключ ntor необходимо преобразовать из формы Монтгомери в форму Эдвардса. Спецификация задаёт преобразование [dir-spec § Converting a curve25519 public key to an ed25519 public key]:

$$y = \frac{u - 1}{u + 1} \bmod p.$$

Координата u определяет y, но не знак x: одной координате Монтгомери соответствуют два симметричных открытых ключа Эдвардса. Восстановить знак по открытому ключу Монтгомери невозможно, поэтому дескриптор передаёт его отдельно в первом аргументе поля перекрёстного сертификата: 0 означает положительный, 1 — отрицательный знак. Ретранслятор вычисляет его из закрытого ключа.

Модуль `labs/ru/код/torlab/conversion.py` реализует преобразование в нескольких строках:

```
P = 2**255 - 19
def montgomery_to_edwards(u_bytes, sign):
    u = int.from_bytes(u_bytes, "little") & ((1 << 255) - 1)
    y = (u - 1) * pow(u + 1, P - 2, P) % P      # деление с помощью обратного элемента
    по модулю p
    compressed = bytearray(y.to_bytes(32, "little"))
    compressed[31] |= (sign & 1) << 7          # знак координаты x в старшем бите
    return bytes(compressed)
```

Затем модуль проверяет перекрёстный сертификат с помощью декодера сертификатов из лабораторной работы 1. Применительно к дескрипторам лабораторной сети (ключ сокращён для удобства чтения; ключи меняются с каждой новой сетью):

```
$ PYTHONPATH=labs/ru/код python3 -m torlab.conversion labs/ru/лаборатория/net/nodes
    /000a/cached-descriptors test004r
test004r: ntor-onion-key DaxPE+Grnjex2Hmv+5qutl5dIX4La2jF6MIL7IK6+3E
  ключ Эдвардса (знак 0): c369d08f8083002a15fe22f65dca14bf...
  crosscert действителен; подтверждённая
    идентичность: kjPIgDB2OFsGJzuwsT+KMS/El9qrlE2/4P5WvIvmzC8
```

В приведённом примере подпись действительна, а подтверждённая идентичность совпадает с записанной в файле `fingerprint-ed25519` ретранслятора. Это же наблюдалось у остальных ретрансляторов лабораторной сети. При неверном знаке проверка завершается неудачей — такой случай предусмотрен отрицательным автоматизированным тестом модуля. Деление по модулю p вычисляется с помощью малой теоремы Ферма: умножением на $(u+1)^{p-2}$.

B.3. Изменяемость шифртекста потокового шифра

AES в режиме счётчика (разд. 5.2) создаёт поток байтов, который объединяется с сообщением операцией XOR. Изменив бит шифртекста, атакующий изменяет *соответствующий бит открытого текста*, даже не зная ключа: достаточно знать или угадать исходное содержимое. Пример на Python:

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
k, n = os.urandom(16), bytes(16)
cipher = lambda d: Cipher(algorithms.AES(k), modes.CTR(n)).encryptor().update(d)
c = bytearray(cipher(b"BEGIN www.example.org:443"))
delta = bytes(x ^ y for x, y in zip(b"www.example.org", b"www.example.net"))
for i, d in enumerate(delta):
    c[6 + i] ^= d                 # целевое изменение без знания ключа
print(cipher(bytes(c)))            # b'BEGIN www.example.net:443'
```

Исходное послойное шифрование ретрансляционных ячеек Tor (tor1) использует AES-CTR; целостность проверяется на конечном узле цепи по четырёхбайтовому дайджесту. Поэтому вредоносный ретранслятор способен предсказуемо изменить ячейку, а сообщник дальше по маршруту — распознать изменение. Это атака маркировки из разд. 10.3 и одна из причин создания CGO (разд. 10.4), где изменённая ячейка и последующие становятся невосстановимыми.

Упражнения

Упражнение B.1 ⋆ Сколько различных значений может принимать закрытый скаляр X25519 после фиксации битов? Сравните ответ с 2^{256}.

Упражнение B.2 ⋆⋆ Выведите и реализуйте обратное преобразование от координаты y открытого ключа Ed25519 к координате Монтгомери u. Сохраните исходный бит знака x, а затем с помощью функции `montgomery_to_edwards` из `conversion.py` проверьте, что обе операции восстанавливают исходный открытый ключ Ed25519 (при $y \neq 1$).

Упражнение B.3 ⋆⋆⋆ В tor1 ячейка содержит четырёхбайтовый дайджест и двухбайтовое нулевое поле `recognized`. Какова вероятность, что случайно изменённая ячейка будет принята последним узлом как допустимая? Почему атака маркировки всё же возможна?

Приложение C

Руководство по спецификациям torspec

Последняя проверка: 23 сентября 2026 г. — коммит torspec `8e1f245`

Спецификации Tor Project являются основным источником почти для каждой главы этой книги. Они исчерпывающи, но создавались не как учебник. Более двадцати лет их поддерживали как отдельные текстовые файлы, и лишь сравнительно недавно объединили на одном сайте. Даже собственные соглашения документации признают, что работа над единообразием ещё продолжается. Это приложение служит картой для читателей, которые после книги хотят перейти к непосредственному изучению спецификаций.

C.1. Где их найти

Спецификации опубликованы по адресу `https://spec.torproject.org/` и поддерживаются вместе с предложениями по развитию протокола в GitLab-репозитории `torspec` Tor Project. В репозитории спецификации находятся в Markdown-файлах внутри `spec/`: небольшим темам соответствуют отдельные файлы, более крупным документам — каталоги. Файл `permalinks.md` перечисляет стабильные адреса документов, например `https://spec.torproject.org/tor-spec`; при цитировании следует использовать именно их. Во всей книге документы цитируются как [document § section], причём название раздела сохраняется таким, каким оно приведено на сайте, а предложения — как «Предложение NNN».

C.2. Соглашения

Вводный документ с соглашениями устанавливает правила, действующие для спецификаций, если явно не указано иное:

- `MUST`, `SHOULD`, `MAY` и их отрицательные формы имеют значения RFC 2119: соответственно обязательное требование, сильная рекомендация или разрешённая возможность.
- Длины измеряются в восьмибитных байтах, а двоичные префиксы вроде «kibibyte» означают кратные 1024.
- Многобайтовые целые записываются в big-endian, или «сетевом», порядке байтов: десятичное число 4660 в двух байтах записывается как `12 34`.

Два важных исключения уже встречались в книге: ключи и точки Curve25519 и Ed25519 используют little-endian (прил. B), а некоторые документы системы каталогов имеют текстовый формат с двоичными данными, закодированными в base64.

C.3. Карта документов

Документ	Содержимое	Главы
`tor-spec`	Основной протокол: каналы, ячейки, цепи, рукопожатия, послойное шифрование на ретрансляторах, потоки, управление потоком и перегрузкой	7–11
`cert-spec`	Сертификаты Ed25519 в Tor	7
`dir-spec`	Центры каталогов, голосования, консенсус, дескрипторы, микродескрипторы и документы extra-info	12, 24
`path-spec`	Выбор маршрута и ограничения	13
`guard-spec`	Алгоритм выбора входных узлов (guard)	13
`vanguards-spec`	Полноценные и облегчённые защитные промежуточные узлы (vanguards)	13
`rend-spec`	Onion-сервисы v3: ключи, дескрипторы, точки представления и встречи	14, 21
`hspow-spec`	Доказательство выполнения работы (PoW) для onion-сервисов	15
`dos-spec`	Защита от отказа в обслуживании (DoS)	15
`pt-spec`	Интерфейс между tor и подключаемыми транспортами	16
`padding-spec`	Заполнение каналов и цепей	7, 26
`control-spec`	Протокол ControlPort	18
`socks-extensions`	Специфичные для Tor расширения SOCKS	11, 28
`param-spec`	Сетевые параметры консенсуса и их значения по умолчанию	11, 15
`address-spec`	Специальные имена, например адреса `.onion`	14
`srv-spec`	Совместное случайное значение центров каталогов	14

C.4. Как читать раздел спецификации

Несколько рекомендаций, выработанных при написании книги:

1. **Начинайте с формата.** Разделы о бинарных сообщениях приводят имена и размеры полей в таблицах. Прежде чем читать правила, восстановите формат на бумаге, как мы делали в схемах форматов книги.

2. **Отличайте актуальный материал от устаревшего.** Многие документы сохраняют исторические функции, например TAP, версии протокола связи 1–3 и onion-сервисы v2. Устаревший материал часто сгруппирован в разделах или файлах вроде `obsolete-channels.md` либо `historical/`, но не всегда. При сомнении исследуйте код и проверьте, существует ли соответствующая ветвь реализации.

3. **Сравнивайте с реализациями.** Спецификации выражают намерение; кодовые базы tor и Arti показывают фактическое поведение. Во время исследований для книги несколько расхождений оказались особенно информативными: отсутствие заполнения канала в одноузловых цепях (гл. 7) и наблюдаемая на выходном ретрансляторе причина закрытия потока (разд. 22.4).

4. **Проверяйте статус предложений.** Предложение (proposal) может быть реализовано до включения в спецификацию либо всё ещё обсуждаться и не иметь реализации (разд. 31.1).

5. **Проверяйте экспериментально.** Приватная сеть вроде лаборатории книги позволяет проверить, ведёт ли себя поле так, как описано. На этом методе основаны все лабораторные работы.

C.5. Arti и спецификации

Код Arti часто ссылается на спецификации в комментариях и документации и отмечает неоднозначные требования или различия с tor. Таблица поддерживаемых подпротоколов (разд. 30.3) показывает, какие функции поддерживают обе реализации и, следовательно, какие части спецификаций описывают «живой» протокол.

Приложение D

Решения упражнений

Глава 1: анонимность, наблюдение и метаданные

Решение 1.1. При HTTPS интернет-провайдер видит IP-адрес веб-сайта и потенциально его доменное имя — из DNS-разрешения и часто из поля TLS SNI. Он также видит временные характеристики, длительность соединения и объём трафика. При использовании VPN провайдер видит только соединение с VPN-сервером; сам VPN-провайдер, однако, может видеть всё то, что раньше видел интернет-провайдер. При использовании Tor провайдер видит соединение с ретранслятором Tor или мостом, а также его временные характеристики и объём. Ни одна отдельная точка внутри сети Tor не видит одновременно пользователя и назначение.

Решение 1.2. Учётная запись под вымышленным именем — пример псевдонимности. Неподписанное сообщение на доске объявлений — пример анонимности, если ничто другое не идентифицирует автора. Письмо, подписанное инициалами, — слабая псевдонимность и нередко позволяет установить личность. Анонимное пожертвование наличными — пример анонимности.

Решение 1.3. Множество анонимности сокращается от пятидесяти пользователей Tor до тех, кто говорит по-итальянски и одновременно бодрствует в 3 часа ночи; потенциально это всего несколько человек. Tor скрывает IP-адрес, а не поведение. Язык, время, стиль письма и личные сведения способны идентифицировать пользователя даже тогда, когда сетевой трафик этого не позволяет.

Решение 1.4. VPN-провайдер видит и пользователя, и его назначения. Всё доверие сосредоточено у одного оператора, который может вести журналы, быть юридически обязанным передать данные или быть скомпрометирован. Tor распределяет доверие между тремя независимо управляемыми ретрансляторами, ни один из которых не видит оба конца. Для связывания пользователя и назначения пришлось бы скомпрометировать или одновременно наблюдать первый и последний переход.

Глава 2: истоки

Решение 2.1. При случайном выборе вероятность составляет 1 из 100. Если наблюдатель сам отправил 99 из 100 сообщений в партии, он способен распознать собственные исходящие сообщения и определить единственное оставшееся сообщение как сообщение жертвы. Анонимность исчезает. Это *атака* $n - 1$; реальные сети смешивания противодействуют ей более сложным формированием партий, фиктивными сообщениями и контролем поступающих сообщений.

Решение 2.2. Используйте *анонимный обратный адрес* в смысле Чаума. Пользователь подготавливает зашифрованный многослойный «обратную луковицу» для цепочки узлов смешивания. Внутренний слой содержит адрес пользователя и ключ для шифрования ответа. Отвечающий присоединяет сообщение к луковице; каждый узел смешивания снимает один слой, и ни один отдельный узел смешивания не узнаёт конечное назначение. Центральная таблица маршрутизации не требуется.

Решение 2.3. На каждом шаге запрос пересылается другому участнику группы с вероятностью $3/4$ и отправляется веб-сайту с вероятностью $1/4$. Число пройденных участников имеет геометрическое распределение со средним $1/(1/4) = 4$, если учитывать первый шаг пересылки. Участник не может знать, является ли переславший запрос узел его источником, поскольку промежуточная пересылка подчиняется тому же правилу.

Глава 3: от onion routing NRL к Tor Project

Решение 3.1. При трёх переходах вредоносный центральный узел сразу знал две организационные сети на концах, поскольку входной и выходной узлы находились внутри сетей этих организаций. При четырёх переходах тот, кто скомпрометировал узел рядом с одним концом, знал, какой дополнительный узел атаковать для восстановления цепи; при пяти переходах требовалась ещё одна компрометация. Tor использует три перехода, поскольку его ретрансляторы не находятся внутри сетей пользователей и потому что каждый дополнительный переход увеличивает задержку, не уменьшая существенно главную угрозу корреляции входом и выходом. Вместо этого против этой угрозы помогают входные узлы (guard).

Решение 3.2. Единая луковица, зашифрованный долгосрочными открытыми ключами узлов, не обеспечивает прямую секретность. Тот, кто позднее получит эти ключи, сможет расшифровать ранее записанные луковицы. При телескопическом построении на каждом переходе выполняется эфемерный обмен ключами, а временные секреты после использования удаляются. Клиент также строит цепь по одному переходу и может сразу определить не отвечающий ретранслятор и заменить его. При единой луковице отказ становится заметен только после попытки пройти весь маршрут.

Решение 3.3. Это упражнение с открытым аргументированным ответом. К проверяемым элементам относятся опубликованные исходный код и спецификации, воспроизводимые сборки, независимый анализ в научной литературе и сеть, управляемая независимыми операторами. Доверия требуют, например, соответствие распространяемых сборок опубликованному исходному коду, хотя это частично проверяемо; отсутствие сговора центров каталогов; отсутствие неизвестных уязвимостей. Безопасность следует оценивать по проверяемости и распределённому доверию, а не выводить из источников финансирования проекта.

Глава 4: модель угроз

Решение 4.1. (a) Да: оператор сети видит только соединение с ретранслятором Tor. (b) Да: веб-сайт видит адрес выходного ретранслятора. (c) Нет в отношении идентичности: пользователь сам идентифицирует себя, входя в учётную запись. Tor скрывает только место, откуда выполняется соединение. (d) Вероятно, нет: противник, наблюдающий оба конца — пользователя и газету, — может попытаться выполнить временную корреляцию, то есть именно тот сценарий, который исключён из модели угроз Tor.

Решение 4.2. Сложная система привлекает лишь небольшое число сильно мотивированных пользователей, часто именно тех, кому защита нужна больше всего. Уже само её

использование выделяет их и оставляет небольшое множество анонимности. Простая в использовании система, которой миллионы людей пользуются по обычным причинам, помогает скрываться тем, чьи потребности более чувствительны.

Решение 4.3. Это упражнение с открытым ответом. Возможные противники включают государство или работодателя источника, наблюдающих локальную сеть, провайдера электронной почты или сообщений и организации, контролирующие сети в одной из двух стран. Tor не позволяет локальной сети узнать, с кем общается журналист, а сервису — увидеть IP-адрес журналиста. Он не защищает содержимое сообщений без сквозного шифрования, метаданные, видимые сервису, например учётные записи и временные метки, скомпрометированные устройства или корреляцию трафика противником, наблюдающим оба конца. Onion-сервис вроде SecureDrop уменьшает несколько из этих рисков.

Глава 5: необходимая криптография

Решение 5.1. $C_1 \oplus C_2 = P_1 \oplus P_2$, поскольку повторно использованный поток ключа сокращается. Зная P_1, можно восстановить $P_2 = C_1 \oplus C_2 \oplus P_1$.

Решение 5.2. 16-байтовый результат совпадает с первыми 16 байтами 32-байтового результата: SHAKE генерирует один поток, который можно считывать до любой требуемой длины. Поэтому KDF может запросить длинный поток и разделить его на части. Однако запрос различных длин для разных целей *не* обеспечивает разделения доменов; для этого всё равно нужны разные доменные метки.

Решение 5.3. Оба общих секрета совпадают. Полностью нулевой открытый ключ представляет точку малого порядка и дал бы полностью нулевой общий секрет независимо от закрытого ключа, то есть секрет, известный всем. Библиотека `cryptography` отказывается его возвращать. ntor явно требует эту проверку (разд. 9.6); без неё атакующий мог бы навязать предсказуемый общий секрет.

Решение 5.4. Без разделения доменов значение, вычисленное в одном контексте, может быть принято в другом. Например, MAC, созданный для аутентификации сообщения рукопожатия, можно попытаться представить как ключ или доказательство в другой фазе либо в другом протоколе, использующем ту же функцию и ключи. Доменные метки по конструкции делают входы разных назначений непересекающимися.

Глава 6: архитектура сети

Решение 6.1. Входной узел (guard): IP-адрес пользователя, временные характеристики и объём трафика, но не назначение. Промежуточный ретранслятор: ни пользователя, ни назначения. Выходной ретранслятор: назначение и любое содержимое, не зашифрованное приложением, но не пользователя. HSDir: никакой информации о пользователе, посещающем обычный веб-сайт. Центр каталогов: обычно никакой информации о конкретном посещении сайта. Она может видеть IP-адрес клиента, загружающего документы каталогов *напрямую*, особенно при первоначальном подключении. Если клиент загружает их через резервный сервер каталогов или кэш, запрос видит этот сервер, а не центр каталогов.

Решение 6.2. `Link=3-5` объявляет версии протокола связи вплоть до версии 5 (гл. 7). `Relay=2-6` включает ntor, ntor-v3 (`Relay=4`) и CGO (`Relay=6`). `FlowCtrl=1-2` означает аутентифицированные SENDME и управление перегрузкой. Ретранслятор может выступать выходным узлом с CGO при условии, что также имеет флаг `Exit` и согласует управление перегрузкой вместе с CGO (разд. 9.7).

Решение 6.3. 100 KB прикладных данных требуют примерно $102400/498 \approx 206$ ячеек DATA, то есть около $206 \times 514 \approx 106$ KB на канале. Нужно добавить несколько ячеек для запроса, открытия потока и сообщений SENDME — примерно семь, по одному на каждые 31 ячейку данных, — а также служебных данных записей TLS. В сумме это примерно на 5–7% больше полезной нагрузки, без учёта начальных рукопожатий.

Глава 7: каналы и протокол связи

Решение 7.1. Клиент всегда сам инициирует соединения: с входным узлом (guard) и ретрансляторами, предоставляющими сведения системы каталогов. Никому не требуется подключаться к клиенту, а прослушивание порта лишь облегчило бы его идентификацию. Ретранслятор инициирует каналы главным образом тогда, когда получает запрос `EXTEND2` и должен связаться со следующим переходом цепи, либо когда, как любой узел, создаёт собственные цепи для загрузки документов каталогов или проверки своей доступности.

Решение 7.2. `0x000772F7` $= 488183$ часа, что соответствует 9 сентября 2025 года, 23:00 UTC. 32-битный беззнаковый счётчик секунд переполнился бы в 2106 году. Подсчёт часов вместо секунд даёт запас в сотни тысяч лет, а точности до часа достаточно для сертификатов, действующих дни или недели. Одно примечание спецификации требует осторожности: `cert-spec` утверждает, что четырёх байт достаточно до 10136 года. Это ошибка, явно отмеченная в Предложении 338; она возникла из-за интерпретации счётчика как измеряющего *минуты*, поскольку 2^{32} минут — примерно 8 166 лет.

Решение 7.3. $38 = 32 + 2 + 2 \times 2$ описывает проверочный вызов с двумя методами. Если N_Methods равно 2, тело должно содержать $32 + 2 + 4 = 38$ байт. Тело ячейки длиной 36 байт объявляет больше методов, чем фактически содержит, и является некорректным, поэтому должно быть отклонено. Спецификация разрешает игнорировать *лишние* байты, но не отсутствующие.

Решение 7.4. Сравнение ключа, сертифицированного сертификатом типа 5, с отпечатком SHA-256 TLS-сертификата атакующего завершится неудачей: он отличается от сертификата настоящего ретранслятора. Если атакующий предъявит тот же подлинный TLS-сертификат, он не сможет завершить рукопожатие TLS без соответствующего закрытого ключа.

Решение 7.5. `695661c0` (1 января 2026 года, 12:00:00 UTC); `04 04 c6336407` (адрес клиента, 198.51.100.7); `01`; и `04 04 cb007105` (адрес ретранслятора, 203.0.113.5), затем заполнение нулевыми байтами до общей длины 509 байт.

Решение 7.6. Во-первых, входной узел видел бы один и тот же идентификатор, когда пользователь переключается между домашней, рабочей и мобильной сетями, связывая сеансы и IP-адреса во всех этих средах. Во-вторых, идентификатор также передавался бы ретрансляторам, с которых клиент загружает документы каталогов. Сотрудничающие ретрансляторы либо атакующий, наблюдающий их, могли бы связывать активность пользователя между разными ретрансляторами. В обоих случаях постоянный идентификатор разрушал бы предполагаемую Tor несвязываемость.

Решение 7.7. Общий план: разобрать сертификат типа 2 библиотекой X.509, например `cryptography.x509`. Извлечь его открытый RSA-ключ, проверить самоподпись и убедиться, что показатель степени равен 65537. Затем разобрать перекрёстный сертификат типа 7 согласно формату. Убедиться, что сертифицированный Ed25519-ключ совпадает с уже проверенной идентичностью, и проверить RSA-подпись с PKCS#1 v1.5 без OID алгоритма хэширования над SHA-256-хэшем предписанного префикса и полей. Отпечаток ретранслятора — дайджест SHA-1 DER-кодированного представления открытого ключа RSA в формате PKCS#1.

Решение 7.8. Нет. Трафик клиента `test008c` проходит через *собственный* канал к его входному узлу, а не через наш канал. Наш канал никогда не использовался для цепи, поэтому `channelpadding_decide_to_pad_channel` решает не добавлять заполнение. Чтобы наблюдать заполнение канала, нужно передавать многоузловую цепь через собственный канал, как в гл. 10.

Решение 7.9. TLSSECRETS выводится из секретов TLS-сессии между A и посредником, и подпись A покрывает это значение. B повторно вычисляет TLSSECRETS из секретов *своей* сессии с посредником. Эти секреты различаются, поэтому проверка не проходит даже тогда, когда посредник честно пересылает каждую ячейку и SLOG с CLOG совпадают. Без TLSSECRETS и SCERT посредник мог бы представиться B как A и переслать подпись A, заставив B ошибочно считать, что у него аутентифицированный канал с A.

Глава 8: ячейки

Решение 8.1. **`CREATE2`** — ячейка фиксированной длины: 512 байт при протоколе связи версии 3 с двухбайтовым CircID или 514 байт при версии 5 с четырёхбайтовым CircID. Ячейка **`VERSIONS`** всегда использует двухбайтовый CircID, поэтому её длина равна $2 + 1 + 2 + 4 = 9$ байт.

Решение 8.2. 0 (**`PADDING`**) — фиксированной длины; 7 (**`VERSIONS`**) — переменной; 9 (**`RELAY_EARLY`**) — фиксированной; 12 (**`PADDING_NEGOTIATE`**) — фиксированной; 129 (**`CERTS`**) — переменной. Команда 200 по правилу ≥ 128 имела бы переменную длину, но она не определена и поэтому недопустима. Получатель может отбросить её и закрыть канал.

Решение 8.3. Только `0x92345678`. CircID, равный нулю, зарезервирован для ячеек вне цепи. При протоколе связи версии ≥ 4 сторона, инициирующая канал, здесь клиент, должна выбирать CircID со старшим битом, равным 1; у `0x12345678` старший бит (MSB) равен 0.

Решение 8.4. Первая ячейка: CircID `00000000`, команда `0x81` = 129 (**`CERTS`**), следовательно, длина переменная; длина `0003`, тело `aa bb cc`. Вторая ячейка: CircID `0x80000007`, команда `0x04` (**`DESTROY`**), фиксированной длины, поэтому имеет тело 509 байт. Первый байт тела, `00`, означает причину NONE. Тело `aabbcc` само по себе не является корректным объектом CERTS: упражнение посвящено только формату обрамления ячеек.

Решение 8.5. `02 0000 0005 00000000 012c`: команда DATA (2), Recognized 0, StreamID 5, Digest временно равен нулю, длина $300 =$ `0x012c`. Оставшиеся $509 - 11 - 300 = 198$ байт — заполнение: первые четыре байта нулевые, остальные 194 случайные.

Решение 8.6. Для ячейки, всё ещё зашифрованной ещё одним слоем, Recognized равен нулю с вероятностью 2^{-16}: в среднем примерно $10^6/65536 \approx 15$ таких ячеек на миллион. Одного Recognized недостаточно, поскольку возможны ложные совпадения. Только сравнение с накопительным дайджестом подтверждает, что ячейка адресована ретранслятору: для узла, не являющегося адресатом, случайное совпадение дайджеста потребовало бы дополнительной вероятности 2^{-32}.

Решение 8.7. Предположим, атакующий контролирует входной и выходной ретрансляторы. Когда выходной ретранслятор хочет пометить цепь, он закрывает её редкой причиной, например `NOSUCHSERVICE`. Если промежуточный и входной ретрансляторы пересылали бы эту причину без изменений к клиенту, входной ретранслятор мог бы одновременно увидеть необычное значение в одной из своих цепей и связать известного ему клиента с назначением, известным выходному ретранслятору. Если все ретрансляторы передают и распространяют только NONE, байт причины не может служить таким сигналом. Корреляция по времени закрытия цепи всё ещё возможна, но этот канал намного более шумный.

Решение 8.8. Традиционный формат: $\lceil 1048576/498 \rceil = 2106$ ячеек, занимающих $2106 \times 514 = 1\,082\,484$ байт в канале, то есть избыточные данные составляют 3,2%. CGO: $\lceil 1048576/488 \rceil = 2149$ ячеек, занимающих $1\,104\,586$ байт, то есть избыточные данные составляют 5,3%. Сообщения SENDME и открытия потока не учитываются.

Решение 8.9. Общий план: использовать Hypothesis для генерации списков кортежей, содержащих CircID, команду и тело ячейки. Команды выбирать из `CellCommand`, а тела ячеек фиксированной длины ограничивать 509 байтами. Объединить результаты `encode`, разбить полученную последовательность на фрагменты случайных длин и постепенно добавлять их в буфер. Многократно вызывать `decode`, извлекая ячейки до тех пор, пока не останется полной ячейки. Для фиксированных ячеек сравнивать тела с исходными, дополненными нулями до 509 байт.

Решение 8.10. В Arti 2.6.0 кодек канала — `ChannelCodec` в `crates/tor-cell/src/chancell/codec.rs`, инициализируемый версией протокола связи через `ChannelCodec::new`. Его методы `write_cell` и `decode_cell` сериализуют и извлекают ячейки. Подобно нашему `decode`, `decode_cell` обрабатывает неполные ячейки без ошибки, возвращая `Ok(None)`, пока не поступит достаточно байт, и удаляя данные из буфера только для полной ячейки. Arti, однако, работает с изменяемым буфером `BytesMut` и использует обобщённый тип сообщения канала (`M: ChanMsg`), позволяя сузить набор принимаемых сообщений. Ещё одно заметное отличие: `decode_cell` читает команду по смещению 4 и предполагает фиксированные ячейки длиной 514 байт, то есть четырёхбайтовые CircID. Arti поддерживает только протоколы связи версий 4 и 5, как задаёт `LINK_PROTOCOLS` в `crates/tor-proto/src/channel/handshake.rs`; начальная **`VERSIONS`** с двухбайтовым CircID обрабатывается отдельно во время рукопожатия.

Глава 9: построение цепи

Решение 9.1. ntor требует $20 + 32 + 32 = 84$ байта. ntor-v3 с трёхбайтовым сообщением требует $32 + 32 + 32 + 3 + 32 = 131$ байт. HDATA может содержать максимум $509 - 4 = 505$ байт, поэтому CM может занимать не более $505 - 128 = 377$ байт.

Решение 9.2. Секрет для **`CREATE_FAST`** равен $X \| Y$ и передаётся открытым текстом внутри канала. Для второго перехода значения X и Y прошли бы через цепь через первый ретранслятор. Этот ретранслятор знает ключи собственного слоя и смог бы прочитать значения, вывести ключи второго перехода и расшифровать всё, чем клиент обменивался с ним, уничтожая разделение между переходами.

Решение 9.3. AES-CTR шифрует, выполняя XOR открытого текста с потоком ключа, зависящим только от ключа и счётчика. При одном и том же ключе в обоих направлениях ячейка клиента номер n и ячейка ретранслятора номер n использовали бы один и тот же поток ключа. Промежуточный ретранслятор, наблюдающий оба шифртекста, мог бы выполнить XOR и получить XOR двух открытых текстов, что нередко позволило бы восстановить оба сообщения.

Решение 9.4. Ретранслятор `test004r` использует ORPort 5104 = `0x13f0`: `03` означает три спецификатора соединения; спецификатор IPv4-адреса и порта равен `00 06 7f000001 13f0`. Полное сообщение содержит $1 + (2 + 6) + (2 + 20) + (2 + 32) = 65$ байт спецификаторов, плюс $2 + 2 = 4$ байта для HTYPE и HLEN и $32 \cdot 3 + 1 + 32 = 129$ байт HDATA. Итого 198 байт, что свободно помещается в 498-байтовую полезную ёмкость ретрансляционного сообщения.

Решение 9.5. Без $\mathrm{EXP}(B, x)$ ретранслятор не аутентифицирован. Промежуточный ретранслятор, пересылающий рукопожатие, мог бы сгенерировать собственные Y и $AUTH$ со своим

y, выдать себя за следующий переход и читать последующий трафик, предназначенный ему: это *man-in-the-middle*-атака. Без $\mathrm{EXP}(Y, x)$ теряется прямая секретность. Любой, кто позднее получит долгосрочный ключ onion b, сможет вычислить $\mathrm{EXP}(X, b)$ из записанных сообщений рукопожатия и вывести все сеансовые ключи.

Решение 9.6. `VER` привязывает рукопожатие к его контексту, для Tor это `"circuit extend"`. Поскольку значение участвует во всех вычислениях ключевого материала, рукопожатие, созданное в одном контексте, не даёт ни ключей, ни значений $AUTH$, допустимых в другом. Без `VER` атакующий мог бы повторно использовать сообщение рукопожатия из другого протокола против ретранслятора Tor или наоборот, если обе стороны принимали одинаковые вычисления. Это *межпротокольные* атаки.

Решение 9.7. В лабораторной сети `test003r`, поддерживающий `Relay=2-6`, отклоняет запрос, состоящий только из `[02 06]`, ячейкой **`DESTROY`** с причиной `PROTOCOL`. Комбинированный запрос с `CC_FIELD_REQUEST` он принимает и отвечает `CC_FIELD_RESPONSE`. Код tor включает CGO на переходе только вместе с управлением перегрузкой: `enable_cgo = supports_cgo && use_congestion_control` в `src/core/or/extendinfo.c`. Предложение 359 объясняет, почему новый формат ретрансляционного сообщения, где StreamID имеется лишь у некоторых команд, несовместим с прежним механизмом управления потоком через SENDME на уровне потоков. Он требует управления потоком из Предложения 324, то есть `FlowCtrl=2`.

Решение 9.8. Это практическое упражнение. `ntor_ref.py` содержит клиентскую и серверную функции. Использование фиксированных ключей вместо случайной генерации новых даёт воспроизводимые значения для сравнения с `ntor.py`.

Решение 9.9. Общий план: `server_part1` получает HDATA, проверяет `ID` и B, вычисляет $X_b = \mathrm{EXP}(X, b)$, выводит ключи первой фазы, проверяет MAC и расшифровывает CM. Затем `server_part2` генерирует (y, Y), вычисляет key_seed и $verify$, выводит ключ для SM и вычисляет $AUTH$. Если y зафиксировать на значении тестового вектора из предложения, ответ должен совпасть с `server_handshake`.

Глава 10: луковица

Решение 10.1. Клиент обновляет накопительный дайджест прямого направления второго перехода, шифрует с Kf_2, а затем с Kf_1. Третий переход ничего не видит: второй переход распознаёт ячейку и не пересылает её дальше.

Решение 10.2. Каждая ячейка потребляет 509 байт потока ключа. После 1 000 ячеек позиция в потоке ключа равна $509 \times 1000 = 509\,000$ байт, то есть после AES-блока 31 812, поскольку блоки имеют длину 16 байт. Повторное использование ключа для новой цепи со счётчиком, начинающим снова с нуля, привело бы к повторному использованию потока ключа; XOR двух шифртекстов, зашифрованных им, раскрыл бы XOR исходных открытых текстов. Поэтому каждая цепь выводит свежие ключи через рукопожатие с эфемерными значениями.

Решение 10.3. Накопительный дайджест ретранслятора включил бы ячейку, которую клиент не учитывал для этого перехода. После этого состояния дайджеста разошлись бы, и ни одна следующая ячейка не распознавалась бы, уничтожая цепь. Для каждой транзитной ячейки, не адресованной ретранслятору, проблема возникает с вероятностью 2^{-16}: в среднем один раз на $65\,536$ таких ячеек, что соответствует примерно 33 MB трафика.

Решение 10.4. Атакующему следует изменять байты, ожидаемый открытый текст которых известен или предсказуем, чтобы сообщник на выходном ретрансляторе мог обнаружить конкретное изменение: например команду ретрансляционного сообщения, `02` для `DATA`, StreamID либо четыре нулевых байта в начале заполнения. До проверки дайджеста сотрудничающий

выходной ретранслятор смотрит, соответствует ли значение ожидаемой модификации. Честный выходной ретранслятор видит лишь ячейку с неверным дайджестом и уничтожает цепь.

Решение 10.5. При CGO после снятия слоя шифрования выходного ретранслятора изменённая ячейка даёт псевдослучайные тег и содержимое. Узнаваемой метки нет: изменение непредсказуемо распространяется по ячейке. Поскольку тег участвует в обработке следующей ячейки, последующие ячейки этой цепи также становятся нерасшифровываемыми, и цепь приходится отбросить. Поэтому атакующий может лишь уничтожить цепь, что он и без того способен сделать.

Решение 10.6. Наблюдаемые интервалы: 4,1; 3,0; 5,7; 8,7 и 6,8 секунды, среднее 5,7. Для максимума двух независимых равномерных величин на $[a, b]$ математическое ожидание равно $a + \frac{2}{3}(b - a) = 1.5 + \frac{2}{3} \cdot 8 \approx 6.8$ секунды. Пять наблюдений — слишком малая выборка, чтобы разница была удивительной. Кроме того, таймер перезапускается при прохождении через канал *любой* ячейки, а не только ячеек заполнения.

Решение 10.7. В лабораторной сети промежуточный ретранслятор не отвечает на `DROP`; сообщение распознаётся и молча отбрасывается. Косвенное доказательство — цепь остаётся пригодной: после двух сообщений `DROP` клиент всё ещё может расширить её до выходного ретранслятора. Если бы промежуточный ретранслятор не распознал ячейки, будучи в тот момент последним переходом цепи, он уничтожил бы цепь. Если бы клиент и промежуточный ретранслятор по-разному учитывали ячейки, их накопительные дайджесты рассинхронизировались бы и последующее расширение завершилось бы неудачей.

Решение 10.8. В тесте приватной сети изменение одного бита в ячейке `EXTEND2`, предназначенной входному узлу, который в тот момент является единственным переходом цепи, приводит к неудаче проверки дайджеста. Как последний узел этой цепи входной ретранслятор отвечает ячейкой **`DESTROY`** с причиной 1 (`PROTOCOL`). В более длинной цепи изменение, внесённое клиентом в ячейку для третьего перехода, было бы переслано через первые два перехода и обнаружено только третьим.

Глава 11: потоки, SENDME и управление перегрузкой

Решение 11.1. Имя хоста должно быть в нижнем регистре: `example.com:443`, затем нулевой байт, то есть `6578616d706c652e636f6d3a343433 00`, после чего FLAGS с установленным битом 1 для разрешения IPv6: `00000001`.

Решение 11.2. IPv4-адрес `c0000201` — это 192.0.2.1; TTL `0000012c` равен 300 секундам. Клиент может игнорировать адрес. Если он сохраняет его, нельзя повторно использовать значение между цепями: выходной ретранслятор мог передать ложные сведения или вернуть уникальный адрес, чтобы впоследствии распознать клиента.

Решение 11.3. Без опережающей отправки данных (optimistic data) клиент ждёт `CONNECTED` перед отправкой запроса, что добавляет ещё один обмен туда и обратно примерно в 600 мс к операции, и без того требующей как минимум двух RTT. Если соединение завершается неудачей, данные в очереди нужно отбросить, поскольку назначения, способного их получить, нет. Их нельзя передавать через другое соединение.

Решение 11.4. Поток может иметь в полёте не более 500 ячеек, то есть примерно $500 \times 498 \approx 249$ KB на RTT. При RTT 400 мс это около 620 KB/s. Даже при множестве потоков цепь ограничена 1 000 ячейками на RTT, то есть примерно 1,2 MB/s. При управлении перегрузкой окно начинается всего со 124 ячеек, около 62 KB, но растёт до тех пор, пока измерения RTT не покажут перегрузку, без фиксированного верхнего предела.

Решение 11.5. Вредоносный клиент открывает много потоков к серверу, отправляющему большой объём данных, возможно к собственному серверу атакующего, почти ничего не читает и многократно отправляет поддельные сообщения `SENDME`. Выходной ретранслятор продолжает передачу, а ячейки накапливаются в очередях входного и промежуточного ретрансляторов, а также клиента, расходуя память ретрансляторов. Аутентифицированное сообщение `SENDME` требует доказательства, что клиент получил ячейку, вызвавшую это сообщение. Если бы содержимое ячейки было предсказуемым, например при загрузке знакомого файла, клиент мог бы вычислить дайджест, не получая саму ячейку; случайные байты заполнения предотвращают такой обход.

Решение 11.6. Общий план: отправить `RESOLVE` с новым StreamID и именем хоста, завершённым нулём, затем интерпретировать `RESOLVED` как последовательность кортежей, содержащих тип, длину, значение и TTL. Для несуществующего имени хоста выходной ретранслятор отправляет запись с кодом ошибки (`0xF0` или `0xF1`), содержимое которой следует игнорировать.

Решение 11.7. Выходной ретранслятор отвечает `END` с причиной 4 (`EXITPOLICY`), после которой идут адрес назначения, четыре байта для IPv4 и четырёхбайтовый TTL. Клиент может использовать эту информацию, чтобы не повторять попытку через тот же выходной ретранслятор для данного назначения.

Решение 11.8. Мы измеряем время, за которое `SENDME` достигает выходного ретранслятора, плюс время, необходимое разблокированному выходному ретранслятору для отправки дополнительных ячеек и их доставки клиенту. Когда выходной ретранслятор ждёт `SENDME`, это приближённо соответствует RTT цепи с точки зрения клиента. Вторая одновременная загрузка удлиняет очереди и повышает измеренные значения — именно этот сигнал используется механизмом управления перегрузкой для уменьшения окна.

Глава 12: система каталогов

Решение 12.1. Интервал голосования равен одному часу: от `valid-after` до `fresh-until`. Клиент выбирает случайный момент между $11{:}00 + \frac{3}{4} \cdot 60' = 11{:}45$ и $11{:}45 + \frac{7}{8} \cdot 75' \approx 12{:}50$. Если к 13:00 нового консенсуса всё ещё нет, существующий формально истёк. Однако клиент продолжает использовать его как *reasonably live*, то есть в течение 24 часов после окончания срока действия, согласно `REASONABLY_LIVE_TIME` в `networkstatus.c`. Он может продолжать строить цепи, многократно пытаясь получить более новый консенсус. Примерно через 24 часа после истечения клиент перестаёт полагаться на старый документ.

Решение 12.2. Вероятнее всего, `Running` и `Valid`, а также `Fast`, поскольку 500 KB/s больше 100 KB/s. Возможно `V2Dir`, если ретранслятор также работает как кэш каталогов. Но не `Stable`, `Guard` или `HSDir`, для которых требуются дни или недели работы. И не `Exit`, поскольку политика выхода разрешает только порт 443, тогда как для этого флага требуется выходной трафик и на порту 80, и на 443.

Решение 12.3. Кэш отправляет выбранному клиенту двухчасовой, но всё ещё действительный консенсус, в котором отсутствуют недавно добавленные ретрансляторы. Поэтому клиент выбирает маршруты из немного иного множества, чем остальные, что потенциально делает его трафик узнаваемым. Атака ограничена трёхчасовой действительностью консенсуса, необходимостью действительной подписи центров каталогов — кэш не может придумать собственный консенсус — и случайным выбором кэшей клиентами. Загрузка микродескрипторов по дайджесту, а не по идентичности ретранслятора, не позволяет кэшу предоставить конкретному клиенту другую версию ключа ретранслятора, поскольку дайджест зафиксирован подписанным консенсусом.

Решение 12.4. При трёх измерениях медиана равна 5200. Одно завышенное измерение её не меняет. Если бы третий центр каталогов была единственной, измерявшим этот ретранслятор, система не имела бы трёх измерений, необходимых для медианы. Вместо этого консенсус использовал бы заявленную или указанную в голосах пропускную способность с `Unmeasured=1` и верхним ограничением, поэтому завышенное измерение не повлияло бы на результат.

Решение 12.5. В документе голосования присутствуют строки, отсутствующие в консенсусе или отличающиеся от него. Среди примеров — значения пропускной способности `Measured` от центров измерения пропускной способности, строки `m` с поддерживаемыми методами консенсуса, флаги, назначенные конкретным центром каталогов, её `flag-thresholds` и обязательства для совместного случайного значения. Консенсус сохраняет только объединённые результаты.

Решение 12.6. Хэшируйте текст от строки `onion-key`, открывающей микродескриптор, до перевода строки перед следующим микродескриптором либо до конца документа, включая строки PEM-кодированного RSA-ключа. Дайджест SHA-256, закодированный base64 без завершающих символов `=`, должен совпасть со строкой `m` консенсуса.

Решение 12.7. При остановке одного центра каталогов две оставшиеся продолжают голосовать и создают консенсус с двумя подписями, чего достаточно для требуемого большинства $\lfloor 3/2 \rfloor + 1 = 2$. При остановке двух центров каталогов единственная оставшаяся не может получить большинство, и новый консенсус не создаётся. Тем не менее клиент продолжает использовать последний консенсус как *reasonably live* примерно 24 часа после окончания срока действия. В лабораторной сети консенсус действует только 40 секунд, но срок допустимого использования устаревшего консенсуса всё равно равен 24 часам, поэтому нарушение не видно немедленно.

Решение 12.8. Консенсус `ns` обычно содержит подписи SHA-1; спецификация для этого варианта разрешает только их. Его строка `r` дополнительно содержит полный дайджест дескриптора ретранслятора, а каждая запись ретранслятора также включает строку `p` со сводкой политики выхода.

Глава 13: выбор маршрута

Решение 13.1. К выходному ретранслятору предъявляются самые строгие требования: он должен разрешать запрошенное назначение. Выбор выходного ретранслятора первым устанавливает, что подходящий узел вообще существует. Если сначала выбрать входной узел, ограничения семейства или сети могут исключить единственный подходящий выходной ретранслятор и потребовать повторного выбора. Есть и статистическая причина: исключения применяются к последующим позициям, поэтому они должны сильнее влиять на позиции с более крупными множествами кандидатов.

Решение 13.2. Как входной узел: $20000 \times 0/10000 = 0$ (`Wgd=0`). Как промежуточный узел: $20000 \times 0 = 0$ (`Wmd=0`). Как выходной узел: $20000 \times 10000/10000 = 20000$ (`Wed=10000`). При весах этого консенсуса ретранслятор используется исключительно как выходной узел.

Решение 13.3. Без постоянных входных узлов вероятность компрометации превышает 50%, когда $1 - (1 - c^2)^C > 0.5$, то есть $C > \ln 0.5 / \ln(1 - c^2)$: примерно 6932 цепи для $c = 1\%$ и 17 цепей для $c = 20\%$. С одним фиксированным входным узлом вероятность остаётся близкой к c, то есть 1% и 20% соответственно. Входные узлы недостаточны против очень мощного противника, но различие всё равно существенно.

Решение 13.4. Атакующий должен предоставить реальную пропускную способность: вероятности выбора зависят от пропускной способности, измеренной центрами каталогов, а не только от заявленных значений. Выходные ретрансляторы также тщательно контролируются, в том числе через `BadExit`. Ограничение в два ретранслятора на адрес вынуждает атакующего получать множество адресов. Объявление семейства не позволяет двум его ретрансляторам одновременно попасть в одну цепь; отсутствие объявления создаёт риск обнаружения и исключения центрами каталогов. Наконец, веса резервируют дефицитную выходную пропускную способность для настоящих выходных ретрансляторов, и именно там учитывается пропускная способность атакующего.

Решение 13.5. Без ограничения выборки атакующий, контролирующий локальную сеть клиента, мог бы блокировать соединения ко всем честным входным узлам. Клиент продолжал бы выбирать новые входные узлы, пока не нашёл доступный, и в конце концов выбрал бы вредоносный. При ограниченной выборке клиент исчерпывает попытки и перестаёт добавлять входные узлы, возможно теряя связность вместо автоматического перехода на входной узел атакующего.

Решение 13.6. Это практическое упражнение. Получите микродескрипторы из CollecTor вместе с соответствующими консенсусами. Для каждого ретранслятора постройте множество идентификаторов семейства через `family-ids` или взаимные списки `family` и отклоняйте кандидата, если он принадлежит к одному семейству с любым уже выбранным ретранслятором.

Решение 13.7. Это исследовательское упражнение. Крупные семейства обычно дают заметно большую долю вероятности выбора выходного узла, чем любой отдельный крупный ретранслятор. Поэтому фактический риск концентрации зависит от операторов, а не только от числа ретрансляторов. Именно этим объясняется внимание сообщества к разнообразию операторов и необъявленным семействам ретрансляторов (гл. 27).

Глава 14: onion-сервисы v3

Решение 14.1. $32 + 2 + 1 = 35$ байт, то есть 280 бит. Base32 кодирует по пять бит на символ, поэтому требуется $280/5 = 56$ символов без символов заполнения. Случайная ошибка ввода имеет примерно вероятность 2^{-16} случайно пройти проверку контрольной суммы. Байт версии также должен остаться `0x03`, что делает незамеченную случайную ошибку ещё менее вероятной.

Решение 14.2. В 09:00 UTC 1 января 2026 года с начала эпохи Unix прошло 29 454 300 минут. После вычитания смещения 720 минут остаётся 29 453 580, поэтому $\lfloor 29453580/1440 \rfloor = 20453$. Период 20 453 начался в 12:00 UTC 31 декабря 2025 года, поскольку 09:00 всё ещё раньше полудня.

Решение 14.3. Атакующему нужны ослеплённый ключ периода и, следовательно, знание адреса сервиса, а также SRV этого периода и позиции контролируемых ретрансляторов в кольце. Эти значения используются для генерации идентичностей ретрансляторов, располагающихся непосредственно после индексов сервиса. SRV становится известен только при его публикации центрами каталогов, примерно за половину суток до начала соответствующего периода. Атакующий не может подготовить ретрансляторы с точно подобранным положением за несколько дней, а новым ретрансляторам ещё требуется получить флаг `HSDir`, что требует минимум 96 часов работы.

Решение 14.4. HSDir видит ослеплённый ключ, а не адрес сервиса, и получает запросы через анонимные цепи. Точка представления видит ключ аутентификации и цепь, приходящую от сервиса, и может подсчитывать запросы на представление. Точка встречи видит

идентификатор cookie, две цепи и зашифрованный трафик между ними. Точка представления особенно полезна для оценки популярности, поскольку наблюдает каждую попытку соединения, хотя не знает соответствующий адрес. Однако сторона, уже знающая адрес, может загрузить дескриптор и установить эту связь.

Решение 14.5. В цепи из одного перехода HSDir является первым ретранслятором и видит IP-адрес клиента. Он мог бы связать *того, кто* ищет сервис, с *тем, какой* сервис ищется, как минимум по ослеплённому ключу, а если адрес уже известен — и непосредственно с адресом. Отклонение неанонимных запросов вынуждает клиентов загружать дескрипторы через другие ретрансляторы.

Решение 14.6. Это практическое упражнение. Когда HSDir становится больше, чем 2×4, сервис уже не публикует дескриптор на каждом. Запрос дескриптора у каждого HSDir через трёхузловую цепь должен находить его именно на HSDir, рассчитанных для текущего или следующего периода, поскольку сервис публикует дескрипторы для обоих периодов.

Решение 14.7. Значение base64 содержит число спецификаторов соединения, после которого идут сами спецификаторы в формате `EXTEND2` (разд. 9.2.2). Например, тип `00` задаёт адрес и ORPort, тип `02` — отпечаток RSA, тип `03` — идентичность Ed25519. Ретранслятор можно определить по ORPort, который равен 5100 плюс номер узла. Строка `auth-key` содержит сертификат типа 9, подписанный ключом подписи дескриптора и удостоверяющий ключ аутентификации сервиса в этой точке представления.

Глава 15: защита от DoS

Решение 15.1. В первую минуту полный резервуар даёт 60 токенов, а пополнение добавляет $2 \times 60 = 120$, то есть в сумме примерно 180 цепей; фактически несколько меньше, потому что клиент помечается, как только резервуар опустеет. За час при установившейся скорости получится примерно $60 + 2 \times 3600 = 7260$ цепей. Обычный клиент создаёт лишь несколько десятков цепей.

Решение 15.2. Только первый ретранслятор видит настоящий IP-адрес клиента: на следующих переходах «адрес» канала принадлежит предыдущему ретранслятору. Если промежуточный ретранслятор ограничивает число цепей для каждого адреса, он фактически ограничивает все цепи входного узла, который обслуживает тысячи пользователей, и наносит ущерб всем, а не атакующему. Поэтому ограничения применяются только к неаутентифицированным соединениям клиентов.

Решение 15.3. Клиент отправляет сообщение `INTRODUCE1` по цепи, которую может использовать повторно. Сервис должен расшифровать сообщение, выполнить рукопожатие hs-ntor и построить новую цепь как минимум из трёх переходов к точке встречи. Это требует соответствующих рукопожатий ntor с участвующими ретрансляторами, а также ячеек создания и расширения для каждого перехода. Одна ячейка клиента тем самым вызывает несколько операций с открытым ключом и десятки ячеек в сети: усиление нагрузки на один или два порядка.

Решение 15.4. При требовании k нулевых бит можно задавать только уровни трудоёмкости, равные степеням двойки: 1, 2, 4 и так далее. При переходе на следующий уровень вычислительная стоимость удваивается. Линейная проверка позволяет задать любое целое значение трудоёмкости E. Регулятор с обратной связью, изменяющий трудоёмкость малыми шагами, нуждается в более точной шкале: при удвоениях он колебался бы между слишком низкими и слишком высокими значениями.

Решение 15.5. Нет. Трудоёмкость E входит в задачу, по которой вычисляются и решение Equi-X, и хеш R. При объявлении $E = 5000$ задача меняется, поэтому решение, найденное для $E = 50$, больше недействительно. Клиент должен был выбрать $E = 5000$ до начала поиска.

Решение 15.6. Лабораторное упражнение. Ячейки накапливаются в очереди цепи промежуточного ретранслятора, пока общий объём не превысит `MaxMemInQueues`. Тогда диспетчер памяти закрывает цепи с наиболее старыми данными в очередях, в том числе нашу, и записывает в журнал объём освобождённой памяти.

Решение 15.7. Упражнение по программированию. Ожидается, что запросы с высокой трудоёмкостью будут иметь короткое и устойчивое время ожидания, а запросы с нулевой трудоёмкостью будут обслуживаться только при почти пустой очереди или отбрасываться при её разгрузке. Это намеренный результат: он защищает пользователей, готовых затратить немного вычислительных ресурсов.

Глава 16: цензура, мосты и подключаемые транспорты

Решение 16.1. Мост (bridge) выполняет роль входного узла (guard), первого перехода цепи, но не фигурирует в публичном консенсусе. Клиент узнаёт его адрес из полученной частным образом строки конфигурации моста. При `UseBridges` клиент использует мосты *вместо* публичных входных узлов: добавление обычного входного узла означало бы прямое подключение к опубликованному адресу, то есть именно то, что цензор блокирует или отслеживает.

Решение 16.2. «Не быть похожим ни на один известный протокол»: obfs4 (для блокирования цензору пришлось бы запретить весь нераспознаваемый трафик). «Быть похожим на разрешённый трафик»: meek (потребовалось бы заблокировать крупную сеть доставки содержимого), Snowflake (заблокировать WebRTC, брокер и множество добровольных прокси), WebTunnel (блокировать HTTPS к неизвестным сайтам либо сайтам, на которых размещены мосты), Conjure (блокировать широкие диапазоны адресов у сотрудничающих провайдеров).

Решение 16.3. Цензор может искать долгие подключения к адресам без связанного доменного имени, высокую энтропию байтов уже в первом пакете, характерные распределения размеров и интервалов передачи (например, всплески, связанные с ячейками Tor), а также отношение входящего и исходящего трафика, напоминающее просмотр веб-страниц. `iat-mode` добавляет случайные задержки и фрагментацию, затрудняя распознавание размеров и времени ценой снижения производительности.

Решение 16.4. Цензор блокирует полученные мосты, и заблокированный мост не позволяет продвинуться на следующий уровень: соответствующие учётные данные остаются на низком уровне и не дают права приглашать других. Цензор может не блокировать найденные мосты, пока не достигнет высокого уровня, а затем заблокировать их все сразу. Однако для этого пришлось бы месяцами поддерживать множество терпеливых идентичностей, а мосты всё это время обслуживали бы обычных пользователей. Кроме того, каждые учётные данные высокого уровня дают доступ лишь к небольшому числу мостов.

Решение 16.5. Оба захвата имеют энтропию, близкую к восьми битам на байт: TLS после рукопожатия тоже передаёт зашифрованные данные. Различия заключаются в нескольких байтах: заголовках записей TLS (`16 03 01`, `17 03 03`, затем длина) и первоначальном незашифрованном рукопожатии. Система глубокой проверки пакетов (DPI) распознаёт эти признаки без статистического анализа.

Решение 16.6. Лаборатория 1 завершается по времени ожидания (`TimeoutError`), потому что мост не отвечает. При неверном параметре `cert` клиентская программа `obfs4proxy` не может завершить рукопожатие; мост не распознаёт метку аутентификации клиента, не отвечает и через некоторое время закрывает соединение. В журнале моста могут появиться только сообщения о неудачном рукопожатии, в зависимости от уровня журналирования.

Глава 17: установка и torrc

Решение 17.1. tor сообщает о неизвестной опции сообщением вида `Unknown option '...'. Failing.` и завершает работу, не запускаясь. Команда `--dump-config short` для клиентского файла `torrc` показывает лишь параметры, отличающиеся от значений по умолчанию: в лабораторной сети это, среди прочего, `TestingTorNetwork`, записи `DirAuthority` и порты.

Решение 17.2. Если почта и форум используют одну цепь, выходной ретранслятор видит примерно в одно время по этой цепи соединение с почтовым сервером пользователя (часто связанное с его именем) и с форумом под псевдонимом. Он может связать псевдоним с человеком. Решения: два разных SOCKS-порта, по одному для каждого приложения (разные порты изолированы по умолчанию, если им не назначена общая `SessionGroup`); либо разные учётные данные SOCKS с `IsolateSOCKSAuth`; либо `IsolateDestAddr` на общем порту.

Решение 17.3. Атакующий, наблюдающий сети Италии, мог бы увидеть и пользователя, входящего в сеть Tor, и итальянский выходной ретранслятор, подключающийся к итальянскому сайту. Оба конца находятся в одной наблюдаемой области — это как раз сценарий корреляции, от которого модель угроз Tor не гарантирует защиты. Кроме того, итальянских выходных ретрансляторов мало; ограничение выбора делает пользователя более узнаваемым и облегчает атаки со стороны злонамеренных ретрансляторов.

Глава 18: порт управления и Stem

Решение 18.1. `552` — постоянная ошибка в ответ на команду: ключ `pippo` не существует. `650` — не ответ на команду, а асинхронное событие, передаваемое после подписки клиента с помощью `SETEVENTS`. Оно может поступить в любой момент, в том числе во время ожидания ответа на другую команду.

Решение 18.2. План: вызвать метод контроллера `create_ephemeral_hidden_service`, сопоставив виртуальный порт 80 с локальным портом 8080 и указав `await_publication`; метод возвращает адрес onion-сервиса. Подключиться к нему можно командой `curl --socks5-hostname 127.0.0.1:9008 http://<address>.onion/`. Ключи временного сервиса tor хранит в памяти и возвращает контроллеру, который может сохранить их для последующего восстановления того же адреса. При закрытии управляющего соединения сервис исчезает, если не задать режим независимой работы (detach).

Решение 18.3. План: зарегистрировать обработчик для `EventType.BW`, получающий каждую секунду количество прочитанных и записанных байтов (`event.read` и `event.written`), и выводить значения во время загрузки. Скорость приёма растёт до предела, определяемого механизмом управления перегрузкой и ресурсами компьютера.

Решение 18.4. При `__LeaveStreamsUnattached` tor не прикрепляет потоки к цепям автоматически. Команда `EXTENDCIRCUIT 0 test004r,test005r,test006e` строит цепь и возвращает её идентификатор. Когда поступает событие `STREAM` со статусом `NEW`, вызывают

ATTACHSTREAM. Ручной выбор маршрута опасен: он обходит взвешенный выбор ретрансляторов и правила использования входных узлов, делает пользователя узнаваемым и может позволить вредоносной программе направить его трафик через подконтрольные противнику ретрансляторы.

Глава 19: приватная сеть Tor

Решение 19.1. `test006e` — узел номер 6: ORPort 5106, ControlPort 8006. У `test009h` нет ORPort: onion-сервис не является ретранслятором и не принимает соединения от других узлов сети. Он сам строит исходящие цепи к точкам представления и встречи.

Решение 19.2. Это параметр `TestingDirAuthVoteGuard *` в файлах torrc центров каталогов. В публичной сети флаг Guard требует измеренных за некоторое время стабильности и пропускной способности (гл. 12). В лаборатории любой ретранслятор может быть выбран входным узлом: статистика выбора первого перехода не соответствует реальной сети.

Решение 19.3. При `--socks5` программа `curl` самостоятельно разрешает имя `localhost` и передаёт клиенту Tor IP-адрес. При `--socks5-hostname` передаётся имя, которое разрешает выходной ретранслятор. При посещении обычных сайтов в публичной сети первый способ выдавал бы доменные имена локальному DNS-резолверу и его наблюдателям, хотя сам трафик шёл бы через Tor. Это классическая утечка DNS.

Решение 19.4. Каждые 20 секунд согласно `V3AuthVotingInterval`. За сутки это $86400/20 = 4320$ консенсусов в лаборатории против 24 в публичной сети, где интервал голосования равен одному часу.

Решение 19.5. При остановке одного центра каталогов сеть продолжает выпускать консенсусы, поскольку достаточно большинства подписей (две из трёх). При остановке двух единственный оставшийся центр не достигает большинства, и новый действительный консенсус не создаётся. Клиенты, однако, продолжают использовать старый консенсус как *reasonably live* — ещё примерно 24 часа после истечения его срока (гл. 12) — и могут долго строить цепи. В лаборатории срок консенсуса составляет 40 секунд, но допустимое использование после окончания срока остаётся равным 24 часам.

Решение 19.6. План: `c.get_network_status(fingerprint)` возвращает запись консенсуса с полями `address` и `or_port`. Вызовите метод для каждого отпечатка из `circ.path`, то есть для первого элемента каждой пары.

Решение 19.7. Определение сети сохраняет структуру из книги, но использует `getN(5)`, `getN(10)` и `getN(4)`. Время начальной загрузки зависит от компьютера. При увеличении числа узлов особенно растёт ожидание появления ретрансляторов в консенсусе и загрузки микродескрипторов клиентами.

Решение 19.8. Исследовательское упражнение: найдите сообщения о выборе маршрута, отправке **`CREATE2`**/EXTEND2, получении ответов и завершении построения цепи. Уровень журналирования *info* показывает не все ячейки; более подробные сведения можно получить, включив *debug* в torrc клиента.

Решение 19.9. Нет: после рукопожатия TLS ячейки передаются зашифрованными, а `tcpdump` видит только записи TLS. При этом остаются видимыми адреса, порты, размеры записей и время — данные, на которых основаны атаки анализа трафика (гл. 26).

Глава 21: публикация onion-сервиса

Решение 21.1. Шесть символов заданного префикса требуют в среднем $32^6 \approx 1{,}07 \times 10^9$ попыток, около 107 секунд при десяти миллионах попыток в секунду. Восемь символов требуют $32^8 \approx 1{,}1 \times 10^{12}$ попыток, около $1{,}1 \times 10^5$ секунд, чуть более 30 часов. Каждый дополнительный символ увеличивает время в 32 раза.

Решение 21.2. Страница ошибки раскрывает программное обеспечение и версию веб-сервера (это помогает искать уязвимости) и, что ещё важнее, IP-адрес 192.0.2.10, который может оказаться настоящим публичным адресом сервера: тогда сервис можно локализовать. Необходимо скрыть сигнатуры сервера в заголовках и страницах ошибок, разрешить ему слушать только localhost или Unix-сокет и проверить ответы лабораторным клиентом.

Решение 21.3. Удалите `alice.auth` и перечитайте конфигурацию. Сервис публикует новый дескриптор без записи Алисы, зашифрованный новым *descriptor cookie*, который она не может расшифровать. Клиенты со старым дескриптором могут пользоваться известными точками представления, пока они действуют; для более полного отзыва стоит сменить точки представления, например перезапустив сервис. Крайний случай: если Алиса была единственным авторизованным клиентом, каталог окажется пустым, а HSDir отклонят дескриптор (разд. 21.3). Для настоящего отключения ограниченного обнаружения удалите каталог `authorized_clients/` и перечитайте конфигурацию.

Решение 21.4. Задайте `HiddenServicePort 80 unix:/path/socket` и настройте веб-сервер на прослушивание этого Unix-сокета. Помимо отсутствия открытого TCP-порта, сокет защищается правами файловой системы, недоступен через сетевой интерфейс и исключает случайное открытие сервиса на публичном интерфейсе.

Глава 20: наблюдение за трафиком

Решение 20.1. TLS не выравнивает границы записей по ячейкам Tor. tor помещает ячейки в буфер, а библиотека TLS объединяет их в записи длиной до 16 KB (отсюда значение `sendme_inc`, равное 31 ячейке, разд. 11.3) и добавляет заголовок и тег аутентификации. TCP может делить или объединять эти записи в сегментах. Фиксированная длина относится к ячейкам, а не к сетевым пакетам.

Решение 20.2. План: после **`CREATED2`** расшифруйте ячейки **`RELAY_EARLY`** и **`RELAY`** лабораторной цепи, снимая слои шифрования по порядку с помощью ключей переходов из `circuit.py`. Состояние потока AES-CTR должно продвигаться ровно в порядке использования. При построении цепи видны `EXTEND2` и `EXTENDED2`, затем возможные `BEGIN`, `CONNECTED`, `DATA` и `SENDME`.

Решение 20.3. На клиентском канале, по которому проходят цепи из нескольких переходов, записи TLS могут появляться через несколько секунд даже без прикладного трафика. Это ячейки заполнения канала **`PADDING`**, передаваемые через случайные интервалы от 1,5 до 9,5 секунды (разд. 10.5). На канале, не использовавшемся для многошаговых цепей, такого трафика нет.

Решение 20.4. Задание по программированию: анализатор Lua, зарегистрированный как *эвристический диссектор* расшифрованных данных TLS, читает первые два байта CircID начальной ячейки **`VERSIONS`** или четыре байта в последующих ячейках, затем байт команды и поле длины переменных ячеек. Нужно сохранять состояние отдельно для каждого соединения и направления, как в `traffic.py`.

Глава 22: управление ретранслятором или мостом

Решение 22.1. Да. Первое подходящее правило `accept *:80` разрешает соединение, последующие правила уже не проверяются. Достаточно изменить порядок: `reject 192.0.2.0/24:*`, `accept *:80`, `reject *:*`.

Решение 22.2. Сводка содержит только порты, без адресов, и является приближённой. Порт считается разрешённым для «большинства» адресов согласно точному правилу dir-spec, а не просто при доле свыше 50%: выходной ретранслятор не должен блокировать более 2^{25} IPv4-адресов (одна сеть /7) или 2^{112} IPv6-адресов (одна сеть /16), не считая непубличных адресов. Получив `END` с причиной `EXITPOLICY`, клиент на время TTL запоминает отказ выходного ретранслятора для указанного адреса и порта и повторяет запрос в другой цепи через другой выходной узел.

Решение 22.3. При `MyFamily` учитываются только взаимные объявления: пять прежних ретрансляторов принадлежат к одному семейству, но шестой — нет, поскольку ни один из прежних его не указал. Поэтому клиент может выбрать шестой узел вместе с другим членом семейства в одной цепи. С ключом семейства достаточно установить этот ключ на новом ретрансляторе: подписанный сертификат связывает его с семейством, не требуя изменений на остальных узлах.

Решение 22.4. В журналах последовательно появляются «in-progress connect failed …(Connection refused)» и «Didn't recognize errno 115». Если причину окончания потока вычислять по ошибке, которую возвращает `getsockopt(SO_ERROR)`, клиент получит `CONNECTREFUSED`. Аналогично можно верно передавать `TIMEOUT` при `ETIMEDOUT`, `NOROUTE` при `ENETUNREACH` или `EHOSTUNREACH` и `CONNRESET`, вместо `MISC` для соединений, завершившихся ошибкой после `EINPROGRESS`. Для клиента меняется сообщение об ошибке, а не сам порядок обработки неудачи.

Решение 22.5. При 10 MB/s ретранслятор передаёт около 864 GB в сутки; учитывается максимум между отправленным и полученным трафиком. Лимит 1 TB исчерпывается чуть более чем за день. Ретранслятор, выключенный большую часть месяца, теряет флаги `Stable` и `Guard` и разрывает цепи при остановке; медленный узел, непрерывно работающий весь месяц, гораздо полезнее. Поэтому `RelayBandwidthRate` стоит настроить так, чтобы лимита хватило на весь период.

Глава 23: Tor Browser, Tails и Whonix

Решение 23.1. Функция «Новая цепь для этого сайта» (New Circuit for This Site) меняет только цепь сайта активной вкладки, но не удаляет файлы cookie, историю и состояние других сайтов: сайт может по-прежнему узнавать пользователя по сохранённым данным. «Новая личность» (New Identity) закрывает вкладки, очищает состояние браузера и создаёт новые цепи для дальнейшей работы. Ни одна из команд не меняет входной узел (guard), поскольку частая замена повышает вероятность рано или поздно выбрать враждебный узел (разд. 13.3); при этом входной узел не знает посещаемых сайтов.

Решение 23.2. Дополнительное расширение может блокировать загрузку некоторых отслеживающих элементов, но Tor Browser уже изолирует их по сайтам и ограничивает устойчивые идентификаторы. Цена — потеря единообразия: сайты могут обнаружить само расширение или его списки фильтрации и выделить пользователя среди других пользователей Tor Browser. Кроме того, расширение добавляет программный код с доступом ко всем посещаемым страницам.

Решение 23.3. Если две вкладки одного сайта используют одинаковые учётные данные SOCKS, tor направляет их потоки через одну цепь; третья вкладка с другими данными получает другую цепь. Команда `GETINFO stream-status` позволяет увидеть идентификатор цепи каждого потока.

Решение 23.4. (a) Достаточно Tor Browser, при необходимости с мостом. (b) Tails: загружается с USB-накопителя, не оставляет обычных следов на чужом компьютере и сохраняет только то, что помещено в зашифрованное постоянное хранилище; сохраняется риск подменённого оборудования. (c) Whonix: приложения Workstation проходят через tor по устройству системы, которая подходит для длительной работы без обязательного удаления следов при завершении сеанса.

Решение 23.5. Упражнение с открытым ответом. Примеры характеристик: размеры экрана и окна, часовой пояс, язык, доступные шрифты, результат отрисовки на холсте (canvas). В обычном браузере они могут выдавать от нескольких бит информации (например, часовой пояс) до многих (шрифты и canvas). Tor Browser округляет размеры окна с помощью letterboxing, сообщает UTC и единый язык, ограничивает шрифты и унифицирует либо блокирует наиболее раскрывающие интерфейсы. Остаются различия, связанные с операционной системой, уровнем безопасности и округлёнными размерами экрана.

Глава 24: измерение сети

Решение 24.1. Количество запросов к каталогам уменьшилось бы вдвое, а при постоянном делителе 10 вдвое упала бы и оценка числа пользователей. Графики показали бы почти одновременное снижение в разных странах по мере распространения новой версии tor. Постепенный глобальный спад, совпадающий с внедрением версии, свидетельствует об изменении поведения программы, а не числа пользователей; метод оценки нужно исправить, изменив делитель.

Решение 24.2. $20\,000\,000/(0{,}6 \cdot 10) \approx 3\,333\,333$ пользователя. При доле отчётности 8% оценка отбрасывается, поскольку она ниже порога 10%.

Решение 24.3. План: отберите ретрансляторы с обоими флагами и разделите суммарную пропускную способность этих узлов на общую пропускную способность сети. Для вероятности выбора выходного узла возьмите `probabilities(consensus, "exit")` из Лаборатории 7 и сложите вероятности ретрансляторов, чьи сети /16 входят в пятёрку наиболее часто встречающихся.

Решение 24.4. Упражнение с открытым ответом. Обычно число прямых пользователей резко падает в день блокировки, а число пользователей мостов, в частности Snowflake или WebTunnel, может расти в следующие дни по мере перехода на подключаемые транспорты. Индикатор Metrics отмечает дату первоначального падения.

Решение 24.5. Без округления противник, знающий, что является единственным пользователем определённой страны на данном ретрансляторе (или точно знающий число других пользователей), может по статистике extra-info определить, появился ли ещё один пользователь этой страны: счётчик изменится на единицу. Округление скрывает такие небольшие изменения. Противник, контролирующий много ретрансляторов, вообще не нуждается в опубликованной статистике: он непосредственно видит подключения к своим узлам.

Глава 25: правовые и этические аспекты

Решение 25.1. (a) Выходной ретранслятор, самостоятельно отправляющий запросы на сайты, например для «смешивания» трафика, сам инициирует передачу. (b) Выходной ретранслятор, перенаправляющий соединение на адрес, отличный от запрошенного в ячейке `BEGIN`, выбирает получателя. (c) Выходной ретранслятор, вставляющий содержимое в страницы HTTP или фильтрующий их по смыслу, выбирает либо изменяет информацию. Именно за подобное поведение центры каталогов могут присвоить флаг `BadExit`.

Решение 25.2. Кухонный нож как инструмент законен, нападение с ним — нет. Чтение запрещённой цензурой газеты с помощью Tor может быть законным применением законного инструмента, но неправомерный доступ к компьютеру остаётся правонарушением и с Tor, и без него. Оператор ретранслятора предоставляет канал передачи, не выбирая содержимое: он ближе к перевозчику, чем к автору, если сам не участвует в незаконной деятельности. Конкретные правовые выводы зависят от страны и обстоятельств.

Решение 25.3. Следует объяснить, что адрес принадлежит выходному ретранслятору Tor, что проверяется публичными инструментами Tor Project; что оператор не знает и не записывает личность клиентов и не может их установить; что сайт при желании может отдельно обрабатывать подключения с публичных выходных адресов. На практике оператор может добавить в политику выхода запрет для адреса жалующегося сайта, если польза оправдывает ограничение.

Решение 25.4. Предложение предполагает наблюдение за трафиком реальных пользователей без их согласия, сбор сведений, которые нельзя безопасно публиковать (посещённые домены), и лишь небольшую пользу. Оно нарушает несколько принципов безопасного исследования и в Италии может создавать проблемы уголовного права и защиты персональных данных. Альтернативы: использовать уже опубликованную агрегированную статистику Metrics, самостоятельно проверять доступность новостных сайтов через собственный клиент Tor либо предложить Совету по безопасности исследований (Research Safety Board) схему подсчёта с агрегацией и добавлением шума, как в статистике ретрансляторов.

Решение 25.5. Исследовательское задание: ищите судебные решения в официальных базах (в Италии — решения Кассационного суда и других судов, в ЕС — документы Суда Европейского союза), а не только в сторонних пересказах. Для каждого решения укажите суд, дату, применённую норму и результат; различайте дела об операторах ретрансляторов и о пользователях Tor.

Глава 26: корреляция, анализ отпечатков и Sybil-атаки

Решение 26.1. Корреляция трафика требует наблюдения за обоими концами соединения (ретрансляторами или сетями); защиты: входные узлы, ограничения маршрутов, взвешивание по измеренной пропускной способности. Определение посещаемого сайта по отпечатку трафика требует лишь наблюдения канала клиент—входной узел; защиты: ограниченное заполнение, исследования продолжаются. Обнаружение входного узла требует контроля промежуточного ретранслятора и возможности вызывать построение цепей; защиты: постоянные входные узлы и vanguards. Атаки Sybil используют множество ретрансляторов; защиты: измерение пропускной способности, семейства, ограничение /16 и наблюдение за сетью. Атаки маршрутизации требуют контроля автономной системы (AS) или точки обмена трафиком (IXP); внешняя защита — мониторинг BGP.

Решение 26.2. На 100 000 посещений приходится в среднем около 0,95 правильного сигнала и $0{,}001 \cdot 99\,999 \approx 100$ ложных. Доля правильных сигналов составляет $0{,}95/(0{,}95+100) \approx 0{,}9\%$. Почти все предупреждения оказываются ложными.

Решение 26.3. При двухсекундных интервалах за пять секунд остаётся только две или три точки на ряд, поэтому корреляция становится случайной. Если задать всем клиентам один ритм трафика (например, одинаковое зерно генератора случайных чисел), ряды становятся почти неразличимыми и сопоставление затрудняется. Именно выравнивание ритмов сильнее всего мешает корреляции: такова идея постоянной скорости передачи в PipeNet (разд. 2.4).

Решение 26.4. При постоянном темпе передачи входные ряды перестают меняться. Корреляция Пирсона тогда математически не определена (нулевая дисперсия) либо определяется шумом сети: такая атака не работает. Цена — постоянная передача с максимальной выбранной скоростью, даже когда полезных данных нет. В лаборатории это десятки или сотни KB/s на клиента, намного больше полезного трафика.

Решение 26.5. С постоянным входным узлом противник выигрывает, только если этот узел принадлежит ему (вероятность около 5%); тогда примерно в 5% цепей выходной узел тоже принадлежит противнику. После шести месяцев интенсивного использования вероятность хотя бы одной компрометации приближается к 5% в целом, а при честном входном узле остаётся нулевой в этой упрощённой модели. Если выбирать входной узел заново для каждой цепи, риск одной цепи около $0{,}05 \cdot 0{,}05 = 0{,}0025$; после нескольких тысяч цепей хотя бы одна компрометация почти неизбежна. Это аргумент в пользу постоянных входных узлов; проверку моделированием позволяет класс `Adversary` из Лаборатории 7.

Глава 27: реальные атаки на сеть

Решение 27.1. В направлении к клиенту каждый ретранслятор добавляет свой слой шифрования к содержимому ячейки (гл. 10). Входной узел видит содержимое, зашифрованное ключами промежуточного и собственного переходов, и не может прочитать данные, записанные HSDir. Команда самой ячейки, напротив, находится во внешнем заголовке, который каждый ретранслятор читает и пересылает без изменений.

Решение 27.2. При трёх входных узлах вероятность, что хотя бы один из них враждебен, примерно втрое выше, и каждый такой узел видит часть трафика клиента. При одном входном узле риск выбора противника ниже; если узел честен, клиент защищён долгое время. Минус: единственный враждебный входной узел наблюдает весь трафик клиента, а медленный или недоступный узел снижает производительность.

Решение 27.3. Лаборатория уже использует ячейки `DROP`, а не запросы на расширение цепи: счётчик в `command.c` проверяет внешнюю команду ячейки до расшифрования, поэтому не зависит от её содержимого. В обратном направлении программа записывает предупреждение и закрывает цепь с причиной `PROTOCOL`, независимо от того, принадлежит ли цепь клиенту или ретранслятору.

Решение 27.4. Признаки: ретрансляторы появились в консенсусе в одну минуту; в дескрипторах совпадают редкая версия tor и операционная система; адреса находятся в одной сети или у одного провайдера; похожи порты, имена узлов, отсутствуют контакты. Ложные совпадения возможны у честных крупных операторов, одновременно обновляющих узлы, распространённых хостинг-провайдеров и автоматических установок с одинаковой конфигурацией. Одного признака недостаточно: нужны разные свидетельства и, по возможности, связь с оператором.

Решение 27.5. Промежуточный ретранслятор видит незашифрованные внешние данные: CircID (на каждом переходе меняется, поэтому через всю цепь не проходит); команду ячейки (**`RELAY`** либо **`RELAY_EARLY`**): этот канал был использован в 2014 году, теперь он закрыт в направлении к клиенту и ограничен восемью ячейками в обратном направлении; ячейку **`DESTROY`** и причину закрытия, время и число ячеек. Последние две величины всегда доступны для пассивного подтверждения и не имеют полной защиты. Содержимое зашифровано послойно, но в tor1 один злонамеренный ретранслятор может изменять его для распознавания другим (маркировка, разд. 10.3); CGO (разд. 10.4) делает такие изменения причиной невосстановимого содержимого на следующих переходах.

Глава 28: ошибки операционной безопасности

Решение 28.1. Трафик вне Tor, документы, открытые при подключении к сети, и утечки DNS относятся к уровню приложений; меры защиты — Tor Browser, Tails или Whonix, `SafeSocks` и открытие файлов без сетевого подключения. Метаданные, вход в учётные записи и смешение идентичностей относятся к уровню содержимого; нужны инструменты очистки метаданных и раздельные сеансы. Стиль письма, время публикации и использование Tor в небольшом множестве анонимности относятся к поведению; здесь помогают изменение привычек, мосты и задержка перед публикацией.

Решение 28.2. Запрос DNS уходит с компьютера пользователя на настроенный в системе сервер разрешения имён, обычно принадлежащий провайдеру связи, и содержит доменное имя сайта. Провайдер и любой наблюдатель на этом участке узнают, какой сайт пользователь намеревается посетить, даже если само соединение затем пойдёт через Tor.

Решение 28.3. Через Stem подпишитесь на `EventType.STATUS_CLIENT`. Запрос с одним лишь IP-адресом вызывает событие уровня `WARN` с действием `DANGEROUS_SOCKS` и аргументами `PROTOCOL` и `ADDRESS`. Графический интерфейс мог бы предупреждать с указанием приложения и адреса назначения, предлагая исправить его настройки SOCKS.

Решение 28.4. При использовании obfs4 наблюдатель увидел бы зашифрованное соединение без узнаваемой структуры с обычным на вид адресом и не смог бы уверенно определить, что оно связано с Tor. Остались бы другие признаки: объём и время передачи, необычная длительность соединения, похожие подключения, а также содержание сообщений и мотив, сужающие круг подозреваемых до тех, кому предстояло сдавать этот экзамен.

Решение 28.5. Упражнение с открытым ответом. Хороший анализ следует структуре гл. 4: противники и их возможности, что каждый способен наблюдать на уровнях рис. 28.1, выбранные средства защиты и обоснование их выбора, а также явный перечень неустранённых рисков — например, корреляция трафика наблюдателем обоих концов или изъятие устройств.

Глава 29: академические исследования Tor

Решение 29.1. $H = \frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + 2\cdot\frac{1}{8}\cdot 3 = 1{,}75$ бита; $H_M = \log_2 4 = 2$ бита; $d = 0{,}875$. Эквивалентное число равновероятных пользователей: $2^{1{,}75} \approx 3{,}4$.

Решение 29.2. (a) Моделирование сети с помощью Shadow или, в небольшом масштабе, эмуляция через Chutney: необходимо поведение настоящего tor, но привлекать реальных пользователей не нужно. (b) Измерение публичной сети по опубликованным агрегированным данным, как в Tor Metrics (разд. 24.2). (c) Моделирование поведения по образцу TorPS с использованием настоящих консенсусов за длительный период, при необходимости с последующим теоретическим анализом.

Решение 29.3. При `guard_lifetime` в 30 дней (фактический срок 18–30 дней из-за случайного сдвига даты в прошлое) за 180 дней происходит примерно семь или восемь выборов входного узла, а моделирование даёт 32,1% скомпрометированных клиентов. При стандартных 60 днях (фактически 48–60) получается примерно три выбора и 18,4%; при 180 днях — один или два выбора и 8,6%. Приближённо после k независимых выборов доля скомпрометированных приближается к $1 - (1 - q)^k$ при $q = 5\%$: для $k \approx 7{,}5$ это около 31%, что соответствует результату для 30 дней. Чем дольше служит входной узел, тем реже выборы и тем меньше новых клиентов подвергаются риску; однако клиент, выбравший враждебный узел, остаётся под наблюдением дольше.

Решение 29.4. Ни один из двух противников не компрометирует клиентов согласно критерию лаборатории, для которого необходимо видеть оба конца соединения. Однако получаемые ими сведения различаются: выходной ретранслятор видит назначения и трафик, не защищённый прикладным шифрованием, но не знает личности клиентов; входной ретранслятор видит IP-адреса выбравших его клиентов, но не знает их назначения и может пытаться определить посещаемые сайты по отпечаткам трафика (разд. 26.4).

Решение 29.5. Исследовательское упражнение. Повторное использование цепей сокращает число новых выборов за день и замедляет кривую компрометации при отсутствии постоянных входных узлов. Ограничение /16 обычно слабо влияет на результат, кроме случаев, когда враждебные ретрансляторы сосредоточены в одной сети. Противник на уровне автономной системы меняет модель: он наблюдает трафик без управления ретрансляторами, поэтому нужны данные о топологии Интернета, как в [Joh+13].

Глава 30: Arti

Решение 30.1. Глава 5: `tor-llcrypto`; глава 7: `tor-proto` (каналы) и `tor-chanmgr`; глава 8: `tor-cell`; главы 9–11: `tor-proto` (цепи, шифрование, потоки и управление потоком); глава 12: `tor-netdoc`, `tor-dirmgr`, `tor-dirclient`; глава 13: `tor-circmgr`, `tor-guardmgr`, `tor-relay-selection`; глава 14: `tor-hsclient`, `tor-hsservice`, `tor-hscrypto`; глава 15: `equix` и `hashx` для доказательства выполненной работы; глава 16: `tor-ptmgr`.

Решение 30.2. Преимущества: меньше кода и поверхность атаки, отсутствие особых случаев (например, двухбайтовых CircID) и необходимости поддерживать слабую устаревшую криптографию. Недостаток: Arti несовместим с очень старыми ретрансляторами. Однако публичная сеть Tor сама уже исключает нерекомендуемые версии, поэтому это важно главным образом для частных сетей и давно не обновлявшихся установок.

Решение 30.3. `decode_cell` читает команду из пятого байта и определяет, имеет ли ячейка переменную длину. В этом случае два следующих байта задают длину полезных данных, а общая длина составляет 7+ указанная длина; иначе ячейка занимает 514 байт. Если байтов пока недостаточно, функция возвращает `Ok(None)`, а не ошибку: ячейка не повреждена, а лишь ещё не получена целиком. Вызывающая сторона повторяет попытку при поступлении следующих байтов. Так же должен работать потоковый декодер из Лаборатории 2.

Решение 30.4. Да: SOCKS-прокси Arti по умолчанию изолирует потоки с разными учётными данными SOCKS и разделяет разные прослушивающие сокеты. Проверить это можно, наблюдая цепи со стороны ретрансляторов (например, через ControlPort выходного узла) или в подробных журналах Arti. При использовании Arti как библиотеки явную изоляцию обеспечивают `TorClient::isolated_client` и токены изоляции `StreamPrefs`. На момент Arti 2.6.0 эквивалентную настройку для отдельных портов файл `cfg.rs` помечает как будущую возможность.

Решение 30.5. Упражнение по программированию: создайте `TorClientConfig` из TOML-файла лаборатории, запустите клиент методом `TorClient::create_bootstrapped` и откройте поток с помощью `connect(("127.0.0.1", 8080))`, затем передайте по нему HTTP-запрос. Конфигурация должна разрешать локальные адреса, как файл, создаваемый Лабораторией 20.

Глава 31: открытые проблемы

Решение 31.1. Примеры на дату, принятую в главе: Предложение 332 (ntor-v3) имеет состояние *Closed*: реализована и включена в спецификации. Предложение 324 (управление перегрузкой) — *Finished*: реализована, но текст ещё не целиком перенесён в спецификации. Предложение 359 (CGO) — *Open*: реализация уже присутствует в коде, однако предложение ещё обсуждается. Состояние можно узнать в заголовке каждой инициативы и в указателе по состояниям репозитория.

Решение 31.2. В традиционном формате расчёт по размерам даёт $\lceil 1184/498 \rceil = 3$ ячейки для открытого ключа ML-KEM-768 длиной 1184 байта и ещё $\lceil 1088/498 \rceil = 3$ для шифртекста длиной 1088 байт: всего шесть. Однако действующий протокол даже не позволяет так фрагментировать одно ретрансляционное сообщение. Предложение 340 предлагает необходимое обрамление: одно сообщение может занимать несколько ячеек, а несколько сообщений — помещаться в одной. Тело ретрансляционной ячейки в новом формате содержит 493 байта, а не 498: 16 байт отведены под тег аутентификации, $509 - 16 = 493$, как объясняется в гл. 8. Первый фрагмент содержит не более 490 байт полезных данных, поскольку в нём есть и заголовок обрамления. Пока предложение ограничивает обычные тела сообщений 493 байтами; чтобы передавать более длинный постквантовый `EXTEND2`, потребуется новая версия подпротокола `Relay`, снимающая ограничение. Даже при таком расширении открытый ключ займёт три ячейки, а шифртекст — ещё три. Значения передаются в *противоположных* направлениях (клиент — ретранслятор и обратно), поэтому объединить их нельзя. Всего остаётся шесть ячеек.

Решение 31.3. Если бы каждый клиент загружал собственное подмножество ретрансляторов, противник, знающий или влияющий на подмножество конкретного пользователя, мог бы распознавать его цепи: цепь через узлы, известные лишь этому пользователю, раскрывала бы, кто её создал. Даже без предварительных сведений наблюдение за тем, какие ретрансляторы используются со временем, сужало бы множество возможных клиентов.

Решение 31.4. Отберите цепи с назначением `CONFLUX_LINKED` и сгруппируйте по `CONFLUX_ID` (гл. 18), после чего сравните маршруты каждой пары. Выходной ретранслятор всегда один и тот же. Другие позиции в лабораторной сети часто тоже совпадают: например, две ветви `[test007e, test003r, test006e]` и `[test003r, test007e, test006e]` состоят из тех же трёх ретрансляторов в другом порядке. Это не ошибка tor: в лаборатории мало подходящих узлов, а Предложение 329 выражает пожелание через `SHOULD NOT`, а не строгий запрет. В публичной сети с тысячами ретрансляторов такое совпадение гораздо реже.

Решение 31.5. Упражнение с открытым ответом. Образец структуры задаёт Предложение 001: заголовок с номером, названием, автором, датой и состоянием, затем мотивация, описание решения, анализ безопасности и совместимости, план реализации и внедрения.

Приложение B: дополнительная криптография

Решение B.1. Из 256 бит пять фиксируются (три младших, старший и предпоследний), поэтому остаётся 2^{251} значений вместо 2^{256}. Это не ослабляет ключ: безопасность кривой всё равно составляет примерно 128 бит.

Решение B.2. Из $y = (u-1)/(u+1)$ следует $u = (1+y)/(1-y) \bmod p$. В Python значение y читают из 32 байтов в порядке от младшего к старшему (little-endian), очистив бит знака; затем вычисляют `(1 + y) * pow(1 - y, P - 2, P) % P` и записывают результат в 32 байта в том же порядке. Функция `montgomery_to_edwards` с исходным битом знака x восстанавливает исходный ключ Эдвардса (при $y \neq 1$).

Решение B.3. Случайно изменённая ячейка проходит проверку примерно с вероятностью $2^{-16} \cdot 2^{-32} = 2^{-48}$: два нулевых байта `recognized` и четыре верных байта дайджеста. Однако при атаке маркировки не требуется, чтобы изменённую ячейку *приняли*. Сообщник, видящий её, или конечный узел, отвергающий её, узнаёт об изменении по последствиям — например, закрытию цепи или предсказуемому содержимому — и этого достаточно для связывания двух участков цепи.

Литература

[03] *Decreto legislativo 30 giugno 2003, n. 196, Codice in materia di protezione dei dati personali.* Сводный текст, опубликованный итальянским органом по защите персональных данных (Garante per la protezione dei dati personali). 30 июня 2003. URL: https://www.garanteprivacy.it/ (дата обр. 23.09.2026).

[17] *Legge 20 novembre 2017, n. 167, art. 24 (termine di conservazione dei dati di traffico per finalità di accertamento e repressione dei reati).* Gazzetta Ufficiale, Serie Generale № 277 от 27.11.2017. 20 нояб. 2017. URL: https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:2017-11-20;167.

[22] *Regolamento (UE) 2022/2065 del Parlamento europeo e del Consiglio, del 19 ottobre 2022, relativo a un mercato unico dei servizi digitali (regolamento sui servizi digitali).* Официальный журнал ЕС L 277 от 27.10.2022. 19 окт. 2022. URL: https://eur-lex.europa.eu/eli/reg/2022/2065/oj.

[Bau+07] Kevin Bauer, Damon McCoy, Dirk Grunwald, Tadayoshi Kohno и Douglas Sicker. «Low-Resource Routing Attacks Against Tor». В: *ACM Workshop on Privacy in the Electronic Society (WPES)*. 2007.

[Boc+24] Cecylia Bocovich, Arlo Breault, David Fifield, Serene и Xiaokang Wang. «Snowflake, a censorship circumvention system using temporary WebRTC proxies». В: *Proceedings of the 33rd USENIX Security Symposium*. 2024.

[BPW13] Alex Biryukov, Ivan Pustogarov и Ralf-Philipp Weinmann. «Trawling for Tor Hidden Services: Detection, Measurement, Deanonymization». В: *IEEE Symposium on Security and Privacy*. 2013.

[Cha81] David L. Chaum. «Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms». В: *Communications of the ACM* 24.2 (1981), с. 84—88.

[Cha88] David Chaum. «The Dining Cryptographers Problem: Unconditional Sender and Recipient Untraceability». В: *Journal of Cryptology* 1.1 (1988), с. 65—75.

[DDM03] George Danezis, Roger Dingledine и Nick Mathewson. «Mixminion: Design of a Type III Anonymous Remailer Protocol». В: *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. 2003, с. 2—15.

[Día+02] Claudia Díaz, Stefaan Seys, Joris Claessens и Bart Preneel. «Towards Measuring Anonymity». В: *Privacy Enhancing Technologies (PET 2002)*. LNCS. 2002.

[Din14] Roger Dingledine. *Tor security advisory: «relay early» traffic confirmation attack.* 30 июля 2014. URL: https://blog.torproject.org/tor-security-advisory-relay-early-traffic-confirmation-attack/ (дата обр. 23.09.2026).

[DM06] Roger Dingledine и Nick Mathewson. «Anonymity Loves Company: Usability and the Network Effect». В: *Proceedings of the Fifth Workshop on the Economics of Information Security (WEIS 2006)*. 2006.

[DMS04] Roger Dingledine, Nick Mathewson и Paul Syverson. «Tor: The Second-Generation Onion Router». В: *Proceedings of the 13th USENIX Security Symposium*. 2004, с. 303—320.

[EDG09] Nathan S. Evans, Roger Dingledine и Christian Grothoff. «A Practical Congestion Attack on Tor Using Long Paths». В: *Proceedings of the 18th USENIX Security Symposium*. 2009.

[Ele] Electronic Frontier Foundation. *Legal FAQ for Tor Relay Operators*. URL: `https://community.torproject.org/relay/community-resources/eff-tor-legal-faq/` (дата обр. 23.09.2026).

[Fif+15] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann и Vern Paxson. «Blocking-resistant communication through domain fronting». В: *Proceedings on Privacy Enhancing Technologies* 2015.2 (2015), с. 46—64.

[Fro+19] Sergey Frolov, Jack Wampler, Sze Chuen Tan, J. Alex Halderman, Nikita Borisov и Eric Wustrow. «Conjure: Summoning Proxies from Unused Address Space». В: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2019.

[GRS96] David M. Goldschlag, Michael G. Reed и Paul F. Syverson. «Hiding Routing Information». В: *Information Hiding, First International Workshop*. Т. 1174. LNCS. Springer, 1996, с. 137—150.

[GSU13] Ian Goldberg, Douglas Stebila и Berkant Ustaoglu. «Anonymity and One-Way Authentication in Key Exchange Protocols». В: *Designs, Codes and Cryptography* 67.2 (2013), с. 245—269.

[Jan+14] Rob Jansen, Florian Tschorsch, Aaron Johnson и Björn Scheuermann. «The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network». В: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2014.

[JH12] Rob Jansen и Nicholas Hopper. «Shadow: Running Tor in a Box for Accurate and Efficient Experimentation». В: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 2012.

[JJ16] Rob Jansen и Aaron Johnson. «Safely Measuring Tor». В: *ACM Conference on Computer and Communications Security (CCS)*. 2016.

[Joh+13] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr и Paul Syverson. «Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries». В: *ACM Conference on Computer and Communications Security (CCS)*. 2013.

[Jua+14] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz и Rachel Greenstadt. «A Critical Evaluation of Website Fingerprinting Attacks». В: *ACM Conference on Computer and Communications Security (CCS)*. 2014.

[Jus14] Just Security. *Video clip of former director of NSA and CIA: «We kill people based on metadata»*. Johns Hopkins Foreign Affairs Symposium, 1 апреля 2014 г. 2014. URL: `https://www.justsecurity.org/10318/video-clip-director-nsa-cia-we-kill-people-based-metadata/` (дата обр. 24.09.2026).

[MD05] Steven J. Murdoch и George Danezis. «Low-Cost Traffic Analysis of Tor». В: *IEEE Symposium on Security and Privacy*. 2005.

[NBH18] Milad Nasr, Alireza Bahramali и Amir Houmansadr. «DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning». В: *ACM Conference on Computer and Communications Security (CCS)*. 2018.

[NIS24] NIST. *FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA)*. три постквантовых стандарта, опубликованные одновременно. 13 авг. 2024. URL: `https://csrc.nist.gov/pubs/fips/203/final` (дата обр. 24.09.2026).

[OON] OONI. *Specification: Tor (ts-023) and Tor using Snowflake (ts-030).* URL: https://github.com/ooni/spec/tree/master/nettests (дата обр. 23.09.2026).

[ØS06] Lasse Øverlier и Paul Syverson. «Locating Hidden Servers». В: *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 2006, с. 100—114.

[Pan+11] Andriy Panchenko, Lukas Niessen, Andreas Zinnen и Thomas Engel. «Website Fingerprinting in Onion Routing Based Anonymization Networks». В: *ACM Workshop on Privacy in the Electronic Society (WPES)*. 2011.

[Per+18] Mike Perry, Erinn Clark, Steven Murdoch и Georg Koppen. *The Design and Implementation of the Tor Browser*. Черновик. 15 июня 2018. URL: https://2019.www.torproject.org/projects/torbrowser/design/ (дата обр. 23.09.2026).

[PH10] Andreas Pfitzmann и Marit Hansen. *A terminology for talking about privacy by data minimization: Anonymity, Unlinkability, Undetectability, Unobservability, Pseudonymity, and Identity Management*. Версия 0.34. 2010.

[RR98] Michael K. Reiter и Aviel D. Rubin. «Crowds: Anonymity for Web Transactions». В: *ACM Transactions on Information and System Security* 1.1 (1998), с. 66—92.

[San13] Runa A. Sandvik. *Harvard Student Receives F For Tor Failure While Sending «Anonymous» Bomb Threat*. 18 дек. 2013. URL: https://www.forbes.com/sites/runasandvik/2013/12/18/harvard-student-receives-f-for-tor-failure-while-sending-anonymous-bomb-threat/ (дата обр. 24.09.2026).

[Sir+18] Payap Sirinam, Mohsen Imani, Marc Juarez и Matthew Wright. «Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning». В: *ACM Conference on Computer and Communications Security (CCS)*. 2018.

[Sun+15] Yixin Sun, Anne Edmundson, Laurent Vanbever, Oscar Li, Jennifer Rexford, Mung Chiang и Prateek Mittal. «RAPTOR: Routing Attacks on Privacy in Tor». В: *USENIX Security Symposium*. 2015.

[Syv11] Paul Syverson. «A Peel of Onion». В: *Proceedings of the 27th Annual Computer Security Applications Conference (ACSAC)*. 2011, с. 123—137.

[Taia] Tails. *How Tails works*. URL: https://tails.net/about/ (дата обр. 23.09.2026).

[Taib] Tails. *Warnings: Tails is safe but not magic!* URL: https://tails.net/doc/about/warnings/ (дата обр. 23.09.2026).

[TG23] Lindsey Tulloch и Ian Goldberg. «Lox: Protecting the Social Graph in Bridge Distribution». В: *Proceedings on Privacy Enhancing Technologies* 2023.1 (2023), с. 494—509.

[Thea] The Tor Project. *History*. URL: https://www.torproject.org/about/history/ (дата обр. 23.09.2026).

[Theb] The Tor Project. *Onionbalance: documentazione e guida per gli onion service v3*. URL: https://onionservices.torproject.org/apps/base/onionbalance/ (дата обр. 23.09.2026).

[Thec] The Tor Project. *Reproducible Metrics*. URL: https://metrics.torproject.org/reproducible-metrics.html (дата обр. 23.09.2026).

[Thed] The Tor Project. *Tor Browser User Manual*. URL: https://tb-manual.torproject.org/ (дата обр. 23.09.2026).

[Thee] The Tor Project. *Tor Specifications*. URL: https://spec.torproject.org/ (дата обр. 23.09.2026).

[Thef] The Tor Project. *Tor Support: Staying anonymous*. URL: https://support.torproject.org/faq/staying-anonymous/ (дата обр. 23.09.2026).

[Theg] The Tor Project. *Tor Support: Using Bridges; Unblocking Tor*. URL: https://support.torproject.org/little-t-tor/circumvention/using-bridges/ (дата обр. 23.09.2026).

[The22] The Tor Project. *Malicious relays and the health of the Tor network*. 25 апр. 2022. URL: https://blog.torproject.org/malicious-relays-health-tor-network/ (дата обр. 23.09.2026).

[The23a] The Tor Project. *Introducing Proof-of-Work Defense for Onion Services*. 23 авг. 2023. URL: https://blog.torproject.org/introducing-proof-of-work-defense-for-onion-services/ (дата обр. 23.09.2026).

[The23b] The Tor Project. *Tor is slow right now. Here is what is happening*. 7 февр. 2023. URL: https://blog.torproject.org/tor-network-ddos-attack/ (дата обр. 23.09.2026).

[The24a] The Tor Project. *Hiding in plain sight: Introducing WebTunnel*. https://blog.torproject.org/introducing-webtunnel-evading-censorship-by-hiding-in-plain-sight/. 12 марта 2024 г. 2024.

[The24b] The Tor Project. *Making new connections: from BridgeDB to Rdsys*. 2024. URL: https://blog.torproject.org/making-connections-from-bridgedb-to-rdsys/ (дата обр. 24.09.2026).

[The24c] The Tor Project. *Uniting for Internet Freedom: Tor Project & Tails Join Forces*. 26 сент. 2024. URL: https://blog.torproject.org/tor-tails-join-forces/ (дата обр. 24.09.2026).

[Tor] Tor Research Safety Board. *Tor Research Safety Board*. URL: https://research.torproject.org/safetyboard/ (дата обр. 23.09.2026).

[Tor25] Tor Project Forum. *Feedback from China [April 2025]: increased GFW censorship; obfs4/meek/snowflake unusable, webtunnel connects but is quickly blocked*. 2025. URL: https://forum.torproject.org/t/feedback-from-china-april-2025-increased-gfw-censorship-obfs4-meek-snowflake-unusable-webtunnel-connects-but-is-quickly-blocked/18233 (дата обр. 24.09.2026).

[WBU13] WBUR News. *Student Charged For Bomb Threats At Harvard*. 17 дек. 2013. URL: https://www.wbur.org/news/2013/12/17/harvard-bomb-hoax-student-charges (дата обр. 23.09.2026).

[Who] Whonix. *onion-grater*. URL: https://www.whonix.org/wiki/Onion-grater (дата обр. 23.09.2026).

[WL14] Philipp Winter и Stefan Lindskog. «Spoiled Onions: Exposing Malicious Tor Exit Relays». В: *arXiv:1401.4917*. позднее вошла, с участием других авторов, в *Privacy Enhancing Technologies Symposium* (PETS 2014). 2014.

Предметный указатель

AUTH_CHALLENGE, 39, 44, 47, 49, 56, 60, 165
AUTHENTICATE, 39, 45, 49, 50, 56
AUTHORIZE, 56

CERTS, 33, 39, 41–45, 47, 49, 56, 60, 165, 265
CREATE, 56
CREATE2, 33, 55, 56, 60, 64, 67–69, 71, 75, 76, 265, 275
CREATE_FAST, 28, 56, 70, 76, 132, 266
CREATED, 56
CREATED2, 56, 60, 67–69, 76, 276
CREATED_FAST, 56

DESTROY, 56, 57, 60, 64, 69, 76, 81, 90, 91, 94, 95, 132, 217, 265, 267, 268, 281

NETINFO, 39, 45–49, 54, 56, 60, 165

PADDING, 46, 49, 54, 56, 76, 85, 86, 265, 276
PADDING_NEGOTIATE, 56, 265

RELAY, 46, 56, 57, 60, 83, 171, 173, 215, 250, 276, 281
RELAY_EARLY, 46, 56, 57, 60, 64, 65, 69, 83, 85, 86, 215–219, 265, 276, 281

torrc
- __LeaveStreamsUnattached, 153, 156
- AccountingMax, 182, 217
- AssumeReachable, 162
- AutomapHostsOnResolve, 148
- BridgeDistribution, 136, 186, 187
- ClientOnionAuthDir, 177, 179
- ContactInfo, 182
- ControlPort, 148, 151
- CookieAuthentication, 148
- DirAuthority, 161, 274
- DirPortFrontPage, 203
- DNSPort, 91, 148
- EntryNodes, 148
- ExcludeNodes, 148
- ExitNodes, 148
- ExitPolicy, 187
- ExitPolicyRejectPrivate, 183
- ExitPolicyRejectPrivate 0, 183
- ExitRelay 0, 182
- FamilyId, 187
- HashedControlPassword, 148
- HiddenServiceDir, 175, 176, 179, 180
- HiddenServiceNonAnonymousMode, 179
- HiddenServicePort, 122, 176, 180
- HiddenServiceSingleHopMode, 179
- HSLayer2Nodes, 112
- HSLayer3Nodes, 112
- IsolateSOCKSAuth, 190, 211
- KeepalivePeriod, 38
- Log, 173
- MaxCircuitDirtiness, 147
- MaxMemInQueues, 128, 133, 134, 273
- MyFamily, 185, 187, 277
- OfflineMasterKey, 187
- OfflineMasterKey 1, 183
- ReducedExitPolicy, 183, 187
- RelayBandwidthBurst, 182
- RelayBandwidthRate, 128, 182, 277
- SafeLogging, 149, 173, 204, 239
- SafeSocks, 223, 224, 281
- SessionGroup, 147, 274
- SigningKeyLifetime, 44
- SocksPort, 147, 149
- StrictNodes, 148
- TestingDirAuthVoteGuard, 162, 275
- TestingLinkCertLifetime, 44
- TestingTorNetwork, 161, 166, 239, 274
- TestSocks, 223, 224
- TransPort, 148
- UseBridges, 273
- V3AuthDistDelay, 100
- V3AuthNIntervalsValid, 100
- V3AuthVoteDelay, 100
- V3AuthVotingInterval, 100, 161, 275

VERSIONS, 39, 47, 48, 53, 54, 56, 60, 62–64, 164, 165, 172, 237, 250, 265, 266, 276
VPADDING, 39, 48, 56

Предложения Tor
001, 243, 248, 283
214, 55
216, 71, 75
220, 39
224, 116
236, 110
271, 110
289, 58, 83, 92, 95, 96
291, 110
300, 244, 246, 248
302, 212
323, 246, 248
324, 59, 73, 92–94, 96, 243, 267, 283
327, 130
329, 59, 164, 244, 245, 248, 283
332, 72, 75, 77, 243, 283
333, 111
338, 46, 264
339, 59
340, 59, 245, 247, 283
346, 34
348, 246
350, 70, 244, 245
355, 244, 248
359, 59, 77, 82, 244, 267, 283
362, 246
инициатор, 37
ответчик, 37